

TRABAJO FIN DE MÁSTER

Pricing Intelligence en APIs de correo: catálogo de datasheets y comparación sistemática de la capacidad

Realizado por

Daniel Vela Camacho

Para la obtención del título de

Máster en Ingeniería del Software: Cloud, Datos y Gestión TI

Dirigido por

Antonio Ruiz Cortés

Manuel Resinas Arias de Reyna

Tercera convocatoria, curso 2025/26

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Resumen

Las APIs de correo electrónico se han consolidado como la pieza con la que una aplicación moderna envía correo. Proveedores como SendGrid, Resend, Mailgun o Amazon SES ofrecen servicios que permiten el envío transaccional y de marketing a través de interfaces programáticas. Sin embargo, seleccionar la API más adecuada para un proyecto concreto es una decisión mal informada. Cuántas peticiones por segundo admite un *endpoint*, cuántos destinatarios caben en una llamada o qué ocurre al superar la cuota son datos que rara vez están en la página de precios. Aparecen repartidos entre documentación de desarrollador, artículos de soporte y políticas de uso, cuando aparecen. Las comparativas que sí son fáciles de encontrar las elaboran los propios proveedores.

Esta situación dificulta la comparación objetiva y sistemática entre APIs. Pongamos un equipo que necesite enviar 100.000 correos al mes en ráfagas de 1.000 peticiones por minuto, y además con soporte de webhooks y envío en lote. No tiene forma de averiguar qué proveedor las cumple todas a la vez. Existen estándares como OpenAPI para la descripción funcional de APIs REST. Las condiciones de consumo de una API, en cambio, carecen de un equivalente asentado que las recoja de forma estructurada, comparable y procesable por máquina.

Este trabajo se enmarca en el *Pricing Intelligence*, la línea de investigación que persigue transformar la información contenida en los planes de precios en conocimiento útil para la toma de decisiones. Dentro de esa línea, trabajos previos del grupo han conceptualizado la capacidad contractual de las APIs mediante el *API Capacity Model* (ACM) y su serialización ACM2YAML. De ahí nace la *Datasheet*: un documento orientado a máquinas que reúne en una sola representación la información que está dispersa. Sobre ella operan las herramientas del ecosistema: PRIME4API como motor de análisis de la capacidad, HARVEY4API como interfaz de consulta y SPHERE como *hub* de publicación. Dichos artefactos, sin embargo, se han validado hasta ahora sobre tres pricings de dos servicios [1], y su versión operativa captura dos de las cuatro dimensiones del metamodelo.

El objetivo de este Trabajo Fin de Máster es llevar ese instrumental a un dominio real y a escala. Para ello se construye un catálogo de Datasheets de un conjunto representativo de APIs de correo electrónico, documentado de forma rigurosa a partir de la documentación técnica oficial de cada proveedor y con trazabilidad explícita de las fuentes. La confrontación del modelo con el dominio permite, además, identificar y corregir sus carencias de expresividad.

La contribución de este trabajo es cuádruple. En primer lugar, se aporta un catálogo curado y versionado de Datasheets del dominio del correo, preparado como contribución a SPHERE y reutilizable por el resto del ecosistema. En segundo lugar, se propone una evolución del esquema ACM2YAML justificada caso a caso: cada extensión responde a un escenario real que la versión previa no era capaz de representar. En tercer lugar, se caracteriza cómo interactúan los límites de uso y los *rates* en el dominio. Cuándo el *rate* impide llegar a agotar la cuota, cuándo la cuota se agota antes de

que el *rate* llegue a notarse, y qué le cuesta eso al consumidor. La relación estaba definida en el modelo, pero no caracterizada sobre un dominio real. Se analiza con la interpretación geométrica de la capacidad y las operaciones de PRIME4API. En cuarto lugar, se establece un método de comparación sistemática que responde a preguntas de decisión del tipo *"¿cuál es la API más adecuada si mis requisitos son X?"*. El método cruza los requisitos funcionales con los no funcionales en la misma consulta.

El encargo fijaba tres fuentes para seleccionar las APIs. Eran las analizadas en un Trabajo Fin de Máster previo del mismo programa, las ya modeladas en el ecosistema y una consulta documentada a un modelo de lenguaje. La primera no pudo ejecutarse, porque ese trabajo llegó con la selección ya hecha. Las otras dos y el piloto del método dan las ocho APIs del catálogo.

Los resultados muestran que estructurar formalmente el conocimiento sobre las APIs de correo hace visibles diferencias entre proveedores que las comparativas convencionales no dejan ver. Muestran también que una parte significativa de la capacidad contractual de estos servicios no llega a publicarse: eso marca el límite de lo que puede conocerse por documentación y deja el resto en manos de la medición empírica. Cabe esperar que tanto el instrumento como el método sean extensibles a otros dominios de APIs más allá del correo, aunque esa extensibilidad queda sin probar.

Palabras clave: Pricing Intelligence, APIs de correo electrónico, *datasheets* de APIs, capacidad de APIs, *rate limits*, cuotas de uso, comparación sistemática de APIs, ACM2YAML, propiedades no funcionales, selección de servicios

Abstract

Email APIs have become the component a modern application uses to send email. Providers such as SendGrid, Resend, Mailgun, and Amazon SES offer services that enable transactional and marketing email delivery through programmatic interfaces. However, choosing the most suitable API for a given project is a poorly informed decision. How many requests per second an endpoint admits, how many recipients fit into a single call, or what happens once a quota is exceeded are figures that rarely appear on the pricing page. They are spread across developer documentation, support articles, and acceptable-use policies, when they appear at all. The comparisons that are easy to find are written by the providers themselves.

This situation makes objective and systematic comparison between APIs difficult. Take a team that needs to send 100,000 emails a month in bursts of 1,000 requests per minute, with webhook support and batch sending. It has no way of finding out which provider meets all of that at once. Standards such as OpenAPI exist for the functional description of REST APIs. An API's consumption conditions, by contrast, lack an established equivalent that captures them in a structured, comparable, and machine-readable manner.

This work is framed within *Pricing Intelligence*, the research line that aims to turn the information contained in pricing plans into knowledge that supports decision-making. Within that line, previous work by the group has conceptualised the contractual capacity of APIs through the *API Capacity Model* (ACM) and its ACM2YAML serialisation. From it comes the *Datasheet*: a machine-oriented document that brings together in a single representation the information that is scattered. The tools of the ecosystem operate on it: PRIME4API as the capacity analysis engine, HARVEY4API as the query interface, and SPHERE as the publication hub. These artefacts, however, have so far been validated against three pricings of two services [1], and their operational version captures two of the four dimensions of the metamodel.

The goal of this Master's Thesis is to take that instrumentation to a real domain and at scale. To this end, a catalogue of Datasheets is built for a representative set of email APIs, rigorously documented from each provider's official technical documentation and with explicit source traceability. Confronting the model with the domain also makes it possible to identify and correct its expressiveness gaps.

The contribution of this work is fourfold. First, it delivers a curated, versioned catalogue of Datasheets for the email domain, prepared as a contribution to SPHERE and reusable by the rest of the ecosystem. Second, it proposes an evolution of the ACM2YAML schema justified case by case: every extension responds to a real-world scenario that the previous version was unable to represent. Third, it characterises how usage limits and rates interact within the domain: when the rate keeps the quota from ever being exhausted, when the quota runs out before the rate is even felt, and what that costs the consumer. The relationship was already defined in the model, but not characterised on a real domain. It is analysed through the geometric interpretation of capacity and the analysis operations provided by PRIME4API. Fourth, it establishes

a systematic comparison method that answers decision questions such as *"which API is most suitable if my requirements are X?"*. The method weighs functional and non-functional requirements in the same query.

The brief set three sources for selecting the APIs. They were those analysed in a previous master's thesis from the same programme, those already modelled within the ecosystem, and a documented query to a language model. The first could not be used, because that thesis arrived after the selection had been made. The other two, plus the pilot of the method, yield the eight APIs of the catalogue.

The results show that formally structuring knowledge about email APIs makes visible differences between providers that conventional comparisons do not reveal. They also show that a significant part of these services' contractual capacity is never published: this marks the limit of what can be known from documentation and leaves the rest to empirical measurement. Both the instrument and the method are expected to extend to API domains beyond email, although that extensibility remains untested.

Keywords: Pricing Intelligence, email APIs, API datasheets, API capacity, rate limits, usage quotas, systematic API comparison, ACM2YAML, non-functional properties, service selection

Índice general

1. Introducción **1**
- 1.1. Contexto . . . 1
  - 1.1.1. Los dos canales de un servicio SaaS . . . 1
  - 1.1.2. Pricing Intelligence y su ecosistema . . . 1
- 1.2. Motivación . . . 2
- 1.3. Objetivos y preguntas de investigación . . . 3
- 1.4. Contribuciones . . . 4
- 1.5. Estructura de la memoria . . . 4

2. Metodología y planificación **6**
- 2.1. Metodología . . . 6
- 2.2. Planificación . . . 8
- 2.3. Presupuesto . . . 10
  - 2.3.1. Presupuesto temporal . . . 10
  - 2.3.2. Presupuesto económico . . . 11
- 2.4. Desviaciones . . . 13
  - 2.4.1. Calendario . . . 13
  - 2.4.2. Horas . . . 13
  - 2.4.3. Alcance y método . . . 14

3. Estado del arte **17**
- 3.1. Modelado de pricings de SaaS . . . 17
- 3.2. Modelado de la capacidad de las APIs . . . 18
- 3.3. Análisis de la capacidad de las APIs . . . 19
- 3.4. El canal API del ecosistema: ACM, Datasheets y motor de análisis . . . 20
- 3.5. El dominio de aplicación: las APIs de correo electrónico . . . 22
- 3.6. Análisis del estado del arte . . . 23

4. Construcción del catálogo de Datasheets de APIs de correo **26**
- 4.1. Criterio de selección del dominio . . . 26
  - 4.1.1. Fuentes de inclusión . . . 26
  - 4.1.2. Criterios de inclusión y de exclusión . . . 27
  - 4.1.3. Lista de reserva y regla de reemplazo . . . 30
  - 4.1.4. Ventana de toma de datos . . . 30
- 4.2. El ciclo de construcción por API . . . 31
- 4.3. Trazabilidad y confianza . . . 31
- 4.4. Validación: de la inferencia a la evidencia . . . 33
- 4.5. Conformidad estricta y evolución del esquema: cómo una carencia llega a propuesta . . . 34
- 4.6. Calibración del método: el piloto Postmark . . . 35
- 4.7. Respuesta a la pregunta de investigación . . . 38

5. Hallazgos y comparación **41**

5.1. Introducción . . . 41
5.2. Lo que revela cada API . . . 42
5.2.1. Postmark . . . 43
5.2.2. Resend . . . 46
5.2.3. SendGrid . . . 49
5.2.4. MailerSend . . . 53
5.2.5. Brevo . . . 59
5.2.6. Amazon SES . . . 65
5.2.7. Mailgun . . . 72
5.2.8. Mailchimp Transactional (Mandrill) . . . 78
5.3. La relación entre límites de uso y *rates* . . . 83
5.4. Catálogo de preguntas de Pricing Intelligence . . . 100
5.5. Comparación sistemática sobre el catálogo . . . 106
5.5.1. Comparación funcional por lectura . . . 111
5.6. Conclusiones del capítulo . . . 113

6. Evolución del esquema ACM2YAML 117
6.1. De los elementos no soportados a las propuestas . . . 117
6.2. Dos clases de carencia . . . 118
6.3. Inventario consolidado de elementos no soportados . . . 119
6.4. Propuestas de expresividad . . . 124
6.4.1. P-01. La cuota declara volúmenes contratados . . . 124
6.4.2. P-02 y P-03. El `workload` deja de ser una terna escalar . . . 125
6.4.3. P-06. El estado de la suscripción y la reputación se declaran una vez . . . 126
6.4.4. P-07. El sobrecoste declara su tope . . . 130
6.4.5. P-09. La cuota declara qué hace con lo que la supera . . . 130
6.4.6. P-08. Un cupo que cuenta destinatarios distintos y no se repone 131
6.4.7. F-07 recurre y queda fuera . . . 132
6.5. Defectos de contrato . . . 133
6.5.1. Un defecto que no produce error: la cuota que no cambia de unidad 134
6.5.2. Una ambigüedad previa: la forma de `workload` . . . 135
6.5.3. P-04. Distinguir «sin límite» de «límite no publicado» . . . 135
6.5.4. P-05. Declarar un plan cuyo precio existe y no es público . . . 137
6.6. Observaciones que no son carencias del esquema . . . 138
6.7. Compatibilidad y vía de contribución . . . 139
6.8. Lo que queda fuera . . . 140
6.9. Respuesta a la pregunta de investigación . . . 143

7. Conclusiones 145
7.1. Respuesta a las preguntas de investigación . . . 145
7.2. Evaluación de las contribuciones frente a los objetivos . . . 147
7.3. Amenazas a la validez . . . 149
7.3.1. Validez de constructo . . . 149
7.3.2. Validez interna . . . 150
7.3.3. Validez externa . . . 152
7.3.4. Validez de conclusión . . . 153
7.4. Paquete de replicación . . . 154

7.5. Trabajo futuro 156
7.6. Conclusiones técnicas 157

8. Bibliografía **159**

A. Uso de inteligencia artificial **164**
A.1. Alcance del uso 164
A.2. La consulta de la selección 165
A.3. Prompt literal 165
A.4. Respuesta íntegra 166
A.5. Cómo se aplicó 167

Índice de figuras

2.1. Cronograma previsto frente al real 10

4.1. Estructura comercial de Postmark frente a su proyección en el bloque `plans` de ACM2YAML v0.3 37

5.1. Curva de capacidad $C(t)$ de Resend Pro, SendGrid Essentials 50K, MailerSend Hobby y Amazon SES 90
5.2. Curva de capacidad de MailerSend Hobby en la dimensión de peticiones 99

Índice de extractos de código

4.1. Serialización del semáforo de confianza en la Datasheet del piloto (fragmento). 32
6.1. P-01: volúmenes contratados dentro de un plan, con precio y sobrecoste propios (caso Resend, plan Scale; se muestran tres de los seis). 124
6.2. P-02 y P-03: composición de dimensiones anidadas y cota expresada en otra métrica. 126
6.3. P-06: los estados de la suscripción y de la reputación declarados una vez, con su disparador y con la carga por petición que la guía no documenta (caso MailerSend). 128
6.4. P-09: una cuota que retiene lo que la supera hasta un tope fijo (caso Brevo, plan gratuito, correo transaccional). 131

1. Introducción

1.1. Contexto

Este trabajo se ha realizado en el contexto de las líneas de investigación del grupo ISA (Ingeniería del Software Aplicada) de la Universidad de Sevilla, y en particular de la línea de *Pricing Intelligence* [2].

1.1.1. Los dos canales de un servicio SaaS

Un servicio ofrecido bajo el modelo *Software as a Service* (SaaS) puede ser consumido a través de distintos canales de interacción. El más visible es habitualmente un *frontend* web, orientado a usuarios humanos, en el que las condiciones de contratación se publican de forma tabular mediante planes, características (*features*) y complementos (*add-ons*). Junto a él, el mismo servicio puede exponer una API (REST o GraphQL) para que otros sistemas consuman sus capacidades de forma programática.

Aunque ambos canales describen aspectos del mismo servicio, cada uno utiliza fuentes, estructuras y terminologías distintas. El canal web habla de planes y funcionalidades; el canal API habla de *endpoints*, límites de velocidad (*rates*), cuotas de volumen y reglas de consumo. La Sección 3.4 precisa estos términos: con la definición del NIST, lo que aquí se llama canal es una interfaz del servicio.

En el dominio del correo electrónico esta dualidad es especialmente marcada. Proveedores como SendGrid, Mailgun, Postmark, Resend o Amazon SES publican una página de precios orientada a la contratación y, en paralelo, una documentación técnica donde residen las condiciones que realmente gobiernan el consumo programático. La información determinante para un integrador no siempre aparece en la página de precios: cuántas peticiones por segundo admite un endpoint, cuántos destinatarios caben en una llamada, qué ocurre al superar la cuota. Está dispersa entre documentación de desarrollador, artículos de soporte, especificaciones OpenAPI y políticas de uso aceptable. El Capítulo 5 mide, caso por caso, cuánto de esa información falta.

1.1.2. Pricing Intelligence y su ecosistema

El *Pricing Intelligence* se define como una disciplina dirigida por datos cuyo objetivo es transformar la información contenida en los planes de precios en conocimiento útil para la toma de decisiones [2]. Recopilar precios es solo el punto de partida. El objetivo es responder de forma automática a preguntas como qué plan se ajusta mejor a un patrón de consumo dado, o qué diferencias relevantes existen entre dos proveedores.

Alrededor de esta idea, el grupo ha construido un ecosistema de herramientas interoperables. Para el canal web, el metamodelo *iPricing* y su serialización Pricing2YAML representan los planes de un SaaS [3]. Sobre ellos operan A-MINT, ex-

tractor de pricings desde páginas web mediante modelos de lenguaje [4], y PRIME, validador y analizador de los modelos resultantes [3]. HARVEY, presentado en CAiSE 2026, es el orquestador de estas capacidades tras una interfaz conversacional [2]. Finalmente, SPHERE es el *hub* del ecosistema: ahí se publican los artefactos, ahí se ven y desde ahí se comparten [5].

La extensión de este ecosistema al canal API es reciente. El Trabajo Fin de Máster de Gavira Sánchez [1], en coordinación con el de Ruiz López [6], conceptualiza la capacidad contractual de una API en el *API Capacity Model* (ACM) y define ACM2YAML, la serialización que da lugar a la **Datasheet**. La Datasheet es un documento orientado a máquinas que reúne en una sola representación la información tabular del pricing y la que aparece dispersa en la documentación técnica. Sobre ella operan PRIME4API [7], motor de análisis de la capacidad, y HARVEY4API [6], el agente que lo consume.

1.2. Motivación

El responsable técnico que debe elegir entre una docena de proveedores rara vez encuentra en la página de precios los tres datos de consumo que enumera la Sección 1.1.1. Se enfrenta, además, a documentaciones heterogéneas y a comparativas elaboradas por los propios vendedores [8, 9]. Y le falta de forma sistemática la información que más le importa: la que determina si el servicio soportará su patrón de consumo real.

El ecosistema descrito en la sección anterior ofrece el instrumental necesario para abordar el problema de forma rigurosa. Sin embargo, ese instrumental se ha validado hasta ahora sobre tres pricings de dos servicios [1], y su versión operativa captura dos de las cuatro dimensiones de capacidad del metamodelo que la sustenta. Existe, por tanto, una distancia entre lo que el modelo promete y lo que se ha demostrado que soporta.

Esa distancia es la que motiva este trabajo, y es también la que determina su dificultad. Los motores de análisis ya existen, así que el reto no está en la complejidad algorítmica. Está en tres frentes de naturaleza distinta. El primero es de reconstrucción. La capacidad contractual de una API real rara vez está publicada en un único lugar, y en ocasiones sencillamente no está publicada. Documentarla exige localizar, integrar y contrastar fuentes heterogéneas y con frecuencia incompletas. El segundo es de modelado. Cada servicio que no encaja en el esquema obliga a discernir si es un caso particular que puede simplificarse o una carencia real del modelo que debe corregirse, y esa distinción no admite automatización. El tercero es de coordinación. Este trabajo se construye sobre artefactos que otros trabajos desarrollan en paralelo, y cualquier cambio propuesto debe verificarse contra el resto del ecosistema.

1.3. Objetivos y preguntas de investigación

Del contexto y la motivación expuestos se deriva el objetivo global de este Trabajo Fin de Máster.

Objetivo del Trabajo Fin de Máster
Construir un espacio de conocimiento sobre las APIs de correo electrónico mediante Datasheets conformes al ACM, y establecer un método de comparación sistemática que responda a necesidades funcionales y no funcionales, evolucionando el modelo con las carencias que el dominio revele.

Este objetivo se articula en torno a cuatro preguntas de investigación.

RQ1. ¿Qué APIs de correo deben incluirse en un análisis representativo del dominio, y bajo qué criterios reproducibles?

RQ2. ¿Es ACM2YAML suficientemente expresivo para representar la capacidad contractual de las APIs de correo? ¿Qué extensiones requiere?

RQ3. ¿Cómo se relacionan los límites de uso y los *rates* en las APIs de correo, y qué implicaciones tiene esa relación para el consumidor?

RQ4. ¿Cómo pueden compararse las APIs de correo de forma sistemática para responder a preguntas de decisión que involucran simultáneamente requisitos funcionales y no funcionales?

De ellas se derivan los siguientes objetivos específicos.

OBJ-01 Selección justificada del dominio. Definir y aplicar un criterio de inclusión reproducible que determine el conjunto de APIs analizadas, documentando el proceso de forma que un tercero pueda replicarlo.

OBJ-02 Construcción del catálogo de Datasheets. Documentar la capacidad contractual de cada API seleccionada conforme a ACM2YAML, con trazabilidad explícita de las fuentes de cada dato.

OBJ-03 Evolución del esquema. Identificar las carencias de expresividad que el dominio revele y proponer las extensiones correspondientes, justificando cada una con el caso real que la motiva.

OBJ-04 Análisis de la capacidad. Caracterizar cómo se relacionan los límites de uso y los *rates* en el dominio, con la lectura geométrica de la capacidad y las operaciones de análisis de PRIME4API.

OBJ-05 Método de comparación. Establecer un catálogo de preguntas de decisión propias del dominio y un procedimiento para responderlas sobre el conjunto de Datasheets.

El componente de investigación del trabajo reside principalmente en OBJ-03 y OBJ-04. La construcción del catálogo es condición necesaria, pero es un medio. El resultado investigable es lo que la confrontación entre un modelo y un dominio real

revela sobre las limitaciones del primero, y el conocimiento que puede extraerse del segundo una vez formalizado.

1.4. Contribuciones

El desarrollo de estos objetivos se materializa en cuatro contribuciones.

C-01 Catálogo de Datasheets de APIs de correo. Un conjunto curado y versionado de Datasheets del dominio, con trazabilidad de fuentes y preparado como contribución a SPHERE, reutilizable por el resto de herramientas del ecosistema.

C-02 Evolución de ACM2YAML. Una propuesta de extensión del esquema justificada caso a caso, en la que cada elemento nuevo responde a un escenario real que la versión previa no era capaz de representar.

C-03 Caracterización de la relación entre límites y *rates*. Un análisis de cómo interactúan ambas dimensiones en el dominio del correo y de sus consecuencias para el consumidor. La relación estaba definida en el modelo, pero no caracterizada sobre un dominio real (Sección 3.6).

C-04 Método de comparación sistemática. Un catálogo de preguntas de decisión funcionales y no funcionales, junto con el procedimiento para resolverlas sobre las Datasheets.

Estas contribuciones tienen aplicación académica e industrial. En el ámbito académico, el catálogo de Datasheets es un banco de pruebas sobre un dominio real para las herramientas del ecosistema, que hasta ahora se han validado sobre tres pricings de dos servicios [1]. Las extensiones propuestas al esquema revierten directamente sobre el modelo. En el ámbito industrial, el instrumento aborda una decisión que los equipos de desarrollo toman de forma recurrente y sin base comparable. Cabe esperar que el método y el esquema se extiendan a otros dominios de APIs más allá del correo electrónico, aunque esa extensibilidad queda sin probar (Sección 7.3.3).

1.5. Estructura de la memoria

El **Capítulo 1** introduce el contexto del trabajo, presenta los dos canales de acceso a un SaaS y el ecosistema de Pricing Intelligence, formula la motivación y los objetivos, y enumera las contribuciones realizadas.

El **Capítulo 2** describe la metodología seguida y la planificación del proyecto, con su estructura de paquetes de trabajo, el presupuesto estimado y las desviaciones observadas.

El **Capítulo 3** revisa el estado del arte en el modelado de pricings de SaaS, en el modelado y análisis de la capacidad de APIs, y en las herramientas del ecosistema de Pricing Intelligence sobre las que se construye este trabajo.

El **Capítulo 4** presenta el método con el que se construye el catálogo (criterio de selección del dominio, ciclo de construcción por API, reglas de trazabilidad y confianza) y su calibración sobre un piloto real. Cierra la RQ1 con la condición que debe cumplir el conjunto de APIs del catálogo (Sección 4.7).

El **Capítulo 5** recoge lo que las APIs del catálogo revelan. Reúne los hallazgos de cada una, la relación observada entre límites de uso y *rates*, y el catálogo de preguntas de Pricing Intelligence con el que se comparan de forma sistemática. Las Datasheets completas no se reproducen en la memoria. Se conservan, con sus fuentes y evidencias, en el repositorio de datos que acompaña al trabajo.

El **Capítulo 6** formula la evolución del esquema ACM2YAML. Recoge el inventario de elementos no soportados levantado sobre las ocho APIs del catálogo, aplica los criterios de promoción y propone las extensiones que el dominio justifica, cada una con el caso real que la motiva.

El **Capítulo 7** cierra la memoria. Responde a las cuatro preguntas de investigación y evalúa las contribuciones frente a los objetivos. Después agrupa las amenazas a la validez y describe el paquete de replicación. Termina con las líneas de trabajo futuro y con las conclusiones técnicas.

El Anexo A declara el uso de inteligencia artificial en el trabajo, con la consulta que completó la selección de APIs.

2. Metodología y planificación

Este capítulo describe cómo se ha desarrollado el trabajo y se organiza del siguiente modo. En primer lugar, la Sección 2.1 presenta el enfoque metodológico adoptado y lo relaciona con los objetivos y las contribuciones enunciados en el Capítulo 1. A continuación, la Sección 2.2 recoge la planificación del proyecto. Después, la Sección 2.3 presenta el presupuesto, previsto y real. Por último, la Sección 2.4 recoge las desviaciones observadas respecto al plan.

2.1. Metodología

El trabajo adopta el enfoque de investigación en ciencia del diseño (*Design Science Research*, DSR) formulado por Hevner et al. [10]. Se elige porque el objetivo de la Sección 1.3 pide construir artefactos que resuelvan un problema relevante y evaluar con rigor su utilidad, y no describir un fenómeno. Para operacionalizar el enfoque se sigue el proceso DSRM de Peffers et al. [11], que ordena la investigación en seis actividades y admite iterar entre ellas.

En términos de DSR, este trabajo produce artefactos de tres tipos. El catálogo de Datasheets es una instanciación (contribución C-01): un conjunto de documentos conformes a ACM2YAML v0.3 salvo dos formas que la guía no define (Sección 4.5), escritos para que las herramientas del ecosistema los consuman sin adaptación. El método de comparación sistemática es un método (contribución C-04). El método de construcción del catálogo lo es también, y la memoria lo cuenta dentro de C-01. Las extensiones propuestas al esquema son un modelo (contribución C-02). Cómo se relacionan límites y *rates* (contribución C-03) es el conocimiento que resulta de analizar la instanciación con la lectura geométrica de la capacidad y las operaciones de análisis de PRIME4API. Ese modelo (C-02) no es un metamodelo nuevo, y el trabajo tampoco crea un motor de análisis nuevo (Capítulo 3): aplica el ACM y su serialización a un dominio completo y propone las extensiones que ese dominio justifica.

Las seis actividades del proceso DSRM se corresponden con los capítulos de la memoria del siguiente modo.

A1. Identificación del problema y motivación. El Capítulo 1 plantea el problema: la capacidad contractual de una API de correo está dispersa entre fuentes heterogéneas y el instrumental del ecosistema se ha validado sobre pocos servicios. El Capítulo 3 sitúa ese problema respecto a los trabajos previos sobre modelado de pricings y de capacidad de APIs.

A2. Definición de los objetivos de la solución. La Sección 1.3 formula las preguntas RQ1 a RQ4 y los objetivos OBJ-01 a OBJ-05 que se derivan de ellas. Cada actividad posterior se evalúa contra esos identificadores.

A3. Diseño y desarrollo. El Capítulo 4 define el criterio de selección del dominio (OBJ-01) y el ciclo de cinco fases con el que se documenta cada API (Sección 4.2).

Las reglas de trazabilidad y el semáforo de confianza (Sección 4.3) y la escalera de validación (Sección 4.4) hacen revisable cada dato (OBJ-02). El desarrollo produce el catálogo, que parte de ocho APIs, y, junto a él, el inventario de elementos del dominio que el esquema no soporta, del que parte OBJ-03. Sobre ese inventario el Capítulo 6 diseña las extensiones del esquema, en una segunda pasada por esta actividad. La Sección 5.4 diseña el método de comparación (C-04): el catálogo de preguntas de decisión y la clase que dice con qué instrumento se responde cada una.

A4. Demostración. Las Datasheets del catálogo se ejecutan con PRIME4API y se preparan como contribución a SPHERE. La ejecución comprueba, dentro del ecosistema al que se destinan, cuánto de cada una consume el motor, y ninguna de las ocho resulta plenamente consumible (Tabla 6.5). El catálogo cubre un dominio completo y no un servicio aislado, y sobre él el Capítulo 5 aplica el análisis de capacidad y el método de comparación (OBJ-04 y OBJ-05).

A5. Evaluación. La conformidad de cada Datasheet se comprueba con el validador de formato de este trabajo, y su consumo, con la verificación por uso sobre PRIME4API (Sección 4.5 y Capítulo 6). Las fuentes de cada dato las contrasta un verificador que vuelve a abrir cada URL citada. Es un agente de inteligencia artificial distinto del que construye la Datasheet (Anexo A). Los elementos no soportados se someten a un criterio de promoción de dos vías: la recurrencia en al menos dos APIs del dominio, o una sola API que cumple tres condiciones. El hecho está citado o medido, la consecuencia está ejecutada sobre el consumidor de referencia y la carencia es propia del esquema. Solo el elemento promovido cuya forma se contrasta con la guía llega a propuesta (Sección 4.5). El método de comparación (C-04) se evalúa aplicándolo sobre las ocho APIs (OBJ-05). Cada pregunta del catálogo se clasifica por el instrumento que la responde, sea la Datasheet, PRIME4API, HARVEY4API o la matriz de *features*, o por lo que falta para responderla. La caracterización de C-03 se apoya en las operaciones de análisis del mismo motor sobre las mismas Datasheets (OBJ-04). Esa evaluación se reúne y se confronta con las contribuciones en el Capítulo 7.

A6. Comunicación. Esta memoria, con sus conclusiones en el Capítulo 7, es el vehículo principal. El catálogo se publica en un repositorio abierto, con sus registros de fuentes y de elementos no soportados, y se propone a SPHERE como colección. Las extensiones del esquema se elevarán como propuestas al repositorio de la guía del ecosistema.

El proceso no se recorre una sola vez: un piloto (Postmark) calibró el ciclo de construcción antes de escalarlo al resto del dominio (Capítulo 4). Ese piloto fijó también el formato de presentación. El catálogo es mutable y cada sustitución se anota con fecha y motivo (Sección 4.1). Las decisiones de criterio que surgen al modelar (qué producto se modela cuando un proveedor vende varios, qué se considera cuota y qué no) se registran y se convierten en reglas de un libro de criterios. Así una pregunta resuelta en una API no vuelve a plantearse en la siguiente, y el catálogo se construye con un criterio uniforme. Ese libro forma parte del paquete de replicación como `03_datos/CRITERIOS.md` (Tabla 7.2). Las decisiones de alcance quedan registradas con fecha y motivo en un registro de decisiones, que es la fuente de la sección de desviaciones

de este capítulo.

El enfoque tiene un límite conocido: DSR exige evaluar con rigor la utilidad del artefacto, y este trabajo la establece por uso de las herramientas del ecosistema, sin un estudio con integradores.

2.2. Planificación

El Trabajo Fin de Máster tiene asignados 12 créditos ECTS [12]. A razón de 25 horas por crédito, el mínimo que fija la norma [13], la dedicación esperada es de 300 horas. El alcance del trabajo se fijó en la reunión de arranque con los tutores del 13 de febrero de 2026. Con ese alcance, la línea base del proyecto reparte la dedicación esperada entre siete paquetes de trabajo (Tabla 2.1).

Dos paquetes son transversales: la gestión del proyecto (PT0) y la memoria (PT6) acompañan a todo el trabajo. Los otros cinco siguen el orden del método. El estudio previo (PT1) prepara la selección y el piloto (PT2), y el piloto calibra la construcción del catálogo (PT3). El análisis de límites y *rates* (PT4) y la comparación sistemática (PT5) trabajan sobre el catálogo construido. Cada paquete tiene un rol, que fija su tarifa en el presupuesto (Sección 2.3).

Paquete	Contenido	Objetivo	Rol	Horas
PT0. Gestión y seguimiento	Reuniones con los tutores, planificación y seguimiento, registro de decisiones y control de versiones	—	Gestor de proyecto	20
PT1. Estudio previo	Estudio del ecosistema de Pricing Intelligence y de los trabajos previos sobre modelado de pricings y de capacidad de APIs	—	Analista de sistemas	30
PT2. Selección y piloto	Criterio de inclusión, selección de las APIs del dominio y Datasheet piloto con la que se calibra el ciclo	OBJ-01	Analista de sistemas	25
PT3. Construcción del catálogo	Datasheet de cada API con sus fuentes, su observación de panel y sus sondeos, registro de las fricciones con el esquema y extensiones que proponen	OBJ-02 y OBJ-03	Desarrollador	90
PT4. Límites y *rates*	Ejecución de las Datasheets en PRIME4API y caracterización de cómo se relacionan límites y *rates*	OBJ-04	Analista de sistemas	30
PT5. Comparación sistemática	Preguntas de decisión del dominio y método para responderlas con el instrumental del ecosistema	OBJ-05	Analista de sistemas	25
PT6. Memoria y defensa	Redacción de la memoria, depósito y preparación de la defensa	—	Gestor de proyecto	80
Total				**300**

Tabla 2.1: Paquetes de trabajo de la línea base, con el objetivo de la Sección 1.3 al que responde cada uno, su rol y las horas previstas.

La construcción del catálogo es el paquete mayor porque la línea base preveía diez a doce APIs, la cifra del encargo. La memoria es el segundo: incluye la defensa y empieza en marzo, sin esperar a que acabe el catálogo. El trabajo de Gavira Sánchez [1] concluye que dejar la síntesis y el análisis para después de la implementación desgasta, y que el orden inverso habría sido más productivo. El reparto por paquete es nominal. Parte del alcance acordado en la reunión de arranque y de la dedicación esperada, y el primer plan escrito es posterior, del 2 de agosto de 2026 (Sección 2.4).

La línea base repartía el trabajo entre febrero y septiembre, en paralelo a las asignaturas del segundo cuatrimestre, con el depósito en la convocatoria de septiembre. La Figura 2.1 muestra la ventana prevista de cada paquete y, debajo, las horas reales de cada mes. El estudio previo y la selección ocupaban el primer tramo, hasta mediados de abril. La construcción del catálogo iba de abril a mediados de julio, y el análisis de límites y la comparación se solapaban con su final. La Sección 2.4 explica por qué las

horas reales caen en otros meses.

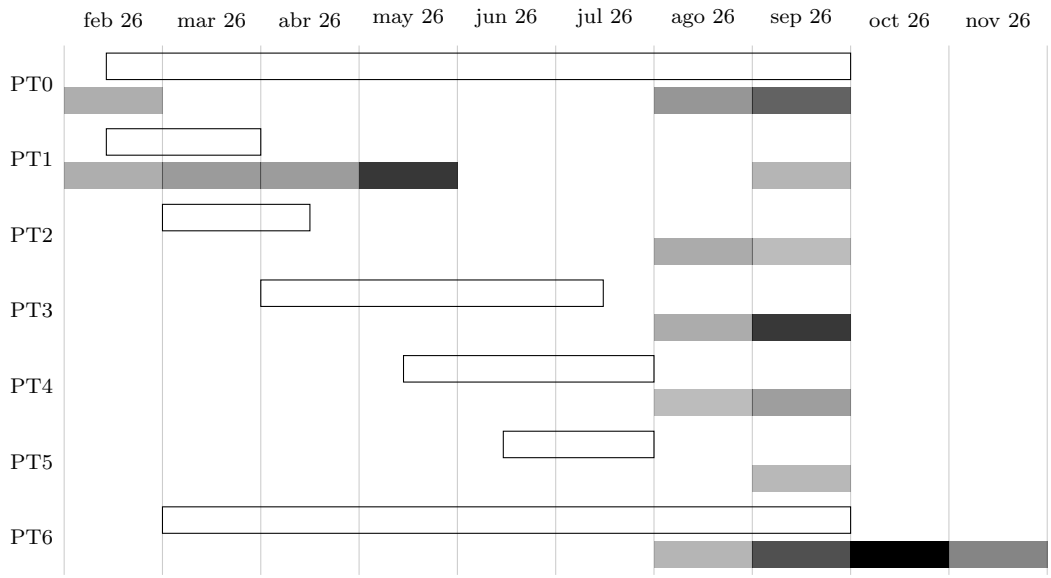


Figura 2.1: Cronograma del proyecto. Por cada paquete, la barra sin relleno es su ventana en la línea base, y las celdas rellenas de debajo son las horas reales de cada mes, más intensas cuantas más horas. Las de octubre y noviembre son previsión.

2.3. Presupuesto

El presupuesto sigue el modelo de Gavira Sánchez [1]. El temporal se da en horas por paquete, y el económico suma tres partidas: el personal, la amortización del equipo y los costes fijos, con una contingencia encima. El personal se calcula dos veces, con las horas de la línea base y con las horas reales. La amortización y los costes fijos no dependen de las horas: se calculan una vez, sobre el calendario real, y entran igual en las dos columnas. Así, la diferencia entre previsto y real es solo la de las horas.

2.3.1. Presupuesto temporal

Las horas previstas son las de la línea base (Tabla 2.1). Las reales no se llevaron en un parte de horas, y se reconstruyen de tres formas según el tramo.

Medidas. Del 7 de septiembre de 2026 al 29 de septiembre de 2026, las horas salen de los registros de sesión de Claude Code, el asistente de programación con el que se ha trabajado. Solo cuentan los mensajes escritos por el autor. No cuentan los avisos que el propio asistente inserta, ni las peticiones automáticas de revisión de seguridad. Una sesión agrupa los mensajes separados por 45 minutos o menos, y dura del primero al último más 15 minutos. El resultado son 62 horas en 15 días. La media es de 4,3 horas por día con actividad, sobre 14 de esos días: del 7 de septiembre de 2026 solo se midió la noche, y no entra.

Estimadas. Antes del 7 de septiembre de 2026 no hay registros de sesión utilizables. El 8 y el 9 de septiembre se trabajó en otro entorno, que no los deja, y lo mismo ocurrió durante el día del 7 de septiembre de 2026. Con esa media se valoran 11 jornadas de agosto y septiembre con trabajo registrado en el repositorio fuera de las sesiones medidas. Suman 47 horas. Una es el día del 7 de septiembre de 2026, que se suma a su noche. La fase de febrero a mayo se estima por tramos: la reunión de arranque (4 horas), la lectura gradual del estado del arte hasta abril (20 horas) y la puesta al día de mayo (30 horas).

Pendientes. Lo posterior al corte es previsión. La revisión de la memoria llega hasta su cierre, el 29 de octubre, para el depósito del 2 de noviembre (40 horas). La preparación de la presentación y de la defensa, del 9 al 13 de noviembre, suma 15 horas.

Cada sesión medida se reparte entre paquetes por el vocabulario de sus mensajes. Cada paquete tiene una lista de términos, y la sesión se divide en proporción a las apariciones de cada uno. Los días estimados se reparten por las actividades que el repositorio registra ese día. La medición tiene límites en los dos sentidos. No ve el trabajo fuera del asistente, como la lectura o la revisión. Tampoco ve el trabajo en otro entorno, que se estima aparte cuando el repositorio lo registra. Y cuenta como trabajo las pausas de hasta 45 minutos dentro de una sesión y la cola de 15. En total salen 219 horas reales frente a 300 previstas, y la Sección 2.4 las desglosa por paquete.

2.3.2. Presupuesto económico

Costes de personal. Las tarifas por rol salen de la Instrucción 1/2023 de la Agencia Digital de Andalucía [14], que fija los precios medios de referencia de los perfiles TIC en las contrataciones de la Junta de Andalucía. El trabajo de Gavira Sánchez [1] usa un informe de 2018 de la Junta, y aquí se usa esta instrucción porque es posterior. Sus precios son los que cobra una empresa contratista, e incluyen sus gastos generales (14,91 %) y su beneficio industrial (7,63 %). La propia instrucción deja los costes directos en el 77,46 % del precio, y la tarifa de cada rol es esa parte (Tabla 2.2). La instrucción calcula su precio mínimo sumando los gastos generales y el beneficio al salario con Seguridad Social. Con esa misma estructura, la parte de costes directos ya incluye la cotización, y no se suma aparte.

Rol	Precio medio	Tarifa	Horas previstas	Coste previsto	Horas reales	Coste real
Gestor de proyecto	56,65	43,88	100	4.388,00	116	5.090,08
Analista de sistemas	45,30	35,09	110	3.859,90	69	2.421,21
Desarrollador	35,80	27,73	90	2.495,70	34	942,82
Total personal			300	10.743,60	219	8.454,11

Tabla 2.2: Costes de personal por rol. El precio medio es el de referencia de la Instrucción 1/2023, sin IVA, y la tarifa es su parte de costes directos, en euros por hora. Los costes van en euros.

Amortización del equipo. El trabajo se ha hecho con un portátil Huawei MateBook D 16 (2024), con procesador Intel Core i5-13420H y 16 GB de memoria RAM. Como coste de adquisición se toma el precio de partida de la gama en España, 799,00 € [15], con una vida útil de 60 meses. La amortización lineal sobre los 9 meses que van del arranque a la defensa da el coste imputable:

$$C_{\text{amortización}} = \frac{799{,}00}{60} \times 9 = 119{,}85 \text{ €} \tag{2.1}$$

Costes fijos. Recogen la suscripción al asistente y los suministros del puesto de trabajo (Tabla 2.3). La suscripción a Claude Max 5x se imputa por el único mes pagado, con el importe de su factura. Para la conexión a internet y la electricidad se toman las mismas cuotas de referencia que Gavira Sánchez [1], imputadas en los 8 meses con trabajo: de febrero a mayo y de agosto a noviembre.

Concepto	**Coste mensual (€)**	**Meses**	**Total (€)**
Suscripción a Claude Max 5x	90,00	1	90,00
Conexión a internet	30,00	8	240,00
Electricidad	20,00	8	160,00
Total costes fijos			490,00

Tabla 2.3: Costes fijos del proyecto.

Los proveedores del catálogo no suman coste, porque en ninguno se contrató un plan de pago. Resend, Mailgun y Brevo se usaron con su plan gratuito, y Amazon SES en su entorno de pruebas. Postmark y MailerSend se usaron con una cuenta sin aprobar, y SendGrid y Mailchimp Transactional con una cuenta de prueba. La infraestructura tampoco suma: PRIME4API y una instancia local de SPHERE se ejecutaron en el mismo equipo, y la memoria se compila en local.

Presupuesto total. Como en Gavira Sánchez [1], no se aplica beneficio industrial, porque es un trabajo de investigación. Sí se añade una contingencia del 10 % sobre el subtotal previsto (Tabla 2.4).

Partida	**Previsto (€)**	**Real (€)**
Personal	10.743,60	8.454,11
Amortización del equipo	119,85	119,85
Costes fijos	490,00	490,00
Subtotal	11.353,45	9.063,96
Contingencia	1.135,35	—
Total	**12.488,80**	**9.063,96**

Tabla 2.4: Presupuesto total del proyecto. El previsto aplica las horas de la línea base y suma la contingencia. El real aplica las horas reales, con las pendientes incluidas, y no la necesita.

Respecto al subtotal previsto, el coste real se desvía −2.289,49 €, sin tocar la contingencia. La diferencia sale entera del personal: las horas de analista y de desarrollador bajan más de lo que suben las de gestor (Tabla 2.2).

2.4. Desviaciones

Las desviaciones se analizan en tres planos: el calendario, las horas de cada paquete y las decisiones que cambiaron el alcance o el método. El calendario se lee en la Figura 2.1, las horas en la Tabla 2.5, y las decisiones salen del registro de decisiones (Sección 2.1).

2.4.1. Calendario

El trabajo no siguió el calendario de la línea base. Durante el segundo cuatrimestre, la dedicación al trabajo se compaginó con las asignaturas del máster y con un empleo, y se quedó en el estudio previo. Hubo 20 horas de lectura gradual hasta abril y una puesta al día de 30 horas en mayo, y en junio y julio no consta trabajo. El grueso se concentró en agosto y septiembre, cuando la compaginación lo permitió. Del 11 al 31 de agosto la actividad bajó por un viaje, y no consta trabajo en esas fechas.

El autor elaboró el plan al retomar el trabajo. El 2 de agosto de 2026, su plan por fases ordenó lo pendiente: arranque, selección, construcción, análisis de límites, comparación y redacción. No tenía fechas, porque la convocatoria aún no estaba decidida. El 1 de septiembre, con la de septiembre ya fuera de alcance, se fijó la tercera convocatoria y, con ella, el primer calendario con fechas. Ese calendario reparte las seis semanas hasta el 12 de octubre entre las tareas del catálogo y las de la memoria.

La construcción del catálogo cumplió ese calendario. Ninguna Datasheet se cerró después de su semana prevista, y las de SendGrid, Brevo y Mailchimp Transactional se cerraron antes. El catálogo quedó completo el 21 de septiembre, una semana antes de lo previsto.

2.4.2. Horas

La Tabla 2.5 compara las horas previstas y las reales de cada paquete, con las reales separadas en medidas, estimadas y pendientes (Sección 2.3.1).

Paquete	**Previstas**	**Medidas**	**Estimadas**	**Pendientes**	**Reales**	**Desviación**
PT0. Gestión	20	15	19	0	34	+14
PT1. Estudio previo	30	2	50	0	52	+22
PT2. Selección	25	1	5	0	6	−19
PT3. Catálogo	90	22	12	0	34	−56
PT4. Límites	30	5	4	0	9	−21
PT5. Comparación	25	0	2	0	2	−23
PT6. Memoria	80	17	10	55	82	+2
Total	300	62	102	55	219	−81

Tabla 2.5: Horas previstas y reales por paquete de trabajo, con el nombre abreviado de la Tabla 2.1. Las medidas van del 7 de septiembre de 2026 al 29 de septiembre de 2026, y las pendientes son previsión hasta la defensa.

El total real queda en 219 horas, una desviación de −81 horas sobre las 300 previstas (−27,0 %). Por paquete, la diferencia se concentra en tres puntos.

El primero es la construcción del catálogo (PT3), con −56 horas. La línea base preveía diez a doce APIs, y el 1 de septiembre el catálogo se fijó en ocho (Tabla 2.6). Además, parte del trabajo de cada API, como la recolección de fuentes, los sondeos y la validación de formato, se ejecutó con el asistente de programación (Sección 2.3.2), y eso reduce las horas que exige cada una.

El segundo es el análisis de límites y *rates* (PT4) y la comparación sistemática (PT5), que se quedan en 9 y 2 horas propias. Los dos se hicieron sobre cada Datasheet y no como fases aparte. Los sondeos de cada una ya miden sus límites. La matriz de *features* con la que se compara se captura en el ciclo de cada API desde MailerSend, y en las tres primeras se construyó después, desde sus fuentes (Sección 5.5.1). El reparto por vocabulario no separa ese trabajo: una sesión que construye una Datasheet habla de la Datasheet aunque mida sus límites, y cuenta en PT3. Escribir sus secciones cuenta en PT6. Por eso la desviación de PT4 y PT5 se lee junto a la de PT3.

El tercero son los paquetes que suben. El estudio previo (PT1) crece +22 horas. De sus horas, 50 son la estimación del autor para febrero a mayo, y la puesta al día de mayo cae fuera de la línea base, que cerraba el estudio previo a mediados de abril. La gestión (PT0) crece +14, por el seguimiento del propio proceso: el registro de decisiones, el control de versiones y la coordinación del trabajo con el asistente. La memoria (PT6) cierra cerca de lo previsto, +2, con 55 horas todavía por delante. La selección y el piloto (PT2) ocuparon sobre todo tres días, el piloto de Postmark el 2 y el 3 de agosto y la selección el 1 de septiembre, y se quedan en 6 horas.

2.4.3. Alcance y método

Las decisiones de modelado, como qué se considera cuota, pasan a reglas del libro de criterios y no se cuentan como desviaciones: su efecto está en los Capítulos 4 a 6. La Tabla 2.6 recoge los cambios de alcance y de método respecto a la línea base, al plan del 2 de agosto y al calendario del 1 de septiembre, con la fecha de la decisión que los fija.

Fecha	Previsto	Real	Motivo registrado
3 ago	Una versión nueva del esquema (v0.8 o v1.x)	Datasheets conformes a v0.3. Las carencias se registran como fricciones, y las que cumplen el criterio de promoción se proponen como extensiones (Capítulo 6)	Mantener las Datasheets consumibles por las herramientas del ecosistema sin bifurcar el esquema
5 ago	Observación de panel solo en parte de las APIs, con el peldaño de validación alcanzado declarado en cada una	Observación de panel con cuenta propia en las ocho APIs. Sin ella una Datasheet no se cierra, lo que exige una cuenta por proveedor	Sin la misma observación en todas las APIs, el semáforo de confianza deja de medir lo que dice medir
1 sep	Depósito en la convocatoria de septiembre	Tercera convocatoria, con la entrega el 16 de octubre y la defensa del 9 al 13 de noviembre de 2026, y un calendario semanal hasta la entrega	La dedicación del segundo cuatrimestre, compaginada con las asignaturas y con un empleo, no alcanzó para depositar en septiembre
1 sep	Diez a doce APIs	Ocho APIs, las ocho cerradas y sin ninguna sustitución (Sección 4.1)	Sin motivo propio. La cifra se fija en la misma decisión que el calendario hasta el depósito
1 sep	Tres fuentes de selección	Se ejecutan la segunda y la tercera, y la tercera sustituye a la primera (Sección 4.1)	La memoria del trabajo previo llegó el 14 de septiembre, con el proyecto ya avanzado y la selección hecha
6 sep	Las Datasheets, en anexos de la memoria	Las Datasheets, en el repositorio de datos que acompaña al trabajo. La memoria recoge los hallazgos y el catálogo de preguntas	La memoria razona sobre los hallazgos y los datos van aparte, como en el artículo de Pricing Intelligence [2]
7 sep	Comparación por necesidades funcionales, además de por precio, con el instrumental del ecosistema	Matriz de *features* por plan y comparación funcional por lectura, fuera del esquema (Sección 5.5.1)	Las preguntas funcionales del catálogo de preguntas no tenían respuesta con ese instrumental
7 sep	Ejecución y publicación sobre el despliegue del ecosistema	PRIME4API en local, donde se ejecutan las preguntas P (Sección 5.4). Las preguntas H, las que exigen HARVEY4API, quedan para el despliegue público del ecosistema. El catálogo se prepara como contribución a SPHERE, y su publicación depende de la plataforma	Sin acceso al despliegue público del ecosistema. El motor sí corre en local
22 sep	La evaluación de las contribuciones, en un capítulo propio	Una sección del Capítulo 7. La memoria queda en siete capítulos	El índice anunciaba un capítulo de evaluación sin escribir, y la numeración impresa no coincidía con la anunciada
15 oct	Entrega de la memoria el 16 de octubre	Memoria cerrada el 29 de octubre, para el depósito del 2 de noviembre	La reunión con el tutor del 15 de octubre dejó cambios por incorporar. El tutor permitió entregar cuando el autor quisiera, y este mantuvo el depósito del 2 de noviembre
15 oct	Sin colección en SPHERE: Pricing2YAML quedaba fuera del alcance	Contribución preparada a SPHERE: los ocho pricings en Pricing2YAML y una maqueta de la colección con el histórico de cada pricing (Sección 3.6)	Reestimación del alcance tras la reunión con el tutor del 15 de octubre, que nombró la colección de APIs en SPHERE como resultado del trabajo

El catálogo de ocho APIs reduce el alcance previsto. Las preguntas H sin ejecutar también lo reducen, mientras sigan abiertas. La observación de panel en todas las APIs lo amplía, porque cada API suma un alta con cuenta propia. La comparación funcional cumple lo previsto por otro medio: la lectura de una matriz de *features* por API, capturada en su ciclo desde MailerSend y construida después en las tres primeras. La reestimación del 15 de octubre suma la contribución a SPHERE, que el plan inicial dejaba fuera. El resto cambia cómo se ejecuta o se presenta lo previsto, y no cuánto se hace.

En conjunto, las horas reconstruidas quedan por debajo de las previstas y el trabajo termina más tarde, con la defensa en noviembre en lugar de septiembre y un catálogo menor que el del encargo. Las horas de menos se explican por ese catálogo menor y por el trabajo con el asistente, y el retraso, por una dedicación que hasta agosto no pudo ser continua.

3. Estado del arte

Este capítulo revisa los trabajos existentes más relevantes en relación con los objetivos formulados en el Capítulo 1. La revisión se organiza del siguiente modo. En primer lugar se revisa el modelado de pricings de SaaS en el canal web, de donde proceden el metamodelo y las herramientas cuyo patrón replica después el canal API. A continuación, el modelado de la capacidad de las APIs y el análisis de esa capacidad. Después, la extensión del ecosistema de Pricing Intelligence al canal API: el ACM, la Datasheet y su motor de análisis, que son la plataforma inmediata sobre la que este trabajo se construye. Por último, el dominio de aplicación, las APIs de correo electrónico. El capítulo se cierra con un análisis del estado del arte que sintetiza qué resuelve cada propuesta y qué limitaciones concretas permanecen abiertas, de las que se derivan las contribuciones enumeradas en la Sección 1.4.

Conviene advertir una particularidad de esta revisión. El bloque dedicado al canal API procede de dos Trabajos Fin de Máster coordinados del mismo grupo de investigación. Uno aporta el modelo de capacidad, su serialización y el motor de análisis, y se defendió pocos meses antes que este trabajo [1]. El otro, desarrollado en paralelo, se centra en el agente conversacional del ecosistema y comparte la autoría del modelo [6]. Ninguno de los dos es aquí una referencia lejana. Ambos son el punto de partida operativo de este trabajo, y por eso se revisan con un detalle mayor del habitual. Una de las preguntas de investigación de este TFM, la de la expresividad de ACM2YAML, responde a una línea que Gavira Sánchez deja pendiente de forma explícita [1]. Las demás se derivan del análisis de la Sección 3.6.

3.1. Modelado de pricings de SaaS

El pricing de un servicio SaaS es la estructura de planes, características (*features*), complementos (*add-ons*) y límites de uso con la que el proveedor regula el acceso a su servicio. Ha dejado de ser un elemento puramente comercial para convertirse en un artefacto que condiciona el diseño, la operación y la evolución del propio sistema. La evidencia empírica de este desplazamiento la aportan los estudios de García-Fernández et al. El primero analiza más de 150 pricings de una treintena de SaaS comerciales a lo largo de seis años y documenta espacios de configuración de miles de combinaciones y un crecimiento sostenido de su complejidad [16]. El segundo es un estudio posterior sobre el grado de soporte industrial para la operación dirigida por pricings (*pricing-driven DevOps*), publicado en *IEEE Transactions on Services Computing* [17]. Gestionar manualmente estructuras de ese tamaño no es viable, y esa constatación es la que motiva su formalización.

La respuesta del grupo ISA a este problema sigue una tradición que le es propia: convertir un artefacto informal en un modelo estructurado para someterlo a análisis automatizado. Es el mismo camino que en su día recorrieron los modelos de características [18]. Su materialización para los pricings es el metamodelo **iPricing**, pro-

puesto originalmente bajo el nombre Pricing4SaaS [19] y consolidado después con su denominación actual [3]. El metamodelo estructura un pricing en planes, complementos y características. Estas últimas están sujetas a límites de uso renovables o no renovables. Su serialización, Pricing2YAML, permite representar pricings de forma portable e interoperable [17]. Sobre esta base, el trabajo presentado en CAiSE 2025 define siete operaciones de análisis (cardinalidad del espacio de configuración, filtrado, enumeración de suscripciones, coste, validación del pricing y de la suscripción, y suscripción óptima). Las siete se resuelven traduciendo el pricing a un problema de satisfacción de restricciones con optimización (CSOP). El trabajo las valida sobre más de 150 pricings reales y detecta inconsistencias en 35 de ellos [3].

En torno a iPricing se ha construido un ecosistema de herramientas interoperables, articulado por la línea de *Pricing Intelligence* [2]. A-MINT automatiza la extracción: toma la URL de la página de pricing de un SaaS y devuelve un iPricing válido en Pricing2YAML. Lo hace con un pipeline de modelos de lenguaje que valida y corrige su propia salida en bucle. Alcanza una F1 del 70,51 % y valida con éxito el 98,73 % de los modelos que genera [4], evaluado sobre SPECTRA, un corpus público con los pricings de una treintena de SaaS entre 2019 y 2024 [20]. PRIME (*PRicing Intelligence and Modelling Engine*) es el motor que ejecuta las operaciones de análisis sobre el modelo [3]. HARVEY, presentado en CAiSE 2026, es el agente conversacional que traduce preguntas en lenguaje natural a llamadas sobre esas herramientas y sintetiza los resultados [2]. SPHERE, por último, actúa como *hub* del ecosistema: repositorio compartido de pricings organizados en colecciones, editor con validación formal y renderizado en tiempo real, y punto de acceso público a las demás herramientas [5].

De este bloque interesan dos rasgos para los objetivos de este trabajo. El primero, el papel del artefacto central. El pricing formalizado concentra la información en una representación común e independiente de la implementación. Sobre esa fuente compartida de conocimiento operan extracción, análisis, consulta y publicación, y el canal API replicará después ese patrón con la Datasheet. El segundo, la especialización de canal. iPricing fue concebido para el canal web y no representa conceptos del canal API como los *endpoints*, los *rates* por ventana temporal o las reglas de consumo por petición. No es una deficiencia: Gavira Sánchez concluye de forma explícita que canales distintos requieren modelos distintos [1]. Aun así, esa especialización delimita dónde termina su alcance y dónde debe empezar otro instrumental.

3.2. Modelado de la capacidad de las APIs

En el canal API la situación de partida es asimétrica. La superficie funcional de una API REST [21] cuenta con un estándar de descripción maduro y universalmente adoptado, la especificación OpenAPI [22], que documenta rutas, métodos, parámetros y respuestas. Las condiciones de consumo, en cambio, carecen de un equivalente asentado: cuántas peticiones admite un endpoint por unidad de tiempo, qué volumen mensual permite cada plan, qué ocurre al superarlo. Y es la información determinante para quien integra el servicio.

Que esas condiciones son ubicuas y heterogéneas está documentado empírica-

mente. El análisis de Gámez-Díaz et al. sobre 69 ofertas comerciales de APIs RESTful muestra que las cuotas y los *rates* son las limitaciones dominantes y que la métrica más habitual es el número de peticiones [23]. Muestra también que los modelos de negocio se articulan en planes escalonados, con y sin sobrecoste por exceso. Un estudio posterior, realizado junto a actores industriales del sector de la gestión de APIs, sitúa esas limitaciones y los acuerdos de nivel de servicio que las contienen en el centro de la industria del API [24]. El mismo estudio constata la ausencia de una representación estándar y procesable de esas limitaciones.

Los intentos de dotar a esas condiciones de una representación formal heredan de la larga tradición de especificación de acuerdos de nivel de servicio, cuyo exponente clásico es WS-Agreement [25]. SLA4OAI traslada esa línea al ecosistema OpenAPI como extensión de la especificación [26]. Ancla las limitaciones (cuotas y *rates*, con umbrales, periodos y costes por operación o por exceso) a las mismas rutas y métodos que el documento funcional ya describe. El respaldo conceptual de esa sintaxis es **Pricing4APIs**, el modelo riguroso de pricings de APIs RESTful de Fresno-Aranda et al. [27]. Organiza la información en planes, limitaciones y capacidad, entendida como el techo máximo que el proveedor puede ofrecer, del que las limitaciones son fracciones. El modelo incorpora además una operación de validación con criterios jerárquicos (del límite individual al pricing completo) que detecta inconsistencias como límites inalcanzables o planes más caros menos generosos que otros más baratos [27].

Tres de los rasgos que Gavira Sánchez destaca en esta familia de propuestas [1] resultan determinantes para este trabajo, porque son los que motivaron el modelo que se revisa en la Sección 3.4. Primero, la capacidad es unidimensional. Se reduce a un valor máximo de *throughput* sobre la métrica de peticiones, de modo que restricciones reales como la concurrencia o el tamaño de la respuesta quedan fuera de su semántica. Segundo, la distinción entre cuota y *rate* se delega en el *limitador*, el componente del proveedor que lleva la cuenta de las peticiones recibidas y rechaza las que exceden lo permitido. Que un límite sea cuota o *rate* depende de cómo ese componente cuente: sobre una ventana de tiempo fija o sobre una ventana deslizante. Es, en palabras de Gavira Sánchez, «un detalle de implementación que la mayoría de proveedores no publica» [1]. Dos límites publicados pueden resultar inclasificables desde la documentación. Y tercero, el plan es el único eje de segmentación: cualquier variación en las condiciones de dos consumidores exige planes distintos [1, 27].

3.3. Análisis de la capacidad de las APIs

Modelar la capacidad es condición necesaria pero no suficiente: el valor está en poder razonar sobre ella. La gestión de la capacidad es una preocupación clásica de la gestión de servicios TI [28]. Trasladada a las APIs cristaliza en las *operaciones de análisis de la capacidad*, cómputos formales sobre una noción de capacidad que evalúan el uso de una API bajo condiciones de carga determinadas. En la literatura coexisten dos aproximaciones, que difieren en cómo entienden la capacidad y en cómo resuelven el análisis.

La primera, de Fresno-Aranda et al., opera sobre arquitecturas de microservicios

[29] que consumen APIs externas reguladas por planes con limitaciones, bautizadas LAMA (*Limitation-Aware Microservice Architecture*). La capacidad es el máximo volumen de trabajo que la arquitectura completa puede atender sin exceder ninguna limitación externa. Smart LAMA, la API REST que los autores construyen sobre ese estilo, traduce la topología (un grafo de servicios propios y APIs consumidas) a un CSOP. Resolverlo responde a operaciones como el máximo de peticiones servibles en un periodo y coste dados, el coste mínimo para un volumen dado o el tiempo mínimo para completarlo [30, 31, 32]. La misma línea aporta además un catálogo de once operaciones de análisis centradas en el consumidor individual de una API [33]. Sus cinco escenarios van del suscriptor que calcula tiempos de espera al prosumidor que busca la suscripción que soporte su propio modelo de precios. Para Gavira Sánchez, frente a Smart LAMA, el catálogo convierte por primera vez el pricing de una API en objeto directo del análisis, aunque se definió sin implementación de referencia [1].

La segunda aproximación es geométrica y parte de la visión de la capacidad bajo un enfoque de caja negra planteada por Antonio Ruiz-Cortés y Miguel Ruiz-López [34]. El *rate* se interpreta como la velocidad máxima de consumo y la cuota como el volumen máximo acumulable en un periodo. Ambos límites operan a la vez, y su intersección define la capacidad efectiva. Esta idea se materializa en la biblioteca Pricing4API, cuyo primer prototipo desarrolló Daniel Ruiz López y que Gavira Sánchez extendió en su Trabajo Fin de Grado [35, 36]. El *BoundedRate* agrupa un rate y una o varias cuotas interdependientes. Sobre él se define la **curva de capacidad** $C(t)$: el máximo número de peticiones realizables desde el instante inicial hasta t sin violar ninguna restricción. Su ventaja es doble. Se lee de un vistazo en una gráfica, y evaluarla en un punto cuesta mucho menos que resolver un problema de restricciones. Sobre ella la biblioteca responde preguntas de tiempo mínimo, capacidad acumulada, agotamiento de cuota o rentabilidad entre planes [36].

Leídas desde los objetivos de este TFM, ambas aproximaciones comparten dos características que delimitan su alcance. La primera es la entrada. Smart LAMA asume los pricings ya descritos conforme a Pricing4APIs, y la biblioteca Pricing4API ni siquiera contaba con un artefacto de entrada estructurado: cada plan debía codificarse a mano en Python, trasladando uno a uno los parámetros de la documentación del proveedor. Ninguna de las dos responde de dónde sale, de forma trazable y a escala, la información de capacidad sobre la que analizan. La segunda es que la interacción entre límites de uso y *rates* queda definida como semántica del modelo (la intersección de rectas que da lugar al BoundedRate), pero no caracterizada sobre ningún dominio real. Qué formas adopta esa relación en las APIs de un sector concreto, y qué consecuencias tiene para el consumidor, es una pregunta que ninguno de estos trabajos aborda.

3.4. El canal API del ecosistema: ACM, Datasheets y motor de análisis

Esta sección revisa los dos Trabajos Fin de Máster presentados al principio del capítulo, que extienden el ecosistema de Pricing Intelligence al canal API. Son el de Gavira Sánchez [1] y el de Ruiz López, centrado en el agente HARVEY4API [6].

El **ACM** (*API Capacity Model*) reconceptualiza la capacidad para superar las tres limitaciones señaladas al final de la Sección 3.2. Frente a la visión unidimensional, el ACM define la capacidad como un conjunto de *dimensiones* independientes, cada una expresada sobre su propia métrica: *Volume* (las cuotas), *Frequency* (los *rates*), *Concurrent Requests* y *Results Per Request*. A ellas se añade el CRF (*Consumption Request Factor*), que no es un límite sino la regla que declara cuántas unidades de cada métrica gasta una petición. En una API de correo esa distinción importa. Una misma llamada consume una petición y, a la vez, tantos correos de la cuota mensual como destinatarios incluya [1]. Frente a la semántica delegada en el limitador, el ACM decide la clasificación comparando los límites entre sí. De los que una API publica, el de menor unidad temporal es el *rate*, que gobierna la velocidad, y el de mayor unidad es la cuota, que gobierna el volumen. Los 900 puntos por minuto y las 5.000 peticiones por hora de la API de GitHub [37] quedan así clasificados sin necesidad de conocer el mecanismo interno del proveedor [1]. La interpretación geométrica de la Sección 3.3 queda integrada en el modelo como semántica oficial de la capacidad.

La serialización del modelo, **ACM2YAML**, da lugar a la **Datasheet**: un documento orientado a máquinas que reúne en una sola representación la capacidad de una API. En el canal API esa capacidad aparece dispersa entre páginas de precios, documentación de desarrollador, especificaciones OpenAPI y políticas de uso. Su estructura combina los conceptos de capacidad con los elementos de pricing que los contextualizan. Los primeros son los catálogos de cuotas y rates (bloques `capacity` y `max_power`), el CRF serializado como `workload` y el sobrecoste como `overage_cost`. Los segundos, la jerarquía Plan → Endpoint → Alias. Frente al plan como único eje de segmentación, esa jerarquía asocia cada restricción al nivel al que aplica, sin crear planes artificiales [1]. La versión operativa, ACM2YAML v0.3, captura dos de las cuatro dimensiones del modelo (Volume y Frequency, además del CRF) y deja fuera concurrencia y resultados por petición. Su propio autor la califica de versión operativa parcial, priorizada para poner en marcha el ecosistema antes que para agotar la expresividad del metamodelo [1].

Sobre la Datasheet operan las herramientas del canal. **PRIME4API** refactoriza la biblioteca Pricing4API como API REST y convierte el análisis en un proceso dirigido por la Datasheet: cada pregunta se reduce a operaciones sobre el BoundedRate y la curva de capacidad [1]. Junto al motor, Gavira Sánchez define un catálogo de diez preguntas de análisis (Q1–Q10) en tres categorías: análisis de la capacidad de una suscripción, selección de la suscripción óptima y equilibrio entre cuotas heterogéneas. Lo instancia sobre tres pricings de dos servicios (SendGrid, en dos versiones, y DBLP), y esas respuestas de referencia funcionan como *ground truth* para validar el ecosistema [1]. La tercera categoría es especialmente relevante para este trabajo. Q8 (cuál de dos cuotas se agota primero dado un CRF), Q9 (qué CRF iguala el agotamiento de dos cuotas) y Q10 (cuánto cuesta completar un volumen incurriendo en sobrecoste) razonan sobre la interacción entre límites heterogéneos. Esa categoría no tiene correspondencia en el catálogo de Fresno-Aranda precisamente porque su modelo carecía del factor de conversión [1]. Por último, HARVEY4API expone estas capacidades tras una interfaz conversacional, publicando las operaciones del motor como herramientas mediante el *Model Context Protocol* [6, 38]. La extensión de SPHERE al canal API aporta colecciones de Datasheets, renderizado en tiempo real y edición validada [1, 5].

La extracción automática es tarea de otra herramienta del canal, A-MINT4API, y el editor de SPHERE sirvió para validar visualmente las Datasheets que genera [1].

Ruiz López ha desarrollado también dos *skills* para caracterizar cómo se accede a un servicio SaaS [39]. Una *skill* es un conjunto de instrucciones empaquetadas que un modelo de lenguaje sigue para una tarea. La primera, `provider-service-resolver` (versión 1.1), recibe el nombre de un SaaS e identifica a su proveedor. Después clasifica cada candidato de su catálogo como servicio autónomo, servicio dependiente, complemento, característica u oferta, y cada afirmación lleva su fuente oficial con el extracto literal. La segunda, `saas-interface-channel-analyzer` (versión 3.0), parte de ese resultado y separa la interfaz del canal. Una interfaz es un mecanismo definido por el proveedor con un solo paradigma de interacción, y el canal es ese paradigma: programático, por eventos, interactivo, conversacional o agéntico. Con esas reglas, el envío por SMTP y la API REST de un proveedor de correo son dos interfaces del mismo canal programático, porque cambian el protocolo y la autenticación pero no el paradigma.

Esta distinción es un avance reciente del trabajo. Hasta octubre de 2026, esta memoria llamaba canal a cada vía de acceso a un servicio: el canal web, donde se publica el pricing, y el canal API, que es el que se modela. Esa palabra mezclaba dos cosas, la vía de acceso y la forma de interactuar a través de ella. En la revisión con el tutor del 15 de octubre de 2026 se acordó usar interfaz con la definición del NIST: la frontera común entre sistemas o módulos independientes en la que tienen lugar las interacciones [40]. Con ella, la API REST, el relé SMTP y el panel web de un proveedor son interfaces distintas del mismo servicio. El canal queda para el paradigma de interacción, como en las *skills* de Ruiz López. La multicanalidad de un servicio de correo se lee entonces como el conjunto de sus interfaces y el canal de cada una. Las expresiones canal web y canal API que siguen en esta memoria nombran, por tanto, la interfaz web y las interfaces programáticas del servicio.

El resultado es una cadena completa y operativa: extracción, modelo, motor, agente y hub funcionan sobre ACM2YAML v0.3. Sus límites, sin embargo, están declarados con la misma claridad en la propia memoria de Gavira Sánchez [1]. La validación descansa sobre tres pricings de dos servicios. Dos de las cuatro dimensiones de capacidad carecen de serialización. La integración entre capacidad y estructura comercial se resolvió de forma pragmática y se señala como trabajo futuro. Y tanto completar la serialización como someterla a validación formal figuran literalmente entre sus líneas pendientes. Con la plataforma construida, falta confrontarla con un dominio completo.

3.5. El dominio de aplicación: las APIs de correo electrónico

El dominio elegido para esa confrontación es el del correo electrónico programático, presentado en la Sección 1.1.1. Es un dominio maduro y competitivo, en el que varios proveedores ofrecen el mismo servicio por API. La información de capacidad (*rates* por endpoint, cuotas por plan, factores de consumo por destinatario o por lote)

resulta especialmente dispersa y heterogénea. Eso lo convierte en un banco de pruebas exigente para el instrumental recién revisado.

Sobre cómo se comparan estas APIs se han revisado dos comparativas públicas del sector. Las dos las publican actores que ofrecen servicios de correo y que se incluyen a sí mismos en la comparación [8, 9]. Adoptan el formato de listas de recomendaciones: útiles como panorámica del mercado, pero elaboradas con criterios no declarados, sin trazabilidad de fuentes, no reproducibles y no procesables por máquina. En la literatura académica revisada en este capítulo, las APIs de correo aparecen únicamente como casos de ilustración o validación. SendGrid, en dos versiones, forma parte del *ground truth* sobre el que se validó el canal API del ecosistema, y MailerSend sirvió como banco de pruebas en el diseño de ACM2YAML [1]. En los trabajos revisados en este capítulo no se ha identificado un tratamiento sistemático del dominio como objeto de comparación. Tampoco ofrecen un catálogo estructurado, trazable y procesable de las condiciones de consumo de las APIs de correo, ni un método asentado para compararlas combinando requisitos funcionales y no funcionales.

3.6. Análisis del estado del arte

La Tabla 3.1 sintetiza la revisión anterior. Para cada propuesta se indica el canal que modela, cómo entiende la capacidad, qué análisis ofrece y sobre qué se ha validado.

Propuesta	**Canal**	**Capacidad**	**Análisis**	**Validación**
iPricing / Pricing2YAML [16, 19]	Web	Límites de uso sobre *features*	7 operaciones sobre CSOP [3]	>150 pricings; corpus SPECTRA [20]
Pricing4APIs / SLA4OAI [26, 27]	API	Unidimensional (peticiones); cuota/*rate* según ventana del limitador	Validación de consistencia (VC1–VC4)	Casos de ilustración
Smart LAMA [30, 31]	API (arquitectura)	Máximo *workload* de la LAMA	CSOP: max. peticiones, min. coste, min. tiempo	Notebook con 10 preguntas
Catálogo de operaciones [33]	API	La del modelo Pricing4APIs	11 operaciones formuladas, sin implementación	—
Pricing4API (biblioteca) [35, 36]	API	Curva de capacidad $C(t)$; BoundedRate	Q1–Q7, programático y visual	Casos sin entrada estructurada
ACM / Datasheet / PRIME4API [1, 6]	API	4 dimensiones + CRF (v0.3 serializa 2)	Q1–Q10 dirigido por la Datasheet	3 APIs (*ground truth*)

Tabla 3.1: Síntesis del estado del arte revisado.

De esta síntesis se desprenden cuatro observaciones, que son los cuatro huecos

sobre los que se articula este trabajo.

Primera: el instrumental existe; su validación a escala de dominio, no. La cadena completa del canal API funciona, pero se ha ejercitado sobre tres pricings de dos servicios [1]. Ningún trabajo ha confrontado todavía el modelo con un conjunto representativo de las APIs de un sector, elegido con un criterio declarado, incluidas las que no encajan con naturalidad. Con una confrontación así se contrasta si la Datasheet es un formato general o un formato que funciona donde ya se probó. Este es el hueco que aborda el catálogo de Datasheets de correo (contribución C-01).

Segunda: la expresividad de ACM2YAML v0.3 es parcial, y sus carencias están declaradas pero no inventariadas. Se sabe qué dimensiones faltan (concurrencia, resultados por petición), pero no qué extensiones concretas exige un dominio real ni con qué prioridad. Ese inventario solo puede producirlo la confrontación sistemática con casos reales, documentando cada carencia con la API que la revela. Es el hueco de la evolución justificada del esquema (C-02).

Tercera: la relación entre límites de uso y *rates* está definida, pero no caracterizada. La semántica existe (la intersección que define el BoundedRate) y las operaciones Q8–Q10 la ejercitan sobre casos concretos. Lo que ningún trabajo ofrece es una caracterización de esa relación sobre un dominio real: qué patrones adopta, cuándo el *rate* enmascara a la cuota o la cuota al *rate*, y qué implicaciones tiene para el consumidor. Es el hueco de la contribución C-03.

Cuarta: no hay método de comparación entre APIs. Los catálogos de preguntas existentes (las once operaciones de Fresno-Aranda, las diez de Gavira Sánchez) razonan sobre la capacidad de un pricing. La decisión que motiva este trabajo, *qué API conviene a unas necesidades dadas*, exige además preguntas funcionales y no funcionales propias del dominio y un procedimiento sistemático para responderlas sobre un conjunto de Datasheets comparables. Ese método, con su catálogo de preguntas de decisión, es la contribución C-04.

Este trabajo, conviene subrayarlo, no propone un modelo nuevo ni un motor nuevo. Aplica y extiende los instrumentos revisados en este capítulo (la Datasheet, PRIME4API, SPHERE), llevándolos a un dominio real y a escala. El catálogo resultante queda en el repositorio de datos que acompaña al trabajo (Sección 7.4), preparado como contribución a SPHERE, y el Capítulo 6 propone las extensiones del esquema que el dominio justifica. Los capítulos siguientes desarrollan cómo.

Este trabajo aporta a SPHERE dos piezas. La primera es una colección con las ocho APIs del catálogo como *API products*, es decir, interfaces programáticas con un pricing propio. Cada una lleva su Datasheet y su pricing en Pricing2YAML [17]. La segunda es una propuesta de sección de términos de consumo para la ficha de cada API. Los términos de consumo (*API consumption terms* [41]) son los términos y condiciones del proveedor que afectan a la velocidad y a la capacidad de consumo de su API, como sus *rates*, sus cuotas y su política de uso. La sección llevará una pestaña con las versiones registradas de esos términos y la procedencia de cada dato: URL, tipo de fuente y fecha. El repositorio del trabajo incluye una maqueta de la colección con esa pestaña y con las versiones anteriores de cada pricing, tomadas sobre todo de capturas de Wayback Machine. SPHERE y Pricing2YAML son del grupo ISA: la colección los

usará tal como son, y la sección se propone como extensión de la plataforma.

Las *skills* de Ruiz López se aplican sin modificarlas, con el procedimiento de la Sección 4.1.2.

4. Construcción del catálogo de Datasheets de APIs de correo

Este capítulo presenta el método con el que se construye el catálogo de Datasheets de APIs de correo electrónico (la contribución C-01 de la Sección 1.4) y su calibración sobre un piloto real. Se organiza en cinco piezas. El criterio de selección del dominio. El ciclo de construcción por API. Las reglas de trazabilidad y confianza que hacen revisable cada dato. La escalera de validación que convierte inferencias en evidencia. Y el marco de conformidad y evolución del esquema. Le sigue el piloto (Postmark) con los hallazgos que validan el método y anticipan la evolución del esquema (contribución C-02). Esa evolución no se formula aquí: se pospone al cierre del catálogo, conforme al criterio de promoción de la Sección 4.5, y ocupa el Capítulo 6. El capítulo se cierra con la respuesta a la RQ1 (Sección 4.7).

4.1. Criterio de selección del dominio

El dominio del catálogo son las APIs de correo transaccional. Postmark, una de las ocho, define el correo transaccional como un mensaje individual que el destinatario espera recibir y que suele desencadenar el propio usuario [42]. Pone como ejemplos el restablecimiento de contraseña, el recibo de compra o el aviso de pago fallido. Excluye el correo con el mismo contenido para muchos destinatarios que no desencadena ningún evento, al que llama de *marketing*. Una API de correo transaccional es la interfaz con la que una aplicación entrega esos mensajes a través del proveedor. El producto de *marketing* que varios proveedores venden en la misma cuenta queda fuera (Sección 4.1.2). Hasta dónde valen los resultados fuera de este dominio lo discute la Sección 7.3.3.

El conjunto de APIs analizadas se determina con el criterio fijado en el encargo del trabajo, que combina tres fuentes de inclusión. La selección se completó el 1 de septiembre de 2026 con la tercera fuente, una consulta a un modelo de lenguaje con cinco criterios fijados antes de hacerla. El Anexo A registra la fecha, el modelo, el prompt con sus criterios y la respuesta íntegra de la consulta, y señala lo que no consta. Con ese registro un tercero puede repetir la consulta con el mismo prompt y contrastar lo que obtenga con lo que recoge el anexo. Los apartados que siguen desarrollan el protocolo, y el último fija la ventana en que se tomaron los datos del catálogo.

4.1.1. Fuentes de inclusión

La Tabla 4.1 recoge las tres fuentes del encargo y lo que aportó cada una al catálogo. El catálogo resultante parte de ocho APIs: Postmark, Resend, SendGrid, MailerSend, Amazon SES, Mailgun, Brevo y Mailchimp Transactional.

Fuente	Qué es	Qué aportó al catálogo
Primera	Las APIs analizadas en un Trabajo Fin de Máster anterior señalado en el encargo	Ninguna: no se ejecutó
Segunda	Las APIs ya modeladas en el ecosistema	SendGrid y MailerSend, modeladas en el trabajo de Gavira Sánchez [1] y en el repositorio de la línea, y Resend, modelada en v0.1 en ese repositorio [43]
Tercera	Una consulta a un modelo de lenguaje con cinco criterios fijados antes de hacerla	Amazon SES, Mailgun, Brevo y Mailchimp Transactional, más cuatro APIs de reserva (Apartado 4.1.3)
Piloto del método	No es fuente del encargo. Es la API con la que se calibró el método	Postmark (Sección 4.6)

Tabla 4.1: Fuentes de inclusión del catálogo. Las tres primeras filas son las del encargo. La última recoge el piloto con el que se calibró el método, que ya formaba parte del catálogo cuando se hizo la consulta. Resend figura en la segunda fuente porque el repositorio de la línea la tenía modelada en v0.1, aunque este trabajo empezó a construirla antes de la consulta.

La primera fuente no pudo ejecutarse. La memoria del trabajo previo llegó el 14 de septiembre de 2026, con el proyecto ya avanzado y la selección del 1 de septiembre hecha. Se deja constancia y se sustituye por la tercera.

Las tres APIs de la segunda fuente se construyeron en v0.3 con datos del proveedor recogidos en la ventana de toma de datos (Apartado 4.1.4). El modelado previo sirvió de punto de partida en Resend y de contraste en MailerSend, y SendGrid se construyó sin consultarlo.

4.1.2. Criterios de inclusión y de exclusión

La consulta pidió cuatro APIs de correo electrónico transaccional para completar un catálogo de ocho, con cinco criterios ordenados por peso. El encargo pedía que la consulta buscara APIs populares, diversas y diferenciadoras, y cuatro de los criterios desarrollan esa petición. El segundo criterio añade una condición propia del método: que la Datasheet pueda construirse con trazabilidad. Los criterios, en el orden y con el alcance del prompt, son estos:

1. Diversidad del modelo de precios. El catálogo debe cubrir cuatro formas de cobro: planes por volumen mensual, pago por uso puro, producto vendido como añadido de otro y planes con límite diario además del mensual. Tienen prioridad las APIs que añadan una forma aún no cubierta.
2. Documentación pública de precios y de límites (como los *rates*, los tamaños máximos o los destinatarios por petición), consultable sin darse de alta. Sin ella, la Datasheet no puede construirse con trazabilidad.
3. Adopción real en producción, para que la comparación tenga interés práctico.

4. Un rasgo diferenciador para analizar la relación entre límites de uso y *rates*, como un *rate* publicado por *endpoint*, un caudal negociado no publicado o un límite diario.

5. Diversidad geográfica y de tamaño de proveedor, con al menos una API europea.

La respuesta justifica tres de las cuatro seleccionadas por la forma de cobro que añaden al catálogo. A Amazon SES le atribuye el pago por uso puro, y a Brevo el límite diario además del mensual. A Mailchimp Transactional le atribuye el producto vendido como añadido de otro. Mailgun tiene, según la respuesta, la misma forma de cobro que tres de las cuatro APIs ya presentes, y entra por su rasgo de límites, que describe como «el caso limpio de rate documentado». La misma respuesta sitúa en Brevo la API europea que pide el quinto criterio. En la reserva, anota contra Mailjet que pertenece al mismo grupo que Mailgun, y contra SparkPost, «menos adopción reciente». La Sección 4.7 contrasta estas caracterizaciones con lo que registró la construcción.

La única exclusión de APIs aplicada fue la del propio prompt, que impedía proponer las cuatro ya presentes en el catálogo. El orden de la reserva no excluye ninguna API: fija cuál entraría primero si una del catálogo no pudiera documentarse con trazabilidad.

Un último criterio, que no está entre los cinco del prompt, acota qué se modela de cada proveedor. El catálogo modela solo el canal API de cada proveedor, y el canal web, que el ecosistema representa con *iPricing* [3], queda fuera del alcance. Varios venden más de un producto bajo la misma cuenta. Resend, por ejemplo, ofrece un producto transaccional facturado por correos enviados y otro de *marketing* facturado por contactos, con catálogos de precios independientes. El catálogo modela en cada API el producto transaccional, que es el comparable entre proveedores y el que consume la API de envío. El otro producto se documenta en el registro de fuentes y queda registrado como elemento no soportado (Capítulo 6).

Las ocho APIs son *API products* en la taxonomía de Ruiz López (Sección 3.4). Un servicio SaaS se ofrece por interfaces de cliente o por interfaces programáticas, y una interfaz programática con pricing es un *API product*. Amazon SES era el caso dudoso. Su página de precios publica tres planes y una tarifa por uso de 0,10 USD por cada mil correos [44], así que también cumple la definición. Por eso las ocho caben en una colección de SPHERE.

Dos *skills* de Ruiz López, descritas en la Sección 3.4, contrastan este recorte con lo que publica cada proveedor. La palabra canal tiene en ellas otro sentido que en esta memoria: allí nombra el paradigma de interacción, y el canal API de este trabajo corresponde a sus interfaces programáticas. El resolvedor, `provider-service-resolver`, dice si un proveedor de correo ofrece varias colecciones de APIs y si cada una lleva su pricing o uno solo las cubre. El analizador, `saas-interface-channel-analyzer`, enumera las interfaces de cada servicio, como la API REST, el relé SMTP o los *webhooks*, y asigna a cada una su canal. Los fines eran dos: decidir si la colección de SPHERE necesita mostrar el árbol de proveedor y servicio, y contrastar las reglas de las *skills* en un dominio distinto del de su validación. Se aplicaron sin modificarlas, con Claude Opus 5.5 en esfuerzo medio, sobre las ocho APIs. Primero se probaron con Resend y Postmark. Después el resolvedor se aplicó a los cuatro proveedores multiproducto, y al

final las dos *skills* se aplicaron a las seis APIs restantes. Todos los YAML de salida pasan sin errores el validador que trae cada *skill*. El nombre de entrada de cada API se fijó antes de abrir ninguna página, porque el catálogo que devuelve el resolvedor depende de ese nombre. Con «SendGrid», por ejemplo, el resolvedor devolvía una agrupación de productos, y con «Twilio SendGrid Email API», el producto.

En tres de los cuatro proveedores multiproducto hay varias colecciones de APIs, cada una con su pricing. Brevo es el contraejemplo: su correo transaccional va incluido en todos los planes de la plataforma de *marketing*. Por eso el analizador no encontró en Brevo un servicio que analizar, y sus interfaces son las de la plataforma que contiene el producto. El analizador encontró entre cinco y diez interfaces por API. El cruce con las Datasheets deja interfaces en un solo lado. El analizador registra el servidor MCP, la CLI y el panel, que no tienen límites de consumo publicados. Las Datasheets registran los planes, las cuotas y el caudal, que el analizador no está hecho para recoger.

Las reglas del analizador no cubren un mecanismo que tienen todos los proveedores: el correo entrante. En todos llega de forma parecida: el proveedor recibe el mensaje y lo entrega al cliente, casi siempre en una URL suya. Aun así, el analizador lo clasificó como interfaz programática en cinco APIs y como canal de eventos en SendGrid y Mailgun, y en Brevo lo dejó como evento pendiente. Las dos clasificaciones son compatibles con las reglas, que no dicen cuál corresponde a este mecanismo. Una nota de dominio para el correo fijaría una sola. Ningún dato de los Capítulos 5 y 6 procede de las *skills*.

El uso de las *skills* se plantea como una colaboración con su autor. Las *skills*, sus reglas, sus esquemas y sus validadores son de Ruiz López [39]. Este trabajo las aplicó sin modificarlas al correo, un dominio que no estaba entre los de su validación: GitHub, GitHub Copilot, Claude Platform y Claude. Después cruzó cada salida con la Datasheet de la misma API, construida antes y sin las *skills*, como se ha descrito en los párrafos anteriores.

La colaboración da resultados en las dos direcciones. A este trabajo le da las interfaces de cada API y la respuesta sobre las colecciones: tres de los cuatro proveedores multiproducto tienen varias, cada una con su pricing. A las *skills* les da ocho APIs de un dominio nuevo, con todos los YAML válidos y cada extracto encontrado literal en la página descargada. También les da cinco propuestas, cada una con la ejecución que la motiva. La primera es comprobar los extractos, porque el validador revisa la estructura y una copia con un extracto inventado pasó sin error. La segunda admite varios servicios base, o un código de duda, para un dependiente que vale en dos planes, como Automations en Resend. La tercera es una regla para cuando el nombre de entrada es el de la empresa y no el de un producto, como pasó con SendGrid y Brevo. La cuarta es la nota de dominio sobre el correo entrante descrita arriba. La quinta es un atributo opcional que enlace cada interfaz con sus límites de consumo, que es la línea de trabajo futuro de la Sección 7.5.

Las cinco propuestas están redactadas en el informe del piloto, junto a los YAML de las ocho APIs y la salida de los validadores.

4.1.3. Lista de reserva y regla de reemplazo

La consulta pidió cuatro APIs seleccionadas y cuatro de reserva, ordenadas. Para cada una pidió su forma de cobro y lo que aportaba al catálogo, además del riesgo principal para construir su Datasheet. También pidió que no inventara cifras y que describiera el tipo de modelo y no los precios. La respuesta da el riesgo solo para las cuatro seleccionadas. No contiene ningún precio ni límite con cifra, así que ninguno de los que cita esta memoria procede de ella. La Tabla 4.2 recoge la reserva en su orden.

Orden	**API**	**Forma de cobro según la respuesta**	**Lo que anota la respuesta**
1	Mailjet	Planes por volumen con *rate* documentado	Europea, del mismo grupo que Mailgun, con menos diversidad que Brevo
2	Zoho ZeptoMail	Cobro por créditos prepagados	Otra forma de pago por uso
3	SparkPost	Planes por volumen	Muy documentada, con menos adopción reciente
4	SMTP2GO	Planes por volumen con límites diarios	Proveedor pequeño, útil como caso de diversidad de tamaño

Tabla 4.2: Lista de reserva de la consulta, en su orden. Las dos últimas columnas resumen la respuesta y no se verificaron, porque ninguna de estas APIs llegó a construirse.

El catálogo es mutable. Si una API no puede documentarse con la trazabilidad que exige la Sección 4.3, entra en su lugar la primera de la lista de reserva. La sustitución queda registrada con fecha y motivo. La regla sigue el orden de la lista y no la forma de cobro de la API que sale. Solo la dispara la falta de trazabilidad: que la construcción corrija una caracterización de la respuesta no basta.

La regla no llegó a aplicarse, porque las ocho APIs cerraron el ciclo con cero errores de validación, que es su criterio de salida (Sección 4.2). Las ocho APIs de partida son, así, las ocho que presenta el Capítulo 5.

4.1.4. Ventana de toma de datos

Los datos del catálogo se tomaron en una sola ventana, del 2 de agosto al 21 de septiembre de 2026. Abarca la lectura de la documentación y la observación del panel, y dentro de ella se ejecutaron los sondeos, del 3 de agosto al 21 de septiembre. Hasta el 1 de septiembre (día de la consulta) solo se tomaron datos del piloto y de Resend. Las otras seis APIs empezaron a construirse entre el 6 y el 21 de septiembre, en el orden en que las presenta el Capítulo 5.

Los precios, planes y límites que describe esta memoria son los de esa ventana. Una comparación entre dos APIs puede cruzar fechas distintas dentro de la ventana de toma de datos, porque la construcción se repartió a lo largo de ella. El esquema v0.3 lleva una sola fecha por documento, en el campo `date`, y no una por dato. La fecha

de acceso de cada valor se conserva en el registro de fuentes (Sección 4.3). Un valor de otra fecha, como el de un modelado previo, se escribe con su fecha.

4.2. El ciclo de construcción por API

Cada API del catálogo pasa por un ciclo de cinco fases, y el ciclo separa a quien construye de quien verifica: recolección, borrador, validación, revisión y consolidación. Los dos papeles los desempeñan agentes de inteligencia artificial distintos, y el verificador solo ve la Datasheet terminada (Anexo A). En la *recolección* se localizan y registran las fuentes: página de precios, documentación de desarrollador, especificaciones, artículos de soporte y políticas de uso. En la *construcción del borrador* se elabora la Datasheet conforme a ACM2YAML v0.3 [1], siguiendo la guía de referencia del repositorio del ecosistema [43]. La *validación* es automática y comprueba dos cosas: que el documento cumple el esquema (campos obligatorios, enumeraciones, unidades declaradas y referencias entre planes, cuotas y *rates*) y que cada dato remite a una fuente registrada. El criterio de salida por API es cero errores, con los huecos deliberados señalizados como avisos justificados. La *revisión* examina el borrador campo a campo con las fuentes a la vista. Resuelve las ambigüedades que la documentación no cierra y convierte en preguntas explícitas lo que requiere criterio o decisión. La *consolidación*, por último, incorpora el resultado de la revisión y de los sondeos empíricos que la revisión haya motivado.

El ciclo produce, por cada API, cinco artefactos. Cuatro son la Datasheet en ACM2YAML, el registro de fuentes con sus citas, el registro de elementos no soportados y el acta de revisión con las decisiones de representación adoptadas. El quinto es una matriz de *features* por plan (*webhooks*, IP dedicada, retención, dominios, usuarios, entrada de correo, SMTP y plantillas). La recolección la captura con las mismas fuentes, y como v0.3 no la admite se conserva fuera del YAML para las preguntas funcionales del Capítulo 5. Los cinco se versionan junto al catálogo. Ninguno se reproduce en esta memoria. El conjunto íntegro (fuentes y elementos no soportados incluidos) está en el repositorio de datos que acompaña al trabajo (Sección 7.4), de modo que cualquier dato sea auditable sin acceso a material privado. Los registros de elementos no soportados y de decisiones son, además, la materia prima de la contribución C-02. El catálogo de preguntas de C-04 se construye sobre la gramática de Pricing Intelligence [2], y varias de las que no pueden responderse coinciden con elementos no soportados del registro.

4.3. Trazabilidad y confianza

La regla central del método es que ningún dato entra en una Datasheet sin fuente verificable ni se rellena por estimación. Cada valor lleva asociada la URL de procedencia, la fecha de acceso y la *cita literal* del texto que lo respalda. Las páginas de precios cambian con frecuencia, y una URL sin cita no permite verificar el dato meses después. El esquema ya prevé parte de este registro: `sources_visited` y `data_provenance`, en el bloque de metadatos [43]. El método añade dos elementos. Una lista `not_found`, que

documenta lo que se buscó y no se encontró, para que la ausencia quede como dato y no como descuido. Y un registro de *discrepancias*, que enfrenta con sus citas las fuentes que se contradicen entre sí o con el comportamiento observado del servicio. El registro admite también la entrada que tiene por lado una ausencia de fuente.

La página de precios del proveedor fija la verdad del dato, porque es lo que el cliente compra. Las demás fuentes (documentación, ayuda, blog, panel o sondeo) detectan incoherencias del *pricing*, pero no lo corrigen. Si una de ellas da otro valor para el mismo dato, se modela el de la página y el otro se registra como incoherencia, con las dos citas. Si la propia página se contradice, no se elige: el campo queda en ámbar, el color intermedio del semáforo que se describe a continuación, mientras la discrepancia siga abierta.

Sobre esa base, este trabajo introduce como aportación metodológica propia un *semáforo de confianza*: cada campo de la Datasheet se etiqueta con el respaldo que tiene su valor. Verde, si el dato aparece explícito y literal en una fuente oficial. Ámbar, si se ha inferido o la fuente es ambigua. Rojo, si el proveedor no lo documenta. Si dos fuentes oficiales discrepan y una de ellas es la página de precios, el verde va al valor de la página. La etiqueta se serializa como comentario YAML con el color y nada más, de modo que no altera la conformidad del documento ni lo carga de prosa. El detalle de qué fuente respalda qué dato vive en el *provenance* del propio documento y en el registro de fuentes, no en el comentario.

El semáforo cumple una doble función. La operativa: dirige el esfuerzo de revisión, porque con ocho APIs y decenas de campos por API no todo puede verificarse con la misma intensidad. Y la analítica. Que el bloque de *rates* de Postmark estuviera en rojo en el piloto es un hueco de un proveedor. Que lo esté en todas las APIs del catálogo es un rasgo del dominio. Agregar los colores por API y por dimensión convierte esa diferencia en un dato medible: qué proporción de su capacidad contractual publican los proveedores de correo, qué dimensiones ocultan de forma sistemática y si la opacidad es común o propia de algunos. Es la lectura que alimenta el análisis de los capítulos posteriores. De ahí la segunda regla del método: cuando una dimensión de capacidad existe pero su valor no está publicado, el hueco se registra en `not_found` y en el registro de elementos no soportados, con la búsqueda documentada, y el documento no incorpora ninguna forma que el esquema no acepte. Un hueco visible es un hallazgo sobre el dominio. Un hueco disimulado se confunde con la inexistencia del límite.

El Extracto de código 4.1 ilustra ambas reglas sobre un fragmento real de la Datasheet del piloto, anterior a la adopción de su *rate* el 7 de septiembre de 2026. Recoge una cuota cuyo valor procede del volumen mínimo observado (ámbar) junto a un sobrecoste publicado literalmente (verde), y el bloque de *rates* vacío, con el hueco marcado en rojo. Desde esa fecha el bloque declara 5.000 peticiones por segundo en ámbar, y el Apartado 5.2.1 recoge el estado de la Datasheet al cierre del catálogo. En la memoria, los emojis de color del fichero se representan con los círculos 🟢, 🟡 y 🔴.

Extracto de código 4.1: Serialización del semáforo de confianza en la Datasheet del piloto (fragmento).

```yaml
capacity:
  quota_email_included_basic:
    value: 10000                                          # 🟡
```

```
    unit: emails
    period:
      value: 1
      unit: MONTH
    window_type: fixed_billing_cycle                          # ●
    overage_cost:
      price: 0.0018                                           # ●
      value: 1

# ● rate no publicado -- hueco deliberado (ver metadata.not_found)
max_power: {}
```

4.4. Validación: de la inferencia a la evidencia

Los campos en ámbar no se quedan en inferencias: el método define una escalera de validación de cuatro peldaños que se recorre en orden de coste. El primero es el *contraste documental*: buscar una segunda fuente oficial que confirme o contradiga la primera. Si un lado de la contradicción es la página de precios, gana la página y el otro valor se registra como incoherencia del *pricing*. Si la página de precios no interviene, o si la contradicción está dentro de ella, no se elige entre las dos y se registra como discrepancia. El segundo es la *observación de panel* con cuenta propia: darse de alta en el servicio y confrontar lo publicado con lo que el panel de control muestra, archivando capturas fechadas como evidencia. El panel no sobrescribe el precio publicado: si muestra otra cifra para el mismo dato, la diferencia queda como incoherencia registrada. El tercero es el *sondeo empírico ligero* contra la API real. Prefiere un mecanismo inocuo cuando el proveedor lo ofrece, y si no lo hay admite un envío real a destinatarios propios con un presupuesto declarado en el script del sondeo. Un sondeo que mide menos de lo que promete la página de precios deja el valor modelado como está y documenta una incoherencia del *pricing*. En el caso de Postmark, un token de pruebas documentado valida las peticiones sin enviar correo ni consumir cuota [45], lo que permite comprobar límites de aceptación con peticiones diseñadas al efecto. El catálogo de códigos de error del proveedor actúa aquí como mapa: cada límite que se aplica rechazando la petición tiene un error observable asociado. No todos los límites se aplican así. Brevo publica que, superada la cuota diaria de su plan Free, retiene en cola hasta mil correos transaccionales más [46]. Mailchimp publica que retiene en cola lo que excede su cuota [47]. El lote de MailerSend se acepta con un `202`, y el error solo aparece al consultar su estado (Sección 5.2). El cuarto peldaño es la *medición empírica formal* con el banco de pruebas del ecosistema [48], para lo que ninguno de los anteriores alcanza (los umbrales no publicados, el cómputo real de consumo, el comportamiento bajo carga). Esa medición excede el ciclo documental que construye el catálogo. Este trabajo deja identificados los campos candidatos en la Sección 7.3.1: el *rate* adoptado desde una cabecera en Postmark y Mailgun, el caudal de SendGrid y la unidad de la cuota en cuatro APIs. Así la medición puede ejecutarse sobre el catálogo terminado sin rehacer trabajo de documentación.

4.5. Conformidad estricta y evolución del esquema: cómo una carencia llega a propuesta

El método adopta una posición deliberadamente conservadora respecto al esquema: todas las Datasheets del catálogo se escriben contra ACM2YAML v0.3, sin inventar campos nuevos. Dos formas quedan, sin embargo, al margen de lo que la guía define, y las ocho las comparten: la unidad `MBs` (con `MiBs` en Mailgun) para los topes de tamaño, y el registro de ausencias `not_found`. Así el catálogo está escrito para que las herramientas del ecosistema lo consuman sin adaptación, y constituye una aportación compatible con la base documental existente [43]. En el ecosistema, PRIME4API analiza las Datasheets y SPHERE las publica [1, 5]. El Capítulo 6 documenta hasta dónde las consume PRIME4API. El catálogo se entrega a SPHERE como contribución, y su publicación depende de los responsables de la plataforma. Cuando el dominio exhibe información que el esquema no puede representar, esa carencia no se resuelve bifurcando el formato. Se registra como «elemento no soportado»: un hecho del dominio que ACM2YAML v0.3 no puede expresar, o que expresa sin que el consumidor de referencia lo lea bien. Cada uno lleva un número, F-nn, con el caso real que lo motiva, la consecuencia para el análisis y una propuesta de resolución. En el repositorio de datos estos registros se llaman «fricciones» (`fricciones.md`), y de ahí la F de F-nn.

Un elemento no soportado se promueve por una de dos vías. La primera es la *recurrencia*: los elementos que reaparecen en al menos dos APIs del dominio quedan promovidos. Quedar promovido no equivale a ser propuesta: el Capítulo 6 solo formula como propuesta de extensión, con número P-nn, el promovido cuya forma se contrasta con el texto de la guía. La segunda vía admite una sola API y exige tres condiciones. Primera: el hecho está citado con literal o medido en un sondeo, con su evidencia archivada. Segunda: la consecuencia está ejecutada sobre el consumidor de referencia, es decir, el motor devuelve un valor falso o deja sin responder lo que respondería si el esquema expresara la carencia. Un caso de control tiene que mostrar que el motor sabe responder bien. Tercera: ningún mecanismo ya existente de la guía cubre esa carencia, y no la explica un defecto del motor ya conocido. El veredicto de cada elemento no soportado se anota en el inventario consolidado del Capítulo 6 (Tablas 6.1 y 6.2). Con el catálogo cerrado, el paquete completo queda formulado: nueve propuestas (Tabla 6.3) y doce elementos promovidos con una forma esbozada, sin número de propuesta (Tabla 6.6). Esta regla protege dos cosas a la vez. La utilidad inmediata del catálogo, que no depende de que las propuestas se acepten. Y la fuerza de las propuestas que nacen de un elemento no soportado, respaldadas por frecuencias observadas sobre el dominio completo o, cuando entran con una sola API, por la consecuencia ejecutada sobre el motor.

La regla hace explícito un criterio del método: solo entra en el modelo lo que generaliza. Un elemento que aparece en una sola API y no deja traza sobre el motor no se añade al esquema, aunque esté bien documentado, y queda en el inventario sin promover, con la condición que no cumple en la columna de vía de su fila. En la vía de traza, la tercera condición exige además descartar que el elemento ya estuviera soportado y que la carencia saliera de una mala lectura de la guía. Con el catálogo cerrado, el criterio promueve veinticuatro de los treinta y cinco elementos no soportados

del inventario y deja once sin promover (Sección 6.3).

Para comprobar si el criterio dejaba fuera casos generales, se preguntó a un modelo de lenguaje cómo se generalizaría cada uno de los doce elementos que entonces no promovía o dejaba en suspenso (Anexo A). Uno de ellos, F-17, se retiró después del inventario, y la respuesta lo contaba entre los generalizables. La consulta recibió la descripción de cada caso, sin su veredicto. La respuesta da F-23, F-29 y F-35 por expresables con el esquema actual. En F-23 coincide con el inventario, donde la tercera condición falla, y en F-35, donde falta. En F-29 apunta en la misma dirección, aunque la comprobación de esa condición sigue pendiente. Los otros nueve los da por generalizables y los reduce a seis formas, como una lista de cuotas que se aplican a la vez o un precio por tramos. En F-25 la respuesta discrepa en parte del inventario, que da la tercera condición por fallida. Según la respuesta, la guía de ACM2YAML expresa los dos modos con alias, aunque no documenta un `workload` por alias. El criterio mide la generalidad por la recurrencia en el catálogo, y la respuesta sugiere que nueve casos se repetirían fuera de él. Sus ejemplos de otras APIs no se han comprobado, así que el criterio y las cifras no cambian. Revisarlo queda como decisión abierta.

4.6. Calibración del método: el piloto Postmark

El método anterior se calibró aplicándolo íntegramente a un piloto: Postmark, un proveedor de correo transaccional [49] no modelado previamente en el ecosistema. Esta sección describe la Datasheet del piloto tal como estaba antes de los sondeos del 7 de septiembre de 2026. Su estado al cierre del catálogo está en el Apartado 5.2.1, y el de sus elementos no soportados en la Tabla 6.1. El resultado es una Datasheet conforme a v0.3 con todos los campos tratados, con valor y fuente o con hueco explícito y trazable. Se construyó sobre trece fuentes registradas: diez documentales, la observación del panel con cuenta propia, la especificación que alimenta el explorador de la API del proveedor y un sondeo empírico. La validación estructural da cero errores y cinco avisos, correspondientes uno a uno a los huecos deliberados.

La Datasheet completa no se reproduce en la memoria. Se conserva, con su registro de fuentes, el inventario de elementos no soportados y las evidencias fechadas, en el repositorio de datos que acompaña al trabajo. La Tabla 4.3 ofrece su lectura resumida, pensada para leer el resultado del piloto sin recorrer el YAML, con cada valor acompañado del nivel de confianza que lleva en el documento.

Propiedad	Free	Basic	Pro	Platform
Precio (USD/mes)	0 ●	15 ●	16,50 ●	18 ●
Cuota incluida (emails/mes)	100 ●	10.000 ●	10.000 ●	10.000 ●
Sobrecoste en 10.000 (USD/1.000 extra)	no admitido ●	1,80 ●	1,30 ●	1,20 ●
Rate de la API REST	no publicado ●			
Destinatarios por mensaje	50 en total ●			
Mensajes por lote (`/email/batch`)	500 ●			
Envío masivo (`/email/bulk`)	sin tope numérico ● (carga $\leq$ 50 MB ●)			

Tabla 4.3: Lectura resumida de la Datasheet del piloto antes de los sondeos del 7 de septiembre de 2026. Cada valor conserva su nivel del semáforo: ● explícito en fuente oficial, ● inferido, ● no documentado. Los precios y el sobrecoste corresponden al volumen mínimo de cada *tier*, que es el que la página de precios publica; su variación con el volumen contratado se recoge en la Figura 4.1. La Datasheet completa, en el repositorio de datos del trabajo.

La misma lectura hace visible lo que el proveedor no publica. Además del límite de velocidad, la Datasheet registra como huecos la ausencia de un límite de concurrencia documentado para la API REST y la ventana exacta de la cuota en los planes de pago (confirmada solo en el gratuito, por panel, como ciclo de cuenta). También la URL de la especificación que alimenta el explorador oficial de la API, el precio de cada volumen en los *tiers* superiores y la disponibilidad del envío masivo por plan. Cada uno figura en `not_found` con su búsqueda documentada, de modo que la ausencia queda afirmada y fechada, no supuesta.

El piloto validó el método precisamente porque el dominio se resistió al esquema en puntos concretos. Cuatro hallazgos lo ilustran. Primero, el límite de velocidad de la API REST resulta contractualmente inobservable. El proveedor documenta el error de *rate limit* pero no publica umbral alguno [45]. Por eso el bloque correspondiente de la Datasheet queda vacío de forma deliberada, y la determinación del valor pasa a la fase empírica (Apartado 5.2.1).

Segundo, la estructura comercial real difiere de la publicada. La página de precios presenta tres planes. El panel revela que contratar el servicio exige dos decisiones independientes: qué *tier* funcional se quiere y qué volumen mensual se contrata. El volumen tiene su propia lista de precios, que llega hasta 1.500.000 correos por 775 USD al mes. Por encima de esa cifra el precio deja de publicarse y se negocia. El *tier* decide cuántos recursos se pueden usar (usuarios, servidores, *streams*) y qué partes de la API están disponibles: las de estadísticas y de seguimiento de aperturas y clics son exclusivas de Pro y Platform. El sobrecoste por correo excedido depende de los dos ejes a la vez: la página de precios lo hace variar por *tier* y el panel por volumen. Su valor en cada cruce no es observable sin una cuenta aprobada, de modo que la Datasheet recoge lo que cada fuente publica por separado y registra el cruce como hueco. La Figura 4.1 contrasta esa estructura con la proyección que admite el bloque `plans` de v0.3, que es

la adoptada en el catálogo: un plan por tier, valorado en su volumen mínimo, con la lista completa registrada en el *provenance*.

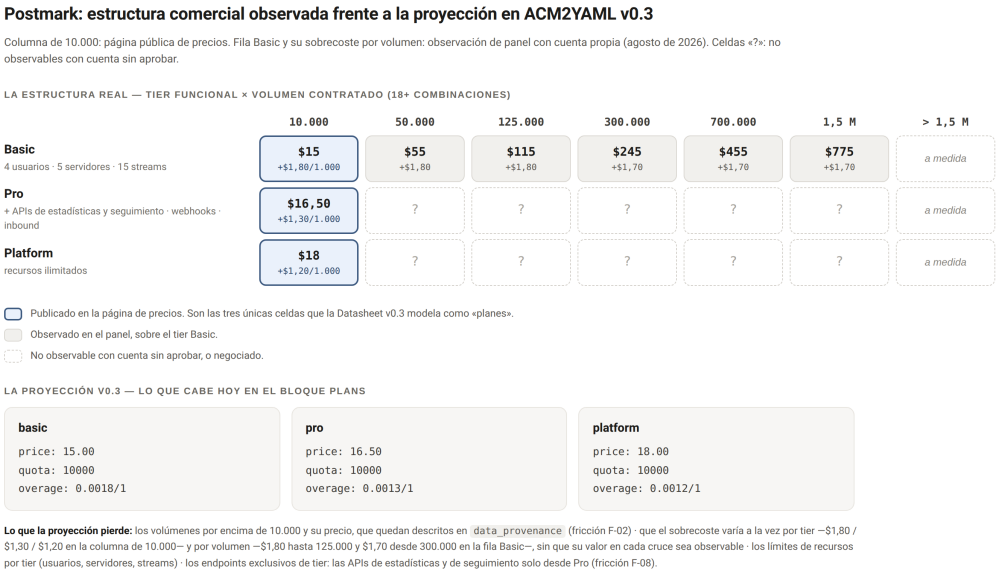


Figura 4.1: Estructura comercial de Postmark (*tier* funcional × volumen contratado) frente a la proyección representable en el bloque `plans` de ACM2YAML v0.3. La columna de 10.000 procede de la página de precios [49]. La fila Basic y su sobrecoste por volumen proceden de la observación de panel con cuenta propia. Las celdas sin valor no son observables con una cuenta sin aprobar. Elaboración propia (agosto de 2026).

Tercero, el sondeo empírico desmintió a la documentación en un dato que condiciona el análisis de capacidad. La referencia de la API declara un máximo de cincuenta destinatarios *por campo* (To, Cc y Cco), lo que sugiere hasta ciento cincuenta por mensaje. Las peticiones de sondeo demostraron que el límite real es de **cincuenta destinatarios totales por mensaje**: cincuenta se acepta en cualquier reparto entre campos y cincuenta y uno se rechaza en todos, y el propio mensaje de error lo confirma de forma literal. La discrepancia queda registrada con ambas citas. El sondeo del 7 de septiembre acotó después el factor de consumo por petición del envío por lotes: hasta quinientos mensajes por llamada, con cincuenta destinatarios cada uno (Apartado 5.2.1).

Y cuarto, el piloto dejó un inventario de ocho elementos no soportados documentados entre el dominio y el esquema, que la Tabla 4.4 sintetiza junto con su estado en el marco de recurrencia. Ninguno de ellos impide la conformidad del catálogo. Todos eran candidatos a fundamentar las extensiones de la contribución C-02 hasta que el resto del dominio confirmara (o desmintiera) su generalidad. La Tabla 6.1 recoge el veredicto de cada uno sobre las ocho APIs.

Id	Elemento no soportado por ACM2YAML v0.3	Estado
F-01	Un *rate* que existe contractualmente pero sin valor publicado: el bloque vacío se confunde con "sin límite"	candidato
F-02	Cuotas por volumen contratado, con sobrecoste dependiente a la vez del volumen y del *tier*, y tramo superior negociado	candidato
F-03	Factor de consumo multiplicativo (mensajes × destinatarios) que una terna única no puede expresar	candidato
F-04	Restricciones declaradas que el proveedor admite no aplicar o que la respuesta a la petición no refleja	discusión
F-05	La dimensión *Results Per Request* existe documentada en el dominio y v0.3 no la serializa	candidato
F-06	Un *workload* acotado por otra métrica (bytes de *payload*, tras codificación) y no por un entero	candidato
F-07	*Add-ons* que modifican dimensiones de capacidad, condicionados a volumen y a *tier*	candidato
F-08	El "plan" real es *tier* × volumen, con superficie de API dependiente del *tier*	candidato

Tabla 4.4: Elementos no soportados documentados en el piloto. Su promoción requiere recurrencia en al menos dos APIs del dominio, o una sola con la consecuencia ejecutada sobre el consumidor de referencia (Sección 4.5).

El piloto cumple así las dos funciones que se le pedían. Ha calibrado la mecánica del ciclo antes de escalarlo al dominio completo. Y ha mostrado que el método produce a la vez lo que este trabajo persigue: Datasheets conformes sin extender el esquema y conocimiento fundamentado sobre los límites del modelo que las define. La verificación por uso del Capítulo 6 mide si el motor de referencia puede consumirlas, y ninguna de las ocho resulta plenamente consumible por PRIME4API (Tabla 6.5). Fija además el formato de presentación del catálogo. Cada API se expondrá como aquí (caracterización breve, lectura resumida en tabla con su semáforo, huecos documentales y elementos no soportados nuevos o recurrentes respecto al inventario vigente), con la explicación del método ya dada por este capítulo, de modo que los epígrafes sucesivos resulten más breves, uniformes y comparables. Los capítulos siguientes aplican este ciclo al resto del catálogo y explotan sus resultados.

4.7. Respuesta a la pregunta de investigación

La RQ1 pregunta qué APIs de correo deben incluirse en un análisis representativo del dominio, y bajo qué criterios reproducibles. La primera mitad se responde con una condición: el conjunto debe cubrir las cuatro formas de cobro del primer criterio, y sus APIs deben diferir en los rasgos de límites que pide el cuarto. En la ventana de toma de datos (Apartado 4.1.4), las ocho APIs de la Tabla 4.5 forman un conjunto que la cumple. La condición fija qué debe cubrir el conjunto, y otro que cubriera lo mismo

también la cumpliría. La segunda mitad la responde el protocolo de la Sección 4.1, y el Anexo A registra la consulta.

API	**Fuente**	**Forma de cobro según la respuesta**	**Lo que registró la construcción**
Postmark	Piloto	Planes por volumen mensual	El plan real es *tier* × volumen contratado (Sección 4.6)
Resend	Segunda	Planes por volumen con caudal negociado no publicado	Planes por volumen mensual, con Enterprise sin precio, volumen ni *rate* publicados y Free con 100 correos al día y 3.000 al mes (Apartado 5.2.2)
SendGrid	Segunda	Planes por volumen mensual	Planes por volumen mensual, cuyo sobrecoste tiene techo desde el 26 de marzo de 2026 (Apartado 5.2.3)
MailerSend	Segunda	Planes por volumen mensual	Planes por volumen mensual, y es la única con cuota diaria de peticiones junto a la mensual de correos (Apartado 5.2.4)
Brevo	Tercera	Planes por volumen con límite diario adicional	Límite de 300 correos al día en Free y volumen mensual en los planes de pago (Apartado 5.2.5)
Amazon SES	Tercera	Pago por uso puro	Pago sin plan y, desde el 21 de julio de 2026, planes (Apartado 5.2.6)
Mailgun	Tercera	Planes por volumen mensual	Planes por volumen mensual, sin umbral en la documentación de la API y con un único caudal publicado para la API REST, en las preguntas frecuentes y por IP de cliente (Apartado 5.2.7)
Mailchimp Transactional	Tercera	Añadido de otro producto, por bloques	Complemento de los planes Standard y Premium o del mensual heredado, sin planes con nombre, en bloques de 25.000 correos (Apartado 5.2.8)

Tabla 4.5: Las ocho APIs del catálogo. La segunda columna indica la fuente de inclusión (Apartado 4.1.1). La tercera reproduce la clasificación de la respuesta a la consulta (Anexo A). La cuarta dice qué forma de cobro encontró la construcción en la ventana de toma de datos y añade un rasgo de la API.

La consulta pedía cubrir las cuatro formas de cobro, y la construcción las encontró todas en el catálogo. También corrigió tres afirmaciones de la respuesta. La respuesta describía Amazon SES sin planes, y el proveedor los tenía desde el 21 de julio de 2026, antes de la consulta. Atribuía a Mailgun *rates* publicados por endpoint, y la documentación de su API no publica ningún umbral. El único caudal que el proveedor publica para su API REST está en sus preguntas frecuentes, acotado a una IP

de cliente. Presentaba la combinación de cuota diaria y mensual como aportación de Brevo, y esa combinación ya estaba en el plan Free de Resend. La primera y la tercera corrección muestran además que las formas de cobro no son excluyentes. Ninguna de las tres correcciones activa la regla de reemplazo, que solo depende de la trazabilidad (Apartado 4.1.3).

El Capítulo 5 muestra qué aporta el conjunto frente a sus cuatro primeras APIs. Postmark, Resend, SendGrid y MailerSend tenían planes con cuota mensual. Todas tenían además sobrecoste decreciente y tope de destinatarios. Amazon SES, Brevo y Mailchimp Transactional, que la respuesta justificó por su forma de cobro, carecen de al menos uno de esos rasgos. Mailgun, que según la respuesta comparte forma de cobro con Postmark, SendGrid y MailerSend, es la única del catálogo que contesta por escrito sobre la concurrencia de su API REST. La Sección 5.3 recoge seis afirmaciones que solo son visibles con las ocho APIs. En esta memoria, un análisis del dominio es representativo cuando cumple la condición con que se abre esta sección, fijada con los criterios antes de la consulta. La cuota de mercado de cada API no se midió y no entra en esa condición.

Este cierre de la RQ1 tiene limitaciones conocidas, y las primeras están en las fuentes. La primera fuente no se ejecutó. Postmark entró como piloto del método y no por una de las tres fuentes. Los cinco criterios solo decidieron cuatro de las ocho APIs: la consulta clasificó las otras cuatro por su forma de cobro, sin evaluarlas. El prompt ya fijaba un catálogo de ocho APIs, y ese tamaño no se deriva de la cobertura.

Otras están en los criterios. La respuesta da por cumplidos dos de ellos sin aportar evidencia: la documentación pública del segundo y la adopción real del tercero. La documentación pública no cubrió todos los límites, y la construcción necesitó el panel con cuenta propia para algunos, como la cuota mensual del plan Free de Mailgun. Las cuatro formas de cobro se fijaron en el primer criterio antes de la consulta, sin un censo del mercado ni una lista cerrada de candidatas. Las diferencias en los límites que muestra el Capítulo 5 se observaron después, sobre las ocho APIs, y describen la muestra sin garantizar que esté completa. Mailchimp Transactional fue una de las dos últimas en construirse. Resultó ser la única API sin planes y la única sin tope de destinatarios. Una novena API podría añadir otro rasgo.

La regla de reemplazo es ordinal, y de haberse aplicado no garantizaba conservar la forma de cobro de la API sustituida. Si hubiera salido Amazon SES, habría entrado Mailjet, y el catálogo habría perdido la única API que la respuesta clasificó como pago por uso puro.

Las últimas están en la consulta, cuya respuesta no se repite con garantía. La consulta se hizo en la sesión de trabajo del proyecto, y la respuesta cita dos decisiones previas que el prompt no recoge. Un tercero con el mismo prompt y sin ese contexto puede obtener otra respuesta. El registro tampoco recoge cuántas veces se lanzó la consulta. El protocolo permite a ese tercero contrastar la respuesta que obtenga con los criterios y con lo que registre la construcción, como hace esta sección con la consulta del 1 de septiembre de 2026.

5. Hallazgos y comparación

5.1. Introducción

De las 25 preguntas del catálogo que tienen respuesta con el instrumental del canal API, 12 se responden en las ocho APIs a la vez. Con cuatro Datasheets eran 17, y las cinco que caen son las cinco de Brevo, por dos huecos de su página de precios. Ninguno de sus planes de pago documenta qué ocurre al superar la cuota, la página de precios niega que exista precio de exceso y los escalones intermedios de su deslizador de volumen no tienen precio. Esas cinco preguntas se quedan sin el sobrecoste o sin el precio del escalón que necesitan. Por API, Resend responde 22, Amazon SES y MailerSend 21, SendGrid, Mailgun y Mailchimp 20, Postmark 18 y Brevo 16. De las 18 de Postmark, 7 dependen de un *rate* que el proveedor no publica y que la Datasheet adopta en ámbar desde la cabecera medida por sondeo. Este capítulo lee las Datasheets del catálogo con dos preguntas de investigación delante.

La primera es RQ3: cómo se relacionan los límites de uso y los *rates* en las APIs de correo, y qué implicaciones tiene esa relación para el consumidor. Su respuesta es la contribución C-03: cómo se relacionan ambos límites en el correo y qué se sigue de ello para quien consume la API. La segunda es RQ4: cómo pueden compararse las APIs de forma sistemática para responder a preguntas de decisión que mezclan requisitos funcionales y no funcionales. Su respuesta es la contribución C-04: un catálogo de preguntas de decisión con el procedimiento para resolverlas sobre las Datasheets.

El capítulo no responde a RQ2. Los elementos que el esquema ACM2YAML no soporta y que dejó la construcción del catálogo aparecen aquí solo por su id, cuando un hecho depende de ellos, y se tratan en el Capítulo 6. Los identificadores que usa el capítulo son estos. F-nn es un elemento que el esquema no soporta, y va en el inventario que abre el Capítulo 6. O-nn es una observación que no es carencia del esquema, y el mismo capítulo le dedica una sección. P-nn es una propuesta de extensión del esquema del Capítulo 6. Y Q1 a Q10 son las operaciones de PRIME4API [1], que la Sección 5.3 nombra por su número.

Los hechos proceden de las Datasheets construidas con el método del Capítulo 4: documentación pública, panel con cuenta propia y sondeo contra la API, con semáforo y *provenance* por campo. En este capítulo «hechos» son los valores de la Datasheet y los de su registro de fuentes y evidencias, que guarda lo que ningún campo del esquema admite. Las Datasheets no se reproducen en la memoria, y se conservan con sus fuentes y evidencias en el repositorio de datos que acompaña al trabajo.

A la fecha de este capítulo hay ocho Datasheets construidas y sondeadas. Postmark [49] fue el piloto, y después se construyeron Resend [50], SendGrid [51], MailerSend [52], Brevo [53], Amazon SES [44], Mailgun [54] y Mailchimp Transactional [55]. Todo recuento y todo porcentaje de este capítulo se refiere a esas ocho. Tres sondeos midieron un estado de la suscripción y no el plan: el de MailerSend sobre una cuenta Sandbox sin aprobar, el de SES sobre el entorno de pruebas y el de Mailchimp sobre

una cuenta de prueba. El Apartado 6.4.3 define el estado de la suscripción y lo separa de la reputación. Ningún valor de plan de esas tres Datasheets cambia de semáforo por medición. Las cifras llevan punto de miles y coma decimal, también los precios. Solo el literal del proveedor entre comillas conserva su forma.

El capítulo se organiza del siguiente modo. En primer lugar, la Sección 5.2 recoge lo que revela cada API con el mismo guion de cuatro bloques (documentación, panel, sondeo y lo que se aprende del proveedor). A continuación, la Sección 5.3 calcula sobre las ocho APIs la relación entre límites de uso y *rates*, que es la respuesta a RQ3. Después, la Sección 5.4 presenta el catálogo de preguntas de Pricing Intelligence y la Sección 5.5 lo ejercita sobre las ocho Datasheets, que es la respuesta a RQ4. Por último, la Sección 5.6 resume las dos respuestas junto con lo que queda pendiente.

5.2. Lo que revela cada API

Cada subsección sigue el formato que fijó el piloto en la Sección 4.6, reordenado en cuatro bloques por fuente, porque cada API se distingue por lo que cada fuente añade sobre la anterior. Una apertura anuncia los hallazgos de la API y el estado de la suscripción que acota lo medido. El primer bloque recoge lo que la documentación pública declara sobre planes, cuotas, *rates*, topes por petición y sobrecoste. El segundo recoge lo que añadió el panel con cuenta propia, segundo peldaño de la escalera de validación de la Sección 4.4. El tercero recoge lo que añadió el sondeo, el tercer peldaño: peticiones reales contra la API, con sus códigos de respuesta y sus cabeceras. El cuarto recoge lo que se aprende del proveedor, hallazgo a hallazgo, y cierra con el límite de la lectura. La respuesta de PRIME4API sobre la Datasheet, si la hay, va en el tercer bloque o en el cuarto, junto al hallazgo que sostiene. Importa sobre todo lo que la documentación no dice y el panel o la API sí. Importan también las discrepancias entre fuentes y lo que el proveedor no publica. Entre fuentes se distinguen tres fenómenos, siempre con el mismo nombre. Hay contradicción si dos fuentes dan valores o condiciones incompatibles para el mismo atributo, ausencia si un canal no publica el límite y diferencia de canal si dos canales describen superficies distintas del servicio. Cuando la contradicción enfrenta a la página de precios con otra fuente, se resuelve a favor de la página y el otro valor se lee como incoherencia del *pricing*. Los elementos no soportados se nombran por su id. F-01 a F-22 se tratan en el Capítulo 6, con las observaciones O-nn y las propuestas P-nn, y las posteriores, desde F-23, se definen aquí la primera vez que aparecen. Cada cifra lleva su fuente, y la prosa dice cuándo el campo está en ámbar o en rojo.

Dos reglas gobiernan la lectura de los sondeos. Se aplican igual a las ocho APIs. La primera dice qué valida un sondeo sin envío real. Un sondeo con token de pruebas o en modo *sandbox* valida los topes por petición que la API comprueba al recibir la petición, como los destinatarios o el tamaño del mensaje. No todas lo hacen: el lote de MailerSend valida cada correo después de aceptar la petición, y Mailchimp encola por encima de diez destinatarios sin nombrar ningún tope. Ningún sondeo de este tipo valida la cuota consumida ni el caudal. Cuando la petición se rechaza por validación con un token real, como en los sondeos de caudal de Postmark, MailerSend y Brevo, el contador de caudal sí descuenta, y en MailerSend también la cuota diaria de peticiones.

La segunda dice cuándo un valor observado entra en la Datasheet. Entra cuando el sondeo se diseñó para medirlo y es inocuo. Una cabecera aparecida en un sondeo de otro propósito, si difiere de lo publicado, se anota como valor observado pendiente, y adoptarla exige un sondeo de caudal. Por la misma regla, un valor publicado va en verde si nada lo contradice, aunque no se haya medido. Si el valor lo publica la página de precios y el sondeo mide menos, se mantiene el valor publicado y la medida queda como incoherencia del *pricing*.

Las ocho APIs se presentan en el orden en que se construyeron, el de la Sección 5.1.

5.2.1. Postmark

Postmark [45, 49] deja cuatro hallazgos que se suman a la lectura del piloto (Sección 4.6 y Tabla 4.3, con los semáforos anteriores a septiembre). El primero es de capacidad: la API declara en cabecera un *rate* de 5.000 peticiones que la documentación no publica. El segundo es de topes por petición: el lote de 500 mensajes queda verificado y pasa de ámbar a verde. En el envío masivo el proveedor no pone tope de mensajes: lo que acota es la carga, en megabytes. El tercero es de esquema: la documentación y el panel muestran tres restricciones que v0.3 no recoge, una sobre el relé SMTP y dos sobre la administración de la cuenta. El cuarto es de validez, porque ninguna petición con token real llegó a enviar un correo.

La cuenta propia está en el *tier* gratuito y en modo de prueba. En ese modo solo se pueden enviar 100 correos, y solo a dominios verificados. Está además pendiente de aprobación. Mientras lo esté, el proveedor rechaza con ErrorCode 412 cada mensaje dirigido a un dominio distinto del remitente: «all recipient addresses must share the same domain as the 'From' address». Cada mensaje de los sondeos con token real iba a un dominio no verificado o excedía el tope de destinatarios, y por eso ninguno salió. No se leyó un contador de cuota antes y después, así que el gasto nulo de cuota de correos se infiere del código de rechazo de cada mensaje. Cambiar de plan exige enviar antes una solicitud de aprobación, y lo observado es solo el *tier* gratuito.

Qué dice la documentación. El *rate* de la API REST no está publicado. La referencia describe el error 429, «Rate Limit Exceeded», sin umbral numérico ni ámbito [45].

Los topes por petición que publica la referencia están en la Tabla 5.1, junto a lo que midió el sondeo. El proveedor mide el tamaño del correo «after it has been base64 encoded» [45], y la Datasheet deja las cotas en megabytes en el *provenance*. Lo que v0.3 no puede decir es que una cota en megabytes acote el máximo en correos, que en el envío masivo queda vacío (F-06).

El único límite de concurrencia que publica Postmark es de diez conexiones simultáneas por IP, y es una recomendación para el relé SMTP, un canal distinto de la API REST. El proveedor avisa de que «We do not have any systems in place to enforce these limits» y califica esos límites de «somewhat arbitrary, and informal» (F-04). Para la API REST no publica ninguno, una ausencia que el fichero registra en `not_found`.

Qué añadió el panel. El panel aporta dos de las restricciones que v0.3 no recoge, y una es el tope de servidores. En la cuenta gratuita fija en diez los servidores que se pueden crear y pide subir de *tier* para crear más: «Upgrade your Tier to create more servers». La lista de *tiers* del mismo panel no da cifra para el gratuito y da cinco servidores a Basic, diez a Pro e ilimitados a Platform. Ninguna de las dos pantallas explica que la cuenta gratuita admita más servidores que Basic. Las cuotas de v0.3 llevan periodo y se gastan, y un servidor se tiene o no se tiene, así que el tope queda sin campo en el fichero (F-11).

La otra es la gestión de credenciales: en lo que Postmark publica, generar o borrar el token de un servidor ya creado solo tiene canal en el panel. La pestaña de tokens del servidor ofrece «Generate another token» y «Delete», que se observaron sin pulsar. La referencia de la API no publica ninguna operación equivalente, y lo único programable es `POST /servers`, que crea un servidor nuevo con sus tokens. El fichero solo declara el canal del documento entero, `type: api_first`, y no tiene campo para decir que una operación suelta se hace desde la web (F-22).

Los complementos quedan fuera de la Datasheet por alcance: el esquema los deriva al canal web, que este trabajo no modela, y sus precios van al *provenance* (F-07, Apartado 6.4.7). La página de precios publica el de tres de ellos con la fórmula «Starts at» [49].

Qué añadió el sondeo. La cabecera da la cifra que la documentación no publica. En los tres sondeos con token real, las respuestas de los cuatro endpoints de envío probados traen `ratelimit-limit: 5000`, sea el código 200 o 422. Las que se archivaron completas traen cinco cabeceras de *rate* que la documentación no menciona, entre ellas `x-ratelimit-limit-second: 5000` y `ratelimit-reset: 1`. Que el límite sea por segundo es inferencia de esos nombres, no un literal del proveedor. La primera respuesta archivada con esas cabeceras es de un sondeo diseñado para contar destinatarios. Por la segunda regla de esta sección, el valor no entró en el campo hasta el sondeo de caudal.

El sondeo de caudal no alcanzó el 429, pero descarta una lectura del contador. Envió 200 peticiones a `/email` con token real, 25 en paralelo, en 4,57 segundos, y todas se rechazaron con ErrorCode 412. Las respuestas caen en cinco segundos del reloj del servidor, y en cada uno el contador `ratelimit-remaining` marca 4.999 entre cuatro y ocho veces. El mínimo de todo el sondeo es 4.986, en el segundo con 48 respuestas. Un solo contador que descontase cada petición al responder marcaría 4.999 únicamente en la primera respuesta de cada ventana, y un segundo del reloj toca como mucho dos ventanas de un segundo. Ese contador no marcaría 4.999 más de dos veces por segundo, y en el segundo de 48 respuestas habría bajado a 4.976 o menos. Lo observado es compatible con varios contadores en paralelo o con uno que no se actualiza al instante, y el sondeo no distingue entre esas dos explicaciones. Si la cabecera describe el límite, alcanzar el 429 exigiría al menos 5.000 peticiones en un segundo, más de lo que admite el sondeo ligero del tercer peldaño (Sección 4.4).

El sondeo de lote sirve de control sobre qué descuenta el contador: la respuesta a una petición con 500 mensajes marca 4.999, lo mismo que la de una petición con uno. Si el contador descontase los mensajes al responder, como descuenta la propia petición, habría marcado 4.500 o menos. El resultado vale para mensajes rechazados,

y el sondeo no observa si un mensaje enviado descuenta aparte. Es coherente con la unidad `requests` en la que el fichero declara el *rate*.

En `max_power` el valor figura en ámbar, como 5.000 peticiones por segundo, y pasaría a verde solo si un sondeo alcanzase el 429. El fichero lo referencia desde los cuatro planes por decisión de modelado, la de un solo *rate* no condicionado al plan. Esa decisión extrapola a los planes de pago un valor observado solo en la cuenta gratuita, y ninguna cabecera nombra el plan al que pertenece. Algunas respuestas traen además `x-postmark-account` y `x-postmark-server`, y ninguna de las dos cabeceras dice si el límite se aplica a la cuenta o al servidor.

Endpoint	**Referencia**	**Sondeo**	**Datasheet**
`/email`	≪Max 50≫ en cada campo (`To`, `Cc` y `Bcc`). 10 MB por mensaje	Token de pruebas. 50 en total, aceptados en dos repartos. 51 en total, 422 en tres repartos. 150 en total, 422	50 en total ●
`/email/withTemplate`	—	Token real. 51 en `To`, 422 con ErrorCode 300	50 en total ●
`/email/batch` y `/email/batchWithTemplates`	500 mensajes por llamada. 50 MB de carga (solo `/email/batch`)	Token real. 500 mensajes, lote aceptado con 200. 501, 422 con ErrorCode 410	500 ●
`/email/bulk`	Sin tope de mensajes. Destinatarios ≪as many recipients as you need≫. 50 MB de carga. ≪Max 50≫ destinatarios por mensaje	403 con el token de pruebas, sin medir	vacío ●

Tabla 5.1: Postmark: topes por petición de los cinco endpoints de envío según la referencia [45], el sondeo y la Datasheet. El guion marca ausencia: la referencia no publica un tope propio para el envío con plantilla. La fila de `/email` repite el sondeo del piloto (Sección 4.6) para compararla con la de plantilla. En esa fila la referencia da 50 por campo y el sondeo mide 50 en total, que es lo que adopta la Datasheet. La última columna es el máximo del `workload`, que el fichero declara en la unidad `emails` en los cinco endpoints, y las cotas en megabytes quedan en el *provenance* (F-06).

Con token real, `/email/withTemplate` rechaza lo mismo que el piloto rechazó en `/email`. Con 51 destinatarios, todos en `To`, devuelve 422 y el mismo mensaje, ≪Maximum of 50 recipients allowed per email message≫. El reparto no se varió, y el tope en total se apoya aquí en el literal del error, que cuenta por mensaje. Los dos endpoints de lote responden 200 a una petición de 500 mensajes y rechazan la de 501 con ≪You may only send up to 500 messages in a single batched request≫. Ese par de medidas fija el tope exacto. El envío masivo quedó sin medir: el único intento usó el token de pruebas, que ≪may only be used on the /email endpoint≫, y devolvió 403.

Qué se aprende del proveedor. Postmark es el caso de F-01 con el valor a la vista: un *rate* sin publicar que la API declara en cabecera. Mientras el bloque `max_power` no declaró ningún caudal, PRIME4API respondió como si no hubiera límite (Sección 5.3). P-04 propone una marca que separe ≪límite no publicado≫ de ≪sin límite≫ (Apartado 6.5.3). Con el valor en ámbar el fichero declara a la vez la cifra observada, en `max_power`, y que el proveedor no la publica, en `not_found`.

Los topes por petición tocan tres propuestas del Capítulo 6. En el fichero la

unidad `emails` cuenta destinatarios en `/email` y mensajes en el lote, y el proveedor no publica cómo se contabiliza un lote cuyos mensajes llevan sus propios destinatarios. Es F-03, el elemento no soportado que aborda P-02 (Apartado 6.4.2). El envío masivo acota la carga en megabytes y no en mensajes, que es F-06 y el caso de P-03 en ese mismo apartado. Su campo de mensajes queda además presente y vacío, el hueco que describe P-04, y con él fallan las operaciones de cálculo del motor.

Ninguna de las tres restricciones que v0.3 no recoge produce propuesta en el Capítulo 6. La de SMTP es una recomendación que el proveedor dice no hacer cumplir (F-04), y la Sección 6.8 la discute sin forma fijada. La de servidores es un recurso que se tiene y no se consume (F-11). La de credenciales es una operación cuyo único canal publicado es el panel (F-22). Las dos quedan promovidas, con una forma esbozada en la Tabla 6.6.

El límite de esta lectura es el estado de la suscripción descrito al principio. El contador de *rate* solo se ha leído con mensajes rechazados, y el 429 no se ha alcanzado. Tampoco se han observado el precio de cada volumen contratado en Pro y Platform ni el reprecio al cambiar de *tier*, que el panel no muestra con la cuenta sin aprobar. Y queda abierto si la cuota descuenta por destinatario o por mensaje.

5.2.2. Resend

Resend [50, 56] deja cuatro hallazgos, de capacidad, precio, plan negociado y fuentes. La documentación cuenta la cuota por destinatario, y el sondeo fija el tope de 50 destinatarios sobre la suma de `to` y `cc`. Pro y Scale se venden por volúmenes contratados, y una cuota de v0.3 admite solo uno. Enterprise no tiene precio, volumen ni *rate* publicados, y mientras está en la Datasheet el motor no devuelve ningún plan cuando la consulta los compara todos. Resend es la única de las ocho APIs cuyo *rate* coincide en documentación, panel y cabecera de respuesta.

Lo medido sale de una cuenta Free sin dominio verificado, que solo puede enviar a la propia dirección. Los valores de los planes de pago se conocen únicamente por la documentación.

Qué dice la documentación. La unidad de consumo es el destinatario: «Multiple To, CC, or BCC recipients count as separate emails». En `/emails` solo el campo `to` declara tope, «Max 50», sin aclarar si suma `cc` y `bcc`. El lote admite 100 mensajes con ese tope en cada `to`. De los dos literales se infiere que el consumo de una llamada puede llegar al producto de mensajes por destinatarios (F-03). Los adjuntos se acotan en bytes, 40 MB tras la codificación base64 (F-06).

La recepción también gasta cuota: «Both sent and received emails count towards these quotas» (F-12). Un correo que llega por encima de la cuota se recibe, pero su contenido no se puede recuperar.

Pro se contrata en dos volúmenes y Scale en seis (F-02). En Scale el sobrecoste baja con cada volumen, y en Pro no cambia. Cada cuota de v0.3 admite un volumen y un sobrecoste, así que la Datasheet guarda el primero de cada plan (Tabla 5.2).

El *rate* por defecto es «10 requests per second per team» [56], y no se documenta si varía por plan. Como se publica por equipo y ningún plan publica otro, la Datasheet aplica ese valor a Pro y Scale. La página de precios pone «Custom» en el precio, el volumen y el sobrecoste de Enterprise, y lista «Enterprise rate limits» sin cifra (F-01). La Datasheet copia `custom` y deja vacío su *rate*, en rojo.

Plan	**Volumen** (correos/mes)	**Precio** (USD/mes)	**Sobrecoste** (USD/1.000)	***Rate*** (req/s)
Free	3.000 y 100/día ●	0 ●	no disponible	10 ●
Pro	50.000 ●	20 ●	0,90 ●	10 (def.) ●
	100.000	35	0,90	
Scale	100.000 ●	90 ●	0,90 ●	10 (def.) ●
	200.000	160	0,80	
	500.000	350	0,70	
	1.000.000	650	0,65	
	1.500.000	825	0,52	
	2.500.000	1.150	0,46	
Enterprise	custom ●	custom ●	custom	no publicado ●

Tabla 5.2: Resend: volúmenes contratados por plan [50]. Las filas con semáforo son las que la Datasheet serializa, el primer volumen de cada plan. Las demás quedan en el *provenance* (F-02). El semáforo es el de la Sección 4.3, y «custom» es el literal del proveedor. Las ventanas de la cuota mensual y de la diaria están en ámbar (●). En Pro y Scale, «def.» marca el *rate* por defecto de la documentación. Free no admite sobrecoste. El tope de 50 destinatarios por petición no tiene columna porque la documentación no lo ata al plan, y se midió en la cuenta Free.

Qué añadió el panel. Los contadores «Monthly limit 0 / 3000» y «Daily limit 0 / 100» tienen cada uno una fila de envío y otra de recepción (F-12), y el panel repite «Rate limit 10 req/s».

El sobrecoste es opcional (F-13): en el panel es un interruptor apagado, marcado «Paid feature», cuyo texto empieza por «When enabled». El campo `overage_cost` no puede declarar que el sobrecoste existe solo si el cliente lo activa.

La suscripción de marketing que muestra la cuenta se limita por contactos, así que queda fuera por alcance (F-10). Tampoco entran los complementos, que el esquema deriva al canal web (F-07).

Qué añadió el sondeo. El tope de destinatarios es de la petición y no de cada campo. Todas las entradas llevaban la propia dirección. Con 51 en `to`, la API devolvió 422 con «Field `to` has a maximum of 50 items». Con 26 en `to` y 25 en `cc`, que un tope por campo habría aceptado, devolvió otro 422 con «The total number of recipients cannot exceed 50». Con 25 y 25 devolvió 200. Postmark tiene el mismo tope conjunto (Apartado 5.2.1). El campo `bcc` no se probó, así que lo medido suma solo `to` y `cc`.

La cabecera repite el *rate* de la documentación y del panel: las cuatro respuestas traían `ratelimit-limit: 10`. El corte no se provocó: las peticiones iban separadas entre dos y tres segundos, y en todas quedaron nueve disponibles.

Ante una dirección repetida la validación del tope contó cada entrada y la cuota no. La documentación no dice cómo cuenta una dirección repetida. Se aceptaron dos envíos: uno iba a un destinatario y el otro llevaba la propia dirección 25 veces en `to` y 25 en `cc`. El panel pasó de 0 a «Daily limit 2 / 100». Contar cada entrada habría dado 51. Contar por mensaje o fundir las repeticiones da 2, y el experimento no distingue ambas reglas. En SES una dirección repetida en `to` y en `cc` contó dos veces (Apartado 5.2.6).

La cabecera `x-resend-daily-quota` marcó el consumo previo a cada envío aceptado: 0 en el primero y 1 en el segundo. La documentación la describe como el consumo usado, sin decir si incluye la petición en curso. Con solo dos lecturas no se sabe si la cabecera se calcula antes de contar el envío o si el contador se actualiza tras responder.

Las dos ventanas de la cuota siguen en ámbar, porque el panel no muestra fecha de reposición. Para la diaria la documentación solo dice «wait until 24 hours have passed», y ninguna lectura del contador fija cuándo empiezan esas 24 horas.

PRIME4API falla sobre la Datasheet completa cuando la consulta recorre todos los planes. Pedida la suscripción más barata para 40.000 correos, devuelve 500 con `float() argument must be a string or a real number, not 'NoneType'`. El único campo vacío del fichero es el *rate* de Enterprise. Sin ese plan, la misma consulta responde de Pro, a 20 USD. El error apunta al campo vacío, pero retirar el plan retira también sus cadenas `custom` de precio y volumen. Esas cadenas hacen fallar la Datasheet de SendGrid ante la misma consulta, con otro error, y el 500 de Resend puede venir del campo vacío o de ellas.

Qué se aprende del proveedor. Según la documentación, una llamada a Resend consume tantas unidades como destinatarios lleve, y de ahí se infiere que en el lote ese consumo puede llegar a multiplicarse por el número de mensajes. El `workload` de v0.3 solo admite un máximo escalar, así que el factor queda en una descripción (F-03, P-02, Apartado 6.4.2). Tampoco hay dónde declarar que el tope cuenta las direcciones repetidas y la cuota no.

Los volúmenes contratados son un caso de P-01 (Apartado 6.4.1), porque el fichero guarda uno por plan, y la Sección 5.5 mide cuánto se desvía el motor sin los demás.

El *rate* vacío de Enterprise es un caso de P-04 (Apartado 6.5.3), porque v0.3 no marca un valor que existe sin publicarse. Su `custom` de precio y volumen es un caso de P-05 (Apartado 6.5.4). Que un solo plan negociado impida calcular los demás es lo que la propuesta de P-05 para el motor quiere evitar.

Resend es uno de los tres contraejemplos del desajuste entre el *rate* publicado y el observado (Sección 5.3). Lo que coincide en documentación, panel y cabecera es el valor declarado en la cuenta Free, no un corte medido. Sin dominio verificado, la cuenta no puede enviar a destinatarios distintos, y el sondeo dejó sin medir el corte del *rate* y el campo `bcc`.

El repositorio de la guía [43], del 2026-02-24, modela Resend con 2 peticiones por segundo, y la documentación del 2026-08-05 publica 10. Sin la página de febrero, la diferencia no se puede atribuir ni a un cambio de la oferta, que sería un caso de F-09, ni a la lectura de entonces.

5.2.3. SendGrid

SendGrid [51, 57] deja cuatro hallazgos. En el precio, el envío mensual de Essentials y Pro tiene un techo de diez veces el volumen del plan, sobrecoste incluido, y v0.3 declara el sobrecoste como un precio unitario sin máximo. En la vigencia, la prueba gratuita dura 60 días y después se detiene el envío, pero el único dato temporal de un plan en v0.3 es su periodo de facturación, y no hay campo de vigencia. En las fuentes, el modelado necesita dos datos que solo aparecen fuera de las páginas de precios y de límites: el caudal de envío y el techo del sobrecoste. En la capacidad por petición, el proveedor publica el tope total de destinatarios y el sondeo lo verifica, así que el tope escalar de v0.3 basta. El relé SMTP tiene en cambio topes propios que el esquema no sabe declarar.

La cuenta observada está en la prueba gratuita, con un tope duro de 100 correos al día. Los envíos de los sondeos usaron el modo *sandbox*, que valida la petición sin entregarla ni consumir créditos. Por la primera regla de esta sección, miden topes por petición. Las lecturas de otros endpoints sí fueron llamadas reales con la clave de la cuenta. La Datasheet modela solo el Email API, porque el proveedor cuenta aparte los envíos de Marketing Campaigns (F-10).

Qué dice la documentación. El precio depende del volumen contratado: Essentials se contrata en dos volúmenes y Pro en cinco, cada uno con su precio y su sobrecoste (F-02). La Datasheet declara un plan por *tier* en su volumen mínimo, la convención fijada en la Sección 4.6, y la Tabla 5.3 recoge todos los volúmenes.

Plan	**Precio** (USD/mes)	**Volumen incluido** (correos/mes)	**Sobrecoste web** (USD/correo)	**Sobrecoste panel** (USD/correo)	**Techo mensual**
Free Trial	0,00 ●	100 al día ●	no aplica	no aplica	no aplica
Essentials 50K	19,95 ●	50.000 ●	0,0013 ●	0,00133	10X (500.000)
Essentials 100K	34,95	100.000	0,0009	0,0009	10X
Pro 100K	89,95 ●	100.000 ●	0,0011 ●	0,0011	10X
Pro 300K	249,00	300.000	0,0009	0,00091	10X
Pro 700K	499,00	700.000	0,0008	0,00078	10X
Pro 1.5M	799,00	1.500.000	0,0006	0,00059	10X
Pro 2.5M	1.099,00	2.500.000	0,0005	0,00047	10X
Premier	custom ●	custom ●	custom	—	—

Tabla 5.3: SendGrid: precio por volumen contratado [51]. Precio y volumen salen de la página de precios y coinciden con el panel. El sobrecoste va en dos columnas porque el panel lo da con más decimales que la web. A la Datasheet llega el de la web, y el del panel se registra como incoherencia del *pricing*. El semáforo marca los valores que llegan a la Datasheet, que declara un plan por *tier* en su volumen mínimo (F-02). La ventana mensual de Premier es el único campo en ámbar del fichero, porque ninguna fuente consultada la da para ese plan. El guion indica que la fuente no da el dato, y «no aplica», que la prueba gratuita no tiene sobrecoste.

Desde el 26 de marzo de 2026 el sobrecoste tiene techo: el envío mensual de Essentials y Pro queda en «10X times the plan», que en Essentials 50K son 500.000

al mes, sobrecoste incluido. El literal cuenta «requests», y el mismo artículo las define como correos procesados más descartados, «Requests = Processed + Dropped». De las fuentes consultadas, el techo solo aparece en ese artículo, que resuelve el error «Maximum Credits Exceeded», y ni la página de precios ni el artículo de sobrecostes lo mencionan (F-16). El sobrecoste depende además del canal de contratación: si la cuenta se creó a través de Heroku o Google Cloud, «your subscription does not allow for overages» (F-13).

La prueba gratuita tiene fecha de fin. Se anuncia a «$0/mo for 60 days», y al terminar «email send will stop» salvo que antes se elija un plan del Email API (F-14). Mientras dura es un tope duro, «You can't exceed these limitations». Dar de baja un plan de pago lleva al mismo punto: a final de mes el plan pasa a «end of trial», sin envío, y no vuelve a ningún plan gratuito.

La FAQ de la página de precios describe un plan gratuito sin mencionar los 60 días, «100 emails per day on our free SMTP plan». El registro de cambios del 27 de mayo de 2025 anuncia la retirada de los planes gratuitos. La Datasheet modela la prueba de 60 días porque es el plan con el que el panel abrió la cuenta el 2026-09-06. Un modelado que partiera solo de la FAQ declararía un plan gratuito sin fecha de fin, y el fichero no tiene dónde decir qué parte de una fuente sigue vigente (F-09).

Solo una FAQ de `/v3/mail/send` cifra el caudal de envío: «up to 10,000 requests per second». No lo ata a ningún plan ni dice su ámbito, si se aplica por cuenta, por clave o por subusuario. La página de *rate limits* no menciona ese endpoint, y sus dos cifras, 500 y 150, salen de respuestas de ejemplo que no coinciden entre sí. La duración del «refresh period» tampoco se publica (F-01). La Datasheet copia los 10.000 a los cuatro planes en verde documental, por la segunda regla de esta sección.

El tope de destinatarios cubre la petición entera: «no more than 1,000» entre `to`, `cc` y `bcc`, sumados todos los sobres. Un sobre es un objeto `personalizations` con sus propios destinatarios, y una petición admite como mucho 1.000 sobres. Por el literal de la página de precios, cada destinatario descuenta un crédito, «one email to one recipient», de modo que una petición llena gasta 1.000 correos de la cuota. Con el total publicado basta el tope escalar de v0.3, `max: 1000` correos por petición.

El relé SMTP es otra vía de envío del mismo servicio, que en la prueba gasta la misma cuota, «via web API or SMTP», con topes propios: 10.000 direcciones por mensaje frente a 1.000 por petición en la API. Cada tope rige en su vía, y la diferencia es de canal. Como el relé no tiene ruta ni método, sus topes se quedan en el *provenance* (F-15). La Tabla 5.4 reúne los límites de las dos vías.

Límite	Dónde lo publica el proveedor	Qué observó el sondeo	Dónde queda en la Datasheet
Destinatarios por petición	≪no more than 1,000≫ entre `to`, `cc` y `bcc` de todos los sobres	200 con 1.000 en dos repartos, 400 con 1.001 en tres	`workload`: 1.000 correos ●
Sobres por petición	1.000 `personalizations`	400 con 1.001 sobres, con mensaje propio	descripción del `workload`
Tamaño	≪less than 30MB≫, del correo con adjuntos	cuerpo JSON de 31.457.280 bytes: 200, de 31.457.281: 413	`workload`: 30 MBs ●
Caudal de envío	≪up to 10,000 requests per second≫, en una FAQ, sin plan ni ámbito	sin cabeceras `x-ratelimit-*` en *sandbox*	10.000 peticiones/s en los cuatro planes ●
Caudal de lectura	ninguna página consultada lo cifra	600 en seis endpoints, 6 en `/v3/messages`	*provenance*
Forma de `x-ratelimit-reset`	marca de tiempo Unix, en un ejemplo	segundos restantes (inferencia) en tres endpoints, marca Unix en otros tres	*provenance* (O-07)
Ventana diaria de la prueba	≪next day (UTC time)≫, sin hora exacta	fechas de reposición desfasadas en `/v3/user/credits`	medianoche UTC ●, la hora en `not_found`
Relé SMTP	10.000 direcciones por mensaje, 5.000 mensajes por conexión, 10.000 conexiones simultáneas desde un servidor, tamaño no publicado	no sondeado	*provenance* (F-15)

Tabla 5.4: SendGrid: cada límite de capacidad con la fuente que lo publica, lo que observó el sondeo y dónde queda en la Datasheet [57]. ≪Ninguna página consultada lo cifra≫ es una ausencia. Los caudales de lectura quedan fuera del fichero porque la Datasheet solo modela `/v3/mail/send`.

Qué añadió el panel. El sobrecoste del panel lleva más decimales que el de la web en cinco de los siete volúmenes de pago, y en los cinco la web publica el valor del panel redondeado a cuatro decimales (Tabla 5.3). La diferencia queda anotada en el registro de discrepancias (Sección 4.3) y se cierra con el valor de la página de precios. Los decimales del panel quedan como incoherencia del *pricing*. En el fichero la diferencia solo toca un campo, el sobrecoste de Essentials 50K, que declara 0,0013.

El panel pone fecha a la vigencia de la prueba. La cuenta, creada el 2026-09-06, muestra ≪Your plan will be downgraded to End Of Trial on November 5, 2026≫ (F-14).

El panel no completa los huecos de ventana y caudal. El contador diario dice ≪100 left≫ sin hora ni zona de reposición, y ninguna de las pantallas revisadas muestra el caudal ni un registro de 429 (F-01).

Los complementos cambian con el plan: la validación de direcciones empieza en Pro, y los créditos de Email Testing solo se venden sobre un plan de pago. Ninguno pertenece al envío que modela la Datasheet, y la regla de alcance los excluye (F-07). También cambia con el plan la superficie: la prueba gratuita y Essentials no tienen residencia regional de datos ni subusuarios (F-08). En Premier el panel incluye 500 créditos de Email Testing y la web 60. La contradicción se resuelve a favor de la web, que es la página de precios, y no toca el fichero, porque el esquema no modela ese producto.

Qué añadió el sondeo. El sondeo verificó que el tope de destinatarios es total y descartó que se aplique por campo o por sobre. Se enviaron siete peticiones: una de control con un destinatario y cinco repartos alrededor del tope, más una con 1.001 sobres de un destinatario cada uno. Con 1.000 destinatarios, en un sobre o en dos de 500, la API devuelve 200. Con 1.001 devuelve 400 en los tres repartos probados, que son un sobre único, dos sobres de 500 y 501, y un sobre con 500 en `to` más 501 en `cc`. Un tope por sobre habría aceptado el segundo reparto, y uno por campo también el tercero. El mensaje de error nombra la suma, «limit of 1000 total recipients (to + cc + bcc)». La petición de 1.001 sobres de un destinatario devuelve 400 con el mensaje de la suma y con otro que limita los sobres a 1.000 por petición. Como esa petición también suma 1.001 destinatarios, el código de estado no separa los dos topes. Los separa el mensaje propio.

El sondeo de tamaño midió una magnitud distinta de la publicada. La documentación limita el correo, adjuntos incluidos, a «less than 30MB». El sondeo acotó por bisección el cuerpo JSON de una petición de solo texto, con un envío por tamaño. Los cuerpos de 30.000.000, 31.000.000 y 31.457.280 bytes devuelven 200, y los de 31.457.281 y 31.500.000 devuelven 413. El corte cae en 30 MiB exactos, y un tope de 30 MB decimales sobre la petición habría rechazado ya la de 31.000.000. El 413 llega como una página HTML firmada por nginx, y no con el JSON de errores con que la API rechazó los destinatarios. La Datasheet mantiene los 30 MB publicados como segunda entrada del `workload`, junto a los 1.000 destinatarios: el sondeo midió otra magnitud y no los contradice (F-06).

El caudal de envío no se pudo leer en las respuestas. La página de *rate limits* dice que «Each Web API response includes rate-limit headers», pero ninguna de las siete respuestas de `/v3/mail/send`, todas en *sandbox*, trae cabeceras `x-ratelimit-*`. Solo llevan `date` y, en las aceptadas, `x-message-id`. Queda sin comprobar si las trae la respuesta de un envío real.

Los endpoints de lectura sí traen las cabeceras, y ninguna página consultada cifra lo que dicen (F-01). En seis de ellos `x-ratelimit-limit` vale 600, y en `/v3/messages` vale 6. En esos seis, `x-ratelimit-reset` llega en dos formas, un número pequeño en tres y una marca de tiempo Unix en los otros tres, y la documentación solo muestra la segunda (O-07). Leer el número pequeño como segundos restantes es una inferencia, y las propias respuestas la contrastan. Cada uno de los tres valores, 9, 6 y 58, coincide con los segundos que faltaban para el siguiente cambio de minuto al llegar su respuesta. Las tres marcas Unix apuntan a esos mismos cambios de minuto. Eso es compatible con una ventana fija que se repone en cada cambio de minuto. En `/v3/messages` una sola respuesta no basta para saber la longitud de la ventana.

La llamada a `/v3/user/credits` confirma desde la API que la cuota de la prueba es un tope duro: `is_hard_limit: true`, 100 en total y reinicio diario. No completa, en cambio, la hora de reposición. La documentación la fija al día siguiente en UTC, «wait until the next day (UTC time)», sin dar la hora exacta, y el panel tampoco la da. Las fechas de reposición de la API, «2026-09-01» y «2026-09-02», no están al día en una cuenta creada el 2026-09-06, así que ese campo no sirve para fijarla. La Datasheet asume la medianoche UTC, que es inferencia y no literal, y anota en `not_found` que la hora exacta de reposición no se publica.

Qué se aprende del proveedor. El esquema no tiene dónde poner el techo del sobrecoste. La Datasheet declara en Essentials 50.000 correos y 0,0013 USD por cada uno de más. El campo `overage_cost` es un precio unitario sin máximo, así que el fichero se lee como capacidad ilimitada a ese precio, y el proveedor la corta en 500.000 al mes. En el Apartado 6.4.4 el motor ejecuta esa lectura y se propone el campo que falta. El tope duro de la prueba sí cabe, porque su cuota se declara sin `overage_cost`. Lo que no cabe es que la posibilidad de sobrecoste dependa del canal por el que se creó la cuenta (F-13).

Un plan de v0.3 no tiene campo de vigencia: su único dato temporal es el periodo de facturación. El fichero declara entonces 100 correos al día para una cuenta que desde el 5 de noviembre de 2026 no podrá enviar ninguno si antes no contrata un plan. Este elemento no soportado (F-14) queda promovido sin número de propuesta, con la forma esbozada en la Tabla 6.6.

Una recolección limitada a las páginas de precios y de límites no encontraría el caudal de envío, que está en una FAQ, ni el techo del sobrecoste, que está en un artículo de resolución de errores. De la FAQ de precios tomaría además un plan gratuito sin vigencia declarada. La especificación OpenAPI tampoco sustituye a la prosa, porque no lleva el tope total de destinatarios (O-06). Y una herramienta que lea `x-ratelimit-reset` para calcular cuándo reintentar recibe dos formas de valor bajo el mismo nombre, de las que la documentación solo muestra una (O-07). La Sección 6.6 trata las dos observaciones.

En la capacidad por petición, lo que el esquema puede declarar depende de la vía de envío. Por la API basta el escalar de v0.3, `max: 1000`, porque el sondeo verificó el tope total publicado. Es la variante acotada de F-03 (Apartado 6.4.2), el consumo que se multiplica por el número de destinatarios: aquí el proveedor lo encierra en un tope total por petición. Los topes del relé, por mensaje y por conexión, no tienen dónde declararse sin inventar una ruta (F-15). Tampoco este elemento no soportado llega a propuesta: queda promovido, con la forma esbozada en la Tabla 6.6.

Esta lectura tiene dos límites. El caudal real de `/v3/mail/send` solo se contrastaría con una campaña de carga. Es el cuarto peldaño de la escalera de validación (Sección 4.4), fuera del alcance de este trabajo. El otro límite es Premier, que no publica precio, volumen ni techo. La Datasheet lo declara `custom`, y ese precio no público hace fallar trece de las veintiuna comprobaciones de PRIME4API sobre el fichero (Tabla 6.5, Apartado 6.5.4).

5.2.4. MailerSend

MailerSend [52, 58] deja cuatro hallazgos. El primero es de precio: el de Starter y Professional solo existe con JavaScript, y la misma web sirve otra oferta el mismo día. Los términos de uso remiten a `/pricing`, pero no dicen si rige su lectura con JavaScript o sin él. El segundo es de consumo: de las ocho Datasheets del catálogo, la de MailerSend es la única con una cuota diaria de peticiones junto a la mensual de correos. Además, el contador que el cliente consulta va por detrás de lo consumido. El tercero es de capacidad: el caudal de los endpoints de envío y los topes por petición cambian con el

estado de la suscripción y la reputación, y el plan no basta para saber cuáles aplican. El cuarto es una dimensión que v0.3 no tiene, un cupo de dos destinatarios distintos por cuenta que se alcanzó sin que saliera un solo correo.

La cuenta propia está en modo Sandbox, nunca aprobada, sin plan contratado ni dominio propio. La Datasheet aplica a la cuenta propia lo que la documentación publica para la cuenta sin aprobar. Es un supuesto y no una equivalencia publicada: la API la llama «sandbox account», y el centro de ayuda describe la aprobación como un paso dentro del Sandbox. Lo medido describe una cuenta Sandbox sin aprobar y no el plan, cuyos valores conservan su fuente documental. La Datasheet lo lleva aparte, bajo un estado de la suscripción sin aprobar y en ámbar, porque es el dato de una sola cuenta.

Qué dice la documentación. El precio de Starter y Professional lo fija un selector de ocho posiciones de volumen, de modo que cada *tier* se contrata a un volumen concreto (F-08). El selector reescribe las tarjetas con JavaScript, y un lector sin navegador no ve ese precio. La Datasheet declara cada plan en su primer volumen, que recoge la Tabla 5.5, y deja el resto en el *provenance*. Enterprise se contrata por contacto, sin precio ni volumen publicados. El selector de la página de precios lleva Starter hasta 1.000.000 de correos al mes, y ese es el tope modelado. Dos páginas de producto lo detienen en 500.000, cifra que queda registrada como incoherencia del *pricing*.

El sobrecoste por cada 1.000 correos baja con el volumen, y al mismo volumen difiere entre *tiers*: a 250.000 correos vale 1,10 USD en Starter y 1,00 en Professional. Una sola tarifa de sobrecoste por cuota no recoge esa variación (F-02, que lleva a P-01 en el Apartado 6.4.1). El proveedor no publica un tope para el sobrecoste, y el único tope publicado lo configura el cliente y detiene el envío al alcanzarlo. El Free exige tarjeta y aprobación previa, y su cuota es un tope duro, sin sobrecoste (F-13).

Plan	**Precio** (USD/mes)	**Correos** (al mes)	**Peticiones** (al día)	**Lotes** (por minuto)	**Sobrecoste** (USD/1.000)	**Volumen máximo** (correos/mes por USD/mes)
Free	0 • (aprobación)	500 •	100 •	no modelado	no admitido	único
Hobby	7 •	5.000 •	1.000 •	10 •	1,50 •	único
Starter	35 •	50.000 •	100.000 •	15 •	1,30 • a 1,00	1.000.000 por 600
Professional	110 •	50.000 •	500.000 •	30 •	1,30 • a 0,50	2.000.000 por 1.600

Tabla 5.5: MailerSend: valores por plan al primer volumen contratable [52]. La columna de lotes es el *rate* de `/v1/bulk-email`, que sí cambia con el plan. El Free no lo lleva porque tres fuentes del proveedor dan el lote desde Hobby, y el artículo de límites, que le publica 10 por minuto, las contradice. El sobrecoste va del primer al último volumen del selector, y el volumen máximo es su última posición. Dos páginas de producto dan 500.000 en Starter, cifra registrada como incoherencia. La ventana mensual del Free está en ámbar. El *rate* de `/v1/email` y los topes de destinatarios no se publican por plan y van en la Tabla 5.6. Enterprise no se modela.

La web sirve además otra oferta el mismo día. Sin JavaScript, el HTML de `/pricing` da a Hobby 5,60 USD al mes con pago anual, y el mismo documento renderizado dibuja 6,30. Una página de producto da a Starter y a Professional 28 y 88 USD al mes con pago anual, con un sobrecoste de 0,90 en Starter. Esos precios coinciden

con los del 2026-09-08 rebajados un 20 %, el descuento anual que el proveedor anunció en 2024. El 2026-09-08 ese descuento es del 10 %. Según ese anuncio, Starter tenía ese sobrecoste antes del 29 de julio de 2024. Las dos lecturas son inferencias desde las cifras, porque ninguna página presenta esos valores como anteriores. Los términos de uso se reservan además «in-app beta pricing experiments». Una fecha por documento no dice cuál de esas ofertas vio el lector (F-09).

El ejemplo de MailerSend del repositorio de la guía [43], fechado el 2026-03-19, difiere de la lectura del 2026-09-08 en el precio, el sobrecoste, la ventana de la cuota mensual y el caudal del lote. Sus precios son los de esa lectura con aquel 20 % aplicado. Su sobrecoste de 0,95 USD, que pone en Starter y en Professional, es el que el proveedor anunció para Starter con vigencia desde el 29 de julio de 2024. El sobrecoste de Hobby, 1,20, no tiene otra fuente que el ejemplo. Los otros dos campos son defectos del ejemplo, que recoge la observación O-09 (Sección 6.6): declara ventana deslizante, que ninguna página describe, y un solo caudal de lote para los cuatro planes. Ningún cambio desde el ejemplo hasta esa lectura tiene changelog o anuncio fechado (F-09).

El plan de MailerSend lleva dos cuotas en unidades distintas [58], y ninguna de las otras siete Datasheets del catálogo lo hace. La mensual cuenta correos, y la diaria, repuesta a medianoche UTC, cuenta peticiones. La diaria es común a la API y al relé SMTP, con una diferencia de canal: por la API descuenta cada petición, y por SMTP cada mensaje (F-15). El esquema v0.3 expresa las dos: la diaria en el plan y la mensual en el endpoint. Con un destinatario por petición, en Hobby la diaria se agota antes que la mensual (Sección 5.3).

El Free lleva además un tope diario de 100 que una página de ayuda cuenta en correos y la página de precios en peticiones. Manda la página de precios, así que la cifra es la cuota diaria de peticiones que el plan ya lleva. La frase en correos queda en el *provenance* como incoherencia del proveedor. De haber sido de correos, habría sido un tercer tope en la unidad de la mensual sin sitio en v0.3 [43], y por eso F-17 se retiró del inventario.

La ventana de la cuota mensual tiene otra contradicción, sobre el día en que se repone. Unas páginas dicen «resets every 30 days» y otra «next billing day», y las dos solo coinciden si el ciclo dura 30 días exactos, mientras los términos hablan de un mes. Las dos describen una ventana fija ligada a la facturación, así que el tipo de ventana va en verde en los planes de pago. En el Free va en ámbar, porque ninguna página ata su reposición al día de facturación. Para el plan anual no se publica qué día se repone la cuota, una ausencia.

El caudal publicado depende del plan solo en el lote, y cambia además con la reputación de la cuenta. El caudal general, de 60 peticiones por minuto, y el de `/v1/email`, de 120, se publican como valor por defecto y sin desglose por plan, y la Datasheet los aplica a los cuatro. Para el lote, dos páginas dan un único valor, 10 por minuto, y otras dos dan uno por plan. La Datasheet toma el desglose, porque la página del valor único lo llama «default», y un valor por defecto no contradice una tabla por plan.

Con la reputación bajo revisión el lote baja a 1 petición por minuto y `/v1/email` a 10 (F-18). La página de estados cuenta los 429 entre los indicios que pueden llevar a revisión, «increased error rates (e.g., 403, 422, 429 responses)», sin umbral ni ventana.

Una cuenta recién ascendida «may have a lower rate limit», sin cifra. La Tabla 5.6 pone lo publicado para cada estado junto a lo medido.

El proveedor ata los topes de destinatarios a un segundo estado, que publica aparte de la reputación: la aprobación de la cuenta. Con la cuenta aprobada son 50 en `to` y 10 en `cc` y en `bcc`. Ninguna página da el total por petición, así que la Datasheet declara la suma aritmética, 70, y la deja en ámbar (F-03). En el lote el proveedor sí escribe esa cuenta: 500 objetos de hasta 70 destinatarios cada uno, «for a total of 35,000 emails», en un cuerpo de hasta 50 MB. El proveedor escribe la multiplicación para el lote y la calla para el envío simple.

Para la cuenta Sandbox la documentación publica valores propios, que recoge la Tabla 5.6. Publica también un cupo de dos destinatarios distintos que ata a la cuenta y no al mensaje, «You can send to any 2 recipients in Sandbox mode, until your account is approved». No le da ventana: lo levanta la aprobación.

Qué añadió el panel. El panel muestra el estado de la suscripción y no ofrece ninguna vía para aprobar la cuenta. El lateral muestra «Daily requests 100», la cifra que la documentación da al Sandbox y también al Free, y con ella el panel no distingue los dos estados. El botón de subida de plan pide «first add your own domain», y `/billing` redirige al panel principal. Ese botón contrata un plan y no aprueba la cuenta. El centro de ayuda sitúa la aprobación en el panel, «Submit your account for approval from your dashboard». Con el perfil de empresa sin rellenar, ni el panel principal ni los ajustes muestran un botón para solicitar la aprobación.

El DOM guarda dos modales ocultos. Uno anuncia «From December 2, we're introducing a fee of $7/month for the Hobby plan, with 5,000 emails», sin año. Es el único rastro localizado del paso de Hobby gratuito a Hobby de pago, y ninguna página lo fecha (F-09). El otro avisa del fin del *trial*, un plan con fecha de caducidad (F-14).

El panel no muestra el caudal vigente ni la fecha de reposición de la cuota mensual, y en esta cuenta tampoco los precios por volumen ni el tope de gasto. No dice si los límites son por cuenta o por token.

Qué añadió el sondeo. MailerSend deja leer sus topes en los rechazos de validación, y el sondeo los midió sin que saliera un correo. En las tres primeras ejecuciones la API rechazó las peticiones con 422, porque llevaban un remitente de dominio no verificado. La validación informa del primer error de cada campo, así que cada tope se aisló con el par N, N+1: con N el campo no da error, y con N+1 devuelve el mensaje de su tope. Dentro de un campo, un tope puede ocultar a otro. La segunda ejecución repitió la primera y dio los mismos veredictos. La tercera esperó siete segundos entre llamadas para que el caudal no se agotara antes de medir. Una cuarta, el 2026-09-14, buscaba la unidad de la cuota mensual con un envío real desde el dominio de prueba, que sí está verificado.

Límite	**Publicado, con la condición a la que el proveedor lo ata**	**Medido en la cuenta Sandbox**
Rate de `/v1/email` (por minuto)	120 con reputación sana 10 bajo revisión	10, con 429
Rate del lote (por minuto)	10 en Sandbox y Hobby 15 en Starter 30 en Professional 1 bajo revisión	1, con 429
Rate general (por minuto)	60 con reputación sana y bajo revisión	60 en cabecera
Destinatarios en `to`	50 con la cuenta aprobada 10 sin aprobar	1
Destinatarios en `cc` y en `bcc`	10 cada uno con la cuenta aprobada 1 sin aprobar	`cc`: 1 `bcc`: menos de 10, sin acotar
Objetos por lote	500 en Hobby, Starter y Professional 5 en Sandbox	5, el sexto con 422
Destinatarios distintos por cuenta	2 en Sandbox, hasta la aprobación	alcanzado
Cuota mensual (correos)	100 en Sandbox	no medida
Cuota diaria (peticiones)	100 en Sandbox	100, cada petición descuenta

Tabla 5.6: MailerSend: límites que cambian con el estado de la suscripción y la reputación [58]. El proveedor publica dos clasificaciones por separado, la reputación y la aprobación, y la segunda columna dice a cuál ata cada valor. Una condición que no aparece en una celda es una condición para la que el proveedor no publica valor. La última columna recoge lo medido en una sola cuenta Sandbox, sin plan y con la reputación sin registrar. El *rate* general es una cabecera vista en una lectura de otro propósito, sin 429.

Cada petición descuenta una unidad de la cuota diaria, como publica el proveedor, y el sondeo muestra que cuentan también las rechazadas con 422 o con 429. La consulta del contador, `GET /v1/api-quota`, no descuenta según la documentación. Al empezar la primera ejecución del 2026-09-09 devolvió una cuota de 100 con 100 restantes y la reposición anunciada para la medianoche UTC siguiente, los valores publicados. Tras esa lectura, la ejecución hizo diecisiete peticiones: diez recibieron 422, cinco 429, una 202 y otra 200. Al empezar la segunda ejecución, el contador había bajado diecisiete unidades. Si solo contaran las respuestas de éxito, habría bajado dos.

La documentación no dice que el contador expuesto va por detrás del consumo. La cabecera `x-apiquota-remaining` y ese endpoint se actualizan a saltos y con retraso. En la primera ejecución marcaron 100 en todas las respuestas, y en la segunda se quedaron en 83 durante catorce respuestas antes de saltar a 69. La reposición va igual de retrasada. En la ejecución del 2026-09-10, más de once horas después de la medianoche, el endpoint devolvió 66 restantes, el saldo del día anterior, y un `reset` que ya apuntaba al día siguiente. El saldo nuevo apareció entre la segunda y la tercera petición de esa ejecución, y no se midió qué dispara la actualización ni cuánto tarda (O-11, Sección 6.6).

La respuesta al envío tampoco dice si algo salió. Dos lotes de cinco objetos recibieron 202 y «The bulk email is being processed». Llevaban el mismo remitente no verificado que `/v1/email` rechaza en el acto, y la documentación avisa de que cada correo se valida después de la petición. Consultados después, los dos respondían `completed` con `validation_errors_count: 5`, y ningún correo salió. Cada objeto llevaba dos errores, el del remitente y el del cupo de destinatarios distintos. Es F-04, una

restricción que la respuesta a la petición no refleja: el fallo solo aparece al consultar el estado del lote, y la petición descuenta igual de la cuota diaria.

La cuenta Sandbox recibe en los dos endpoints de envío el caudal que la documentación publica para la reputación bajo revisión. Las respuestas de `/v1/email` traen `x-ratelimit-limit: 10`, con el contador de 9 a 0 también en las rechazadas. La undécima petición del minuto devolvió 429 con «Your account reached its rate limit of 10 requests/min». En el lote la cabecera marca 1, y el segundo lote del minuto devolvió 429. La cabecera ya marcaba 10 en la respuesta a la primera petición de envío y seguía en 10 cinco días después, así que ese valor no lo provocaron los rechazos del propio sondeo. En el lote la medición no reproduce lo publicado para Sandbox: la referencia del endpoint da «10 for Sandbox and Hobby plan accounts», y la cuenta devolvió 1. Caben dos lecturas: o la reputación de la cuenta estaba bajo revisión, cosa que ninguna observación registra, o la fila de Sandbox no describe esta cuenta.

El rechazo por caudal no trae la cabecera de espera que anuncia el centro de ayuda, «indicated by the retry-after response header». Las diez respuestas 429 no traen `retry-after` ni `x-ratelimit-*`. Las respuestas 422 de las mismas ejecuciones sí traen las cabeceras de caudal, así que la API las emite y solo faltan en el 429.

En `to` la cuenta admite una sola dirección, y la documentación publica 10 para la cuenta sin aprobar. La petición falla con dos o con cualquier valor mayor de los probados. En `cc` la medida coincide con lo publicado, una dirección, y en `bcc` solo se sabe que el tope queda por debajo de 10. Si el tope de una dirección lo impone el Sandbox, los topes publicados para la cuenta sin aprobar describen un estado que este sondeo no toca. Si lo impone la falta de aprobación, la API no aplicó a esta cuenta el tope que el proveedor publica. La discrepancia queda registrada sin resolver, porque separar las dos lecturas exige una cuenta aprobada.

Un lote de seis objetos devolvió 422 con «The number of email objects must not be greater than 5», el tope publicado para Sandbox. Se envió tras más de un minuto de espera, para que el caudal del lote no lo rechazara antes.

El sondeo topó con el cupo de dos destinatarios distintos de la cuenta Sandbox. Dos direcciones distintas entre `to` y `cc` pasaban el 2026-09-09 la comprobación de destinatarios. Con tres distintas entre `to` y `cc`, o con once entre `to` y `bcc`, saltaba «You have reached sandbox account unique recipients limit». Los cinco objetos de cada lote de ese día ya llevaban ese error, junto al del remitente. El 2026-09-14, con el panel en cero correos enviados, dos direcciones distintas devolvieron el mismo error. Una sola dirección repetida en `to` y en `cc` devolvió ese error más dos avisos de dirección duplicada. Con una sola dirección no se llega a dos, así que el error no responde a lo que lleva la petición: declara que la cuenta ya había alcanzado el cupo.

Ningún correo había salido nunca, y de ahí se infiere que el cupo lo gastaron intentos que no enviaron nada. Qué petición lo gastó no se determinó. La verificación del remitente no explica que dos direcciones distintas pasaran el 2026-09-09 y fallaran el 2026-09-14. El primer día, con el remitente sin verificar, la API ya devolvía el error del cupo con tres. La documentación no le da ventana, y cinco días después seguía alcanzado. El listado de destinatarios de la API devolvió cero, así que el cupo no se lee por ese canal. La unidad de la cuota mensual, que esa ejecución buscaba, quedó sin

medir, porque sus peticiones de envío devolvieron el error del cupo.

Qué se aprende del proveedor. El precio de MailerSend existe en varias versiones el mismo día, y tomar la que dibuja `/pricing` con JavaScript es una decisión del catálogo. En v0.3 cada documento lleva una fecha, que no dice qué oferta vio el lector ni si su cuenta está en un experimento de precios. Es F-09, y el propio ejemplo de la guía también lo muestra. Queda promovido sin número de propuesta, con una forma esbozada en la Tabla 6.6.

Las dos cuotas del plan caben en v0.3. El problema de esta API está en observar el consumo: el contador se pone al día a saltos y con retraso, y el 202 del lote no dice si salió algo (F-04). Una lectura de consumo en esta API exige releer el contador más tarde y consultar el estado final de cada lote.

El caudal de los endpoints de envío y los topes de destinatarios se leen bajo un estado de la suscripción o de la reputación, y el proveedor publica ese estado en dos clasificaciones separadas: la reputación y la aprobación. El esquema v0.3 tiene con qué declararlo: la guía documenta alias de estado dentro de un endpoint, con esta misma API como ejemplo [43], y la Datasheet declara tres. Lo que le falta lo recoge P-06 en el Capítulo 6 (Apartado 6.4.3): el alias no tiene campo para el disparador, y un estado de la suscripción se repite en cada endpoint. El proveedor no publica cómo se combinan las dos clasificaciones, y la Datasheet declara tres estados independientes, sin combinarlas. La cuenta medida no tiene plan, y el fichero repite su estado en los cuatro planes aunque el proveedor pide la aprobación para contratar el Free. Así el estado de la suscripción se repite por plan además de por endpoint.

El cupo de dos destinatarios distintos no cabe en ningún alias: su unidad son direcciones distintas, no correos ni peticiones, y la documentación no le da periodo (F-21). El Apartado 6.4.6 lleva esta carencia al motor. Con la Datasheet del catálogo, PRIME4API declara alcanzable un objetivo de tres destinatarios que el cupo de la cuenta Sandbox no admite.

Desde una cuenta Sandbox sin aprobar no se alcanzan los topes por campo de la cuenta aprobada ni su total de 70. Quedan también sin medir el caudal de 120 de `/v1/email`, el del lote en Starter y Professional y la ventana de la cuota mensual.

Medir un tope por petición en MailerSend puede agotar un cupo de la cuenta, y ninguna fuente leída lo avisa. El sondeo midió los topes sin enviar nada y, según la inferencia del bloque anterior, gastó el cupo de destinatarios distintos.

5.2.5. Brevo

Brevo [53, 59] deja cuatro hallazgos. El primero es de precio: ningún plan publica cuánto cuesta un correo por encima de su volumen. Lo único automático que se documenta es una subida de tramo al alcanzar el tope de contactos, y el esquema v0.3 no tiene campo para un cambio de plan. El segundo es de capacidad: Brevo publica que, superada la cuota diaria del plan gratuito, retiene en cola hasta mil correos transaccionales más. Los siguientes no se entregan, y v0.3 solo distingue el muro del exceso

facturado. El tercero es de fuentes: el caudal se publica ruta a ruta, en tres categorías, y no aparece en el contrato legible por máquina. La medición desmintió además esa documentación en un punto y resolvió una contradicción suya en otro. El cuarto acota la validez: todo lo medido sale de una cuenta del plan gratuito.

Esa cuenta tiene una cuota de 300 correos al día. En ese plan el envío empieza con una aprobación, «Once we approve your account for sending», y ninguna fuente publica qué topes rigen antes de ella. Sobre esa cuenta hubo un envío real deliberado, a dos destinatarios, y una petición aceptada en modo de pruebas. El resto fueron lecturas y peticiones rechazadas, y no se contrató ningún plan de pago.

Qué dice la documentación. La Datasheet modela solo la API de correo, sin el SMS ni el WhatsApp transaccionales. El precio se publica en cinco planes, y la Tabla 5.7 los recoge [53]. En Starter, Standard y Professional el precio lo fija un configurador por volumen mensual. El plan real es el plan elegido más el volumen contratado (F-08). El último tramo de cada configurador se rotula «More» y no publica ni volumen ni precio. La página sirve ocho divisas y elige la de quien la visita, EUR desde España. Sus cifras no son conversiones entre sí: Starter cuesta 9 USD y 7 EUR, y Professional 499 en las dos. Ninguna fuente declara la divisa de referencia, y la Datasheet lleva USD, la del bloque de datos estructurados de la página. Las otras siete divisas quedan fuera porque el esquema admite una sola por fichero (F-26, un precio que depende de la geografía frente a una sola divisa en el fichero).

Plan	Precio (USD/mes)	Volumen de entrada	Último tramo con precio	Sobrecoste
Free	0 ●	300/día ● (ventana ●)	sin tramos	no publicado
Starter	9 ●	5.000/mes ●	100.000 por 82	no publicado
Standard	18 ●	5.000/mes ●	500.000 por 429	no publicado
Professional	499 ●	150.000/mes ●	10 millones por 4.543	no publicado
Enterprise	a medida ●	a medida ●	a medida	no publicado

Tabla 5.7: Brevo: los cinco planes al primer volumen contratable [53]. El último tramo con precio es el que precede a «More». «No publicado» marca una ausencia buscada, no un sobrecoste cero. El caudal va por categorías y está en la Tabla 5.8. Los topes de destinatarios y de tamaño son los mismos en los cinco planes.

El primer hallazgo está en lo que la página de precios no publica. Ningún plan da un precio por correo por encima de su volumen, y dividir el precio de un tramo entre su volumen da un cociente que cambia de un tramo a otro. La ausencia se buscó en la página de precios, en su tabla comparativa y en los dos artículos del centro de ayuda sobre planes y sobre añadir correos. Tampoco se publica qué ocurre al agotar el volumen de un plan de pago. Lo que sí se documenta es un cambio de plan con otro disparador. Si la cuenta alcanza o rebasa el tope de contactos de su tramo, «we will automatically upgrade your plan to the next tier» [60]. Ese tope es un límite de recurso y no de consumo (F-11). Ampliar el volumen se cobra prorrateado en el acto, y los créditos prepago se venden por paquetes. En v0.3 el exceso se escribe en `overage_cost`, un precio por unidad que cuelga de la cuota rebasada. Ningún campo del esquema declara un cambio de plan (F-27, un exceso que cambia el plan en vez de pagarse por unidad).

La cuota del plan gratuito es de 300 correos al día, y «The daily limit resets every day, and unused emails do not roll over to the next day» [46]. Ninguna fuente dice a qué hora ni en qué huso ocurre el reinicio, así que el campo de la ventana queda vacío y en rojo. Es el único rojo del fichero. Las cuotas mensuales de los planes de pago se renuevan el día de facturación, «Your credits are renewed each month on your billing date». Esa ventana va en verde en los tres planes con configurador. El correo de marketing y el transaccional gastan de la misma cuota, «you can use your email credits to send both marketing and transactional emails»: dos productos de precio comparten una única cuota (F-10, en su forma inversa).

El segundo hallazgo es que el borde de esa cuota diaria no es un muro. Para el correo transaccional, «once you reach your daily limit, up to 1,000 additional emails are held in a retry queue. Emails beyond this queue are not delivered» [46]. El literal no dice si el cliente recibe algún error por esos correos. El esquema distingue dos bordes: el muro y el exceso facturado. Este es un tercero (F-24, una cuota que al superarse retiene un número fijo de envíos antes de dejar de entregar). La Datasheet declara 300 al día sin coste de exceso, que en v0.3 se escribe igual que un muro. Un `overage_cost` a cero sería una escritura legal y tampoco diría ni el tope de mil ni el corte posterior.

El caudal abre el tercer hallazgo. Se publica ruta a ruta en tres categorías con nombre, General, Advanced y Extended, con un registro de cambios fechado [59]. La Tabla 5.8 recoge las filas que tocan al correo y a los contactos. Sobre qué planes alcanza cada categoría, la misma página se contradice. La lista de entrada reserva el Advanced a «Professional and Enterprise accounts» y el Extended a «Enterprise accounts». El distintivo que encabeza la tabla de cada uno los da disponibles en los cinco planes. La Datasheet toma la lectura más estrecha, la de la lista, sin darla por buena: la atribución va en ámbar mientras la página se contradiga. Declara las categorías elegibles de cada plan como aliases, con el nombre que les da el proveedor. Ninguna fuente dice cuál rige por defecto ni si hay que pedirlo, y un alias declara que la categoría es elegible, no que esté en vigor.

Ruta	**General**	**Advanced**	**Extended**	**Sondeo, cuenta gratuita**
`POST /v3/smtp/email` y `GET /v3/smtp/blockedContacts`, en una sola fila	1.000/s ●	2.000/s ●	6.000/s ●	anunciado: 1.000, en tres respuestas de la ruta de envío
`GET /v3/smtp/emails`	2/s ●	3/s	18.000/h	alcanzado: 429 en la tercera petición de un segundo
Resto de `/v3/smtp/`	300/h	600/h	1.800/h	sin medir
`/v3/contacts/`	10/s	20/s	60/s	sin medir
«All other endpoints»	100/h ●	200/h	600/h	anunciado: 5.000 en `GET /v3/account`, sin ventana declarada

Tabla 5.8: Brevo: caudal publicado por categoría y lo que midió el sondeo [59]. Cada celda lleva una sola unidad. Cuando el proveedor publica las dos, va la de segundo, porque en todas esas filas la cifra por hora es la de segundo por 3.600. Hay que respetar las dos a la vez, «you must respect both simultaneously». «Alcanzado» es un límite que el sondeo llevó hasta el rechazo. «Anunciado» es una cifra leída en la cabecera sin llegar al rechazo, que se registra y no se adopta. «Sin medir» es una fila que ningún sondeo tocó. El semáforo solo aparece en las celdas que la Datasheet declara. Se omiten las filas de SMS, eventos, pedidos y productos.

La guía de envío por lotes publica otra cifra para el propio endpoint de envío. Bajo el encabezado «Bulk sending limits» dice «Execute the endpoint up to 6000 times per hour», y la página de caudal le da 3.600.000 por hora en la categoría General, un factor de 600. El lote no tiene endpoint propio, es el mismo `POST /v3/smtp/email`, y por eso las dos cifras se tratan como el mismo atributo. Por esta contradicción, que sigue abierta, y por la de las categorías, el caudal de la ruta de envío va en ámbar en las tres categorías.

Entre las rutas con fila propia y «All other endpoints» hay un nivel intermedio, publicado como prefijo con excepción. Todas las rutas bajo `/v3/smtp/` van a 300 por hora en la categoría General. Una nota al pie exime a dos, «*Excludes POST /v3/smtp/email and GET /v3/smtp/blockedContacts, which have dedicated limits». El esquema v0.3 admite endpoints enumerados y un solo comodín global, así que ese nivel solo se escribe enumerando las rutas del prefijo sin fila propia. La Datasheet no las enumera y se queda en la lectura conservadora: su comodín lleva los 100 por hora de «All other endpoints», un tercio de lo publicado para esas rutas. La nota tampoco nombra a `GET /v3/smtp/emails`, con fila propia como las dos exentas, y el sondeo resuelve esa contradicción.

Los topes de la petición de envío viven dentro de la descripción del campo `messageVersions`, «Maximum total recipients per API request is 2000. Maximum recipients per message version is 99». El tope por petición cae a 99 destinatarios cuando lleva adjunto, «If you add an attachment in your API script, you can only send your transactional email to a maximum of 99 recipients» [61]. El `workload` de v0.3 no admite condiciones, y su guía no documenta un `workload` propio por alias. La Datasheet lleva los 2.000 del caso general en ámbar, porque que valgan para un envío con `to` es inferencia. El caso con adjunto no tiene dónde escribirse.

El tamaño del correo lo publica solo el centro de ayuda, «The size limit for transactional emails is 20MB, including the attachments and email content» [61]. La documentación de desarrollador no da ninguno. La página de producto [62] da como «Max attachment size» «25 MB per email», un adjunto que no cabría en un correo de 20 MB. La Datasheet lleva los 20 MB en ámbar, porque la carga de v0.3 se mide por petición y el literal la acota por correo (F-06).

El contrato legible por máquina no recoge nada de este régimen. Sobre la especificación OpenAPI completa, archivada con 207 rutas, la búsqueda de «rate limit», «RPS», «RPH», «per hour» y «x-sib» no devuelve ninguna coincidencia. Tampoco declara tope en `to`, `cc` ni `bcc`. La ruta de envío declara las respuestas 201, 202 y 400, y la de SMS del mismo fichero sí declara 402 y 429. La guía de cabeceras afirma que las tres `x-sib-ratelimit-*` llegan en todas las respuestas, «not just 429 errors», y el contrato no las declara en ninguna. Es O-06 (Sección 6.6).

Qué añadió el panel. El panel confirma la cuota compartida y no cierra la ventana diaria. Muestra un solo contador para los dos productos, «300 / 300 Marketing & Transactional emails left». Muestra también su renovación con fecha y sin hora, «300 left out of 300 until 17/09/2026», el mismo día de la observación. La fecha hace suponer que el reinicio cae en un cambio de día, aunque ninguna pantalla lo escribe, y el huso sigue sin publicarse. La ventana sigue en rojo. En precio, ninguna página del panel

muestra un sobrecoste por correo. El único dato nuevo de la página de planes es el suelo de volumen de Enterprise, «From 1 million emails per month», que la web pública daba como volumen a medida.

El registro de envíos del panel no basta para contradecir la guía del modo de pruebas. La guía promete «No email logs are created in your Brevo account», y la petición aceptada con la cabecera `X-Sib-Sandbox: drop` dejó dos filas, `Sent` y `Error`. Su destinatario estaba en un dominio que no existe, así que la fila de error aparecería con el modo de pruebas o sin él. Quedan dos lecturas, que ese modo crea registros o que la cabecera no se aplicó, y el panel no distingue entre ellas.

El panel enseña una condición de acceso y un canal de gestión. La condición es la de F-28, una lista de IP autorizadas que decide si la llamada entra. Activada, restringe las llamadas, «restrict API calls to authorized IP addresses only», con un interruptor para las claves de API y otro para las SMTP, y la Datasheet no la recoge. El canal es el de la clave de API, que fuera de las subcuentas Corporate solo se genera desde la web (F-22).

Qué añadió el sondeo. El primer sondeo partió de un supuesto que resultó falso. Se diseñó para no enviar nada: dos peticiones, las dos con el modo de pruebas activado y un defecto que el proveedor debía rechazar. La primera, sin remitente, volvió con 400. La segunda llevaba remitente y destinatario en un dominio inexistente y volvió con 201 y un identificador de mensaje. El 201 es coherente con lo que la guía anuncia, «Sandbox mode only validates the API request format»: ese modo valida el formato de la petición y no si el destinatario puede recibir.

La segunda petición no dejó huella en el saldo: las lecturas previas al envío real, la última casi seis horas después, dieron 300. Esa falta de descuento no se atribuye a la cabecera, porque la petición llevaba también un destinatario inexistente. Un reinicio diario entre la petición y esas lecturas lo explicaría igual, pero las dos horas candidatas que deja el panel, medianoche UTC y medianoche de Madrid, caen después de la última.

La cuota se descuenta por destinatario, y el sondeo lo mide con un solo envío. Se leyó el saldo de la cuenta dos veces en reposo, con 300 en las dos. Después se envió una sola petición a dos direcciones distintas, y las dos lecturas de los minutos siguientes dieron 298. Contar por petición habría dejado 299. Según el registro del panel, una de las dos direcciones rebotó con «Soft bounce», y aun así el saldo descontó las dos: en esta medida, un rebote temporal gastó cuota igual que una entrega. Una lectura posterior, a las 22:49 UTC y hecha con otro fin, seguía en 298, y ninguna se hizo para ver si el crédito se devolvía más tarde.

El sondeo resolvió después una contradicción de la página de caudal: en `GET /v3/smtp/emails`, la ruta de actividad, rige su fila propia, y el prefijo de la familia no la sustituye. Se lanzaron tres peticiones sin pausa, con un límite anunciado de 2. Las dos primeras devolvieron 400 y dejaron el contador en 1 y en 0, y la tercera devolvió 429. La ráfaga se repitió media hora después con el mismo resultado. Un prefijo de 300 por hora que rigiera en lugar de la fila habría rechazado en la petición 301 de la hora, no en la tercera del segundo. Si el prefijo se suma además a la fila no se ha medido, igual que el tope por hora de la propia fila.

Los rechazos por validación también consumen caudal en esta ruta. Las dos primeras peticiones se rechazaron por un parámetro que faltaba, y aun así descontaron del contador, como prevé la primera regla de esta sección. En la ruta de envío, la petición rechazada por falta de remitente trajo el contador en 999 de 1.000. Apunta en la misma dirección, pero una sola respuesta con el modo de pruebas activado no basta para fijarlo.

Después del 429, la ruta volvió a admitir peticiones antes de cumplirse un segundo desde la primera de la serie. Lo muestra la segunda ráfaga: su primera muestra tras el 429, a 0,702 segundos de esa primera petición, ya no se rechazó por caudal. Eso descarta una ventana de un segundo que arranque con la primera petición. Con una sola muestra no se sabe cuándo se repuso ni se separa una ventana fija alineada con el reloj de una reposición gradual. El 429 llega sin cuerpo y sin `Retry-After`, así que solo las tres cabeceras de caudal dicen el límite.

La guía de cabeceras promete las tres en todas las respuestas, y la medición la desmiente. Las cabeceras de caudal llegaron o faltaron según la ruta, fuera cual fuera el código de respuesta. Llegaron en el 400 de la ruta de actividad y en el 200 de `GET /v3/account`. Faltaron en los 200 de `GET /v3/smtp/templates` y de `GET /v3/senders`.

En `GET /v3/account` el contador de caudal es de la cuenta y no de la clave. Seis lecturas que alternaban dos claves bajaron en una sola serie, de 4.999 a 4.994. Ni el envío ni la ruta de actividad se midieron con dos claves. Esa misma ruta anuncia un límite de 5.000 donde la página publica 100 por hora, y la cabecera no declara su ventana. Por la segunda regla de esta sección la cifra queda como valor observado pendiente, y el campo sigue en ámbar.

Con esta Datasheet, PRIME4API calcula para uno, cien mil y diez millones de correos el mismo coste en cada plan con precio: 0 USD en Free, 9 en Starter, 18 en Standard y 499 en Professional. Como control, el motor devuelve para el plan Hobby de MailerSend 7, 149,50 y 14.999,50 a esos volúmenes, así que calcula el exceso cuando el fichero lo trae. La curva plana de Brevo, igual que la del plan gratuito de MailerSend, solo dice que el fichero no trae exceso facturado, y no distingue un muro de un cambio de plan. La falta de campo para el cambio de plan sale de la guía del esquema, no de la curva. Enterprise no entra en la medida, porque el motor no evalúa un precio a medida. El motor devuelve para la cuota diaria del plan gratuito 300 correos sin coste de exceso y sin rastro de la cola de reintento.

Qué se aprende del proveedor. En los planes de pago, Brevo no publica un sobrecoste por correo ni qué ocurre al agotar el volumen de correos. Lo único automático que documenta para esos planes es la subida al tramo siguiente del mismo plan, con su precio, al alcanzar el tope de contactos (F-27). Es un disparador distinto del volumen de correos, y el esquema tampoco puede expresarlo: solo admite el exceso como precio por unidad, así que una herramienta que proyecte el coste desde el fichero no ve esa subida. El caso pide declarar el exceso como un mecanismo, con su disparador y su efecto, y no solo como un precio.

Para el correo transaccional, la cuota del plan gratuito tiene un tercer borde que el fichero escribe como un muro. Quien dimensione desde la Datasheet no ve ni la cola

ni el corte de entrega que la sigue. El caso pide que una cuota pueda declarar qué ocurre al superarla: cuántos envíos retiene y qué hace con los siguientes.

El caudal está documentado ruta a ruta para lectura humana y falta entero del contrato legible por máquina (O-06). Una Datasheet construida solo desde la especificación OpenAPI no tendría ni un caudal. La medición resolvió qué rige en la ruta de actividad y desmintió que las cabeceras lleguen en todas las respuestas. Las discrepancias registradas para Brevo son diecisiete (Sección 4.3), y siguen abiertas cinco. De las doce cerradas, cinco se resolvieron con una medición o en el panel, y una con la página de precios. Las otras seis cayeron al comprobar que uno de los lados no publicaba el dato o que los dos no hablaban de la misma magnitud.

Con una cuenta gratuita no se han observado ni las categorías Advanced y Extended ni el volumen de los planes de pago. La cola de reintento tampoco. Verla exige pasar de los 300 correos del día, y ver su tope exige desbordarla. Ese volumen de envíos no entra en los sondeos de este trabajo.

5.2.6. Amazon SES

Amazon SES [44, 63] deja cuatro hallazgos. El primero es de precio: el proveedor lo publica al céntimo, por destinatario y por tramos. El esquema v0.3 solo admite el precio unitario como exceso sobre una cuota incluida, y aquí no existe. El segundo es de capacidad: los límites observados se contextualizan por cuenta y región, y el esquema no tiene un campo donde declarar ese ámbito. El tercero es de fuentes: SES publica sus límites por cuatro canales, y el registro de discrepancias descrito en la Sección 4.3 anota veinte para esta API. El cuarto acota la validez de lo medido: la cuenta no tiene acceso de producción, así que cada cifra se interpreta dentro del entorno de pruebas. Cada bloque de los que siguen dice a cuál de los cuatro aporta.

La cuenta llegó con el plan `ESSENTIALS` puesto, sin que nadie lo eligiera. Un plan que asigna el proveedor puede contaminar lo que se mida después, así que conviene saber qué gobierna. Los apartados siguientes lo acotan al precio con evidencia documental: la página de límites no ata ninguno al plan, y la cuenta solo ha observado Essentials.

Qué dice la documentación. El precio tiene dos partes, y la página de precios las nombra por separado. Llama al producto «pay-as-you-go» y, dos párrafos más abajo, ofrece «three pricing plans to choose from» [44]. Las dos etiquetas describen partes distintas de la misma factura. El plan fija un importe mensual por cuenta y región, y no toca el precio del envío. Hay cuatro modos de contratación, y el cuarto es no tener plan, «If you do not have a plan, you are on à la carte pricing». Essentials pone «-» en la fila del importe, Pro marca 105 y Enterprise 500, y la Datasheet modela el cuarto con precio 0,00. La Tabla 5.9 recoge los cuatro, y su segunda columna es la que este bloque explica.

Plan	**Precio de la suscripción** (USD por cuenta, región y mes)	**Precio por correo** (USD/1.000, por tramos)
à la carte	0,00 •	0,10, sin campo
Essentials	0,00 •	0,16 / 0,14 / 0,11, sin campo
Pro	105,00 •	0,22 / 0,17 / 0,12, sin campo
Enterprise	500,00 •	0,23 / 0,18 / 0,13, sin campo

Tabla 5.9: Amazon SES: precio por plan [44]. La tabla tiene dos columnas y no seis porque ninguna fuente ata al plan nada más que el precio. La cuota, el caudal, el tope de destinatarios y el tamaño máximo valen lo mismo en los cuatro planes, se aplican al ámbito de cuenta y región y están en la Tabla 5.11. La segunda columna no está en el fichero: son los tramos marginales por volumen mensual, con cortes en 10 y en 100 millones de correos, y v0.3 solo admite un precio de exceso sobre una cuota incluida que aquí no existe.

Ninguno de los cuatro modos incluye un correo, así que el precio de enviar queda fuera de la suscripción y se publica aparte, por dos vías independientes. La página de precios lo da por millar: a la carta cobra «$0.10 / 1,000 emails». Por plan hay tres tramos, que bajan de 0,16 a 0,11 en Essentials, de 0,22 a 0,12 en Pro y de 0,23 a 0,13 en Enterprise. El tarifario legible por máquina lo da por destinatario, «Charged for each email sent to an external recipient», a 0,0001000000 USD. Los tramos son marginales por literal, «each rate applies only to the emails within that tier», y corren en dos series de tramos separadas, una para el saliente y otra para el entrante.

Ese precio no llega a la Datasheet, y la causa está en el esquema. En v0.3 el precio unitario vive en `overage_cost`, un atributo de una cuota que describe lo que cuesta excederse de un volumen incluido. SES no incluye ni un correo, así que no hay cuota que exceder ni sitio donde colgar el importe. La v0.2 tenía un campo para el precio unitario, `cost_per_request`, y v0.3 lo sustituyó por `workload`, que describe la carga de una petición y no su coste. Lo que falta en el fichero es la unidad que factura. El propio proveedor muestra que la estructura de v0.3 le sirve cuando la factura tiene esa forma. Su comparador publica volumen incluido con precio de exceso en inquilinos, «1,000 tenants included per account per region; $0.005/month/tenant for additional tenants», y también en validaciones. Es la misma estructura que el esquema modela, aplicada a otro producto. Para el correo saliente, que es lo que la Datasheet describe, esa forma no existe.

Los complementos sí tienen la forma que el esquema sabría escribir: un importe fijo al mes y, en algunos, una tarifa por unidad encima. La Tabla 5.10 los recoge tal como los publica el proveedor, con la IP dedicada estándar a 24,95 al mes por dirección y el correo entrante a 0,15 por cada mil procesados. La Datasheet modela el envío saliente y ningún complemento pertenece a esa superficie, así que la regla de alcance los deja fuera y su exclusión no señala una carencia del esquema (F-07, F-10).

Complemento	Precio fijo (USD)	Tarifa por unidad (USD)
IP dedicada estándar	24,95 al mes por dirección	—
IP dedicada administrada	15 al mes por cuenta	por correo, con tramos propios
Endpoint de entrada abierto	50 al mes	—
Entrega gestionada global	1.250 al mes	—
Direcciones propias (BYOIP)	24,95 por dirección, mínimo 256	—
Correo entrante	—	0,15 por cada 1.000 correos procesados
Archivado	0,19 al mes por GB guardado	2 por GB ingerido
Antivirus y filtros de reputación	—	entre 0,125 y 0,415 por cada 1.000

Tabla 5.10: Amazon SES: complementos y su precio [44]. Todos tienen la forma que v0.3 modela, un importe fijo y a veces una tarifa por unidad, y ninguno entra en la Datasheet porque ninguno pertenece al envío saliente, que es la única superficie que el fichero describe (F-07, F-10). Dos filas guardan una condición que la cifra no lleva escrita: el endpoint de entrada abierto sigue cobrando mientras exista, y las direcciones propias exigen un mínimo de 256. En el tarifario legible por máquina, un tramo de la IP administrada vale diez veces más en Estocolmo que en las otras veintiocho localizaciones.

Sobre la capa gratuita, cuatro fuentes del propio proveedor publican versiones incompatibles, y la página de precios no la menciona. Las preguntas frecuentes la ofrecen en presente, «Receive up to 3,000 message charges for free each month», y una entrada de blog del 21 de julio de 2026 la declara retirada para clientes nuevos. Otras dos fuentes la atan a EC2 con condiciones distintas entre sí. Ninguna de las fuentes enfrentadas es la página de precios, y en ese caso la regla de resolución de la Sección 4.4 no elige fuente: la registra. Por eso la Datasheet no modela ningún plan gratuito. La condición de EC2 es la excepción. El blog de 2012 contradice al tarifario legible por máquina, que es precio publicado, y la contradicción se cierra a favor del tarifario. No mueve ningún campo, porque v0.3 no tiene dónde poner esa franquicia.

Los límites de uso plantean el tercer hallazgo, y lo primero que se ve es por dónde se leen. Ningún canal los publica todos, así que hace falta cruzar los cuatro: la página monográfica de límites, la referencia de la operación de envío, el contrato legible por máquina de la versión 2 y el relé SMTP. La Tabla 5.11 hace ese cruce y separa tres fenómenos, definidos en su pie. Lo que más muestra es ausencia, canales que no publican el límite: de 48 celdas quedan vacías 32, y la columna del contrato legible por máquina lo está entera.

Límite	Página de límites	Referencia de envío	OpenAPI v2	Relé SMTP
Cuota diaria, entorno de pruebas	200 correos por 24 h	—	—	454 ≪Daily message quota exceeded≫
Cuota diaria, producción	sin cifra, ≪varies based on your specific use case≫	—	—	—
Caudal de envío	1 petición por segundo	—	—	—
Caudal del resto de operaciones	1 petición por segundo, ≪all actions≫ salvo tres nombres de la v1	—	—	—
Destinatarios por mensaje	50, sumando `to`, `cc` y `bcc`	—	—	—
Entradas por llamada al lote	—	—	—	—
Tamaño del mensaje	40 MB tras codificar en base64, y 10 MB por la v1 (F-15)	—	—	40 MB, 552 ≪Message is too long.≫
Adjuntos por mensaje	sin cifra de adjuntos, con 500 partes MIME como valor más cercano	—	—	—
Nombre del rechazo por caudal	`ThrottlingException`, 400	`TooManyRequestsException`, 429	—	454
Espera recomendada tras el rechazo	≪up to 10 minutes≫	—	—	5, 10 y 30 segundos hasta 20 minutos
Conexiones simultáneas	—	—	—	421, sin cifra detrás
Techo del producto	centinela, el valor −1 de `Max24HourSend`	—	—	—

Tabla 5.11: Amazon SES: cada límite frente al canal que lo declara [63]. Los cuatro canales son la página monográfica de límites, la referencia de la operación de envío, el contrato legible por máquina de la versión 2 y el relé SMTP. El guion marca ausencia, es decir, que ese canal no publica el límite, y no que el límite no le aplique. Dos celdas con contenido distinto para el mismo atributo son una contradicción, y un canal que describe otra superficie del servicio es una diferencia de canal.

Cuando dos canales sí publican el mismo límite, no lo nombran igual. El rechazo por caudal tiene tres nombres: `ThrottlingException` con un 400 en la página monográfica, `TooManyRequestsException` con un 429 en la referencia de la operación de envío y el 454 del relé. Las dos páginas se contradicen, porque dan valores incompatibles para el mismo atributo. El relé es diferencia de canal, porque describe otra superficie, y aun así obliga a elegir cuál rige. El canal que debería zanjarlo calla: el contrato legible por máquina de la versión 2 no declara ni un límite ni una cabecera de caudal. Queda un único camino documentado para conocer los valores vigentes, que es gastar una llamada en `GET /v2/email/account`. Sobre el tope de entradas por llamada al lote no hay un solo valor en ningún canal, aunque la operación lo tenga.

El tamaño máximo lleva una condición que su cifra no expresa, y el esquema solo tiene sitio para la cifra. Por encima de 10 MB la página añade una restricción de caudal en MB por segundo, ≪you may be throttled to as low as 40MB/s≫ (F-06). Un único campo escalar de v0.3 no representa un tamaño con un caudal condicionado por él, y el

fichero guarda los 40 MB sin la condición. El tope de destinatarios sí dice qué cuenta, «A recipient is any "To", "CC", or "BCC" address», y cabe en el escalar, como en SendGrid (Apartado 5.2.3).

La cuota de producción no tiene cifra publicada, y la del entorno de pruebas tiene cifra pero no definición. Fuera del entorno de pruebas el proveedor escribe que la cuota «varies based on your specific use case». Existe una sola cifra, 50.000 correos al día, y sale de un blog de 2014. No se adopta, porque un valor aislado y sin fecha vigente no basta cuando lo que cuenta sigue sin fijarse. La fórmula que calcula esa cuota calcula también el caudal, así que los dos quedan sin cifra a la vez. De la cuota que sí tiene cifra, los 200 del entorno de pruebas, el número no cierra dos atributos. La unidad sigue abierta: la cabecera de la tabla del proveedor cuenta destinatarios, «Quotas are based on the number of recipients, rather than on the number of messages», y su fila del entorno de pruebas cuenta correos. La ventana también: una página la describe deslizante y otra fija, y el campo queda vacío y en rojo.

La cuota la consume lo que la API acepta, y esa frontera decide qué se factura. Un mensaje dirigido a una dirección de la lista de supresión se acepta, y por eso cuenta contra el tope diario aunque después no se entregue. Una llamada que falla no llega a aceptarse, y por eso no se factura. El propio proveedor traza esa línea entre aceptar y enviar, «It's possible for Amazon SES to accept a message without sending it». Pasarse de la cuota cae del lado del rechazo: la llamada falla, el mensaje se descarta y no hay cola de reintento, «Amazon SES does not queue the emails and send them later». La consecuencia para el cliente es que un envío que llega con la cuota agotada se rechaza y no se recupera después.

En el envío por lotes ese rechazo viaja dentro de una respuesta con código de estado correcto: cada entrada afectada se marca con `ACCOUNT_THROTTLED` o con `ACCOUNT_DAILY_QUOTA_EXCEEDED`, y el código global no cambia. Un cliente que mire solo ese código da por enviado lo que el proveedor ha rechazado. El caso simétrico queda sin publicar: qué devuelve la llamada cuando el estrangulamiento afecta al lote entero no lo dice ninguna página, y la de valores se contradice con su propia prosa.

Tres límites más están publicados y no tienen dónde escribirse en v0.3. La espera recomendada tras el rechazo aparece en dos canales con cifras distintas, «up to 10 minutes» en uno y una serie de esperas de 5, 10 y 30 segundos hasta 20 minutos en otro. El esquema no tiene campo para ninguna de las dos. El techo del producto no tiene campo, y lo más cercano que publica SES es un centinela, el valor -1 de `Max24HourSend`, que ninguna otra API del catálogo usa. El caudal del resto de operaciones queda en ámbar por el alcance de un literal, «all actions», que exceptúa tres nombres de la versión 1. El fichero modela la versión 2, y el comodín no dice qué operaciones entran.

Quedan tres lagunas que la documentación no cierra. Que el envío por lotes cuelgue del mismo alias de caudal que el individual no lo escribe ninguna página, y el fichero lo asume como inferencia. Las otras dos son de frontera entre la API de envío y otros servicios de AWS. Entre las 116 acciones de la versión 2 no hay ninguna de credenciales, porque las del canal SMTP las crea IAM, que es otro servicio (F-22, variante). El correo entrante va más lejos: en la consulta del 2026-09-18 vive en una API aparte, `sesmailmanager`, y ninguna página reconcilia las tres superficies de entrada (F-10).

Qué añadió el panel. Lo primero que aporta el panel no es un límite sino el ámbito, el segundo hallazgo. La ficha de la cuenta muestra el plan y la región en la misma tarjeta, «Plan: Essentials» junto a «Región: Europa (Irlanda)». El proveedor presenta así el ámbito al que pertenece lo que se mide después, y es ese ámbito el que el esquema no sabe nombrar.

El panel mezcla en la misma fila la utilización por hora y la cuota del periodo de 24 horas, y esa mezcla es su única novedad sobre los límites. Los dos topes documentales no cambian: Service Quotas publica el valor aplicado y el predeterminado de AWS, y coinciden. La fila mezclada es un tercer testimonio sobre la ventana, después de las dos páginas que la describen deslizante y fija, y es la que deja el campo vacío y en rojo.

El contador de uso de la consola apunta en la misma dirección que el sondeo. Arranca en cero sobre 200. Después de cuatro mensajes dirigidos a cinco destinatarios marca cinco correos enviados, con 195 restantes. De contar mensajes marcaría cuatro, de modo que la consola llama correos a lo que son destinatarios. El nombre que elige una consola no basta para cerrar la disputa. La nota al pie del plan de precios sí la cierra, porque fija lo que se factura: «se cobra por destinatario».

El panel de Amazon SES no gestiona plantillas: su página solo lista las que ya existen y remite a la API para lo demás. Es el caso inverso de F-22, la credencial que solo se gestiona desde la web porque la API del producto no llega a tocarla. Aquí la operación exclusiva está en la API, y el fichero no tiene dónde declarar por qué canal se gestiona cada capacidad.

Qué añadió el sondeo. El sondeo no llegó a provocar el rechazo por cuota, y el motivo está documentado en su propia salida. Cuatro peticiones de envío devolvieron 403 con `AccessDeniedException`, y el recurso sobre el que IAM evalúa el permiso lleva escrita la dirección de destino. El rechazo por autorización impidió alcanzar el comportamiento que el sondeo pretendía medir, así que por esta vía no se observó el cuerpo del error asociado al caudal ni a la cuota. Ese camino, provocar el rechazo para leer el límite en el error, funciona en otras APIs del catálogo. En SES lo cierra el permiso de IAM, que se evalúa sobre la dirección de destino, y el sondeo no llega a la condición que quería medir.

Lo que sí cerró el sondeo es la unidad de la cuota, y el escenario se puede reconstruir. Se enviaron cuatro mensajes que sumaban cinco destinatarios: uno llevaba un destinatario, otro repetía la misma dirección en `to` y en `cc`, y los dos restantes llevaban uno cada uno. El contador partía de 0,0 y quedó en `SentLast24Hours` 5,0 sobre `Max24HourSend` 200,0. Contar por mensaje habría dado cuatro, y deduplicar direcciones repetidas también, así que el cinco solo sale de contar destinatarios sin fundir repeticiones. El campo que guarda ese resultado sigue llamándose `emails`, y no por descuido: la guía del esquema no lista `recipients` entre sus unidades, y PRIME4API empareja cuota con carga por igualdad de cadena. Corregir el nombre rompería esa igualdad y con ella el cálculo del motor.

La medición no reproduce el límite de una petición por segundo que la documentación publica, en ninguno de los escenarios probados. Doce peticiones secuenciales a `GET /v2/email/account` llegan a 5,7 por segundo sin un solo rechazo, y la docu-

mentación capa por literal las operaciones que no son de envío, con un alcance que el propio literal deja sin cerrar. Treinta peticiones concurrentes a 140,8 por segundo sí encuentran rechazo: nueve de treinta caen en `TooManyRequestsException` con un 429, sin cabecera de espera. Ese 429 nace en una ruta de lectura y fija el nombre del error, no su valor, porque saltar de 5,7 a 140,8 no dice dónde empieza a cortar. En el envío, dos peticiones sin pausa caen en el mismo segundo contra un tope publicado de una. Esa coincidencia no basta para adoptar un valor, porque dos medidas no delimitan un techo: haría falta una serie que lo acote (F-04).

El contador de la cuenta va con retraso, y el sondeo lo acota sin llegar a medirlo. Una lectura a cinco segundos del último envío seguía en cero y otra a minuto y medio ya contaba los cinco: el retraso cabe en cualquier punto de ese intervalo, y más fino no se midió.

PRIME4API hace visible el primer hallazgo. Sobre esta Datasheet devuelve el mismo coste a un correo, a cien mil y a diez millones: 0,00 a la carta, 0,00 en Essentials, 105 en Pro y 500 en Enterprise, con curva plana en los cuatro. Una curva plana admite dos explicaciones, un motor que calcula mal o un fichero sin precio que leer, y hace falta un control para separarlas. El plan Hobby de MailerSend, leído por el mismo motor, sí devuelve un coste que crece con el volumen, como ya se vio en Brevo (Apartado 5.2.5), y esa diferencia descarta el fallo del motor. Lo que el motor lee es un fichero que no tiene dónde guardar el precio por correo, y la curva plana es la consecuencia numérica de ese campo ausente. De todo el precio de SES, al fichero solo llega la suscripción, la parte fija y ajena al volumen que recoge la primera columna de la Tabla 5.9.

Qué se aprende del proveedor. El precio de SES está completo y aun así no llega al fichero, porque el hueco que debe recibirlo solo admite una forma. SES publica su precio al céntimo y por destinatario, en la página de precios y en un tarifario legible por máquina, y publica tres tramos marginales por plan en dos series independientes. El esquema ata el precio unitario al exceso sobre una cuota incluida y admite un solo valor, y esa pluralidad no tiene dónde escribirse. El fichero declara cuatro planes de 0, 0, 105 y 500 USD. Cualquier herramienta que calcule coste desde él aplica un precio por correo de cero, y el coste de cada plan no cambia con el volumen. Ese cero señala un campo concreto del esquema, el precio unitario sin cuota incluida, y no una objeción general al modelo.

El plan interviene en el precio, y la evidencia de que no fija los límites es documental. La página de límites los ata a la región y al entorno de pruebas, «SES quotas are for each AWS Region that you use in your AWS account», y ninguno al plan. El fichero declara la misma cuota en los cuatro planes, y el motor lo refleja: la capacidad acumulada en siete días sale en 1.401 correos en à la carte y en Essentials. Eso muestra que el fichero es coherente con la documentación, no que el proveedor lo sea, porque la cuenta solo observó Essentials. La región aparece en cambio como un ámbito relevante para interpretar la capacidad. Van por región la cuota y el estado de la suscripción, el plan, la verificación de cada identidad, las credenciales de SMTP y la lista de supresión. La cuenta observada pertenece a una de las 29 regiones con endpoint, y v0.3 no ofrece un campo donde declarar ese ámbito (F-31, un límite cuyo ámbito no es la cuenta entera). La Datasheet registra los valores medidos sin poder expresar el contexto regional

al que pertenecen.

SES cierra cuatro elementos no soportados del catálogo por la negativa, con literal del proveedor, y ese cierre no suma ocurrencias: deja cada elemento comprobado y ausente en esta API. Las cuatro son estas. No hay volumen contratado que comprar (F-02) ni sobrecoste en ningún plan (F-13). No hay cola de reintento, porque el proveedor escribe que descarta (F-24). Nada viene incluido de lo que excederse (F-27). La lista de IP autorizadas (F-28) no entra, porque ninguna fuente de SES la afirma ni la niega.

SES es la API con más discrepancias registradas del catálogo, y conviene decir qué se cuenta. El registro de discrepancias de la Sección 4.3 enfrenta fuentes que se contradicen entre sí o con lo observado, y admite también la entrada que tiene por lado una ausencia. El procedimiento es el de la Sección 4.4 y se aplicó igual a las ocho APIs. SES acumula veinte entradas frente a las diecinueve de MailerSend, las diecisiete de Brevo y las cuatro de Resend, y ninguna de las otras cuatro APIs pasa de tres. De las veinte de SES, diecinueve enfrentan una página de AWS con otra página de AWS, y la restante, ya cerrada, enfrenta una página de AWS con la ausencia de página. De las tres cerradas, una se cerró midiendo, otra en el panel y la tercera con la página de precios. Las diecisiete restantes siguen abiertas.

Medir la cuota y el caudal de producción exige sacar la cuenta del entorno de pruebas, un cambio de estado que esta lectura no ha hecho. Todo lo medido se interpreta dentro de ese entorno y no se extrapola fuera de él.

Queda una observación sobre este trabajo y no sobre el proveedor. SES entró en el catálogo el 2026-09-01 por ser pago por uso puro, y los planes existían desde el 21 de julio de 2026, 42 días antes. El criterio de selección describía al proveedor de antes de esa fecha. Es F-09, la fuente que caduca sin avisar, aplicada al propio TFM.

5.2.7. Mailgun

Mailgun [54, 64] deja cuatro hallazgos. El primero es de precio: un selector de volumen cambia el importe y el sobrecoste de los planes altos, y la Datasheet solo puede declarar el tramo de entrada de cada uno. El segundo es de ámbito: la única cifra de caudal que el proveedor publica para la API REST de Mailgun Send se acota a una IP de cliente, y v0.3 no tiene campo para decirlo. El tercero es de capacidad: sobre un mismo envío el proveedor anuncia varios topes a la vez, y el esquema admite uno por nivel. El cuarto enfrenta fuentes y respuesta: la API reproduce la cifra publicada de tres topes por petición, y en el tamaño rechaza con un tope de petición distinto del que se publica para el mensaje.

Solo se modela Mailgun Send, el producto transaccional. Optimize e Inspect aparecen en la misma cuenta como productos aparte, sin suscripción, y quedan fuera por la regla de alcance (F-10).

La cuenta observada es del plan Free, con el dominio de pruebas que asigna el proveedor, y solo entrega a direcciones autorizadas. A cualquier otro destinatario la API responde 403, con un texto que empieza por «Domain sandbox… is not allowed to send» y sigue con «Free accounts are for test purposes only». El texto nombra el

dominio y atribuye la restricción al tipo de cuenta. Desde una cuenta que solo tiene ese dominio no se puede separar si la restricción es del plan o del dominio. Los trece envíos reales del sondeo fueron a la única dirección autorizada, y no se ha contratado ningún plan de pago.

Qué dice la documentación. La página de precios publica cuatro planes, y el importe de los dos más altos lo fija un selector de volumen que pregunta cuántos correos se quieren enviar al mes [54]. Por la convención del catálogo (Sección 4.6), la Datasheet declara el tramo de entrada de cada tarjeta y guarda los demás tramos en el *provenance* (F-02). El plan de empresa, sin precio publicado, no se modela. La Tabla 5.12 recoge los planes con todos sus tramos. La misma página sirve además la lista en tres divisas, fijadas por el proveedor a un cambio casi constante, y la Datasheet declara la de dólares. El esquema admite una sola divisa. En Mailgun, F-26 aparece por el lado de la divisa: tres divisas, pero no tres precios por geografía.

Plan	**Cuota de correos** (al mes, por tramo)	**Precio** (USD/mes)	**Sobrecoste** (USD/1.000)
Free	100/día • y 3.000/mes • (ventana •)	0 •	—
Basic	10.000 •	15 •	1,80 •
Foundation	50.000 •	35 •	1,30 •
Foundation	100.000	75	1,10
Scale	100.000 •	90 •	1,10 •
Scale	250.000	215	0,90
Scale	500.000	400	0,75
Scale	750.000	550	0,60
Scale	1.000.000	700	0,50
Scale	2.500.000	1.250	0,40

Tabla 5.12: Mailgun: planes y tramos del selector de volumen [54]. Los tramos salen del bloque de datos que sirve las tarjetas. La primera fila de cada plan es su tramo de entrada, el único que declara la Datasheet, y es la única que lleva semáforo. El resto de los tramos solo consta en el *provenance*. El guion marca que el proveedor no publica el valor. Las dos cuotas del Free se declaran por separado, porque ninguna fuente dice si son un tope o dos. Ninguna fuente atribuye a un plan los caudales ni los topes por petición, que están en la Tabla 5.13.

El sobrecoste sigue los mismos tramos. La tarjeta lo publica como precio de arranque, «from $1.80 / 1,000 emails» en Basic, y en los planes altos el selector lo abarata tramo a tramo. La Datasheet guarda el del tramo de entrada, que es el más caro de cada plan. En Scale declara 1,10 USD por mil, mientras que el cliente de 2.500.000 correos paga 0,40. El Free no publica sobrecoste (F-13), y el proveedor no publica ningún tope al exceso. Lo único publicado que puede frenarlo es un tope mensual que fija el propio titular con una llamada a la API, y que se trata más abajo.

Parte de lo que separa a los planes es recurso y no consumo, como los dominios de envío, los usuarios o los días de retención de registros. Como no limitan cuánto se envía, esos topes quedan fuera de la capacidad de la Datasheet (F-11).

El segundo hallazgo parte de la única cifra de caudal que el proveedor publica para la API REST de Mailgun Send. La dan las preguntas frecuentes de su sitio comercial, «1,500 API requests per second per client IP», y la documentación de desarrollador no

la recoge. El literal acota el tope a una IP de cliente. Leído al pie de la letra, es un tope por salida de red, y una cuenta con varias salidas tendría varios. La Datasheet declara la cifra en ámbar por esa acotación, porque v0.3 no tiene campo para el ámbito de un caudal. Es una variante de F-31, como la región en SES (Apartado 5.2.6): aquí el ámbito del caudal es una IP de cliente. Para una cuenta nueva, las páginas leídas solo dicen que el proveedor «gradually increases sending rates» conforme crece la reputación, un caso de F-18: la capacidad depende del estado de la suscripción o de la reputación y no del plan.

La documentación de desarrollador describe el mecanismo sin cifrarlo, y para el umbral eso es ausencia [64]. Nombra las cabeceras `X-RateLimit-*` y el 429 con «Rate limits exceeded», y anuncia la reposición con una cabecera propia en lugar de `Retry-After`. Las dos fuentes difieren también en la condición. La documentación avisa de que las ventanas «may vary by API», y las preguntas frecuentes dan una cifra con ventana de un segundo «per API call», sin decir si vale para cada operación o para todas. El registro de discrepancias de la Sección 4.3 la anota como entrada abierta.

Sobre la concurrencia, las mismas preguntas frecuentes descartan el límite por escrito: «There are no concurrent connection or simultaneous request limits». A las ocho APIs del catálogo se les hizo la misma pregunta, y Mailgun es la única que publica esa respuesta para su API REST.

Las mismas preguntas frecuentes dan al relé SMTP su propia cifra, «1,500 SMTP requests per second per client IP». Entre esa cifra y la de la API hay diferencia de canal, y el esquema, que solo cuelga caudales de un endpoint HTTP, no le da sitio en el fichero (F-15).

El tercer hallazgo arranca con ese tope mensual que fija el titular. La documentación lo describe así: «The custom message limit imposes a hard limit on how many messages your account can send during a calendar month». Es una cuota, porque el literal dice que corta los envíos. Su valor sale de la configuración de cada cuenta, no del plan. El endpoint que lo fija entra en la Datasheet sin cuota ni caudal colgados, porque un tope a elección del titular no es capacidad comparable entre planes. Con ese tope puesto, la capacidad efectiva de la cuenta queda por debajo de la del plan, y v0.3 no tiene dónde decirlo. Es F-37, un tope de capacidad que fija el cliente y no el proveedor.

Qué añadió el panel. La cuota del Free tiene en el panel una segunda cifra, y con ella una indeterminación. La pantalla de facturación dice «3000 emails per month», con el contador «Emails sent this month» en «0 / 3000». La página pública del mismo día dice «100 emails/day included». Las dos cifras no se contradicen, porque la cuota diaria sostenida un mes de treinta días da la mensual, pero ninguna fuente dice si son un tope o dos. La tarjeta de Basic incluye «No daily email limit», que trata el tope diario como propio de otro plan, y eso apunta a dos. La Datasheet declara las dos por separado, con la mensual colgada del plan y la diaria del endpoint de envío, como si esta fuera un tope del endpoint. La diaria va en verde porque sale de la página de precios. La mensual solo la da el panel y queda en ámbar y registrada. Si son dos, el esquema no puede ponerlas a la vez en el plan. Ese caso es F-38: dos cuotas concurrentes de la misma unidad con un solo campo de cuota por nivel.

El panel tampoco fija la ventana de ninguna cuota, porque la única fecha que enseña habla del cobro y no del contador: «Autopay on the 1st of each month». De ahí toma el fichero, como inferencia, el ciclo de facturación de los tres planes de pago, en ámbar. En las dos cuotas del Free el campo de ventana queda vacío y en rojo.

Ninguna pantalla del panel enseña el caudal vigente ni un 429, y el caudal solo pudo leerse con el sondeo. Tampoco enseña la lista de direcciones autorizadas ni los tramos de volumen, que no son observables sin contratar un plan.

Qué añadió el sondeo. Desde una sola IP, el sondeo de lectura no llegó al caudal publicado, así que ni lo confirma ni lo refuta. Lanzó 1.570 lecturas de la lista de dominios en seis tandas, de 10 a 400 procesos concurrentes y con el doble de peticiones que procesos en cada tanda, sin obtener ningún 429 (Tabla 5.13). La tasa conseguida subió hasta 303,0 peticiones por segundo con 200 procesos y cayó a 186,0 con 400. Sin latencias ni carga local en el registro del guion, esa caída puede ser del cliente o de la API.

Límite	**Lo que publica el proveedor**	**Lo que respondió la API**	**En la Datasheet**
Caudal de la API REST	1.500 peticiones por segundo por IP de cliente	ningún 429 en 1.570 lecturas, hasta 303,0 por segundo	1.500/1 s ●
Caudal del envío	—	cabecera de 80.000 peticiones por ventana	80.000/10 s ●
Ventana del caudal del envío	—	unos 10 s, abierta por el uso, tipo sin determinar	10 s ●
Destinatarios por ventana	—	cabecera de 2.000.000, con la misma reposición	no cabe: segundo contador
Destinatarios por petición	1.000 por lote, en dos páginas	sin variables de lote: 403 con 1.000 y 400 con 1.001	1.000 ●
Tamaño del mensaje	25 MB, en tres páginas	403 de la cuenta con adjuntos de 26 y 30 MiB	no se declara
Tamaño de la petición	—	413 con un adjunto de 50 MiB, «request size exceeds 52.4MiB limit»	52 MiB ●
Parámetros de opciones	16 kB, sumando cuatro familias	403 con 16 kB y 400 con 17 kB, con un texto que nombra tres familias	no se declara
Entradas por consulta de eventos	300	200 con 300 y 400 con 301	solo en la descripción (F-05)
Conexiones simultáneas de la API REST	sin límite, con literal	no probado	no se declara

Tabla 5.13: Mailgun: cada límite de la API con lo que publica el proveedor, lo que respondió desde una cuenta Free y lo que declara la Datasheet [64]. El guion marca que ninguna fuente publica el límite. Donde la respuesta es el 403 de la cuenta, el límite no llegó a observarse. Los adjuntos se generaron en MiB, y el proveedor publica en MB.

El caudal del envío solo se leyó gastando un envío. En los casos probados, las cabeceras de caudal llegan únicamente en un POST aceptado, y ni una lectura de dominios ni un envío rechazado con 403 las traen.

La respuesta aceptada trae dos contadores con el mismo instante de reposición: `x-ratelimit-limit: 80000` para las peticiones y `x-recipient-limit: 2000000` para los destinatarios. En doce de los trece envíos, todos de un destinatario y repartidos en

tres tandas de cinco, uno y seis, los dos bajaron a la par. Como ninguno se acercó a su tope, no se observó cuál corta antes. Al dividir un límite entre otro, el cruce queda en 25 destinatarios por petición, y por encima de esa media el primero en llegar al tope sería el de destinatarios. El esquema admite un solo caudal por nodo, así que ese segundo contador no tiene dónde escribirse (F-36, dos contadores sobre la misma ventana).

La ventana de esos contadores dura unos diez segundos, que es lo que declara la Datasheet. Caben en ese valor las tres medidas de su duración, dos con el reloj del cliente y una con la cabecera de fecha del servidor, y ninguna afina por debajo del segundo. La abre el uso y no una rejilla del reloj. Las cinco peticiones anteriores a una pausa de 15 segundos comparten un valor de reposición, y las siete posteriores comparten otro. Los dos valores distan 17,975 segundos, casi lo mismo que las primeras peticiones de cada grupo, a 17,957, así que la reposición sigue a la primera petición. Con una rejilla fija de diez segundos, los dos valores distarían un múltiplo de diez.

Las cabeceras registradas no separan, en cambio, el tipo de ventana: encajan con una ventana fija que abre la primera petición y también con una deslizante que anuncie cuándo caduca la petición más antigua. Separarlas exige una petición entre la reposición de la primera y la de la última de una tanda, y el sondeo no la hizo. Por eso no se decide qué tipo de ventana de v0.3 le corresponde. El 429 no se alcanzó y la cifra va en ámbar. Forzarlo pediría 80.000 peticiones en diez segundos, una carga para la que el sondeo no se diseñó.

El cuarto hallazgo empieza por el tope de destinatarios, que se midió sin enviar. La prueba puso los destinatarios en el campo de destino, sin variables de lote. Con 100, 500 y 1.000 destinatarios la respuesta fue el 403 de la cuenta. Con 1.001 y con 2.000 fue un 400 con «Too many recipients, max is 1000». Cuando concurren los dos rechazos, la API devuelve el del recuento. El tope publicado es por lote, y la medida muestra que la misma cifra se aplica también al envío sin variables de lote, que ninguna página cifra.

Un 400 por 1.001 destinatarios y un 403 por destinatario no autorizado no movieron los contadores de aceptados ni de entregados, que seguían en trece. El contador de cuota del panel no se leyó entonces, así que el resultado no dice si esos intentos descuentan cuota.

Con el tamaño, el 403 de la cuenta tapó el tope publicado. Adjuntos de 26 y 30 MiB devolvieron el 403, así que no se pudo observar qué responde la API a un mensaje de más de 25 MB. Para verlo haría falta un envío de ese tamaño a una dirección autorizada. Una de las tres páginas ata el tope a la suerte del mensaje, que «may be permanently lost», sin hablar de rechazo. El rechazo por tamaño apareció con un adjunto de 50 MiB: un 413 con «request size exceeds 52.4MiB limit», que habla de la petición entera. Sobre la misma petición hay entonces dos topes de tamaño (F-34): uno publicado para el mensaje, que no se llegó a observar, y otro aplicado a la petición, que ninguna página publica.

Leído en mebibytes, el literal del 413 no se sostiene si el tope mide la petición, como dice su texto: la de 50 MiB de adjunto queda por debajo de 52,4 MiB y ya se rechaza. Sí es compatible con 52,4 millones de bytes. El fichero copia la unidad del

literal y declara 52 MiB en ámbar, por encima del corte observado: una herramienta aceptaría mensajes de 50 MiB que la API rechaza. Copiar el literal exigiría 52,4, y el esquema tipa el máximo como entero (F-35). En bytes la cifra sería entera, así que el elemento no soportado aparece por copiar la unidad del proveedor.

La consulta de eventos rechaza 301 entradas con el rango dentro del propio error, «Number out of range [1, 300]». La Datasheet documenta ese límite de resultados por petición en la descripción, porque v0.3 no lo serializa (F-05). La profundidad acumulada de esa paginación no se midió, porque con un día de retención la cuenta solo guardaba 42 eventos, y la segunda página ya llegó vacía.

Una contradicción entre fuentes sí se cerró midiendo: la matriz de funciones marca plantillas en los cuatro planes y la tarjeta de Foundation las cuenta entre lo que añade sobre Basic. Un alta de plantilla desde la cuenta Free devolvió 200 con «template has been stored», y da la razón a la matriz en Free. Para Basic se infiere lo mismo, porque ninguna fuente lo separa de Free en ese endpoint.

Qué se aprende del proveedor. Mailgun vende el correo en la forma que v0.3 espera, y de su precio el esquema solo pierde el selector de volumen. Al contrario que en SES, cada plan de pago incluye volumen y publica un sobrecoste. Una cuota solo admite un valor con su sobrecoste, y por eso los tramos del selector se quedan fuera. PRIME4API muestra lo que aporta ese sobrecoste. Preguntado por el tiempo que lleva enviar cien mil correos en Basic, con un destinatario por petición, responde 270 días sin presupuesto de sobrecoste. Con el presupuesto máximo responde un minuto y seis segundos. La misma pregunta no mueve a Brevo ni a SES, cuyas cuotas no declaran sobrecoste, y eso descarta que el cambio venga del motor. El tiempo de esa respuesta es el de cien mil peticiones a 1.500 por segundo, y ese es el límite que manda en ella.

Mailgun expone dos caudales, y ninguna fuente atribuye ninguno de los dos a la cuenta ni al plan. El publicado se acota a una IP de cliente, y el de 80.000 peticiones por ventana solo aparece en las cabeceras del envío, sin que ninguna página diga a qué se aplica. La Datasheet referencia los dos desde los cuatro planes, y el motor aplica el publicado como caudal del plan, así que el minuto del párrafo anterior vale para un cliente con una sola IP. El caudal del envío es un caso de F-01, un caudal que existe y cuyo valor no se publica: la API lo anuncia en cada respuesta aceptada.

Sobre un envío de Mailgun concurren varios topes: dos contadores de caudal en la misma petición, quizá dos cuotas en el Free y el tope mensual que puede fijar el titular. En cada nivel, v0.3 admite un solo tope. Los tres elementos no soportados que describen esa carencia, F-36, F-37 y F-38, se registraron en esta API, y la última depende de que las dos cuotas del Free sean dos topes. La Datasheet de Resend declara su par de cuotas del Free del mismo modo, la mensual en el plan y la diaria en el endpoint, y MailerSend publica un tope que fija el cliente. El recuento de recurrencia cuenta la Datasheet de Resend como caso de F-38, y el tope de MailerSend que fija el cliente como variante de F-37 que no suma.

En los topes por petición, la respuesta de la API reproduce la cifra publicada en destinatarios, opciones y eventos. En el tamaño no, porque lo publicado y lo aplicado son topes distintos, y la Datasheet solo aloja el aplicado. Como el 403 impidió observar

si el tope publicado del mensaje se aplica, Mailgun no suma un caso medido a F-04 (Tabla 6.1).

Quedan sin medir cuánta cuota consume cada destinatario y si los reintentos de la cola descuentan. Tampoco se midió cómo se reprecia al cambiar de tramo ni qué sobrecoste se aplica. Medirlos exige autorizar más destinatarios o contratar un plan, y las dos cosas cambian lo medido. Ninguna fuente de Mailgun escribe que cada destinatario consuma una unidad de la cuota, y el fichero lo declara como inferencia.

Con los mecanismos probados, leer el caudal del envío costó un envío real. Es la vía que el tercer peldaño de la escalera de validación (Sección 4.4) admite cuando no hay un mecanismo inocuo. La documentación describe además un modo de pruebas que procesa el mensaje sin entregarlo. Ese modo no se probó, y queda por ver si ofrece para este límite el mecanismo inocuo que el peldaño prefiere.

5.2.8. Mailchimp Transactional (Mandrill)

Mailchimp Transactional [47, 55], cuyo nombre histórico es Mandrill, deja cuatro hallazgos. El primero es de precio: vende un bloque de 25.000 correos a seis precios según cuántos bloques se compren, sin planes con nombre ni sobrecoste. ACM2YAML v0.3 admite tres formas de escribir esos tramos, y las tres los deforman. El segundo es de capacidad: la franquicia de la cuenta de prueba no caduca ni se renueva, y el periodo que v0.3 le exige queda vacío. Es uno de los dos campos vacíos que hacen fallar a PRIME4API. El tercero es de topes por petición: el proveedor declara que no hay tope de destinatarios y publica dos tamaños que miden objetos distintos. Con esta cuenta ninguno se puede medir provocando el rechazo, porque por encima de diez destinatarios o con un adjunto la API encola. El cuarto es de validez: todo se midió con una cuenta de prueba sin dominio de envío verificado.

Solo se modela el correo transaccional, no el SMS (F-10). El producto no se vende suelto: está disponible «como complemento de los planes Standard y Premium de Mailchimp, o del plan mensual heredado», y «El plan Essentials no es elegible». La Datasheet no modela esa suscripción base, porque ninguna fuente revisada ata a Standard o a Premium una cuota ni un endpoint del producto transaccional. La cuenta observada es la de prueba, «our free trial plan that includes 500 total sends». Su panel marca «No Sending Domains», y el proveedor «won't send any email from unverified domains». El panel y el sondeo miden lo que responde el proveedor, nunca una entrega.

Qué dice la documentación. Lo que se compra es un bloque, «Cada bloque es un crédito por 25,000 correos electrónicos», pagado en dólares [55]. Su precio baja en seis tramos, de 20,00 a 10,00 USD, según cuántos bloques se compren (Tabla 5.14). El eje de la tabla del proveedor es «TOTAL BLOCKS», y Mailchimp es la única de las ocho APIs cuyo producto de envío no se vende por planes con nombre. La Datasheet declara el tramo de entrada como precio, en ámbar, porque el literal fija el precio de una unidad prepagada (F-32, el precio unitario degresivo sin planes que lo sostengan).

Tramo	Bloques (comprados)	Correos al mes	Precio por bloque (USD/mes)	Por 1.000 correos (USD, calculado)
1	1 a 20	1 a 500k	20,00 •	0,80
2	21 a 40	500k a 1M	18,00	0,72
3	41 a 80	1M a 2M	16,00	0,64
4	81 a 120	2M a 3M	14,00	0,56
5	121 a 160	3M a 4M	12,00	0,48
6	161 o más	4M o más	10,00	0,40

Tabla 5.14: Mailchimp Transactional: los tramos de volumen [55]. El ámbar marca que la Datasheet declara el tramo de entrada como precio de plan cuando es precio de unidad. Los otros cinco tramos no tienen campo y quedan en el *provenance*. La última columna no la publica el proveedor. Es el precio del bloque entre sus 25.000 correos, y solo vale como precio por mil si el bloque se gasta entero, porque los correos que no se usan no pasan al mes siguiente. La tabla no tiene columnas de caudal ni de destinatarios porque ninguna fuente las hace depender del tramo. Para volúmenes mayores el proveedor promete «tarifas por bloque más bajas para volúmenes más altos» y remite a contacto comercial, sin cifra.

Agotar el bloque detiene el envío y no cuesta más dinero: «el sistema dejará de enviar correos electrónicos cuando se te acaben los bloques». En lugar de un precio de exceso, se compra otro bloque al precio del tramo, a mano o con la compra automática que el cliente configura «al llegar a tu límite predefinido».

La franquicia de la cuenta de prueba no tiene ventana. Son 500 correos, «up to 500 transactional emails to any email address on your verified domain», que «do not expire or renew». La prueba termina por consumo, «You send 500 emails», o por el estado de la suscripción, nunca en una fecha. ACM2YAML v0.3 exige `period` y `window_type` en toda cuota. La Datasheet deja los dos vacíos y en rojo, y es F-33, la cuota sin ventana. La ventana del bloque está en ámbar por otra razón. El proveedor solo publica que «Los correos electrónicos no utilizados de un bloque no se transfieren de un mes a otro», y el ciclo de facturación del fichero es inferencia.

El tope horario de la cuenta de prueba está en correos, «a fixed hourly quota of 25 emails». Según la documentación general, lo que excede el tope horario no se rechaza, «If you exceed your quota, unsent messages will be placed in the backlog», y la cola empieza a procesarse en la hora siguiente. Aplicar esa regla al tope fijo de la cuenta de prueba es inferencia, y con ella la Datasheet lo modela como caudal. Ninguna de las otras siete Datasheets expresa un caudal en correos. Para la cuenta de pago no hay cifra. Lo que puede enviar depende de «your pricing plan as well as your hourly quota, which is based on your sending reputation», y entre cuentas de volumen distinto «their hourly quotas will be very different». El caudal existe y su valor no se publica (F-01).

En peticiones, el único caudal publicado es el de la búsqueda de mensajes, un endpoint que no envía nada. Su página avisa de que «If you exceed the limit of 20 searches per minute, we'll now return a 429 Too Many Requests response code». El resto de la API REST no publica caudal ni concurrencia, y la tabla oficial de errores no incluye el 429. Es una ausencia, no una contradicción: esa tabla no opone otro código al de la búsqueda. Ninguna página revisada publica cabeceras de caudal, y el sondeo no devolvió ninguna.

La Tabla 5.15 reúne los límites publicados, con una columna por estado de la

suscripción. Los que solo constan en la descripción de su endpoint son casos de F-05. La cuota propia de recepción de la cuenta de prueba, «receive up to 100 inbound emails per day», queda fuera de la capacidad, porque recibir un correo no es una llamada a ninguno de los endpoints modelados. Ninguna fuente revisada dice que descuente de la cuota de envío, como describe F-12.

Límite	**Cuenta de prueba**	**Cuenta de pago**	**En la Datasheet**
Cuota de envío	500 en total, «do not expire or renew»	25.000 por bloque y mes (Tabla 5.14)	cuota de 500 con periodo y ventana vacíos, y cuota de 25.000 al mes con ventana inferida
Tope horario de envío	25 correos por hora	—	caudal de 25 correos por hora con el alias `demo` en los tres endpoints de envío, y el de pago anotado como no publicado
Caudal de la búsqueda	20 búsquedas por minuto, con 429	el mismo	caudal de 20 peticiones por minuto
Caudal del resto de la API REST	—	—	anotado como no publicado
Destinatarios por llamada	«no recipient limitation»	el mismo	tope vacío en los tres endpoints de envío
Tamaño de la llamada	«less than 10MB»	el mismo	10 MB
Tamaño del mensaje	«cannot exceed 25MB»	el mismo	solo en el *provenance*
Resultados de la lista de rechazos	1.000 por petición	el mismo	solo en la descripción del endpoint
Rango por defecto de la búsqueda	7 días si no se indica rango	el mismo	solo en la descripción del endpoint
Correo entrante	100 al día	—	fuera de la capacidad, en el *provenance*
Mensajes del modo de pruebas	10.000 al día, con dos ventanas en la misma frase	el mismo	no se modela
Topes de dominios, webhooks y usuarios	—	—	anotados como no publicados

Tabla 5.15: Mailchimp Transactional: los límites publicados, por estado de la suscripción [47]. Hay una columna por estado y no por plan, porque ninguna fuente publica un límite distinto para Standard y para Premium. El guion marca que el proveedor no publica el límite para esa cuenta, y no que el límite no exista. «El mismo» marca que la fuente no distingue entre cuentas.

El tope del modo de pruebas, 10.000 mensajes al día, no se modela porque la misma frase le da dos reinicios incompatibles: «This limit resets at midnight, or if they hit the limit, 24 hours after that limit is hit». Si el «or» significa lo que llegue antes, quien agota el límite a las tres de la tarde recupera el contador a medianoche, nueve horas después. Si la segunda cláusula sustituye a la primera, tarda veinticuatro horas. El registro de discrepancias lo trata como una contradicción del proveedor y no elige lectura (Sección 4.4).

El proveedor niega el tope de destinatarios y acota el tamaño en la misma frase: «Via the API, there's no recipient limitation, but the JSON provided per API call must be less than 10MB». El literal da un tamaño de llamada donde cabría un número de destinatarios (F-06), y la Datasheet deja presente y vacío, en rojo, el tope de destinatarios de los tres endpoints de envío. No lo omite, porque en v0.3 la omisión se leería igual que la de un límite que existe y no se declara, y los 10 MB ya van en su propia dimensión. Es el caso que distingue P-04 (Apartado 6.5.3). El envío múltiple va

en el array `to` de la misma llamada, sin endpoint de lote. El relé SMTP sí publica un entero, «up to 1,000 recipients at a time». Es una diferencia de canal: el relé es otra superficie del servicio, y la Datasheet modela la API.

La misma página publica un segundo tamaño, el del mensaje: «the total message size cannot exceed 25MB» (F-34). La Datasheet declara los 10 MB de la llamada y deja los 25 del mensaje en el *provenance*, porque `workload` solo aloja una cifra por unidad. Miden objetos distintos y no se contradicen. Según los dos literales, además, por la API no se alcanzan a la vez: un mensaje de 25 MB no cabe en base64 en una llamada de menos de 10 MB.

Lo recolectado no tiene más fecha de vigencia que la del acceso (F-09). El producto no tiene registro de cambios propio, y la página de presentación de Mandrill devolvió HTTP 404.

Qué añadió el panel. El panel no aclara la ventana de la franquicia. La muestra como contador, «Free trial sends» a «0 of 500», bajo el rótulo «Month plan usage», y no dice que se renueve cada mes. Sobre los contadores aparece «Last reset on September 16, 2026», el día del alta, sin decir qué se reinició ni cada cuánto, así que no se adopta como ancla de ninguna ventana. El tope horario figura al lado, «Email sends this hour» a «0 of 25». El mismo rótulo agrupa el complemento de SMS, «SMS credits» a «0 of 0», que se cuenta en créditos y queda fuera de la Datasheet.

En el panel, toda cuenta nueva figura como «Unknown» hasta que el proveedor ve el resultado de algún envío, y después la reputación se usa «in part, to determine your hourly quota». La cuenta propia sigue en «Unknown», pero su tope es fijo y la reputación todavía no interviene.

El panel no muestra ningún tope sobre dominios, webhooks ni usuarios, y ninguna fuente revisada publica uno (Tabla 5.15). Mailchimp es la única de las ocho APIs en ese caso. Tampoco la clave de API lleva tope: sus columnas «Expires» y «Remaining» tienen sitio para una caducidad y un saldo, y salen vacías.

Qué añadió el sondeo. El sondeo leyó por la API el tope horario que muestra el panel: `POST /users/info` devolvió `hourly_quota` a 25 y `reputation` a 0, con la cola vacía. La documentación y el panel dan el mismo tope que la API, y no hay contradicción que registrar.

El sondeo midió después hasta dónde la respuesta de la llamada informa de un rechazo. Con el mismo cuerpo y un remitente sin dominio verificado, variando solo los destinatarios, las llamadas con 2, 3, 5, 9 y 10 destinatarios devolvieron `rejected` con la razón `unsigned`. Las de 11, 20 y 100 devolvieron `queued` con la razón `multiple_recipients`. Para ese cuerpo el borde es exacto: entre diez y once destinatarios. Con un destinatario y un adjunto, la respuesta fue `queued` con la razón `attachments` en los seis tamaños probados, el menor de 5 MB. Por encima de ese borde, o con un adjunto, la respuesta no nombra ningún tope, y con esta cuenta no se puede medir uno provocando su rechazo.

Los dos barridos que cruzaron ese borde no midieron ningún tope. Las seis llamadas del barrido de tamaño devolvieron HTTP 200 sin mencionar el tamaño (F-04). La mayor llevaba unos 40 MB de cuerpo JSON, cuatro veces el «less than 10MB» publicado. El destino de esos seis mensajes en la cola no se registró, así que la medición no basta para desmentir el literal, y el campo de 10 MB sigue en verde.

El barrido de destinatarios probó 500, 1.000, 1.001 y 5.000 destinatarios en una sola llamada, y las cuatro respuestas fueron HTTP 200 con `queued`, sin rechazo. Mailgun, en cambio, rechaza 1.001 destinatarios en la propia llamada (Apartado 5.2.7). Con esta cuenta, Mailchimp no da ese veredicto en la llamada.

Unas dos horas después de cerrar el sondeo, los destinatarios que pasaron por la cola no habían descontado de la franquicia ni del contador mensual. El panel seguía con «Free trial sends» a «0 of 500» y «Sends This Month» a 0. En la única lectura intermedia los contadores daban 4.956 pendientes, 206 rechazos y ningún envío, y al cerrar quedaban en cola unos 7.600 destinatarios, todos en el dominio reservado example.invalid, que no entrega. F-21 recoge el caso opuesto, un cupo de destinatarios distintos que se gasta al probar. No se pudo observar si descuenta un mensaje encolado que llega a entregarse.

Sobre esta Datasheet, el motor devuelve HTTP 500 en catorce de las diecisiete llamadas del catálogo de preguntas de la Sección 5.4, con un error de conversión sobre un valor nulo. Solo contestan las tres que no calculan tiempo: las dos de cuota y la de caudal. Para separar las causas se rellenaron los campos en memoria en una de esas llamadas, sin tocar el fichero. Con el tope de destinatarios relleno, el error pasa a `division by zero`, que sale del periodo vacío de la franquicia. Con los dos rellenos, esa llamada devuelve 200.

Qué se aprende del proveedor. Mailchimp vende el correo transaccional por volumen y no por planes con nombre, y las tres formas que admite v0.3 para escribirlo fallan. Un plan por tramo crea seis planes que el proveedor no vende y que solo difieren en el precio. Declarar solo el tramo de entrada presenta como precio de plan lo que es precio de unidad. Y `overage_cost` declararía un sobrecoste que el proveedor niega por escrito.

La consecuencia para el coste se deduce del fichero, porque el motor no llega a calcularla. La Datasheet describe un solo bloque, 25.000 correos al mes por 20,00 USD. Quien multiplique ese precio por los bloques que necesita acierta hasta 20 bloques y sobrestima el componente transaccional desde el bloque 21, porque los tramos siguientes cuestan menos. Cuánto sobrestima depende de si el precio del tramo alcanzado es marginal o plano, y ninguna página revisada lo publica. El ámbar es un comentario y los tramos viven en el *provenance*, así que nada de lo que lee el motor avisa del error. El coste total queda además incompleto, porque la suscripción base no se modela.

En la cuenta de prueba la capacidad sigue al estado de la suscripción, como en Mailgun (F-18). La Datasheet declara el alias `demo` en los tres endpoints de envío, el caso de P-06 (Apartado 6.4.3). Con una franquicia sin ventana, lo único que pone tiempo es el tope horario: agotar los 500 correos lleva veinte horas (Sección 5.3). Ese tope está en correos: agrupar destinatarios no lo amplía, salvo que una petición con

varios cuente como un solo correo, y ninguna fuente revisada lo dice. Contado con un destinatario por petición, es el caudal de envío más bajo de las ocho Datasheets.

En Postmark el motor falló con un error de conversión ante un tope vacío. En Resend el mismo error aparece con un caudal negociado vacío, aunque allí el control no separó el campo vacío de las cadenas `custom` (Apartado 5.2.2). En Mailchimp el tope vacío da el mismo error, y el periodo vacío añade otro, una división por cero. La distinción entre «sin límite» y «límite no publicado» que propone el Apartado 6.5.3 tendría que alcanzar también al periodo de una cuota. Falta además una forma de declarar una cuota que no se repone.

En los topes por petición, el fichero guarda el tamaño de la llamada y deja vacío el de destinatarios. Como MB no dice qué objeto mide, quien consulte si cabe un correo con sus adjuntos encuentra los 10 MB de la llamada y no los 25 del mensaje.

Lo medido vale para una cuenta de prueba sin dominio de envío verificado. Quedan tres preguntas de la cuenta de pago, que no se ha contratado. Su tope horario no está publicado. Tampoco consta si el precio del tramo es marginal o plano. Y la unidad que descuenta el bloque, por destinatario o por llamada, no se pudo verificar: el artículo del proveedor sobre la facturación del producto devolvió HTTP 403.

Generar la cola fue un fallo del sondeo, y lo causó la forma de lanzar las tandas. El control del guion paró en la primera tanda de 100 destinatarios, pero las de tamaño y de destinatarios se lanzaron después a mano, sin ese control. La lección es de método: en Mandrill la vía de coste cero solo existe con diez destinatarios o menos y sin adjuntos, el mismo borde que midió el sondeo.

5.3. La relación entre límites de uso y *rates*

En el plan Pro de Resend el *rate* es el valor por defecto que el proveedor publica por equipo, 10 peticiones por segundo [56]. En un mes de 30 días ese *rate* admite 25.920.000 peticiones, y la cuota del plan es de 50.000 correos al mes [50]. El *rate* permitiría 518 veces la cuota antes de que se repusiera. En SendGrid Essentials 50K [51] la misma cuenta da 518.400 veces, con un *rate* de 10.000 peticiones por segundo que el proveedor publica una sola vez, en una FAQ, sin condición de plan. En la cuenta de prueba de Mailchimp Transactional esa cuenta no se puede hacer: la franquicia de 500 correos no tiene ventana declarada, y lo único que pone tiempo es un caudal de 25 correos a la hora [47]. Esta sección responde a RQ3 con ese cociente calculado sobre las ocho APIs del catálogo, y con lo que el cociente implica para el consumidor.

La relación se plantea con cuatro magnitudes de la Datasheet: el *rate* R, en peticiones por segundo (bloque `max_power`), la cuota Q, en correos, y su ventana W, en segundos (bloque `capacity`). El factor k es el número de destinatarios que admite una petición (`workload`). Es el «CRF» (*Consumption Request Factor*) de Gavira Sánchez [1], la regla de consumo que dice cuántas unidades de una métrica descuenta cada petición. En correo vale destinatarios por petición, y esa equivalencia está sostenida en cuatro de las ocho APIs: publicada en Resend y en SendGrid, y medida por sondeo en Brevo y en SES. En Postmark, MailerSend, Mailgun y Mailchimp no consta que

un destinatario descuente una unidad de cuota. Para un consumo uniforme a caudal medio c, en correos por segundo, cada límite fija un umbral:

$$c_{\text{cuota}} = \frac{Q}{W}, \qquad c_{rate} = R \cdot k, \qquad t_{\text{mín}} = \frac{Q}{R \cdot k}. \tag{5.1}$$

La restricción activa es la del umbral menor. El cociente $R \cdot W/Q$, calculado con $k = 1$, dice cuántas veces cabría la cuota en su ventana al *rate* máximo. En las tablas, k entra solo en $t_{\text{mín}}$, el tiempo en que una ráfaga a ese *rate* la agota. Las cuatro magnitudes son las que las operaciones de PRIME4API [1] toman de la Datasheet, y el final de esta sección liga cada cálculo a su operación Qn. Aquí se calculan a mano sobre las ocho APIs, y el final de la sección dice qué devuelve el motor sobre los mismos datos. El mes se toma de 30 días (2.592.000 segundos) y el día de 86.400 segundos. MailerSend añade una quinta magnitud, una cuota diaria de peticiones a nivel de plan, y su párrafo dice cómo entra en la cuenta.

Con ocho APIs esa notación se queda corta en tres sitios, y los tres los aportan las APIs nuevas. El primero es Mailgun, que devuelve en la cabecera del envío dos contadores sobre el mismo nodo y la misma ventana: uno de peticiones y otro de destinatarios. Hay entonces dos términos de *rate*, R en peticiones y R_d en destinatarios, y el que corta depende de k. El segundo es el caudal de Mailchimp, declarado en correos por hora y no en peticiones: ahí k no multiplica nada y la comparación se reduce a $c > R$. El tercero es la franquicia de 500 correos de la cuenta de prueba de Mailchimp, que no tiene ventana. Sin W no hay cociente $R \cdot W/Q$ que formar. El saldo se gasta una vez y termina la cuenta. Los tres se resuelven con un rótulo en la fila de la tabla. La columna no se rellena con un número que el proveedor no da.

Resend publica $R = 10$ peticiones por segundo por equipo [56]. El plan Free tiene dos cuotas, 3.000 correos al mes y 100 al día, con el mismo umbral de 0,001157 correos por segundo, porque 3.000 entre 30 son 100 [50]. El *rate* admite 8.640 veces cualquiera de las dos. Ni la ventana diaria ni la mensual están confirmadas por el proveedor, pero una ventana fija en vez de deslizante cambiaría el momento de reposición, no el cociente. En Pro, con 50.000 correos al mes, el cociente baja a 518,4 y $t_{\text{mín}}$ sube a 83,3 minutos con $k = 1$ o a 100 segundos con $k = 50$. El *rate* de Pro y Scale es el valor por defecto, no una cifra publicada por plan, y los cálculos heredan esa reserva. Enterprise no se calcula: la cuota es «Custom» y el *rate* no está publicado.

SendGrid publica $R = 10{,}000$ peticiones por segundo en `/v3/mail/send` y $k = 1{,}000$ destinatarios por petición. Solo el segundo está verificado por sondeo, en modo *sandbox*, y el primero no es observable. Con la cuota diaria de la prueba, 100 correos con tope duro, el cociente es 8.640.000. En Essentials 50K la ráfaga agota el mes en 5 segundos con $k = 1$ o en 50 peticiones con $k = 1{,}000$. Premier no se calcula porque la cuota es «Custom». Todos estos cálculos llevan una reserva: el 10.000 aparece una sola vez, `mail/send` no devuelve cabeceras de *rate* y el caudal real no es observable sin el cuarto peldaño de la escalera.

En Postmark el *rate* de la API REST no está publicado: la documentación solo describe el error 429 «Rate Limit Exceeded», sin umbral [45]. El valor con el que se calcula viene de la API. La respuesta 422 del sondeo de destinatarios del 2026-09-06 trajo las cabeceras `ratelimit-limit: 5000` y `x-ratelimit-limit-second: 5000`, que

la documentación no publica. El sondeo de caudal inocuo del 2026-09-07 envió 200 peticiones que se rechazaron en 4,57 segundos, y todas trajeron el mismo `ratelimit-limit` sin llegar al 429. Con esa medida la Datasheet adopta 5.000 peticiones por segundo en ámbar (●), y el cálculo es del mismo tipo que el de SendGrid. Del lado de la cuota, Free incluye 100 correos al mes con ciclo de cuenta confirmado en el panel, 0,0000386 correos por segundo (uno cada 7,2 horas) [49]. Basic parte de 10.000 al mes (0,00386 correos por segundo), cifra «Starting at» sin tabla pública ni ventana explicitada. El factor k vale 50, verificado por sondeo, y en `/email/bulk` el ritmo lo fija el proveedor («Postmark will optimize the sending speed for you»), tampoco con cifra. Con $R = 5{,}000$ y un mes de 30 días, $R \cdot W$ vale $1{,}296 \cdot 10^{10}$ peticiones: 129,6 millones de veces la cuota de Free y 1.296.000 veces la de Basic. En ráfaga Basic agota sus 10.000 correos en 2 segundos con $k = 1$ (el motor responde 1 segundo, por la convención de la primera ráfaga) y en 0,04 segundos con $k = 50$, y Free sus 100 en 0,02 segundos. La cuota manda por seis órdenes de magnitud en Basic, y el semáforo del *rate* recuerda que el valor está medido y no publicado (Tabla 5.16).

MailerSend publica el *rate* de `/v1/email` por minuto y no por segundo: 120 peticiones por minuto en los cuatro planes, es decir, $R = 2$ [58]. Es el *rate* publicado más bajo de las seis APIs cuya Datasheet lleva caudal y cuota con valor, cinco veces menor que el de Resend. Es también el valor por defecto de una cuenta en estado sano: bajo revisión baja a 10 por minuto, y una cuenta recién ascendida de plan tiene «a lower rate limit» sin cifra. La cuenta Sandbox sin aprobar del sondeo devolvió ese 10 por minuto con 429 alcanzado, así que el 120 no es observable desde ella. El factor k vale 70, suma de los 50 de `to`, 10 de `cc` y 10 de `bcc` que la documentación da por campo sin dar el total, y la Datasheet lo lleva en ámbar (●). Del lado de la cuota hay dos contadores por plan, y no uno. El endpoint tiene una cuota mensual en correos que «resets every 30 days»: Free 500 sin sobrecoste, Hobby 5.000 a 7 USD, y Starter y Professional 50.000 a 35 y 110 USD [52]. El plan tiene además una cuota diaria de peticiones, común a la API y a SMTP y repuesta a medianoche UTC: 100 en Free, 1.000 en Hobby, 100.000 en Starter y 500.000 en Professional. La ecuación 5.1 se queda corta con ese segundo contador, porque está en peticiones y no en correos. Su umbral es $Q_d \cdot k/W_d$, con $W_d = 86{,}400$ segundos, y frente al *rate* se compara sin pasar por k: el *rate* admite 172.800 peticiones al día, y solo Professional tiene una cuota diaria mayor. Con $R = 2$ y un mes de 30 días, $R \cdot W$ vale 5.184.000 peticiones. El cociente es 10.368 en Free, 1.036,8 en Hobby y 103,7 en Starter y en Professional. Baja a 5,18 en el último volumen de Starter (1.000.000 correos a 600 USD) y a 2,59 en el de Professional (2.000.000 a 1.600 USD), el margen más estrecho de las ocho APIs. En ráfaga Hobby agota sus 5.000 correos en 41,7 minutos con $k = 1$ (41 en el motor) y en 36 segundos con $k = 70$. Pero con $k = 1$ la cuota diaria se agota antes: 1.000 peticiones a 120 por minuto son 8 minutos y 20 segundos, el resto del día queda ocioso y los 5.000 tardan 4 días y 8 minutos de calendario. Como la mensual tiene sobrecoste y la diaria no, el techo duro de Hobby con $k = 1$ son 30.000 correos al mes, y no los 5.184.000 del *rate*. Con la cuenta bajo revisión $R \cdot W$ baja a 432.000, y es el caudal que el sondeo midió en la cuenta sin aprobar. Seis de los doce volúmenes publicados lo superan. En Starter 1M el cociente es 0,43 y el *rate* pasa a ser la restricción activa con $k = 1$, y con $k \geq 3$ vuelve a mandar la cuota. Es el primer caso del catálogo en que $R \cdot W < Q$ para un volumen publicado, y no aparece al subir de plan sino al cambiar la reputación de la cuenta, cuyos umbrales no se publican.

Brevo publica $R = 1{,}000$ peticiones por segundo en `POST /v3/smtp/email` [59], y es la primera API del catálogo que publica el caudal por categorías. La categoría General vale para todas las cuentas, la Advanced da 2.000 peticiones por segundo y la Extended 6.000. La atribución de esas categorías a los planes no está cerrada, porque la página se contradice. Una viñeta reserva Advanced a Professional y Enterprise, y el distintivo que encabeza su tabla enumera los cinco planes, Free incluido, así que afirma en vez de filtrar. Los tres valores viven en ámbar (•): los cálculos de Professional y de Enterprise son acotación y no valor. Brevo declara además el caudal en dos unidades y exige respetar las dos: peticiones por segundo y peticiones por hora. En las cinco filas que publican las dos, la de hora es exactamente la de segundo por 3.600, así que la de segundo manda y además prohíbe la ráfaga. El factor k vale 2.000 destinatarios por petición, y baja a 99 si la petición lleva un adjunto. En el Free la cuota son 300 correos al día, y la cuenta del día entero no llena una sola petición al máximo: una llamada lleva 6,7 veces esa cuota [53]. El cociente es 288.000 en Free, 518.400 en Starter y en Standard, y 17.280 en Professional. Baja a 259,2 en el techo publicado de Professional, diez millones de correos al mes, que es el volumen más alto de las ocho APIs. Aun en ese volumen el *rate* queda 259 veces por encima de la cuota. El sondeo cerró una cosa que la documentación no daba y acotó otra. El caudal es de la cuenta y no de la clave, porque seis lecturas alternando dos claves de la misma cuenta devuelven una serie única de contador. Y acotó la ventana de la ruta de actividad, `GET /v3/smtp/emails`, sin fijar su tipo. Tras el 429 la ruta volvió a admitir peticiones antes de cumplirse un segundo desde la primera de la serie, y eso descarta una ventana de un segundo que arranque con ella. Queda abierto cuál de los dos caudales de envío manda, con factor 600 entre ellos: la categoría General da 1.000 peticiones por segundo y la página del lote dice que el endpoint se ejecuta 100 veces por minuto.

En Amazon SES ninguna fuente ata la cuota ni el *rate* a un plan, y es la única de las ocho con planes en que pasa eso. Los planes son cuatro y PRIME4API lee los cuatro y devuelve la misma cuota, 200 correos al día [44]. Los dos límites son del estado de la suscripción, y la Datasheet los modela como un alias `sandbox` colgado de las dos rutas de envío. Los dos se aplican además al ámbito de cuenta y región: las cuotas van separadas por región, y ninguna fuente publica una cuota por identidad. En el entorno de pruebas R vale 1 petición por segundo y k vale 50 destinatarios por mensaje [63]. El cociente es 432, y la restricción activa sigue siendo la cuota: a una petición por segundo, los 200 correos se agotan en 200 segundos. La unidad de la cuota está medida aquí: cuatro mensajes aceptados con cinco destinatarios dejaron el contador `SentLast24Hours` en 5,0 sobre un máximo de 200. Ese cinco descarta el mensaje como unidad y descarta la deduplicación de direcciones, que habrían dado cuatro las dos. Para los cuatro planes de pago la relación no se puede formar. Ni la cuota ni el caudal de una cuenta con acceso de producción se publican, con la misma fórmula en las dos páginas del proveedor, «this number varies based on your specific use case». No es que el cociente salga alto o bajo: no hay numerador ni denominador. Una página describe la ventana de la cuota como deslizante y otra como fija, y el campo se deja vacío a propósito. La consola mezcla además en la misma fila la utilización por hora y la cuota de 24 horas.

Mailgun mide el caudal del envío y no lo publica. Un POST aceptado devuelve `x-ratelimit-limit: 80000`, y la ventana medida dura unos diez segundos. Dividir

una cifra entre la otra da $R = 8{,}000$ peticiones por segundo, el valor con el que se calcula. La misma respuesta trae un segundo contador sobre la misma ventana y con el mismo instante de reposición, 2.000.000 de destinatarios. El punto de cruce entre los dos es exacto: 2.000.000 entre 80.000 son 25 destinatarios por petición. Por debajo de 25 llegaría antes al tope el contador de peticiones, y por encima el de destinatarios, que es el perfil de quien usa la API de lotes. Con $k = 1{,}000$ el tope de destinatarios deja pasar 2.000 peticiones por ventana en vez de 80.000, y la columna de $t_{\text{mín}}$ solo acelera 25 veces y no 1.000. El esquema admite un solo caudal por nodo, y por eso ese segundo contador está declarado como hueco y no como límite. La ventana dura unos diez segundos, lo que declara el fichero. La abre el uso y no una rejilla del reloj. Las cabeceras no separan si es fija o deslizante, así que no se decide qué tipo de ventana de v0.3 le corresponde. La única cifra de caudal que el proveedor sí publica, 1.500 peticiones por segundo, está acotada por IP de cliente [64]. Leída al pie de la letra, es un tope por salida de red, y una cuenta con varias salidas tendría varios. El esquema no tiene campo para ese ámbito. Es además la más estricta de las dos que el fichero lleva, y es la que el motor aplica cuando responde. Del lado de la cuota, el cociente va de 6.912.000 en el Free a 8.294,4 en el último volumen de Scale, y la restricción activa es la cuota en los cuatro planes [54]. Con un matiz que ninguna de las otras siete tiene. En una campaña de lotes con más de 25 destinatarios por petición, de los dos contadores de la cabecera llegaría antes al tope el de destinatarios, que no está en el fichero. El 429 del envío no se alcanzó, y por eso el caudal se adopta en ámbar (●): forzarlo son 80.000 peticiones en diez segundos, que es una campaña de carga y no un sondeo.

En Mailchimp Transactional la relación no se puede formar, y ese es el resultado. No hay caudal en peticiones: el *rate* de la API REST no está publicado, y el único endpoint con caudal propio es la búsqueda de mensajes, 20 peticiones por minuto, que no envía nada. Lo que gobierna el envío es una cuota horaria en correos, 25 a la hora en la cuenta de prueba [47]. Está confirmada en tres superficies documentales y medida en la cuarta. Ahí k no multiplica, y no es que k sea pequeño: la API aceptó 5.000 destinatarios en una sola llamada sin cota alcanzada, y el proveedor escribe que no hay limitación de destinatarios. La franquicia de 500 correos de la prueba no caduca ni se renueva, así que no hay W con la que formar el cociente. Lo único que el par sí da es el tiempo: 500 entre 25 son 20 horas para agotarla a caudal máximo. De ahí sale el único caso de las ocho en que el caudal es la restricción activa con la cuenta en un estado sano. Sobre cualquier periodo sostenido el tope que muerde son 0,006944 correos por segundo, el caudal absoluto más bajo del catálogo. Queda 144 veces por debajo del entorno de pruebas de SES, que a una petición por segundo con $k = 1$ da 3.600 correos a la hora. Para la cuenta de pago la relación queda sin poder evaluarse. La cuota del bloque son 25.000 correos al mes, y el caudal que habría que comparar contra ese umbral no tiene cifra, porque lo fija la reputación [55]. Hay además un límite de observabilidad, y cae justo sobre el régimen que interesa medir. Hasta diez destinatarios la API evalúa en la propia llamada y responde con estado «rejected». Desde once responde «queued», y el borde es exacto. Por encima de ese umbral la API deja de decir si enviará. Unos 7.600 destinatarios quedaron en cola, todos en un dominio reservado que no entrega, y dos horas después no habían descontado de la franquicia ni del contador mensual. La vía de sondeo que este trabajo usa en otras APIs del catálogo, hacer que el proveedor rechace y leer el umbral en el cuerpo del error, no está disponible con la cuenta de

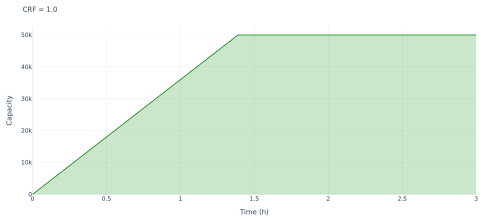
prueba por encima de diez destinatarios.

API · plan	$R \cdot W$	Q	$R \cdot W/Q$	$t_{\text{mín}}$ (k=1 / k **máx**)	**Datos en ámbar o rojo**
Postmark · Free	$1{,}296 \cdot 10^{10}$	100	129.600.000	0,02 s / 0,0004 s	*rate* medido, no publicado ●
Postmark · Basic	$1{,}296 \cdot 10^{10}$	10.000	1.296.000	2 s / 0,04 s	*rate* ●, cuota y ventana ●
Resend · Free (día)	864.000	100	8.640	10 s / 0,2 s	ventana diaria ●
Resend · Free (mes)	25.920.000	3.000	8.640	300 s / 6 s	ventana mensual ●
Resend · Pro	25.920.000	50.000	518,4	83,3 min / 100 s	*rate* por defecto
Resend · Scale	25.920.000	100.000	259,2	2,78 h / 200 s	*rate* por defecto
Resend · Scale 2,5M	25.920.000	2.500.000	10,37	69,4 h / 83,3 min	volumen no serializado
Resend · Enterprise	n.c.	Custom	n.c.	n.c.	*rate* ●
SendGrid · Trial	864.000.000	100	8.640.000	0,01 s / 1 petición	*rate* no observable
SendGrid · Essentials 50K	$2{,}592 \cdot 10^{10}$	50.000	518.400	5 s / 0,005 s	*rate* no observable
SendGrid · Pro 100K	$2{,}592 \cdot 10^{10}$	100.000	259.200	10 s / 0,01 s	*rate* no observable
SendGrid · Pro 2.5M	$2{,}592 \cdot 10^{10}$	2.500.000	10.368	250 s / 0,25 s	volumen no serializado
SendGrid · Premier	$2{,}592 \cdot 10^{10}$	Custom	n.c.	n.c.	cuota custom, ventana ●
MailerSend · Free	5.184.000	500	10.368	250 s / 4 s	ventana ●, k ●, *rate* sin observar. Diaria de 100 pet.: 4 d 50 s (k=1)
MailerSend · Hobby	5.184.000	5.000	1.036,8	41,7 min / 36 s	k ●. Diaria de 1.000 pet.: 4 d 8 min (k=1)
MailerSend · Starter	5.184.000	50.000	103,7	6,94 h / 6 min	k ●, lote 15/min ●
MailerSend · Starter 1M	5.184.000	1.000.000	5,18	5,8 d / 2 h	volumen no serializado. Diaria de 100.000 pet.: 10 d (k=1)
MailerSend · Professional 2M	5.184.000	2.000.000	2,59	11,6 d / 4 h	volumen no serializado
MailerSend · Starter 1M, rev.	432.000	1.000.000	0,43	> 30 d / 24 h	*rate* de 10/min medido en la cuenta sin aprobar, serializado como alias; volumen no serializado

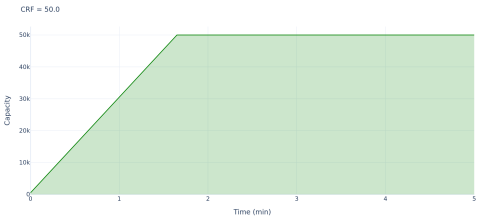
Tabla 5.16: Relación entre *rate* y cuota en las cuatro primeras APIs del catálogo. $R \cdot W$ en peticiones por ventana, Q en correos, $t_{\text{mín}}$ con $k = 1$ y con el k máximo de cada API (50 en Postmark y Resend, 1.000 en SendGrid y 70 en MailerSend, este en ámbar). Mes de 30 días. El *rate* de Postmark es la cabecera medida por sondeo, adoptada en ámbar. El de MailerSend son 120 peticiones por minuto, publicado y no observado: la cuenta Sandbox sin aprobar del sondeo devuelve 10, el caudal de la fila rev. Sus filas anotan además la cuota diaria de peticiones del plan, que la fórmula no recoge. Professional al primer volumen da las cifras de Starter. En todas las filas calculables con la cuenta en estado sano la restricción activa es la cuota. La última fila (rev.), con el *rate* de una cuenta bajo revisión, es la única en que manda el *rate* con $k = 1$, y la cuota no cabe en la ventana. Las cuatro APIs nuevas van en la Tabla 5.17. n.c.: no calculable.

API · plan	$R \cdot W$	Q	$R \cdot W/Q$	$t_{\text{mín}}$ (k=1 / k **máx**)	**Datos en ámbar o rojo**
Brevo · Free (día)	86.400.000	300	288.000	0,3 s / < 1 pet.	*rate* ●, ancla del reinicio diario ●, cola de 1.000 tras el muro
Brevo · Starter	$2{,}592 \cdot 10^9$	5.000	518.400	5 s / 3 pet.	*rate* ●, k ●; 99 destinatarios con adjunto
Brevo · Standard	$2{,}592 \cdot 10^9$	5.000	518.400	5 s / 3 pet.	ídem Starter; misma cuota y distinto precio
Brevo · Professional	$2{,}592 \cdot 10^9$	150.000	17.280	150 s / 75 pet.	categoría Advanced sin atribuir ●
Brevo · Professional 10M	$2{,}592 \cdot 10^9$	10.000.000	259,2	2 h 47 min / 5.000 pet.	volumen no serializado
Brevo · Enterprise	$1{,}5552 \cdot 10^{10}$	Custom	n.c.	n.c.	categoría Extended ●; suelo de 1M solo en el panel
SES · pruebas	86.400	200	432	200 s / 4 s	ventana de la cuota ●; unidad por destinatario, medida; no es cuota de plan
SES · producción	n.c.	no publicada	n.c.	n.c.	cuota ● y caudal ●; ninguna fuente los ata al plan
Mailgun · Free (día)	691.200.000	100	6.912.000	0,0125 s / < 1 pet.	cuota ●; una o dos cuotas, sin decidir ●; ancla de reposición ●
Mailgun · Free (mes)	$2{,}0736 \cdot 10^{10}$	3.000	6.912.000	0,375 s / < 1 pet.	ídem; la diaria por 30 da la mensual
Mailgun · Basic	$2{,}0736 \cdot 10^{10}$	10.000	2.073.600	1,25 s / 0,05 s	ventana del caudal fuera del esquema ●
Mailgun · Foundation	$2{,}0736 \cdot 10^{10}$	50.000	414.720	6,25 s / 0,25 s	ídem; caudal no atribuido por plan ●
Mailgun · Scale	$2{,}0736 \cdot 10^{10}$	100.000	207.360	12,5 s / 0,5 s	ídem
Mailgun · Scale 2,5M	$2{,}0736 \cdot 10^{10}$	2.500.000	8.294,4	312,5 s / 12,5 s	volumen no serializado; el contador de destinatarios corta desde $k = 25$
Mailchimp · demo	en correos/h	500	no formable	20 h (cualquier k)	cuota sin ventana ●; caudal de 25 correos/h; k sin tope ●
Mailchimp · bloque de pago	sin cifra	25.000	no formable	n.c.	caudal de la cuenta de pago sin publicar; ventana del bloque ●

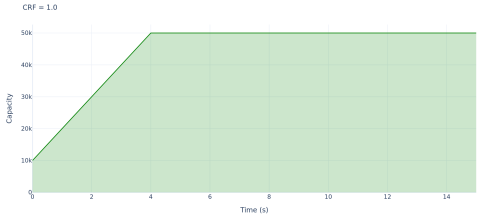
Tabla 5.17: Relación entre *rate* y cuota en las cuatro APIs nuevas, con las mismas columnas y las mismas convenciones que la Tabla 5.16. El k máximo vale 2.000 en Brevo (en ámbar, y 99 si la petición lleva adjunto), 50 en SES y 1.000 en Mailgun. Mailchimp no publica ninguno y el sondeo tampoco lo alcanza. Tres casos se rotulan en vez de numerarse. El caudal de Mailgun es el medido en la cabecera del envío, 80.000 peticiones sobre unos diez segundos, y su columna de $t_{\text{mín}}$ con k máximo la gobierna el segundo contador, el de 2.000.000 de destinatarios sobre la misma ventana: por eso acelera 25 veces y no 1.000. Las dos filas de SES no son de plan sino de estado de la suscripción, y valen para los cuatro planes, que devuelven la misma cuota y el mismo caudal. Las dos de Mailchimp no tienen $R \cdot W$: su caudal está declarado en correos por hora, sin cifra en la cuenta de pago, y la franquicia de la cuenta de prueba no tiene ventana. n.c.: no calculable.



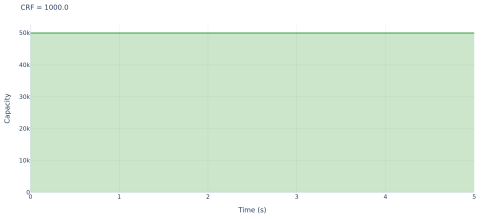
(a) Resend Pro, $k = 1$ (3 horas)



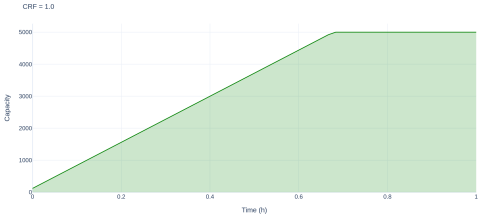
(b) Resend Pro, $k = 50$ (5 minutos)



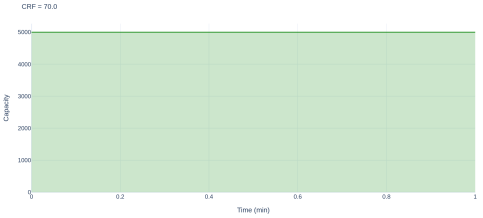
(c) SendGrid Essentials 50K, $k = 1$ (15 segundos)



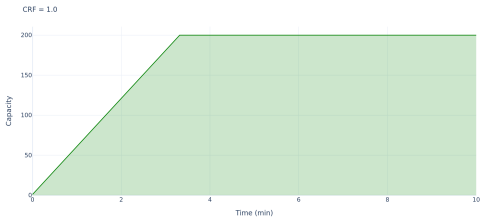
(d) SendGrid Essentials 50K, $k = 1{,}000$ (5 segundos)



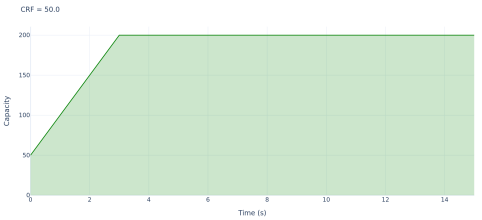
(e) MailerSend Hobby, $k = 1$ (1 hora)



(f) MailerSend Hobby, $k = 70$ (1 minuto)



(g) Amazon SES, los cuatro planes, $k = 1$ (10 minutos)



(h) Amazon SES, los cuatro planes, $k = 50$ (15 segundos)

Figura 5.1: Curva de capacidad $C(t)$ de Resend Pro, SendGrid Essentials 50K, MailerSend Hobby y Amazon SES con $k = 1$ y con el k máximo de cada API, dibujada por PRIME4API [7] en local (endpoint `/datasheet/capacity-curves/chart/inflection`, septiembre de 2026). Cada panel muestra el intervalo que hace visible el codo. A partir de él la curva sigue horizontal hasta que la cuota se repone: al final de la ventana de 30 días en Resend, SendGrid y MailerSend, y cada día en Amazon SES. El área sombreada es la capacidad acumulable. Los ocho paneles son la dimensión de correos, y por eso el panel (e) no incluye la cuota diaria de 1.000 peticiones de Hobby, que el motor evalúa aparte (hallazgo H1 y Figura 5.2). Los paneles (g) y (h) dibujan los cuatro planes de SES a la vez, porque los cuatro devuelven la misma curva: la Datasheet les asigna la misma cuota y el mismo caudal, los del entorno de pruebas. Su codo llega a los 199 segundos con $k = 1$ y a los 3 segundos con $k = 50$, y a partir de ahí la cuenta no envía nada más hasta que la cuota se repone.

Las Tablas 5.16 y 5.17 se leen también con la interpretación geométrica de la capacidad de Ruiz-Cortés y Ruiz-López [34], que el ACM adopta como semántica oficial (Sección 3.4). La curva de capacidad $C(t)$ de Gavira Sánchez [1] es el máximo de correos acumulables desde el inicio de la ventana hasta t sin violar ningún límite. Con un *rate* y una cuota tiene dos tramos. El primero es una recta de pendiente $R \cdot k$, y en él manda el *rate*. El segundo es horizontal a la altura de Q, y en él manda la cuota. El codo entre ambos está en $t_{\text{mín}}$, y la razón entre la ventana y ese codo es $R \cdot W \cdot k/Q$, el cociente de las tablas multiplicado por k. La Figura 5.1 muestra la curva de cuatro casos, Resend Pro, SendGrid Essentials 50K, MailerSend Hobby y Amazon SES, con $k = 1$ y con el k máximo de cada API. La genera el propio PRIME4API [7]: el motor lee las cuatro Datasheets, construye el BoundedRate de cada plan y su renderizador dibuja la curva, ejecutado en local como el servicio que despliega SPHERE (la misma imagen). En SES la figura separa el precio de la capacidad. Sus cuatro planes, de 0 a 500 USD al mes, devuelven la misma curva, así que el precio no mueve el codo ni la meseta. La curva es la del entorno de pruebas, el único estado de la suscripción que el proveedor cifra. En él la cuota de 200 correos se agota en 199 segundos enviando de uno en uno, y en 3 si cada petición lleva sus 50 destinatarios. La ejecución del motor como biblioteca devuelve las mismas trazas punto a punto. El motor cuenta la primera ráfaga en $t = 0$, y por eso da 4.999 segundos donde el cálculo a mano da 5.000. Cada panel muestra el intervalo que hace visible el codo, porque en una ventana de 30 días el tramo inclinado no se distinguiría de una vertical. La lectura es la de la tabla, pero a la vista. En Resend Pro el tramo de *rate* dura 83 minutos con $k = 1$ y 99 segundos con $k = 50$. En SendGrid Essentials dura 4 segundos con $k = 1$ y menos de uno con $k = 1{,}000$, porque la cuota cabe en 50 peticiones y la curva nace ya en el techo. En MailerSend Hobby dura 41 minutos con $k = 1$, con una rampa de 120 correos por minuto, y ninguno con $k = 70$, porque la primera ráfaga (8.400) ya supera los 5.000 de la cuota. El panel de Hobby con $k = 1$ dibuja el par *rate* y cuota mensual, porque los ocho paneles piden la dimensión de correos y la cuota diaria del plan está publicada en peticiones. Con esa cuota la curva es una escalera, y el mismo motor la dibuja si se le pide en peticiones: la Figura 5.2 la recoge junto al hallazgo H1. En los seis paneles de Resend, SendGrid y MailerSend el tramo horizontal ocupa casi toda la ventana de 30 días, y el área sombreada, la capacidad acumulable, crece solo en el tramo inclinado. En los dos de Amazon SES la cuota es diaria: la meseta ocupa casi todo el día, y en la ventana de 30 días la curva es una escalera que llega a 6.000 correos. Lo que el consumidor ve en la gráfica es que la cuota fija un techo, diario en Amazon SES y de ventana en las demás, y el *rate* solo decide cuánto tarda en tocarlo. La figura recoge tres planes de tres APIs y el entorno de pruebas de Amazon SES, que la Datasheet asigna por igual a sus cuatro planes. Las dos formas de las APIs nuevas que no caben en una curva de este tipo ya quedan dichas: los dos contadores de Mailgun sobre la misma ventana y la cuota sin ventana de Mailchimp.

De las dos tablas salen diez afirmaciones sobre RQ3. Cada una se cuenta sobre las ocho APIs, y no sobre planes ni sobre filas de tarifa. Una API cumple cuando su ficha tiene el dato que la afirmación necesita y ese dato va en la dirección que la afirmación enuncia. Hay tres estados más. «No cumple» es que la ficha tiene el dato y apunta en contra, así que es contraejemplo y se nombra. «No comprobable» es que la afirmación tiene sentido en esa API pero el dato no existe en ningún hecho, y entonces nunca cuenta a favor. «No aplica» es que la afirmación presupone una estructura que esa

API no tiene, como los planes en Mailchimp o la cuota incluida en SES, y ahí el hueco es del enunciado. Los denominadores se escriben a favor, en contra, no comprobable y no aplica sobre ocho. Un margen se calcula con $k = 1$, que solo lo multiplica, así que el margen a $k = 1$ es cota inferior. Las cuatro primeras afirmaciones venían de la versión de cuatro APIs, y tres de las cuatro se debilitan al pasar a ocho.

La primera: donde hay *rate* y cuota con valor y la cuenta está en estado sano, la cuota es la restricción activa, por entre 2,6 veces y ocho órdenes de magnitud. Seis a favor, ninguna en contra, una no comprobable y una que no aplica. Los cocientes van de 2,59 en MailerSend Professional 2M a 129.600.000 en el Free de Postmark, y ese rango no se mueve al pasar de cuatro APIs a ocho. Lo que se estrecha es el enunciado: «en ningún plan publicado» valía cuando las cuatro APIs del catálogo tenían las dos cifras en su Datasheet. Con ocho hay que decir en las seis APIs cuya Datasheet lleva *rate* y cuota con valor, publicado o medido. La no comprobable es SES: fuera del entorno de pruebas el proveedor no publica ninguno de los dos números. La que no aplica es Mailchimp, y ahí el signo se invierte. La franquicia son 500 correos sin ventana y el tope horario son 25 correos. Agotarla lleva veinte horas, y lo único que pone tiempo es el caudal. Cinco de los seis márgenes descansan en un *rate* que no está publicado como tal, y el único calculado con documentación, panel y cabecera de acuerdo es el de Resend. Para dimensionar volumen el consumidor puede seguir ignorando el *rate* en seis de las ocho APIs, siempre que la cuenta esté sana. En SES la pregunta no se puede contestar antes de abrir la cuenta, porque los dos números llegan con el acceso de producción.

La segunda: el *rate* de envío no es una variable del plan y la cuota sí. Las dos mitades se cuentan aparte, y las dos se debilitan. La primera va seis a favor, una en contra y una que no aplica. La segunda va seis a favor y dos que no aplican. Sostienen la primera mitad seis APIs: Postmark no publica ningún *rate*, Resend publica un valor por equipo, SendGrid un 10.000 sin condición de plan y MailerSend un 120 por minuto para los cuatro planes. Mailgun no atribuye caudal a ningún plan, y el valor del fichero, medido desde una cuenta Free, se referencia desde los cuatro. Mailchimp no tiene plan del que el caudal pueda depender, porque lo fija la reputación. El contraejemplo es Brevo, y la versión de cuatro APIs no lo tenía. Publica tres categorías de caudal para el mismo endpoint de envío, y una viñeta invita a subir a Enterprise para obtener más. Subir de plan multiplica por seis el caudal del envío individual, y es la primera vez que eso pasa en el catálogo. El contraejemplo no está cerrado, y eso también se cuenta: la misma página restringe Advanced a dos planes en una viñeta y lo ofrece a los cinco en el distintivo de su tabla. La primera mitad no aplica a SES, donde el único caudal publicado es el del entorno de pruebas, un estado de la suscripción, y se aplica por región. PRIME4API lo confirma leyendo el documento: encuentra cuatro planes y ninguno cambia la capacidad. La segunda mitad no aplica a dos APIs, y esto es nuevo. En SES el plan no compra volumen: no hay cuota incluida que comprar, y los cuatro planes devuelven la misma cuota de 200 correos. Lo que el plan compra son funciones y tramos de precio unitario. En Mailchimp no hay planes: lo que se compra es la propia unidad de volumen, el bloque de 25.000 correos, a precio degresivo de 20 a 10 USD [55]. La frase «se compra volumen, no velocidad» deja de valer como regla del dominio. En Brevo se compran las dos cosas, en SES no se compra ninguna y en Mailchimp lo único que hay que comprar es volumen, sin plan alrededor. PI-D04 pregunta si sube el *rate*

al subir de plan. Tenía cuatro noes y ahora tiene seis noes y un sí en ámbar.

La tercera: el *rate* publicado y el observado no coinciden, y la señal para el consumidor es la cabecera de respuesta. La primera mitad va cuatro a favor, tres en contra y una no comprobable, y se cae como afirmación universal. Sostienen el desajuste cuatro APIs, y la primera es Postmark, que no publica *rate* y devuelve 5.000 en la cabecera de cada respuesta. SendGrid publica 10.000 para un endpoint que no devuelve cabeceras. En los endpoints de lectura devuelve 600 y 6, valores que ninguna página documenta. MailerSend publica 120 por minuto y el sondeo midió 10, porque la cuenta no está aprobada. Mailgun documenta las tres cabeceras sin publicar ningún umbral, y lo medido es 80.000 peticiones por diez segundos. Los tres contraejemplos son Resend, que ya coincidía en documentación, panel y cabecera, y dos nuevos. La cabecera del envío de Brevo devolvió exactamente el 1.000 de su tabla General. Las tres superficies documentales de Mailchimp y su medición dan el mismo 25 por hora. Con cuatro APIs el desajuste parecía el estado natural del dominio, y con ocho el enunciado es más estrecho. Las tres que coinciden publican el valor con su ámbito escrito. Resend lo da por equipo, Brevo ruta a ruta y Mailchimp para la cuenta de prueba. En las cuatro que divergen no está publicado, está en una FAQ o es el de otro estado de la suscripción. La segunda mitad va cinco a favor, dos en contra y una con reserva, y cambia de sujeto. SES no emite ninguna cabecera de caudal: la búsqueda sobre su contrato máquina da cero apariciones, y sus nueve respuestas 429 tampoco traen `retry-after`. Mailchimp no tiene ninguna cabecera de caudal en ninguna página oficial ni en las respuestas del sondeo, y su tabla oficial de errores no incluye ni 429 ni 403. La reserva es Mailgun, que emite la cabecera y no deja leerla gratis: solo aparece en un POST aceptado, así que observar el caudal cuesta un envío. La señal fiable ya no es la cabecera, porque dos de ocho no la tienen y una la cobra, y lo que ocho APIs dejan ver va en la sexta afirmación. La documentación miente además sobre su propia señal en dos sitios. Brevo afirma que las cabeceras llegan en todas las respuestas, y llegaron en un 400 y no en dos 200. El ejemplo de 429 de MailerSend lleva 60 en la cabecera y 120 en el mensaje. El tercer peldaño de la escalera de validación (Sección 4.4) es el que ha producido los *rates* observados, y ha llegado a las ocho APIs por dos vías. En seis el número salió de una cabecera de respuesta: Postmark, Resend, SendGrid, MailerSend, Brevo y Mailgun. En las dos que no emiten ninguna cabecera de caudal salió de un endpoint de introspección: el de SES y el de Mailchimp. Y el número observado no siempre responde a la pregunta del plan. En SendGrid son los *rates* de los endpoints de lectura y no el del envío. En Postmark y en Mailgun son valores que ninguna página publica. Y en MailerSend, SES y Mailchimp es el de un estado de la suscripción que la Datasheet modela como alias.

La cuarta: el tope de destinatarios por petición, el CRF, es el factor que conecta el *rate*, en peticiones, con la cuota, en correos. La primera mitad va siete a favor y una en contra. La segunda va cinco a favor, dos en contra y una que no aplica. El tope vale 50 en Postmark, 50 en Resend, 1.000 en SendGrid, 70 en MailerSend, 2.000 en Brevo, 50 en SES y 1.000 por lote en Mailgun. El contraejemplo es Mailchimp, y el proveedor lo escribe en la misma frase: no hay limitación de destinatarios, pero el JSON de la llamada tiene que pesar menos de 10 MB. La cota de una llamada está en megabytes y no en destinatarios, así que no hay k que conecte las dos unidades. La segunda mitad decía que la documentación publica el tope peor de lo que la API

lo aplica, y era cuatro de cuatro. Ahora es cinco de ocho. Los dos contraejemplos son casos en que la documentación acierta. SES publica el tope como suma y con la definición explícita, «50 recipients per message», y aclara que un destinatario es cualquier dirección de `To`, `Cc` o `Bcc` [63]. Resuelve por escrito la ambigüedad que en Postmark, Resend y MailerSend hubo que medir, y ninguna página de SES publica el tope de otra forma. SendGrid cuenta a favor de la afirmación, aunque su referencia también la resuelve, porque su especificación OpenAPI da el tope por campo y sin total. El segundo contraejemplo es Mailgun, que publica 1.000 en dos páginas distintas, y la medición devuelve exactamente eso: el destinatario 1.001 da un 400 con el mensaje «Too many recipients, max is 1000». Lo que queda no es una ley del dominio sino un reparto. Los proveedores que publican el tope por campo lo aplican a la suma, y los que lo publican como suma lo cumplen. Solo cinco de los siete topes están verificados por sondeo: los de Postmark, Resend y SendGrid, el de Mailgun y el contorno del estado no aprobado de MailerSend. En Brevo no hay sondeo de destinatarios, y en SES el 50 no se midió y tampoco aparece en la referencia de la operación, solo en la página de cuotas. Para el consumidor: en SES una petición de 50 destinatarios consume 50 de los 200 diarios del entorno de pruebas, y en Brevo una de 2.000 agota casi siete veces la cuota diaria del plan Free.

Las seis afirmaciones que siguen solo son visibles con ocho APIs.

La quinta: el caudal por cuenta con cifra atribuible a un plan no se publica en siete de las ocho APIs. La única excepción está en ámbar y con la discrepancia abierta. La carencia es el caso general, y ninguna de las siete la declara igual en su fichero. SES la declara como hueco de bloque entero y Mailgun como valor único referenciado desde los cuatro planes. MailerSend la declara como valor por defecto, y Mailchimp como alias de estado repetido en tres endpoints: el tope es de la cuenta y no del endpoint. La excepción es Brevo, con las tres categorías ya descritas. La formulación absoluta, que ningún proveedor publica el caudal por cuenta con cifra atribuible a un plan, no se sostiene: hay uno. Lo que sí se sostiene es el recuento de siete a uno, y que el único que lo publica no ha conseguido publicarlo sin contradecirse en la misma página.

La sexta: la señal fiable del caudal vigente no es la cabecera sino el endpoint de introspección, y solo lo tienen cuatro de las ocho. Tres devuelven el límite en vigor. SES publica los dos topes de la cuenta en un `GET` que no gasta cuota, y su propia documentación señala ese endpoint como la vía de conocerlos, precisamente porque no devuelve ninguna cabecera. Mailchimp devuelve la cuota horaria junto a la reputación y el tamaño de la cola en `POST /users/info`, y MailerSend ofrece `GET /v1/api-quota`, que tampoco consume cuota. El cuarto devuelve menos. El `GET /v3/account` de Brevo da el saldo de correos y el periodo exacto del ciclo, treinta días anclados al alta, pero ninguno de sus campos es de límite ni de consumo, y la medición corrige a la documentación. Los cuatro que no lo tienen son Postmark, Resend, SendGrid y Mailgun. En Mailgun el efecto es el más duro: la cabecera existe y solo llega en un POST aceptado. La relación entre límite y caudal que el consumidor puede verificar antes de enviar existe en la mitad del catálogo. En la otra mitad conoce su capacidad real después de gastarla, o no la conoce. El tercer peldaño del método hereda esa asimetría. Donde hay endpoint de introspección el sondeo cuesta cero envíos, y donde no lo hay medir el caudal exige forzar el rechazo o gastar un envío.

La séptima: la ventana no se declara, y ninguna de las ocho APIs la tiene en verde en todas sus cuotas. Cada API se clasifica por el peor color que lleva alguna de sus cuotas. Hay cuatro en rojo (●), cuatro en ámbar (●) y ninguna en verde. SendGrid es la que más se acerca: publica la medianoche UTC y el periodo de facturación, y solo la ventana de Premier va en ámbar. Los cuatro rojos los aportan las APIs nuevas, y responden a tres modos de fallo. Las dos cuotas del Free de Mailgun y la diaria del Free de Brevo tienen valor y periodo, y ninguna fuente fecha su reposición. La de SES está en contradicción del propio proveedor, deslizante en una página y fija en otra, y el campo se deja vacío a propósito. Y la franquicia de Mailchimp no tiene ventana ninguna, porque el proveedor escribe que no caduca ni se renueva, mientras v0.3 exige periodo en toda cuota. La ventana del caudal no la declara ninguna de las ocho. Dos sondeos la acotaron sin fijar su tipo. En Brevo la ruta de actividad volvió a admitir peticiones antes de cumplirse un segundo desde la primera de la serie. En Mailgun dura unos diez segundos y la abre el uso, pero las cabeceras no separan una ventana fija de una deslizante. Ninguna de las dos medidas decide qué tipo de ventana de v0.3 le corresponde. Saber cuántos correos entran no basta para saber cuándo entran. En cuatro APIs el ancla del contador no existe en ninguna fuente, así que el consumidor no puede calcular cuándo su cuota vuelve a cero sin observarlo en su propia cuenta.

La octava: el estado de la suscripción y la reputación explican más de la capacidad observada que el plan. Seis de las ocho publican al menos un estado que condiciona la capacidad, tres de esas seis publican una cifra distinta por estado, y en tres la capacidad medida es la del estado y no la del plan. MailerSend baja de 120 a 10 peticiones por minuto con la cuenta bajo revisión, y el lote de 10 a 1. SES publica los dos topes del entorno de pruebas y no los de producción. Documenta además tres valores de `EnforcementStatus`: «Healthy», «Probation» y «Shutdown». Mailchimp fija 25 correos por hora en la cuenta de prueba y ata la de pago a la reputación, que su propio panel codifica en cuatro niveles. Los otros tres estados no llevan cifra. Resend amplía el caudal a los remitentes de confianza bajo petición, sin publicar cuánto. Mailgun describe el arranque de una cuenta nueva sin una sola cifra, y Postmark condiciona el cambio de plan a la aprobación de la cuenta. Con cuatro APIs esto era un rasgo de MailerSend, y con ocho el plan ya no basta para leer la capacidad observada. El esquema lo nota. SES modela el estado como alias con el vocabulario del proveedor, `sandbox`, colgado de las dos rutas de envío en los cuatro planes, y Mailchimp repite el alias `demo` en sus tres endpoints de envío por el mismo motivo. Lo que ninguna de las seis publica es el umbral que dispara el cambio de estado.

La novena: el sobrecoste deja de ser universal, porque cinco de las ocho lo tienen y tres no, y cada una de esas tres pone algo distinto en su lugar. En las cinco que lo tienen el precio del exceso baja con el volumen. Postmark va de 1,80 a 1,70 USD por mil, Resend de 0,90 a 0,46 y MailerSend de 1,50 a 0,50. SendGrid lo cobra por correo, de 0,0013 a 0,0005, y Mailgun va de 1,80 a 0,40 [54]. Brevo no publica ninguna cifra de exceso en ninguna de las cuatro páginas buscadas, y lo más cercano es la negación de que haya cargos adicionales [53]. Su lugar lo ocupa un ascenso automático de plan disparado por el número de contactos, que no es una cuota de correo. SES no tiene sobrecoste porque no tiene cuota incluida, y sus tramos de precio bajan al subir de volumen sin corte ni penalización [44]. Mailchimp convierte el agotamiento en un muro, y lo que ocupa el lugar del sobrecoste es comprar otro bloque del mismo

producto. El borde de la cuota tiene ya tres comportamientos documentados. SES descarta el mensaje y no lo reintenta, mientras SendGrid, MailerSend y Mailchimp levantan un muro. Brevo retiene hasta 1.000 correos y descarta en silencio lo que pasa de ahí. El caso que junta las dos formas es Mailchimp: su cuota es muro y su caudal es cola, en la misma cuenta, y el atraso se reprocesa en la hora siguiente. El coste de pasarse de la cuota solo es comparable entre cinco APIs, y en las otras tres la pregunta cambia de forma.

La décima: la unidad de la cuota se confirma midiendo o no se confirma. Cuatro de ocho tienen respaldo, y dos de esos cuatro salieron del tercer peldaño. Con respaldo documental hay dos, Resend y SendGrid, que escriben que cada destinatario consume una unidad. Con respaldo medido hay dos, y los dos son de la tanda de las cuatro APIs nuevas. En Brevo el saldo de la cuenta bajó de 300 a 298 tras una petición con dos destinatarios, y se mantuvo estable noventa segundos después. En SES el contador marcó 5,0 sobre 200 tras cuatro mensajes aceptados con cinco destinatarios, uno de ellos con la misma dirección en `to` y en `cc`. Ese cinco descarta a la vez el mensaje como unidad y la deduplicación de direcciones, que habrían dado cuatro las dos. Sin respaldo quedan cuatro: Postmark no confirma el cómputo por destinatario, MailerSend lo tiene modelado y no publicado, en Mailgun es inferencia declarada y en Mailchimp el artículo del proveedor que lo trataría devolvió un 403. Si la unidad de la cuota no está confirmada, el factor k de la cuarta afirmación no conecta nada. Multiplica peticiones por destinatarios para obtener correos, y en cuatro APIs no consta que un destinatario sea un correo a efectos de contador. La cadena entera de RQ3 descansa en un dato que la mitad del catálogo no publica y que solo se cierra midiendo.

Lo que deja de valer al pasar de cuatro APIs a ocho no es una cifra, sino la lectura de que el catálogo describe un dominio homogéneo. Con cuatro APIs las cuatro tenían planes, cuota mensual, sobrecoste decreciente y tope de destinatarios. Con ocho, una no tiene planes, una no tiene cuota incluida, tres no tienen sobrecoste y una no tiene tope de destinatarios. Las cuatro primeras no eran el dominio: eran su parte regular.

Estos cálculos corresponden a operaciones del catálogo Q1–Q10 de PRIME4API, de Gavira Sánchez [1], motor de análisis de la capacidad sobre las Datasheets. La restricción activa a consumo uniforme responde a Q5 (si una demanda con *rate* y cuota puede sostenerse durante un período), la misma operación que resuelve PI-D01 y PI-D02 en la Tabla 5.20. El tiempo mínimo de agotamiento $t_{\text{mín}}$ responde a Q2 (o a Q1 con el volumen igual a la cuota), y el factor $k^* = Q/(R \cdot W)$ a Q9 (qué CRF hace que ambos límites se agoten a la vez). Q9 se define sobre dos cuotas, y aquí se aplica al par *rate* y cuota tratando el *rate* como un límite con ventana de un segundo. Ese factor es el inverso del cociente de las dos tablas, así que se lee en ellas sin calcular nada aparte. Vale 0,0019 en Resend Pro, $1{,}9 \cdot 10^{-6}$ en SendGrid Essentials, 0,00096 en MailerSend Hobby y 0,0023 en el entorno de pruebas de SES. Las dos tablas tienen una sola fila con cociente menor que uno, MailerSend Starter 1M bajo revisión, y es la única con k^* mayor que uno: 2,31. Ningún $k \geq 1$ iguala los dos límites en ningún plan publicado con la cuenta sana. En Mailchimp Q9 no se puede plantear: el caudal ya está en correos y k no multiplica. El tope de sobrecoste de SendGrid (diez veces el plan, 500.000 correos en Essentials 50K) entra en Q10 (importe del sobrecoste), porque acota el volumen que la cuota admite pagando.

Las mismas operaciones se han ejecutado después contra PRIME4API como servicio REST en local, sobre las ocho Datasheets, con petición y respuesta archivadas por pregunta. Sobre Postmark, Resend y SendGrid el servicio se arrancó a mano, y sobre las otras cinco es el que despliega SPHERE en local. El motor reproduce el cálculo a mano en Resend y SendGrid, con tres convenciones propias y una regla de herencia. Cuenta la primera ráfaga en $t = 0$ (4.999 segundos donde a mano salen 5.000), abre la ventana siguiente en el instante final de un intervalo, y solo conoce el primer volumen contratable de cada plan (F-02). Q9 no tiene operación directa: el motor devuelve el tiempo de agotamiento de cada cuota para cada CRF, y k^* se deriva de esos tiempos. La regla de herencia suma los límites del plan a los del endpoint: un *rate* declarado en el plan y repetido en el endpoint cuenta dos veces. En SendGrid Essentials esa repetición aplanaba la curva con $k = 1$ en el primer segundo y duplicaba el techo a 30 días con $k = 1{,}000$: 100.000 correos donde el proveedor publica 50.000. El catálogo declara desde entonces cada límite una sola vez, en el plan si vale para todos los endpoints y en el endpoint si es propio, y el validador falla cuando se repite. La Figura 5.1 es la que devuelve el motor sobre la Datasheet, sin variante. En Postmark el motor mostró primero lo que hace un `max_power` vacío. Con el *rate* sin adoptar respondió 0 segundos para agotar la cuota de Basic y 270 días para 100.000 correos, nueve renovaciones de 10.000, como si el *rate* fuese infinito. No se equivocaba, hacía lo que el esquema le dejaba hacer, y es la confusión entre «no publicado» y «sin límite» que la propuesta P-04 del Capítulo 6 quiere impedir. Con el valor de 5.000 adoptado en ámbar las mismas preguntas se reejecutaron. El motor responde 1 segundo para agotar la cuota de Basic (2 a mano, por la convención de la primera ráfaga), 270 días y 1 segundo para los 100.000 correos, y devuelve el *rate* donde antes devolvía una lista vacía. La capacidad a un mes con presupuesto sin tope pasa de $1{,}1 \cdot 10^{12}$ correos, acotada solo por el presupuesto, a 12.960.005.000, que es $R \cdot W$ más la ráfaga inicial. El veredicto sobre el pico de 20.000 correos en una hora sigue siendo negativo en los cuatro planes, pero la etiqueta cambia de `rate_exceeded` a `quota_exceeded`. Con el *rate* vacío el motor atribuía al *rate* una violación de cuota.

Sobre MailerSend las diez preguntas P se ejecutaron el 2026-09-08 contra el mismo servicio, con 15 llamadas archivadas. Doce coinciden con el cálculo a mano y tres difieren por convenciones ya conocidas. Sin presupuesto la cuota con sobrecoste es dura, así que 100.000 correos en Hobby tardan 570 días y 41 minutos (19 renovaciones de 5.000), y en $t = 30$ días el motor suma la primera ráfaga de la ventana siguiente.

Sobre las cuatro APIs nuevas las diez preguntas se ejecutaron el 2026-09-21 contra ese mismo servicio. Son 40 instancias y 68 llamadas archivadas. Por HTTP responden 16 de 17 en Brevo, 17 de 17 en Mailgun y en SES, y 3 de 17 en Mailchimp. De las 30 instancias que el motor contesta ninguna difiere del cálculo a mano, y las diferencias que aparecen son las tres convenciones ya descritas. Las catorce llamadas que Mailchimp no contesta salen de dos huecos deliberados del fichero: el tope del `workload` de envío y el periodo de la franquicia de 500 correos. Aislados por sustitución en memoria, rellenar solo el tope cambia el error de `NoneType` a `division by zero`, y rellenar además el periodo devuelve 200. Es la factura medida de un hueco que el esquema obliga a dejar, y va al Capítulo 6. En SES el plan no cambia la respuesta del motor: los cuatro planes devuelven 200 correos al día, una petición por segundo, 1.401 correos en siete días y 6.001 al mes, todo colgado del alias `sandbox`. Y SES deja el primer caso del catálogo en

que la demanda viola en el instante cero y por el caudal. PI-B05 pide 20.000 correos en una hora y el motor responde NO en los cuatro planes con la etiqueta `rate_exceeded`, porque una petición por segundo no sostiene 5,56 correos por segundo.

La cuota dura sin presupuesto no es un detalle de MailerSend: gobierna la respuesta de PI-A06 en las ocho APIs, y el catálogo la reejecutó con los dos regímenes. La Tabla 5.18 los pone al lado. Sin presupuesto el motor cuenta renovaciones de la cuota y el tiempo se mide en meses. Con un presupuesto que no limita expande la cuota con su *overage cost* y el tiempo lo fija el caudal. Las dos cifras son del proveedor: el sobrecoste lo publica él, y ninguna de las dos es una hipótesis del trabajo. La segunda no exige elegir presupuesto, porque la respuesta satura en el coste del propio objetivo. En Hobby, donde 100.000 correos cuestan 149,50 USD, con 149,40 el motor responde 30 días y con 149,50 responde 13 horas y 53 minutos, igual que con mil millones. La tanda de las cuatro APIs nuevas añade los dos primeros casos negativos del régimen. En Mailgun Basic el presupuesto alto baja el tiempo de 100.000 correos de 270 días a 1 minuto y 6 segundos. En Brevo Starter y en SES Pro la misma llamada no mueve un segundo, porque sus cuotas no declaran sobrecoste. El régimen no es entonces una convención del motor: es una propiedad del *pricing*. La distancia entre las dos columnas llega a seis órdenes de magnitud, de 270 días a 19 segundos en Postmark Basic con $k = 1$. Y el esquema no dice cuál de los dos regímenes describe el plan: la cuota lleva su sobrecoste, pero nada declara si el proveedor lo aplica solo, lo exige aprobar o lo tapa un tope. Es el elemento no soportado F-13 visto desde el motor, y va al Capítulo 6.

API y plan	k	**Cuota dura**	**Presupuesto que no limita**
Postmark Basic	1	270 d 1 s	19 s
	50	270 d	0 s
Resend Pro	1	30 d 1 h 23 min 19 s	2 h 46 min 39 s
	50	30 d 1 min 39 s	3 min 19 s
SendGrid Essentials 50K	1	30 d 4 s	9 s
	1.000	30 d	0 s
MailerSend Hobby	1	570 d 41 min	13 h 53 min
	70	570 d	11 min
Brevo Starter	1	570 d 4 s	570 d 4 s
	2.000	570 d	570 d
SES Pro (`sandbox`)	1	499 d 3 min 19 s	499 d 3 min 19 s
	50	499 d 3 s	499 d 3 s
Mailgun Basic	1	270 d 6 s	1 min 6 s
	1.000	270 d	0 s

Tabla 5.18: Tiempo mínimo para enviar 100.000 correos en un plan de pago de cada API, respondido por PRIME4API [7] en los dos regímenes: sin `max_budget`, donde la cuota con sobrecoste se trata como dura y el motor cuenta renovaciones, y con un presupuesto que no limita, donde la expande y manda el caudal. Las respuestas del régimen con presupuesto saturan en el coste del objetivo, así que no dependen del valor que se pase por encima de él. Brevo y SES son los dos casos negativos: sus cuotas no declaran sobrecoste, y por eso las dos columnas coinciden. Las filas de SES son las del alias `sandbox`, el único estado que el proveedor cifra. Mailchimp no aparece porque sus cuatro llamadas devuelven 500. Las treinta y dos llamadas están archivadas en el repositorio de datos.

En MailerSend los veredictos de precio son los del cálculo a mano: Starter a 35 USD para 40.000 correos, y Hobby a 17,5 para 12.000 y a 37 para 25.000. El de demanda también: NO en los cuatro planes al pico de 20.000 correos en una hora, con la etiqueta `rate_exceeded` y la primera violación a los 21,6 segundos. Dos hallazgos sobre el motor son nuevos, y los dos se han reproducido con el código del motor sobre el mismo YAML. El primero (H1): la cuota de plan en peticiones se hereda, y `/quotas` la lista junto a la mensual, pero no entra en la dimensión de correos con la que responden las diez preguntas. El motor multiplica por k los *rates* en peticiones y no hace lo mismo con las cuotas en peticiones. Por eso sus cifras de Hobby con $k = 1$ son las del par *rate* y cuota mensual: 41 minutos para agotar los 5.000, 5.184.120 correos al mes con presupuesto sin tope y 33.786,67 con 50 USD. Con la cuota diaria salen 4 días y 8 minutos, 30.000 y 30.000. La curva de la Figura 5.1 tiene la misma ausencia. El motor sí calcula esa cuota, pero en otra dimensión: pedida la curva en peticiones devuelve la escalera de la Figura 5.2, con la cuota diaria del plan y la mensual del endpoint convertida por k. De las tres conversiones que el factor k permite, el motor hace dos: el caudal de peticiones a correos y la cuota mensual de correos a peticiones. La que omite es la cuota diaria en peticiones llevada a correos. Su renderizador oculta además por defecto la dimensión de peticiones derivada de un *workload*, que hay que pedir con `show_derived_requests`. En SendGrid no se notó porque su cuota de plan ya está en correos, la dimensión en la que responden las preguntas. MailerSend es la primera API del catálogo cuya cuota de plan está en otra unidad que la del endpoint. Ningún veredicto de plan más barato cambia con ello, y el hueco se trata en el Apartado 6.5.1.

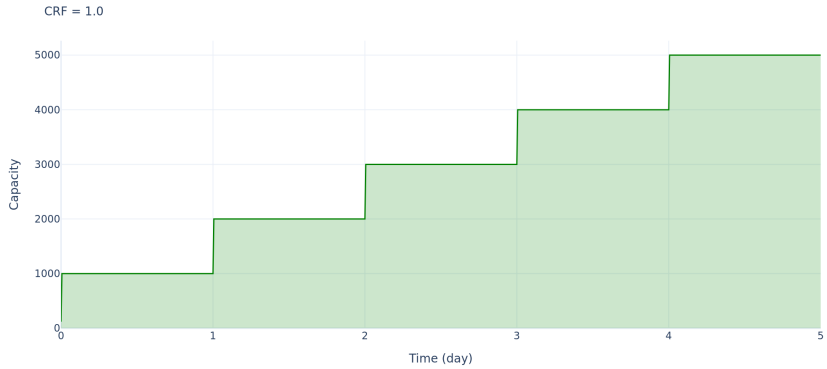


Figura 5.2: Curva de capacidad de MailerSend Hobby con $k = 1$ sobre cinco días, pedida a PRIME4API [7] en la dimensión de peticiones: la misma llamada que los paneles de la Figura 5.1 con `capacity_unit` en peticiones. Aquí sí entra la cuota diaria de 1.000 peticiones del plan, y la curva es una escalera que sube 1.000 cada medianoche UTC hasta los 5.000 de la cuota mensual el quinto día. Con $k = 1$ cada petición lleva un correo, así que la escalera se lee también en correos. El tramo de *rate* (120 por minuto hasta el primer millar, ocho minutos) no se distingue a esta escala.

El segundo (H2): `/capacity-at` con un presupuesto menor que el precio del plan devuelve la cuota base más la ráfaga, 50.120 correos en Professional con 50 USD. No marca el plan como no asequible, cosa que `/budget/recommendation` sí hace. Revisado con ese hallazgo, SendGrid Pro (89,95 USD) había respondido 110.000 con 50 USD de presupuesto en la misma pregunta y no se había anotado. H2 recurre en la tanda de

las cuatro APIs nuevas, y el caso es Brevo Professional, que cuesta 499 USD al mes. Con 50 de presupuesto el motor devuelve igualmente 151.000 y 152.000 correos: aplica el presupuesto a la expansión de la cuota y no al precio del plan. Las dos cifras son la misma capacidad con dos categorías de caudal: 150.000 de cuota más 1.000 con el General y más 2.000 con el Advanced. Los tramos de caudal no cambian el volumen del mes, cambian cuántos correos caben en el primer segundo de la ventana siguiente. Y la etiqueta `unreachable_reason: rate_limit` con causa de cuota aparece ya en cuatro de las ocho, Postmark, Resend, Brevo y SES. Los tres son candidatos a incidencia en el repositorio del motor. Antes hay que comprobar si ya existen.

El límite de esta sección es su base. Son ocho APIs. El par *rate* y cuota con valor en la Datasheet solo se forma en seis. De esas seis, el *rate* lo publica el proveedor en cuatro, y en dos sale de una cabecera medida, Postmark y Mailgun. Ninguno de los cuatro publicados es una cifra firme. El de SendGrid aparece una sola vez en una FAQ sin condición de plan, y el de Resend es el valor por defecto asignado por decisión. El de MailerSend es el de una cuenta en estado sano, y el de Brevo va en ámbar con una contradicción abierta en la misma página. El único margen calculado con documentación, panel y cabecera de acuerdo es el de Resend. En las dos APIs que quedan fuera del par el hueco es del proveedor y no del esquema: SES no publica la cuota ni el caudal de producción, y Mailchimp no publica caudal en peticiones. Y en tres de las ocho lo medido es el estado de la suscripción y no el plan: la cuenta sin aprobar de MailerSend, el entorno de pruebas de SES y la cuenta de prueba de Mailchimp.

5.4. Catálogo de preguntas de Pricing Intelligence

El catálogo de preguntas de este trabajo tiene 32 plantillas, y 25 de ellas (el 78 %) pueden responderse con el instrumental del canal API: 11 leyendo la Datasheet y 10 con una operación de PRIME4API, más 4 con HARVEY4API. Las 7 restantes no tienen respuesta con ese instrumental. Las funcionales PI-E01 y PI-E02 se responden leyendo una matriz de *features* por plan, capturada con las mismas fuentes que la Datasheet y fuera del esquema. Las otras 5 preguntas dejan constancia de qué falta. Esa proporción es la "cobertura", la forma medible de la contribución C-04. Esta sección la construye y la siguiente la ejercita.

Una tabla comparativa responde a las preguntas que su autor decidió poner en las columnas, y un catálogo de preguntas hace lo contrario: fija primero qué quiere saber un consumidor y mide después qué parte de eso puede responderse. Esa métrica es la que usa el paper de Pricing Intelligence [2] para medir el alcance de sus herramientas. Sobre su banco de 56 plantillas obtiene un 54 %. De ese paper se toma también la gramática: cada pregunta es una plantilla con los *placeholders* `SAAS`, `CONFIGURATION`, `FEATURE`, `USAGE LIMIT`, `ACTION`, `PRICE` y `YEAR`. Las preguntas se formulan con esa gramática para que sean compatibles con el banco de plantillas que el grupo construye, y esa compatibilidad la da la gramática, no el formato de la tabla.

Cada pregunta tiene tres partes: la plantilla, un ejemplo instanciado sobre una API del catálogo y una clase, que dice con qué se responde. La clase toma cinco valores.

D significa que se responde leyendo la Datasheet, sin cálculo, contando el *provenance* de cada campo, donde v0.3 guarda lo que el campo no admite. P significa que la ejecuta PRIME4API, anotando la operación Qn del catálogo de Gavira Sánchez [1] que la resuelve. H significa que exige orquestar varias Datasheets o varias operaciones, que es el papel de HARVEY4API [6]. L significa que se responde por lectura de la matriz funcional de la API (Sección 5.5.1), sin instrumento y fuera del esquema. X significa que no es respondible, y entonces la fila dice qué falta: un campo que v0.3 no serializa, un canal fuera del alcance o una operación que el motor no tiene. Ni las clases ni el denominador se mueven al añadir APIs al catálogo. La clase mide con qué se responde la pregunta, no qué publica cada proveedor, y las 32 plantillas siguen repartidas en 25 del canal API, 2 de matriz y 5 fuera del esquema para las ocho columnas. La versión máquina añade a cada pregunta los bloques de la Datasheet que necesita y, cuando procede, el reto abierto del paper al que responde. Las preguntas se agrupan en siete categorías, de la A a la G, con el número de preguntas de cada una en las cabeceras de las tablas. Las cuatro primeras miran la capacidad, desde su lectura en una API hasta la relación entre límites y *rates*. Las otras tres miran alrededor de ella, desde las necesidades funcionales hasta el riesgo de operación.

Las tablas 5.19, 5.20 y 5.21 recogen las 32 preguntas con su plantilla abreviada, su clase y, cuando son X, lo que falta para responderlas. En la clase P se indica la operación Qn que la ejecuta, y en la clase H cómo se compone a partir de operaciones o de lecturas por API.

Id	Pregunta (plantilla abreviada)	Clase	Qn	Qué falta si X
A. Lectura de la capacidad contractual de una API (8)				
PI-A01	Correos al mes que incluye `CONFIGURATION` de `SAAS`, y qué pasa al superarlos	D	—	—
PI-A02	*Rate* máximo con el que `SAAS` acepta peticiones de `ACTION` en `CONFIGURATION`	D	—	—
PI-A03	Destinatarios por petición de `ACTION` en `SAAS`, y si cuentan uno a uno contra la cuota	D	—	—
PI-A04	Si hay límite diario además del mensual en `CONFIGURATION` de `SAAS`	D	—	—
PI-A05	Tamaño máximo de un mensaje (adjuntos incluidos) en `SAAS`	D	—	—
PI-A06	Tiempo mínimo que necesita `CONFIGURATION` de `SAAS` para enviar `USAGE LIMIT` correos	P	Q1	—
PI-A07	Capacidad que acumula `CONFIGURATION` de `SAAS` en una ventana de `USAGE LIMIT`	P	Q3	—
PI-A08	Peticiones simultáneas que admite `SAAS` en `CONFIGURATION`	X	—	Concurrencia: v0.3 no la serializa y solo un proveedor del catálogo la publica para la API REST
B. Selección de suscripción para una necesidad (5)				
PI-B01	Suscripción más barata de `SAAS` para `USAGE LIMIT` correos al mes	P	Q6	—
PI-B02	Con `PRICE` al mes, máximo de correos con `SAAS`	P	Q7	—
PI-B03	Con `USAGE LIMIT` al mes, si sale más barato el sobrecoste o el siguiente plan de `SAAS`	P	Q10	—
PI-B04	Si se puede empezar gratis en `SAAS` y qué limita el plan gratuito para `ACTION`	D	—	—
PI-B05	Plan de `SAAS` que sostiene un pico de `USAGE LIMIT` correos en `USAGE LIMIT 2`	P	Q5	—

Tabla 5.19: Catálogo de preguntas de Pricing Intelligence (v0.1.1), categorías A y B. Clase: D Datasheet, P PRIME4API, H HARVEY4API, L lectura de la matriz funcional, X no respondible.

Id	Pregunta (plantilla abreviada)	Clase	Qn	Qué falta si X
C. Comparación entre APIs (5)				
PI-C01	Para `USAGE LIMIT` correos al mes, cuál sale más barata, `SAAS 1` o `SAAS 2`	H	Q6 por API, más comparación	—
PI-C02	Cuál de `SAAS 1`, `SAAS 2` y `SAAS 3` da más capacidad por euro en el primer plan de pago	H	lectura por API, más comparación	—
PI-C03	Qué API de correo publica su *rate* y cuál no	D	—	—
PI-C04	Cuál envía antes `USAGE LIMIT` correos, `SAAS 1` o `SAAS 2`	H	Q1 por API, más comparación	—
PI-C05	En qué se parecen y en qué se diferencian los pricings de `SAAS 1` y `SAAS 2` para `ACTION`	X	—	Una operación de *diff* entre pricings que el motor no tiene
D. Relación entre límites de uso y *rates* (RQ3) (6)				
PI-D01	En `CONFIGURATION` de `SAAS`, qué se agota antes, la cuota mensual o el *rate*	P	Q5	—
PI-D02	A partir de qué volumen mensual el *rate* de `SAAS` deja de ser el límite efectivo	P	Q5	—
PI-D03	Si `SAAS` cobra por correo y limita por petición, cuántos destinatarios por petición igualan los dos límites	P	Q9	—
PI-D04	Si el *rate* de `SAAS` sube al subir de plan o es el mismo en todos	D	—	—
PI-D05	Si los límites de `SAAS` crecen con el uso (reputación) o son fijos por plan	D	—	—
PI-D06	Qué ocurre en `SAAS` al superar el *rate*: rechazo, cola o degradación	X	—	El comportamiento al exceder no es una dimensión del ACM

Tabla 5.20: Catálogo de preguntas de Pricing Intelligence (v0.1.1), categorías C y D. Misma leyenda que la Tabla 5.19.

Id	Pregunta (plantilla abreviada)	Clase	Qn	Qué falta si X
E. Necesidades funcionales (3)				
PI-E01	Si `CONFIGURATION` de `SAAS` incluye `FEATURE`	L	—	Por lectura de la matriz funcional; con instrumento exigiría el iPricing (canal web), fuera del alcance fijado en el Capítulo 4
PI-E02	Cuál de `SAAS 1` y `SAAS 2` da `FEATURE` al menor precio para `USAGE LIMIT` correos al mes	L	—	Por lectura: *feature* en la matriz, capacidad en la Datasheet, composición a mano. Con instrumento faltaría el vínculo entre canales
PI-E03	Cuánto tiempo conserva `SAAS` los registros de envío en `CONFIGURATION`	X	—	Un límite de uso que no se consume (retención): v0.3 no lo modela
F. Evolución y vigencia (2)				
PI-F01	Si el pricing de `SAAS` ha cambiado desde `YEAR` en cuota o en *rate*	D	—	—
PI-F02	Qué parte de la Datasheet de `SAAS` sigue vigente	X	—	Una Datasheet no dice qué parte sigue vigente (F-09): exige reverificar fuente a fuente
G. Operación y riesgo (3)				
PI-G01	Si el tráfico crece un `USAGE LIMIT` % al mes, cuándo habrá que cambiar de plan en `SAAS`	H	Q6 iterada	—
PI-G02	Coste de un mes de `USAGE LIMIT` correos en `SAAS` sin cambiar de plan	P	Q10	—
PI-G03	Si se puede contratar más volumen en `SAAS` sin cambiar de plan, y a qué precio	D	—	—

Tabla 5.21: Catálogo de preguntas de Pricing Intelligence (v0.1.1), categorías E a G. Misma leyenda que la Tabla 5.19.

Las 5 preguntas X no se descartan. Son evidencia: cada una señala un campo que v0.3 no serializa, un canal fuera del alcance o una operación que el motor no tiene. Las dos L de la categoría E (PI-E01 y PI-E02) son del canal web, y el instrumento no las alcanza por el alcance fijado en el Capítulo 4: se responden leyendo. PI-E03 es un límite que no se consume, la retención, y aunque la matriz funcional recoge

su valor sigue en X porque su sitio es el esquema, y v0.3 no lo modela. PI-D05, los límites por reputación, era la sexta X y ha dejado de serlo. Su motivo decía que un límite por cuenta no tiene sitio en el esquema. La Datasheet de MailerSend muestra lo contrario: declara los estados de reputación con sus aliases, y el caudal de cada estado queda en el fichero con su cifra, de 120 a 10 peticiones por minuto bajo revisión. El sondeo midió ese 10 en la cuenta sin aprobar. La pregunta pasa a responderse leyendo la Datasheet. La clase mide con qué se responde la pregunta, no qué publica cada proveedor. Sobre las ocho APIs la responden dos, MailerSend y Mailchimp, cuyos proveedores atan el caudal a estados de reputación con nombre, y solo MailerSend les pone cifra. En Mailchimp el caso es extremo. No hay planes de los que el caudal pueda depender, y la reputación es lo único que lo mueve. Mailgun y SES la dejan en parcial: el primero dice que los caudales suben con la reputación, sin estados ni cifras, y el segundo publica el criterio del incremento automático sin porcentaje, escalón ni periodo. Postmark, Resend, SendGrid y Brevo no la responden porque no declaran estados de reputación, no porque falte sitio donde escribirlos. Lo que el esquema sigue sin declarar es el disparador del cambio de estado, que MailerSend tampoco publica con umbral, y esa carencia se trata en el Capítulo 6. Otras preguntas se responden, pero solo porque la Datasheet forzó el esquema al construirse. PI-A03 descansa en F-03 (destinatarios por petición) y PI-A05 en F-06 (tamaño del mensaje). PI-G03 se responde desde el *provenance*, porque el volumen contratado no tiene campo en v0.3, y ese hueco es el que motiva la propuesta P-01. PI-A02 y PI-C03 se apoyan en la propuesta P-04, que distingue "no publicado" de "sin límite". Esos elementos no soportados y esas propuestas se explican en el Capítulo 6. Aquí solo interesa el vínculo: una pregunta que un consumidor formula sin esfuerzo tropieza con un hueco concreto del esquema.

El catálogo no repite las operaciones Q1 a Q10 de Gavira Sánchez [1], que son operaciones sobre un plan o sobre un pricing: dado un *rate*, una cuota y un factor de consumo, calcular un tiempo o un coste. Las preguntas de este catálogo son las de un consumidor eligiendo API de correo, y cada una remite a la Qn que la resuelve cuando existe. Las preguntas con esa correspondencia directa son diez de las 32. Las otras 22 preguntas se responden con una lectura o con una composición de operaciones, o esperan algo que el instrumental todavía no da.

La clasificación se hizo primero sobre lo que v0.3 y el catálogo Q1 a Q10 declaran, y después se comprobó ejecutando. Las 10 preguntas P se instanciaron sobre las ocho Datasheets. Sobre Postmark, Resend y SendGrid se ejecutaron contra PRIME4API arrancado como servicio REST en local, 45 llamadas sobre 30 instancias, porque el servicio desplegado no fue accesible en el plazo. Sobre MailerSend se ejecutaron el 2026-09-08 contra el PRIME4API que despliega SPHERE en local, 15 llamadas. Sobre Brevo, SES, Mailgun y Mailchimp se ejecutaron 68 llamadas el 2026-09-21, sobre 40 instancias. Mailgun y SES contestaron 17 de 17, Brevo 16 de 17 y Mailchimp 3 de 17. De las 30 instancias de esa tanda que el motor resuelve, ninguna difiere del cálculo a mano. Los dos fallos tienen nombre y quedan registrados como defectos del motor, no como huecos de cobertura. El plan Enterprise de Brevo devuelve `HTTP 400` porque su precio es `custom` y el motor no lo convierte, y catorce de las diecisiete llamadas de Mailchimp devuelven `HTTP 500` con el mismo texto, por dos campos vacíos que el esquema admite. Las 128 llamadas tienen su petición y su respuesta archivadas. Las 4

preguntas H no se han ejecutado: HARVEY4API exige ese despliegue. Una pregunta pasa a comprobada cuando se instancia sobre una Datasheet real y se ejecuta, o se documenta por qué no. La versión máquina del catálogo (`catalogo.yaml`, con la misma gramática y los mismos ids) se conserva en el repositorio de datos que acompaña al trabajo, junto con las respuestas del motor. El catálogo cumple lo que RQ4 pide, un método de comparación que se pueda repetir, y su límite es que la cobertura del 78 % está medida sobre el motor en las P y declarada sobre el esquema en las H.

5.5. Comparación sistemática sobre el catálogo

De las 25 preguntas respondibles del catálogo, Resend responde 22 sobre sus hechos, SES y MailerSend 21, SendGrid, Mailgun y Mailchimp 20, Postmark 18 y Brevo 16. De las 18 de Postmark, siete se responden solo desde que el *rate* medido por sondeo entró en su Datasheet en ámbar (●). Doce preguntas se responden en las ocho APIs a la vez. Con cuatro APIs eran 17, y las cinco que caen son las cinco que Brevo deja en parcial. El instrumento se ha ejercitado sobre ocho Datasheets y devuelve ocho coberturas distintas, y esa diferencia es el primer resultado de la comparación, antes de cualquier cifra de precio. Tres de las cuatro APIs nuevas quedan por delante de Postmark, que fue el piloto. Aun así, la media de las cuatro nuevas es de 19,25 preguntas, un punto por debajo de las 20,25 de las cuatro primeras.

El procedimiento tiene tres pasos. Se toma cada una de las 32 preguntas y se instancia sobre cada Datasheet con los hechos disponibles, y cuando la clase es P se resuelve a mano con la definición de la operación Qn y se contrasta con la respuesta del motor. Después se anota una de cuatro salidas: respuesta con cifra y con la fuente que la sostiene, registrada en el repositorio de datos, respuesta parcial cuando solo sale una parte de la pregunta, o "no" con el dato que falta. La cuarta salida es "X con respuesta en los hechos", cuando el catálogo la clasifica X pero el registro de fuentes y evidencias de la Datasheet contiene el dato fuera del YAML. Las dos preguntas L se resuelven aparte, leyendo la matriz funcional de cada API (Sección 5.5.1). Dos criterios completan el recuento. "No publicado" cuenta como respuesta cuando la propia pregunta lo admite, como en PI-A02, siguiendo la propuesta P-04. Y en las preguntas de comparación (categoría C) una API cuenta como respondida cuando aporta su lado de la comparación. Un mes se toma como 30 días (2.592.000 segundos). La comparación vale a la fecha de la última reverificación de cada Datasheet: Postmark el 2026-08-03, Resend el 2026-09-01, SendGrid el 2026-09-06, MailerSend el 2026-09-08, Brevo el 2026-09-16, SES el 2026-09-18, y Mailgun y Mailchimp el 2026-09-21. Entre capturas cambiaron textos, no valores modelados.

Cada valor comparado es el de la página de precios cuando esta lo publica. El catálogo registra 82 divergencias entre fuentes, y en 37 la página de precios es uno de los lados. La regla no elige en siete de esas 37, porque la página se contradice a sí misma, ni en las 45 sin página. Cambia el ganador en once. En una cambia un valor modelado: el sobrecoste de Essentials 50K en SendGrid. En otra cambia un color: la cuota diaria del Free de Mailgun. En las nueve restantes solo cambia el registro.

Un tercer criterio aparece con las cuatro APIs nuevas, y es el que sostiene los

corchetes de la tabla. Cuando el eje que la pregunta presupone no existe en el proveedor, la respuesta no se archiva bajo el nombre del eje más parecido. Se ejecuta lo que la Datasheet declara y la celda se rotula con lo que contestó el motor. El rótulo va entre corchetes y con el vocabulario del proveedor. En esta tanda son `[sandbox]` en SES, que es su único estado con cifra, y `[demo]` en Mailchimp, que no tiene planes. El corchete no lleva semáforo. El color mide la fiabilidad del dato, y dos dudas distintas no comparten glifo. La Tabla 5.22 recoge el recuento.

Salida	Postmark	Resend	Send Grid	Mailer Send	Brevo	SES	Mailgun	Mail chimp	Las ocho
Respondible con los hechos	18	22	20	21	16	21	20	20	12
de ellas, con un dato del *provenance* y no de un campo del esquema	4	3	2	1	1	6	1	2	1
Parcial	4	2	4	4	8	4	5	3	2
Respondible en el catálogo, sin respuesta en los hechos	3	1	1	0	1	0	1	2	0
L, respondida por lectura de la matriz funcional	2	2	2	2	2	2	1	2	1
X sin respuesta en ningún sitio	2	2	2	2	2	3	1	2	1
X con respuesta en los hechos (fuera del YAML)	3	3	3	3	3	2	4	3	2
Total	32	32	32	32	32	32	32	32	—

Tabla 5.22: Cobertura del catálogo de preguntas sobre las ocho APIs. El catálogo declara 25 respondibles (D, P y H), 2 L y 5 X, y ese reparto no se mueve al añadir APIs. Sin respuesta en los hechos quedan PI-A04, D05 y F01 en Postmark, PI-D05 en Resend, SendGrid y Brevo, PI-A04 y F01 en Mailchimp, y PI-E02 en Mailgun. El 1 de Mailgun de esa fila es de clase L y es el primer "no" de clase L del catálogo: 200.000 correos al mes no son una posición de su selector de volumen. PI-D05 solo la responden MailerSend y Mailchimp, que atan el caudal a estados de reputación con nombre, y solo MailerSend les pone cifra. Los parciales son PI-A03, B04, D04 y G01 en Postmark, PI-B04 y G01 en Resend, PI-A04, B04, F01 y G01 en SendGrid, PI-A03, A05, B04 y G01 en MailerSend, PI-A01, B01, B02, B04, C01, F01, G01 y G02 en Brevo, PI-B04, D05, F01 y G01 en SES, PI-A03, B04, D05, F01 y G01 en Mailgun, y PI-A03, B04 y G01 en Mailchimp. Las 2 L son PI-E01 y E02. Las X sin respuesta en ningún sitio son PI-A08 y C05 en seis APIs, PI-A08, C05 y E03 en SES, que es la única sin retención publicada, y solo PI-C05 en Mailgun, que es la única que contesta PI-A08. "No publicado" cuenta como respuesta cuando la pregunta lo admite (P-04). Las operaciones P se calcularon a mano y se contrastaron con PRIME4API en local, salvo en Mailchimp, donde catorce de diecisiete llamadas devuelven `HTTP 500`. La columna "Las ocho" cuenta las preguntas con la misma salida en las ocho APIs y no suma 32.

Cuatro preguntas resueltas muestran cómo funciona el instrumento. Cada una dice de qué depende su respuesta. Para 50.000 correos al mes (PI-C01) el orden es SES 5,00 USD, SendGrid 19,95, Resend 20, Mailgun y MailerSend 35 los dos, Mailchimp 40 y Postmark 55. SES queda a la cuarta parte de SendGrid, la siguiente, porque cobra por destinatario y no incluye volumen: 50.000 correos a 0,0001 USD cada uno [44], sin suscripción. En SendGrid [51] Essentials 50K cuesta 19,95 USD y en Resend [50] Pro 20 USD, los dos con esa cuota incluida. En Postmark [49] la respuesta sale del panel: el volumen de 50.000 del *tier* Basic cuesta 55 USD, y ese volumen está en el *provenance* de la Datasheet, no en un campo del esquema. Por sobrecoste desde

los 10.000 de arranque saldría más caro (66 USD en Platform, 68,50 en Pro y 87 en Basic), y esos tres cálculos dependen de una cuota que la Datasheet lleva en ámbar (●). En MailerSend [52] Starter cuesta 35 USD con 50.000 justos, y por sobrecoste desde Hobby saldrían 74,50. Mailgun [54] empata con MailerSend en el importe y por otra vía: Foundation cubre justo la cuota por 35 USD, y por sobrecoste desde Basic saldrían 87. El 35 es el tramo mínimo de una tarjeta con escalera de volumen. En Mailchimp [55] los 50.000 son dos bloques exactos, 40 USD, con dos reservas. El importe no incluye la cuenta Mailchimp de pago que el producto exige por debajo. Y para esa cuenta de pago no hay caudal publicado: el único, 25 correos por hora, es el de la cuenta de prueba, con el que esos 50.000 tardarían 83 días. Brevo no entra con número. Su Starter cubre los 50.000 por algo entre 9 y 82 USD, que es la horquilla del deslizador, y ningún tramo intermedio está cifrado [53]. Su única cifra publicada para ese volumen es el prepago, que cuesta 165 USD. No es una suscripción, y es la vía más cara.

PI-C02 pregunta cuántos correos da cada dólar en el primer plan de pago, y la cuenta es un cociente por API. Resend Pro da 50.000 entre 20, es decir, 2.500, y SendGrid Essentials 50K da 50.000 entre 19,95, es decir, 2.506. Postmark Basic da 10.000 entre 15, es decir, 667, y de nuevo con la cuota en ámbar (●). MailerSend Hobby, su primer plan de pago, da 5.000 entre 7, es decir, 714, y Starter daría 1.429. Mailgun Basic da 10.000 entre 15, los mismos 667 que Postmark Basic y por la misma aritmética, con 15 USD de base y 1,80 por millar de sobrecoste en las dos. Mailchimp da 25.000 entre 20, es decir, 1.250, y es el único cociente que se mueve dentro del mismo producto: en el último tramo de su escalera son 25.000 entre 10, o sea 2.500, que empataría con Resend. Brevo Starter da 5.000 entre 9, es decir, 556, el peor del catálogo, y su Standard cobra 18 USD por esa misma cuota de entrada. En SES la pregunta no tiene instancia sobre el eje que nombra. Ningún plan incluye volumen, así que el numerador del cociente es cero en los cuatro. La magnitud homóloga es el precio unitario, y da 10.000 correos por dólar en la tarifa à la carte, el mejor del catálogo. Lo que sube al subir de plan es la tarifa, de 0,10 a 0,23 USD por millar en el primer tramo. El orden queda SES 10.000, SendGrid 2.506, Resend 2.500, Mailchimp 1.250, MailerSend 714, Mailgun 667, Postmark 667 y Brevo 556. Pero Postmark vende el volumen aparte del *tier* (PI-G03): con el volumen de 50.000 del panel su cociente sube a 909, y ese volumen no está en un campo del esquema.

Con 50 USD al mes (PI-B02, operación Q7) Resend llega a 116.000 correos: el segundo volumen de Pro (100.000 por 35 USD) más 16 bloques de 1.000 a 0,90 USD, que es como el panel cobra el sobrecoste con el interruptor de pago por uso activado. SendGrid llega a 116.722: Essentials 100K (34,95 USD) más 15,05 USD a 0,0009 por correo, lejos del tope de diez veces el plan. Postmark se queda en 36.666 con Platform (18 USD más 32 USD a 1,20 el millar) y en 29.444 con Basic, otra vez sobre la cuota en ámbar (●). MailerSend llega a 61.538 con Starter: 35 USD de plan más 15 USD a 1,30 el millar, 11.538 correos extra. Hobby daría 33.666, pero con $k = 1$ la cuota diaria de 1.000 peticiones lo deja en 30.000. Mailgun llega a 61.538 con Foundation (50.000 más 15 USD a 1,30 el millar) y a 29.444 con Basic. Scale no es asequible, porque su plan cuesta 90 USD. Las otras tres columnas no responden con dinero. En SES los 50 USD compran 500.000 destinatarios a tarifa à la carte, pero el estado declarado solo deja enviar 6.001 correos al mes, y los cuatro planes devuelven esa misma cifra. Es la primera API del catálogo donde la respuesta la fija la capacidad y no el precio. En

Mailchimp los 50 USD compran dos bloques, 50.000 de cuota, pero la cuenta de pago que los compra no tiene caudal publicado. Con los 25 correos por hora de la cuenta de prueba el mes da 18.000, que es la respuesta rotulada `[demo]`. Ese tope dejaría sin enviar 32.000 correos de la cuota, y el saldo no se arrastra. Para la cuenta de pago la respuesta queda sin cifra. En Brevo el tramo de Starter que cabe en 50 USD no tiene precio publicado, así que la respuesta se queda en parcial. Lo que sí está cifrado da el hallazgo: el motor devuelve 9.300 correos en el plan gratuito y 6.000 en el primer plan de pago. De las ocho respuestas solo tres salen enteras de campos del esquema. Las de Resend y SendGrid usan un volumen que la Datasheet no serializa, la de Postmark un dato sin confirmar, la de SES una tarifa que v0.3 no admite y la de Brevo un tramo sin precio. PRIME4API, que solo ve el primer volumen de cada plan, devuelve 83.333 en Resend y 72.594 en SendGrid: la diferencia con el cálculo a mano es lo que el panel vende y el esquema no guarda (F-02, P-01). En MailerSend devuelve 61.658 en Starter, los 61.538 más la ráfaga de la ventana siguiente, y 50.120 en Professional con un presupuesto que no paga el plan (H2 de la Sección 5.3). En Mailgun devuelve 3.100 correos en el plan gratuito, cien por encima de su propia cuota mensual, porque acumula la diaria treinta veces sin aplicar la mensual como techo.

PI-D03 traslada la operación Q9 de Gavira Sánchez [1]: qué número de destinatarios por petición hace que la cuota y el *rate* se agoten a la vez. El cociente que la resuelve es $k^* = Q/(R \cdot W)$, y con la cuota mensual y el *rate* publicados Resend Pro da 50,000/25,920,000 = 0,0019, mientras que SendGrid Essentials 50K da $1{,}9 \cdot 10^{-6}$. Ningún $k \geq 1$ iguala los dos límites: la cuota se agota siempre antes, como recoge la Sección 5.3. En Postmark el *rate* es el medido por sondeo y adoptado en ámbar (●), y Basic da $10{,}000/1{,}296 \cdot 10^{10} = 7{,}7 \cdot 10^{-7}$. En SendGrid el 10.000 por segundo es un dato de una FAQ que las cabeceras de `mail/send` no permiten observar. MailerSend Hobby da 5,000/5,184,000 = 0,00096 con el 120 por minuto publicado (el sondeo midió 10 en la cuenta sin aprobar, otro estado). Bajo revisión Starter 1M da 2,31, el primer k^* mayor que 1 del catálogo (Sección 5.3). Brevo Starter da $1{,}93 \cdot 10^{-6}$, y su tope de 2.000 destinatarios por petición queda nueve órdenes por encima del equilibrio. SES da 200/86,400 = 0,0023, y su tope de 50 destinatarios queda cuatro órdenes por encima del equilibrio. Mailgun Basic da $10{,}000/2{,}0736 \cdot 10^{10} = 4{,}8 \cdot 10^{-7}$ con el caudal medido de la Tabla 5.17. Es la única API que sí tiene un k^* mayor o igual que 1 entre otro par de límites. Su ventana de envío lleva dos contadores a la vez, 80.000 peticiones y 2.000.000 de destinatarios, y con $k = 25$ los dos caen juntos. Ese segundo contador no cabe en el fichero, porque el esquema admite un solo caudal por nodo (F-36). En Mailchimp la pregunta se queda sin variable. Presupone que el proveedor cobra por correo y limita por petición, y Mailchimp limita también por correo, así que k no aparece en la ecuación. En las otras siete la respuesta "ningún $k \geq 1$" sale de un cociente pequeño, y aquí sale de que la pregunta no se puede plantear.

La comparación dice tanto del instrumento como de las APIs. De las 12 preguntas comunes, 5 (PI-A02, B03, C02, C03 y G03) son de cuota, precio y publicación del *rate*, y 7 (PI-A06, A07, B05, C04, D01, D02 y D03) necesitan su valor. Con cuatro APIs las comunes eran 17, y las 5 que caen (PI-A01, B01, B02, C01 y G02) son de cuota, precio y sobrecoste. Las cinco caen por Brevo, y por dos huecos de su página de precios. Sus tres planes de pago no documentan qué pasa al superar la cuota, y la página niega que exista precio de exceso [53]. De la escalera de volumen de cada plan solo están

cifrados los extremos, así que el escalón que cubre la cantidad pedida no tiene precio. Brevo no es la columna más floja por falta de documentación. Sus 8 parciales son el máximo del catálogo y salen de tres huecos documentados: el sobrecoste que no existe en ningún plan de pago, los escalones del deslizador sin precio y la discrepancia sobre sus tres categorías de caudal. Ninguna fuente atribuye esas categorías a un plan sin contradecirse. Son huecos del proveedor, no del método. PI-A02 se responde en Postmark con "no publicado". Las 7 del *rate* las responde Postmark solo desde que la cabecera medida por sondeo entró en su Datasheet en ámbar (●): con el *rate* sin adoptar le costaban cinco enteras y dos a medias. PI-D04, si el *rate* sube al subir de plan, se queda en parcial en Postmark, porque el valor adoptado es uno para los cuatro planes por decisión de modelado y no por observación plan a plan. Brevo la responde que sí, y es la primera API del catálogo cuyo caudal del endpoint de envío escala por plan. En SES el motor la ejecuta plan a plan y devuelve la misma cifra cuatro veces, porque el fichero declara el mismo caudal en los cuatro planes.

La carga de ámbar se cuenta igual en las ocho: celdas respondidas cuya respuesta se apoya en al menos un hecho marcado en ámbar (●). Mailgun 12 de 20, Postmark 13 de 18, Mailchimp 10 de 20, Brevo 9 de 16, MailerSend 7 de 21, SES 2 de 21, Resend 1 de 22 y SendGrid 1 de 20. La versión de cuatro APIs publicaba 14 de 18 en Postmark con otra definición, que sumaba además una celda apoyada en un escalón de panel sin ámbar. Resend y SendGrid responden casi todo en verde, y eso no es virtud del método: es que publican más.

Un recuento aparte mide cuántas respuestas descansan en un eje que no es el que la pregunta pide. En SES son 17 de sus 21: dieciséis llevan `[sandbox]`, porque su cifra es la del estado de la suscripción y no la de un plan, y PI-C02 lleva `[à la carte]`, porque el primer *tier* de pago no tiene instancia con volumen incluido. En Mailchimp son 10 de 20, todas `[demo]`. En las otras seis APIs son cero. El eje que la pregunta presupone existe ahí, y se instancia. Ese 17 de 21 no es cobertura perdida, es cobertura sobre otro eje, y por eso se cuenta aparte en vez de descontarse de la columna. Diez respuestas de Mailchimp están además calculadas a mano y sin confirmación del motor. Son las diez de clase P, y la causa está medida: catorce de diecisiete llamadas devuelven `HTTP 500` con el mismo texto, por dos campos vacíos que el esquema admite. Es la única columna del catálogo cuyas P no están contrastadas contra PRIME4API, y la celda lo dice en cada caso.

Las 5 preguntas X dejan de repartirse igual en todas las APIs, y eso es nuevo. Con cuatro se repartían 3 con respuesta en los hechos y 2 sin ella, idéntico en las cuatro columnas. Con ocho se rompe por los dos lados. Mailgun sube a 4, porque es la única que contesta PI-A08, la concurrencia, y lo hace con literal. SES baja a 2, porque ninguna de sus fuentes publica retención de registros, y es el primer "no" del catálogo a PI-E03. Dos X tienen respuesta en el registro de fuentes y evidencias de las ocho, PI-D06 y PI-F02. La única que sigue sin respuesta en ninguna API es PI-C05, el *diff* entre *pricings*. En las X con respuesta el dato existe en la documentación o en el panel, y lo que falta es sitio en el esquema, que se trata en el Capítulo 6.

5.5.1. Comparación funcional por lectura

Las preguntas funcionales (categoría E) no tienen instrumento en el canal API, y el dato existe en la página de precios y en el panel de las ocho APIs. Por eso el ciclo de cada Datasheet captura además una matriz de *features* por plan, con las mismas fuentes y la misma regla que el resto del catálogo. Cada celda lleva su cita y su escalón, y donde ninguna fuente lo dice la celda queda vacía. No entra en el YAML, porque v0.3 no la admite, y se conserva junto a la Datasheet en el repositorio de datos. Para Postmark, Resend y SendGrid se construyó a posteriori, desde el registro de fuentes ya cerrado, y para las cinco restantes dentro del ciclo de recolección, el mismo día que la Datasheet. Las Tablas 5.23 y 5.24 resumen ocho de sus filas (*webhooks* de eventos, IP dedicada, retención de actividad, dominios, usuarios, entrada de correo, SMTP y plantillas), con una columna por API. La tabla va partida en dos mitades de cuatro columnas por anchura y no por criterio: las mismas ocho filas, en el mismo orden, con la misma cabecera.

Feature	**Postmark** (Free / Basic / Pro / Platform)	**Resend** (Free / Pro / Scale / Enterprise)	**SendGrid** (Trial / Essentials / Pro / Premier)	**MailerSend** (Free / Hobby / Starter / Professional)
Webhooks de eventos	— / — / todos / todos	incluidos en todos; 1 / 5 / 10 / flexible *endpoints*	1 / 2 / 5 / 5	1 / 1 / 50 / ilimitados*
IP dedicada	*add-on* de 50 USD al mes por IP, solo Pro y Platform y desde 300.000 correos al mes	*add-on* de 30 USD al mes, solo Scale o superior y desde 3.000 correos al día	— / — / 1 incluida / 1 incluida; más IPs a 30 USD al mes	solo Enterprise, desde 100.000 correos a la semana y sin precio publicado
Retención de actividad	45 días; en Pro y Platform de 7 a 365 con *add-on* desde 5 USD	30 / 30 / 30 / flexible	3 / 3 / 7 / 7 días; ampliable a 30 con *add-on* sin precio publicado	1 / 1 / 7 / 30 días; hasta 365 con *add-on* cotizado por volumen contratado
Dominios	— / 5 / 10 / ilimitados	3 / 10 / 1.000 / por contrato; 100 más por 20 USD al mes	—	1 / 1 / 10 / ilimitados*; hasta 50 más a 4 USD al mes cada uno desde Starter
Usuarios	— / 4 / 6 / ilimitados	—	1 / 1 / 1.000 / 1.000	1 / 5 / 5 / ilimitados*
Entrada de correo	modo de prueba / — / sí / sí	incluida en todos	documentada sin condición de plan	1 ruta (el panel Sandbox pide subir de plan) / 1 / 50 / 100.000
SMTP	— / sí / — / — (10 conexiones por IP, no aplicado)	incluido en todos	sí en la prueba; 5.000 mensajes por conexión, sin condición de plan	incluido en todos; 5 / 5 / 5 / 50 conexiones concurrentes; 120 peticiones por minuto, y menos por IP según otra página
Plantillas	100 por servidor	—	editor en Free y Essentials; editor y pruebas en Pro y Premier	1 / 10 / 250 / 5.000

Tabla 5.23: Matriz funcional por plan, resumida: Postmark, Resend, SendGrid y MailerSend. Valores literales, con su fuente, en el fichero `features.md` de cada API. "—": ninguna fuente registrada lo cifra para ese plan, que no es lo mismo que "no". En MailerSend manda la página de precios. El asterisco es suyo y remite a una página de límites con cifras finitas (5.000 *webhooks*, 1.000 dominios y 1.000 usuarios). La entrada de correo de Free y Hobby sigue abierta: la página de precios la marca con iconos que no constan como texto. Fecha de validez: la de cada Datasheet.

Feature	**Brevo** (Free / Starter / Standard / Professional / Enterprise)	**SES** (à la carte / Essentials / Pro / Enterprise)	**Mailgun** (Free / Basic / Foundation / Scale)	**Mailchimp** (cuenta de prueba / cuenta de pago)
Webhooks de eventos	salientes en los cinco; de entrada — / — / — / sí / sí; número por plan no publicado	ninguno en ningún plan: los eventos salen por destinos (SNS, EventBridge, Kinesis, CloudWatch, Pinpoint)	incluidos en los cuatro, hasta 3 URL por tipo de evento	— (no observado en el panel de la cuenta de prueba) / por API, `webhooks/add`
IP dedicada	— / — / — / *add-on* de 251 USD al año por IP / incluida	estándar 24,95 USD al mes por IP, *add-on* en los tres de pago y tarifa à la carte, sin mínimo de volumen; administrada incluida en Pro y Enterprise	— / — / acceso desde 50.000 e incluida desde 100.000 / 1 incluida; *add-on* de 59 USD al mes por IP	*add-on* de 29,95 USD al mes, sin mínimo de volumen publicado
Retención de actividad	de 1 a 24 meses, lo fija el cliente; la página de precios dice "unlimited", que no es una duración	— en las cuatro: ninguna fuente publica retención; el archivado de Mail Manager se tarifa por gigabyte y no por días	1 / 1 / 5 / 30 días de *logs*; mensajes — / — / 1 / hasta 7 días por dominio	30 días en la cuenta de prueba / — (sin cifra para la cuenta de pago)
Dominios	sin tope publicado en ningún plan (el panel da el alta y no cifra)	10.000 identidades verificadas por región, no por plan	1 / 1 / 1.000 / 1.000	sin tope publicado; exige verificar el dominio con DKIM y DMARC para enviar
Usuarios	1 / 1 / hasta 3 con *add-on* / 10 incluidos / ilimitados	— (el acceso va por IAM y no por plan)	1 / sin cifra propia / tantos como se necesiten / ídem	—
Entrada de correo	— / — / — / sí / sí; el literal describe eventos externos, no correo entrante	sí en los cuatro (Mail Manager); endpoint autenticado sin cargo y endpoint abierto a 50 USD al mes	1 / 5 rutas / acceso completo sin cifra / hereda de Foundation	100 correos al día en la cuenta de prueba / por API, con rutas
SMTP	incluido en los cinco (puertos 587, 2525 y 465); sin límites de canal publicados; el relay no admite envío por lotes	incluido en los cuatro; 40 MB por mensaje; el rechazo por conexiones concurrentes llega sin cifra	incluido en los cuatro	mismo host y puerto en los dos estados; hasta 1.000 destinatarios por vez
Plantillas	sin tope publicado en ningún plan	20.000 por región, no por plan	incluidas en los cuatro; el alta se midió desde la cuenta Free el 2026-09-21	— (no observado en el panel de la cuenta de prueba) / por API, `templates/add`

Tabla 5.24: Matriz funcional por plan, resumida: Brevo, Amazon SES, Mailgun y Mailchimp Transactional. Mismas filas y misma leyenda que la Tabla 5.23. SES y Mailchimp no reparten por plan lo que las otras seis sí reparten: en SES el ámbito es la región y en Mailchimp el estado de la suscripción, porque no tiene planes con nombre.

Las dos preguntas L se responden sobre las dos tablas. PI-E01 pregunta si el plan gratuito incluye *webhooks* de eventos, y las ocho la contestan. En Postmark la respuesta es "no consta", porque el panel solo los declara en Pro y Platform. Resend Free da un endpoint, la prueba de SendGrid un *Event Webhook*, MailerSend Free un *webhook* y Mailgun los incluye en los cuatro planes, sin recorte en el gratuito. En Brevo la matriz separa dos cosas: los salientes están en los cinco planes y los de entrada solo en Professional y Enterprise. Las dos respuestas que se salen del molde son de las APIs sin plan gratuito nombrado. SES no tiene *webhooks* en ningún plan, y no por precio sino por diseño: los eventos salen por destinos de AWS, con cero coincidencias de `Webhook` en las 116 acciones de la v2. En Mailchimp la cuenta de prueba tampoco los enseña, así que la matriz responde "no consta", y la ruta existe por API para la cuenta de pago.

PI-E02 pregunta qué API da IP dedicada al menor precio para 200.000 correos

al mes, y mezcla una *feature* de la matriz con la capacidad de la Datasheet. Con ocho APIs cambia el ganador. SES cuesta 44,95 USD, que son 20,00 de envío a 0,0001 por destinatario más 24,95 de IP estándar, y es la primera del catálogo sin requisito mínimo de volumen. Detrás van Mailchimp con 189,95 USD (ocho bloques a 20 más 29,95 de IP), Resend con 190 y SendGrid con 199,95. Mailchimp adelanta a Resend por cinco céntimos, y es la comparación más ajustada de la tabla. Brevo cualifica y pierde. Su *add-on* es el más barato de los cifrados, 251 USD al año por IP, unos 20,92 al mes, pero solo existe en Professional, que arranca en 150.000 correos por 499 USD. El suelo queda en unos 520 USD al mes, así que comparar solo el precio del *add-on* daría la respuesta al revés. Postmark no es elegible, porque exige 300.000 correos al mes. MailerSend tampoco: su página de precios reserva la IP a Enterprise, sin precio publicado. Mailgun no la responde, y es el primer "no" de clase L del catálogo: 200.000 no es una posición de su selector de volumen, cuyas paradas publicadas saltan de 100.000 a 250.000. Las dos cifras de cabeza llevan salvedad. La de Mailchimp no incluye la cuenta Mailchimp de pago que el producto exige por debajo, y esa cuenta no tiene caudal publicado. Con los 25 correos por hora de la cuenta de prueba, esos 200.000 tardarían 333 días. La de SES tampoco sale del fichero, porque el precio por destinatario no cabe en v0.3, y con 200 correos al día el volumen es inalcanzable en el estado declarado. PI-E03, la retención, tiene valor en la tabla para siete de las ocho APIs y sigue en X. Postmark 45 días, Resend 30, SendGrid de 3 a 7, MailerSend de 1 a 30, Mailgun de 1 a 30 en los *logs* y Mailchimp 30 en la cuenta de prueba. En Brevo el periodo lo fija el cliente entre 1 y 24 meses, único caso del catálogo en que no lo pone el proveedor. SES es la excepción: ninguna de sus fuentes lo publica, y ahí la X no tiene respuesta en ningún sitio. Sigue siendo un límite que no se consume, y su sitio es el esquema y no la matriz.

La matriz cambia el resultado y deja una asimetría. Con ella el catálogo pasa de 25 a 27 preguntas respondibles, y siete APIs suben dos cada una: 20 en Postmark, 24 en Resend, 22 en SendGrid, 23 en MailerSend, 18 en Brevo, 23 en SES y 22 en Mailchimp. Mailgun sube una sola, hasta 21, porque PI-E02 no tiene respuesta en su selector de volumen. Las comunes a las ocho pasan de 12 a 13, y la que se añade es PI-E01. Pero la capacidad se responde con instrumento (Datasheet, PRIME4API, HARVEY4API) y las *features* a mano, leyendo. Esa asimetría es un hallazgo, no un defecto del procedimiento. Lo que haría a PI-E02 respondible por una herramienta es el vínculo entre el iPricing del canal web y la Datasheet del canal API, que el alcance del Capítulo 4 deja fuera. Queda como trabajo futuro.

La comparación cumple lo que C-04 promete: un procedimiento que se repite pregunta a pregunta, con las no funcionales sobre el instrumento y las funcionales por lectura, y que deja rastro por fuente. Su límite es la base, ocho APIs, y el estado de la suscripción desde el que se midió. En SES es el entorno de pruebas, en Mailchimp la cuenta de prueba, y en MailerSend, Mailgun y Brevo una cuenta gratuita o sin aprobar. Las cuatro preguntas H siguen sin ejecutar.

5.6. Conclusiones del capítulo

RQ3. Donde *rate* y cuota tienen valor en la Datasheet, publicado o medido, y la cuenta está en estado sano, la cuota es la restricción activa. El *rate* admite de 2,6 veces

la cuota (MailerSend Professional 2M) a 129.600.000 (Free de Postmark, con el *rate* en ámbar) en su ventana. Eso vale en seis de las ocho APIs, y las otras dos son las que enseñan el límite del enunciado. En Amazon SES la pregunta no se puede contestar antes de abrir la cuenta, porque el proveedor no publica ni la cuota ni el caudal de producción. En Mailchimp Transactional el signo se invierte: la franquicia de la cuenta de prueba son 500 correos y el tope horario 25, así que agotarla lleva veinte horas y lo único que pone tiempo es el caudal. Pero la relación no se lee en las cifras publicadas sino en dónde se publican y cómo se observan. El *rate* de envío no varía con el plan en seis de las ocho, y Brevo es el contraejemplo. Publica tres categorías de caudal para el mismo endpoint, de 1.000 a 6.000 peticiones por segundo, atribuidas a plan y en ámbar por una discrepancia interna. El publicado y el observado coinciden ya en tres de ocho, así que el desajuste deja de ser universal. Y la cabecera de respuesta deja de ser la señal fiable: dos APIs no emiten ninguna y una cobra la suya con un envío.

RQ4. Las APIs se comparan con un catálogo de 32 preguntas de decisión (29 no funcionales y 3 funcionales), resuelto pregunta a pregunta por API, con la fuente que sostiene cada respuesta. De las 29, 25 se responden con la Datasheet, con PRIME4API (ejecutado en local sobre las ocho Datasheets) o con HARVEY4API (por especificación). De las 3 funcionales, 2 se responden leyendo una matriz de *features* por plan, capturada con las mismas fuentes que la Datasheet y fuera del esquema, y la tercera, la retención, espera sitio en el esquema. El canal API que fija el alcance del Capítulo 4 no tiene instrumento para las funcionales, y esa asimetría es un hallazgo.

Lo primero que dejan las ocho APIs es lo que la documentación no dice y la API sí. Postmark no publica el *rate*, y la API devuelve `ratelimit-limit: 5000` en cada respuesta. Un sondeo de caudal de 200 peticiones no llegó al 429, y la Datasheet adopta el valor en ámbar. SendGrid publica 10.000 peticiones por segundo en una FAQ y `mail/send` no devuelve cabeceras de *rate*, mientras los endpoints de lectura devuelven 600 y 6 sin que ninguna página los cifre. Resend es la única de las ocho en la que documentación, panel y cabecera dicen lo mismo. MailerSend publica 120 peticiones por minuto en tres páginas, como valor por defecto de una cuenta sana, y la cuenta Sandbox sin aprobar del sondeo devuelve 10 con 429 alcanzado. El 120 del plan sigue siendo documental, como el 10.000 de SendGrid, porque el sondeo midió el estado de la suscripción y no el plan. Las cuatro APIs nuevas añaden cuatro formas distintas de lo mismo. Brevo publica el caudal ruta a ruta, en tres categorías, y la misma página se contradice sobre a qué plan pertenece cada una. Amazon SES no publica cuota ni caudal fuera del entorno de pruebas, y tampoco emite cabecera: la vía documentada para conocer los límites en vigor es preguntar por `GET /v2/email/account`, es decir, abrir la cuenta. La Datasheet de Mailgun lleva dos caudales en nodos distintos: el medido en la cabecera, en la ruta de envío, y el documental acotado por IP de cliente, en el plan. El segundo es cinco veces más restrictivo que el primero. Mailchimp declara el suyo en correos por hora y no en peticiones, y es la única fila de caudal del catálogo cuya unidad no son peticiones. Y el tope de destinatarios por petición se aplica en siete de las ocho, con valores que van de 50 a 2.000. Son 50 en Postmark, Resend y SES, 70 en MailerSend, y 1.000 en SendGrid y Mailgun. Brevo admite 2.000, y lo baja a 99 cuando la petición lleva un adjunto. En Mailchimp no existe. El proveedor lo niega por escrito y el sondeo aceptó 5.000 destinatarios en una sola llamada sin rechazo.

De las diez afirmaciones de la Sección 5.3, la primera sigue siendo estructural.

Que la cuota sea la restricción activa no depende de la muestra: el cociente $R \cdot W/Q$ es grande por construcción cuando el proveedor cobra por volumen mensual y publica un *rate* por segundo o por minuto. Lo que las cuatro APIs nuevas han hecho no es refutarla, sino acotar dónde se puede enunciar. Las seis APIs cuya Datasheet lleva las dos cifras la sostienen. Las otras dos no la contradicen, porque no hay con qué construir el cociente. Las afirmaciones segunda, tercera y cuarta dependían de lo observado, y las tres se han debilitado con la muestra ampliada. La segunda tenía un contraejemplo posible, una API que publicara un *rate* de envío distinto por plan, y Brevo lo es. La tercera tenía otro, una API cuyo *rate* publicado coincidiera con el observado, y ahora son tres de ocho. La cuarta pierde su segunda mitad, porque SES publica el tope de destinatarios como suma y con definición, y Mailgun lo publica dos veces y lo aplica exactamente. Las seis restantes solo son visibles con ocho APIs, y ninguna lo sería con cuatro. Siete de las ocho no publican un caudal por cuenta atribuible a un plan. Solo cuatro tienen endpoint de introspección, que es la señal fiable del caudal vigente cuando la cabecera no existe. Ninguna tiene en verde la ventana de todas sus cuotas, y las dos veces que se midió la del caudal su tipo quedó sin decidir. El estado de la suscripción y la reputación explican más de la capacidad observada que el plan. El sobrecoste deja de ser universal, porque tres APIs no tienen precio de exceso y cada una pone algo distinto en su lugar. Y la unidad de la cuota se confirma midiendo o no se confirma: solo cuatro tienen respaldo, y dos de esos cuatro salieron del tercer peldaño de la escalera. Lo que deja de valer con ocho APIs es una lectura, no una cifra. Con cuatro, las cuatro tenían planes, cuota mensual, sobrecoste decreciente y tope de destinatarios. Con ocho, una no tiene planes, una no tiene cuota incluida, tres no tienen sobrecoste y una no tiene tope de destinatarios. Las cuatro primeras no eran el dominio: eran su parte regular.

La comparación dice tanto del instrumento, un catálogo que sigue la gramática del paper de Pricing Intelligence [2], como de las APIs. Sobre ocho Datasheets devuelve ocho coberturas: 22 en Resend, 21 en SES y MailerSend, 20 en SendGrid, Mailgun y Mailchimp, 18 en Postmark y 16 en Brevo. Y 12 preguntas se responden en las ocho a la vez, cinco menos que con cuatro Datasheets. Las cinco que caen son las cinco de Brevo, por dos huecos de su página de precios. Ninguno de sus planes de pago documenta qué ocurre al superar la cuota, la página de precios niega que exista precio de exceso y los escalones intermedios de su deslizador de volumen no tienen precio. La carga de ámbar se reparte muy desigual. En Postmark son 13 de 18 respuestas y en Mailgun 12 de 20, mientras que en Resend y SendGrid es 1 de 22 y 1 de 20. Eso no mide la calidad del método sobre cada API: mide cuánto publica cada proveedor. De las 5 preguntas X, 3 tienen respuesta en el registro de fuentes y evidencias en seis de las ocho APIs. El dato existe en la documentación o en el panel, y lo que falta es sitio en el esquema. Las dos excepciones van en sentidos opuestos y las dos son nuevas. Mailgun sube a cuatro, porque es la única del catálogo que contesta la concurrencia, y lo hace con literal. SES baja a dos, porque ninguna de sus fuentes publica retención de registros. Lo más cercano es el archivado de Mail Manager, que es complemento en los cuatro planes y se tarifa por volumen almacenado y no por días.

Queda pendiente lo que no cabía en el plazo. PRIME4API [1] se ha ejecutado como servicio en local, no en el despliegue del ecosistema, y HARVEY4API no se ha ejecutado: las cuatro preguntas H siguen clasificadas por especificación. Las diez preguntas P de Mailchimp están calculadas a mano y sin contraste, porque catorce

de diecisiete llamadas al motor fallan por dos huecos que el esquema admite: es fallo del instrumento y está declarado como tal. El *rate* de Postmark está adoptado en ámbar desde la cabecera medida en el sondeo de caudal, y solo pasaría a verde si un sondeo alcanzase el 429, fuera de lo inocuo. El de Mailgun está en la misma situación, y alcanzarlo exige 80.000 peticiones en diez segundos, que es una campaña de carga y no un sondeo. Tres sondeos midieron un estado de la suscripción y no el plan, y los tres lo dicen en su Datasheet. El de MailerSend usó una cuenta Sandbox sin aprobar, el de SES el entorno de pruebas y el de Mailchimp una cuenta de prueba. Salir de esos tres estados es alta de cuenta o gasto, y eso no lo decide este trabajo. Y las preguntas funcionales (categoría E) se responden por lectura, sin instrumento: el vínculo entre el iPricing y la Datasheet queda para el trabajo futuro.

Las dos respuestas valen ya para las ocho APIs del catálogo, que es la muestra que el trabajo se propuso construir. El catálogo de preguntas permite repetir el procedimiento sobre cualquier Datasheet nueva sin cambiar nada del método, y lo que este capítulo deja medido es qué parte de una respuesta depende del proveedor y qué parte depende del esquema.

6. Evolución del esquema ACM2YAML

Este capítulo desarrolla la contribución C-02 y responde a la segunda pregunta de investigación enunciada en la Sección 1.3: si ACM2YAML es suficientemente expresivo para representar la capacidad contractual de las APIs de correo, y qué extensiones requiere. La respuesta no se construye por análisis del esquema, sino por confrontación: cada carencia que aquí se documenta apareció al modelar un servicio real o al consumir su Datasheet, y cada propuesta lleva pegado el caso que la obliga.

El capítulo se organiza del siguiente modo. En primer lugar se describe cómo un elemento no soportado registrado al construir el catálogo se convierte en propuesta de extensión, y la segunda clase de evidencia que se añade a la recurrencia, la verificación por uso. A continuación se distinguen las dos clases de carencia que el dominio ha revelado, cada una con su remedio y su prueba. Sigue el inventario consolidado de elementos no soportados con su recurrencia sobre las ocho APIs del catálogo, y sobre él se formulan primero las propuestas de expresividad y luego las de contrato. Después se reúnen las observaciones que el catálogo ha registrado y que no son carencias del esquema. Por último se analiza la compatibilidad de cada propuesta con v0.3, se declara qué queda fuera y por qué, y se formula la respuesta a la pregunta de investigación.

6.1. De los elementos no soportados a las propuestas

El Capítulo 4 fijó el marco: las Datasheets del catálogo siguen ACM2YAML v0.3, las carencias del esquema se registran como elementos no soportados numerados, y un elemento no soportado se promueve por una de dos vías (Sección 4.5). La primera es la recurrencia: el patrón reaparece en al menos dos APIs del dominio. Ese criterio protege la utilidad del catálogo, que no depende de que las propuestas se acepten, y refuerza las propuestas mismas, que llegan respaldadas por frecuencias observadas en lugar de por un caso aislado. Un elemento no soportado que cumple el criterio queda promovido. Entre los promovidos, solo lleva número de propuesta, P-nn, el que además tiene en este capítulo una forma concreta contrastada con el texto de la guía. P-05 es la excepción, porque no sale de ningún elemento no soportado y su prueba es el fallo del motor en tres Datasheets.

La segunda vía, la traza, admite una sola API si se cumplen a la vez tres condiciones. La condición L1 pide el hecho citado con literal o medido en un sondeo ejecutado, con su evidencia archivada. La condición L2 pide la consecuencia ejecutada sobre el consumidor de referencia. El motor, con la Datasheet delante y sin manipularla, tiene que devolver un valor falso o dejar sin responder lo que respondería si el esquema expresara la carencia. Un caso de control tiene que mostrar que sabe responder bien. La condición L3 pide que ningún mecanismo ya existente de la guía cubra esa carencia y que no la explique un defecto del motor ya conocido. La recurrencia muestra el alcance

de una carencia, y la traza, cuando falta recurrencia, muestra el daño. La columna de vía de las Tablas 6.1 y 6.2 dice por cuál entra cada elemento no soportado y, en los que no entran, qué condición da por no cumplida el veredicto de su fila.

Las Datasheets del catálogo siguen v0.3, pero usan dos formas que la guía no define. Una es la unidad `MBs`, con la que todas las Datasheets salvo las de Postmark y Resend expresan sus topes de tamaño, y que la de Mailgun completa con `MiBs`. La otra es el registro de ausencias `not_found`, donde las ocho anotan lo que el proveedor no publica (Apartados 6.4.2 y 6.5.3).

La recurrencia demuestra que una carencia pertenece al dominio, pero no demuestra que la forma adoptada para sortearla funcione. Esa segunda comprobación exige un consumidor, y aquí aparece una particularidad del ecosistema que condiciona el método de este capítulo: **no existe validador oficial de Datasheets**. Se buscó en los cuatro repositorios del ecosistema y no hay esquema JSON, ni analizador estático, ni integración continua que compruebe la conformidad de un fichero `datasheet.yaml`. PRIME4API es un motor de cálculo y no un validador: recorre el documento con accesos directos al diccionario, y sus esquemas describen las peticiones y respuestas de su propia API REST en lugar del artefacto [7]. SPHERE valida *pricings* del canal web y, de los ficheros que recibe, solo el tamaño y el tipo [5]. El repositorio de la guía aporta la norma y los ejemplos, no un comprobador [43].

La consecuencia metodológica es directa: si no hay validador, la conformidad de una Datasheet solo puede establecerse **por uso**, haciéndola consumir por el motor de referencia. Este trabajo ha implementado esa verificación como una herramienta que ingiere una Datasheet en PRIME4API. La herramienta la somete a diecisiete comprobaciones fijas y a una más por cada plan de *pricing* del fichero. Las fijas son cinco de navegación, siete de evaluación y cinco de la superficie de análisis del motor. Esa superficie incluye la recomendación de plan y las curvas de capacidad, que ya documenta el TFM que introdujo el motor [1]. El motor siguió cambiando las curvas después de la entrega de ese TFM. La herramienta se publica con el catálogo y deja la traza completa de cada error. Clona el motor si no tiene copia, sin fijar la revisión, y reutiliza esa copia después. Un tercero puede repetir el procedimiento, pero el resultado depende de la revisión que clone. Las ejecuciones de la Tabla 6.5 se hicieron entre agosto y septiembre de 2026, y no todas registran la revisión del motor que usaron. La copia local del motor se clonó el 4 de septiembre, y su último cambio es del 18 de agosto. La revisión importa porque el motor sigue en desarrollo y sus resultados se mueven con él.

Cada propuesta de este capítulo declara, por tanto, dos cosas separadas: en cuántas APIs del catálogo aparece la carencia, y si existe traza de ejecución que demuestre su efecto sobre el consumidor. La primera es un argumento sobre el dominio y la segunda, sobre el modelo.

6.2. Dos clases de carencia

Los elementos no soportados registrados durante la construcción del catálogo no son todos de la misma naturaleza, y confundirlas debilita las propuestas. El material reunido se separa en dos clases.

Una **carencia de expresividad** se produce cuando el dominio afirma algo que el esquema no tiene dónde poner. Postmark ofrece seis volúmenes contratados para su cuota mensual, cada uno con su precio y su sobrecoste, y una cuota de v0.3 tiene un `value` escalar y un `overage_cost` único: no hay campo donde escribir la lista. Estas carencias se detectan al modelar, se manifiestan como pérdida de información, y su prueba es la recurrencia: que una API venda por volumen contratado puede ser una peculiaridad, que dos lo hagan empieza a ser el dominio.

Un **defecto de contrato** es distinto. Aquí el esquema sí ofrece una forma, la Datasheet la usa correctamente, y el consumidor de referencia falla al leerla. El campo `max` de un `workload` admite quedar vacío, que es lo honesto cuando el proveedor no publica tope, y PRIME4API convierte ese valor a número sin comprobar nulos. La Datasheet es conforme y deja sin responder operaciones enteras del motor. Estas carencias no se detectan modelando, porque el fichero se escribe sin resistencia. Se detectan ejecutando, y su prueba no es la frecuencia sino la traza.

La distinción importa por tres razones. La primera es la carga de la prueba: una carencia de expresividad se discute y un defecto de contrato se reproduce, de modo que este último puede sostenerse con una sola API si la traza es concluyente. La segunda es el remedio: la expresividad se corrige en el esquema, mientras que el contrato admite corrección en el esquema, en el motor o en ambos, y conviene decir en cuál. La tercera es el destinatario, porque las dos clases se comunican en repositorios distintos. La separación no estaba planteada al inicio del trabajo y apareció al ejecutar la primera verificación, lo que también señala su límite: es una distinción útil para ordenar este capítulo, obtenida sobre un solo esquema y un solo consumidor, y no se afirma aquí que valga fuera de ese caso.

6.3. Inventario consolidado de elementos no soportados

Las Tablas 6.1 y 6.2 recogen el inventario sobre las ocho APIs del catálogo y la vía de promoción de cada elemento no soportado. La Tabla 4.4 del Capítulo 4 presentaba los ocho elementos no soportados del piloto. Aquí se suman los veintiocho aparecidos en las siete APIs siguientes, se retira uno que no sobrevivió a la verificación y se aplica el criterio de promoción a los treinta y cinco. Los elementos no soportados F-23 a F-38 aparecieron al construir las cuatro últimas APIs. La segunda tabla los presenta con su número y un nombre corto.

Id	Carencia	PM	RS	SG	MS	BR	SE	MC	MG	Vía	Prop.
F-01	Un *rate* que existe y cuyo valor no se publica	sí	parc.	parc.	parc.	parc.	sí	sí	sí	recurr.	P-04
F-02	Volumen contratado con precio y sobrecoste propios	sí	sí	sí	sí	var.	—	—	sí	recurr.	P-01
F-03	CRF multiplicativo entre dimensiones anidadas	sí	sí	acot.	acot.	acot.	sí	—	—	recurr.	P-02
F-04	Restricciones declaradas que el proveedor admite no aplicar o que la respuesta no refleja	sí	—	—	sí	n/v	sí	sí	n/v	recurr.	discus.
F-05	*Results Per Request* documentada y no serializable	sí	n/v	sí	sí	sí	sí	sí	sí	recurr.	—
F-06	`workload` acotado por una métrica distinta de la suya	sí	sí	sí	sí	sí	sí	sí	sí	recurr.	P-03
F-07	*Add-ons* que modifican dimensiones de capacidad	sí	sí	sí	sí	sí	sí	sí	sí	recurr.	—
F-08	El plan real es *tier* × volumen contratado	sí	parc.	sí	sí	sí	var.	var.	var.	recurr.	—
F-09	El documento caduca entero, y sus datos caducan uno a uno	n/v	sí	sí	sí	sí	sí	parc.	—	recurr.	—
F-10	Dos productos de precio con unidades de consumo distintas	—	sí	sí	parc.	var.	var.	sí	var.	recurr.	—
F-11	Límites de recursos por plan, que no son consumo	sí	sí	sí	sí	sí	var.	—	sí	recurr.	—
F-12	La recepción consume la cuota de envío	n/v	sí	n/v	sí	n/v	n/v	n/v	n/v	recurr.	—
F-13	El sobrecoste es opcional y no existe en el plan gratuito	sí	sí	sí	sí	var.	—	var.	sí	recurr.	—
F-14	El plan tiene fecha de caducidad	—	—	sí	sí	—	var.	—	var.	recurr.	—
F-15	Dos canales, dos `workload` distintos para la misma cuota	n/v	n/v	sí	sí	sí	var.	sí	sí	recurr.	—
F-16	El sobrecoste tiene un tope que impone el proveedor	n/v	n/v	sí	—	—	—	—	—	traza	P-07
F-18	La capacidad depende del estado de la suscripción o de la reputación, no del plan	sí	sí	parc.	sí	var.	sí	sí	sí	recurr.	P-06
F-21	Un cupo de destinatarios distintos que se gasta al probar	—	—	—	sí	—	—	—	—	traza	P-08
F-22	Una credencial que solo se gestiona desde la web	sí	—	—	—	sí	sí	sí	n/v	recurr.	—

Tabla 6.1: Inventario de elementos no soportados sobre las ocho APIs del catálogo, F-01 a F-22, con la vía de promoción de cada uno. Columnas: PM Postmark, RS Resend, SG SendGrid, MS MailerSend, BR Brevo, SE Amazon SES, MC Mailchimp y MG Mailgun. En las columnas de API, sí: el elemento no soportado aparece, también como variante que suma. var.: variante que no suma. parc.: aparece en parte. acot.: aparece en variante acotada. —: comprobado y ausente. n/v: sin fuente que lo afirme ni lo niegue. susp.: en suspenso. El número de «sí» de una fila es su recuento sobre las ocho APIs. En la columna de vía, recurr.: dos APIs o más. traza: una sola API, con las tres condiciones de la Sección 6.1 cumplidas. L2, L3 o L2 y L3: lo que el veredicto de la fila da por no cumplido. Donde falta L2, puede faltar además el veredicto de L3 (Sección 6.8). En la de propuesta, discus.: se discute en la Sección 6.8. Faltan F-17, F-19 y F-20, retirados. F-19 lo absorben F-11 y F-18, y F-20 pasó a observación. F-17 se retiró al cerrarse en peticiones la discrepancia sobre la unidad del tope diario de MailerSend, de la que dependía su única ocurrencia.

Id	Carencia	PM	RS	SG	MS	BR	SE	MC	MG	Vía	Prop.
F-23	Un comodín de prefijo con excepción	—	—	—	var.	sí	var.	—	—	L3	—
F-24	Una cuota superada que retiene hasta un número fijo de envíos y deja de entregar los siguientes	—	—	—	var.	sí	—	—	—	traza	P-09
F-25	Un *endpoint*, dos modos de envío y una carga que depende de otro campo	—	—	—	—	sí	var.	—	—	L3	—
F-26	Un precio que depende de la geografía y una sola divisa en el fichero	—	—	parc.	sí	sí	var.	var.	sí	recurr.	—
F-27	Un exceso que cambia el plan en vez de pagarse por unidad, disparado por otra dimensión	—	—	—	—	sí	—	—	—	L2	—
F-28	Una lista de IP autorizadas decide si la llamada entra	parc.	n/v	parc.	sí	sí	n/v	sí	parc.	recurr.	—
F-29	Un precio por unidad sin cuota incluida de la que colgarlo	—	—	—	—	—	sí	—	—	L3	—
F-30	Precio por tramos marginales sobre el mismo consumo	—	—	—	—	—	sí	—	—	L2 y L3	—
F-31	La unidad de gobierno es el par cuenta × región	n/v	—	var.	—	—	sí	—	var.	L2	—
F-32	Precio unitario degresivo sin planes que lo sostengan	—	—	—	—	—	—	sí	—	L2	—
F-33	Una franquicia que no caduca ni se renueva: la cuota sin ventana	—	—	—	—	—	—	sí	—	L2	—
F-34	Dos topes de tamaño sobre la misma llamada, uno del mensaje y otro de la petición	var.	n/v	var.	parc.	parc.	var.	sí	sí	recurr.	—
F-35	El literal de un tope trae una cifra fraccionaria y el máximo de un `workload` es entero	n/v	n/v	—	n/v	n/v	n/v	n/v	sí	L2 y L3	—
F-36	Dos contadores sobre la misma ventana, y un solo caudal por nodo	n/v	—	n/v	—	—	—	—	sí	L2	—
F-37	Un tope de capacidad que fija el cliente, no el proveedor	n/v	n/v	n/v	var.	sí	var.	n/v	sí	recurr.	—
F-38	Dos cuotas concurrentes de la misma unidad	—	sí	—	—	—	—	—	susp.	L2	—

Tabla 6.2: Continuación de la Tabla 6.1: elementos no soportados F-23 a F-38, con la misma notación.

El criterio promueve veinticuatro de los treinta y cinco elementos no soportados: veintiuno por recurrencia y tres por traza. Los otros once no promueven. F-06 y F-07 aparecen en las ocho APIs, y F-05 en siete. La mayoría de los recurrentes, trece de veintiuno, aparece en cuatro APIs o más. En el otro extremo, F-12, F-14, F-34 y F-37 cumplen el umbral con dos APIs justas. Una celda sin verificar solo puede sumar, así que, respecto de esas celdas, el recuento de cada fila es un mínimo.

La Tabla 6.3 dice de dónde sale cada propuesta. De los veinticuatro promovidos, este capítulo formula extensión para ocho, y P-05, la propuesta restante, no sale de

ningún elemento no soportado. De esos ocho, entran por recurrencia F-01, F-02, F-03, F-06 y F-18, y F-01 lleva además traza. F-16, F-21 y F-24 entran por traza, cada uno con su prueba sobre el motor. En F-16 el motor promete un volumen que el proveedor rechaza, y en F-21 declara alcanzable en cero segundos un objetivo que la medición no consiguió enviar. En F-24 escribe como un muro, sin exceso posible, una cuota que el proveedor publica con una cola de reintento. P-05 no tiene fila en el inventario: su carencia apareció al ejecutar y no al modelar, y sigue sin número F-nn. Un inventario construido al modelar no puede contener lo que solo se ve al usar, y esa asimetría es el motivo por el que la sección anterior separa las dos clases. El resto de los promovidos no lleva número de propuesta, y la Sección 6.8 dice por qué.

Propuesta	**Qué pide**	**Elemento**	**APIs**	**Prueba**	**Clase**	**Apdo.**
P-01	Volúmenes contratados en la cuota, con precio y sobrecoste propios	F-02	5/8	recurr.	expresividad	6.4.1
P-02	Consumo de una petición como producto de dimensiones anidadas	F-03	3/8	recurr.	expresividad	6.4.2
P-03	Tope de una carga expresado en otra métrica	F-06	8/8	recurr.	expresividad	6.4.2
P-04	Marca de valor no publicado junto al campo	F-01	4/8	recurr. y traza	contrato	6.5.3
P-05	Plan con precio que existe y no es público	—	—	motor	contrato	6.5.4
P-06	Estado de la suscripción y reputación, declarados una vez y con su disparador	F-18	6/8	recurr.	expresividad	6.4.3
P-07	Tope del sobrecoste	F-16	1/8	traza	expresividad	6.4.4
P-08	Cuota de entidades distintas, sin periodo y con ámbito de cuenta	F-21	1/8	traza	expresividad	6.4.6
P-09	Comportamiento de la cuota al superarse	F-24	1/8	traza	expresividad	6.4.5
Rectif.	Forma de lista de `workload` en la guía	—	—	—	guía	6.5.2

Tabla 6.3: Propuestas del capítulo y su origen. Elemento: el elemento no soportado del inventario del que sale cada propuesta. APIs: su recuento sobre las ocho del catálogo. Prueba, recurr.: la carencia aparece en dos APIs o más. traza: la consecuencia está ejecutada sobre el motor, con las tres condiciones de la Sección 6.1 cumplidas. motor: el consumidor de referencia falla al leer una forma que la guía admite. P-05 no sale de ningún elemento no soportado del inventario, y el criterio de promoción no se le aplica. La última fila rectifica la guía y no extiende el esquema.

Al pasar de cuatro APIs a ocho, ningún elemento no soportado que promovía deja de hacerlo. F-01 recurre ahora en cuatro APIs y conserva la traza, y F-07 aparece en las ocho sin propuesta. F-18 recurre en seis, así que P-06, que antes quedaba formulada sin promover, entra por recurrencia. F-22, que antes no promovía, recurre ahora en cuatro con la variante que suma de Amazon SES. De los dieciséis elementos no soportados que abren las cuatro últimas APIs, F-26, F-28, F-34 y F-37 recurren y promueven. Los demás cuentan en una sola API, y entre ellos solo F-24 promueve, por traza. La Sección 6.8 dice qué condición de la traza le falta a cada uno de los que no promueven.

Dos anotaciones sobre cómo se ha contado, porque de ellas depende que el recuento signifique algo. F-01 aparece en las ocho APIs y solo cuenta en cuatro. Es plena donde el proveedor admite que el límite existe y no le da cifra. Postmark documenta el error `429` y no publica ningún umbral. Amazon SES escribe que su caudal de envío «varies based on your specific use case» [63]. Mailchimp no cifra la cuota horaria de la cuenta de pago (Apartado 5.2.8). La documentación de desarrollador de Mailgun no cifra el umbral de su límite [64]. El proveedor publica un único caudal en sus preguntas frecuentes, 1.500 peticiones por segundo por IP de cliente. El caudal de la cuenta queda así sin cifra (Apartado 5.2.7). Es parcial donde una cifra cubre una parte y deja otra sin cubrir. Resend publica un *rate* por defecto de diez peticiones por segundo por equipo y un procedimiento para ampliarlo bajo petición. SendGrid publica un caudal de envío que ninguna cabecera deja observar, y devuelve cabeceras de caudal en los endpoints de lectura, que ninguna página cifra. Brevo cifra sus endpoints y no el canal SMTP ni la segmentación. Y MailerSend publica cuatro caudales y deja sin cifra dos de sus cuatro estados de reputación. Contar las parciales como plenas habría subido la recurrencia de cuatro a ocho con situaciones que no son la misma, y le habría restado crédito al recuento.

F-08 es la segunda anotación, y aquí el recuento sí se mueve. En el piloto, F-08 englobaba también los límites de recursos por plan, que desde Resend tienen número propio (F-11). La mitad que suma aquí es que la superficie de la API cambie con el *tier*. Con las ocho APIs, esa mitad aparece en Postmark, SendGrid, MailerSend y Brevo, y como variante que no suma en Amazon SES, Mailchimp y Mailgun. Aun así no produce propuesta, porque el esquema ya admite esa forma. Un endpoint que un plan no tiene se omite de ese plan, que es como la Datasheet de MailerSend modela el lote en su plan gratuito, a reserva de que el proveedor lo confirme. F-08 promueve igualmente, porque la recurrencia cuenta en cuántas APIs aparece el patrón y no comprueba si v0.3 ya lo expresa. Lo que no cabe es que la superficie cambie con el estado de la suscripción o con la reputación en lugar de con el plan, y eso es F-18.

El recuento deja treinta y una celdas sin verificar y una en suspenso. En cuatro filas bastaría con que una de esas celdas pasara a «sí» para cambiar la vía. F-31, F-35 y F-36 pasarían a recurrentes, y F-16 pasaría de la traza a la recurrencia. F-38 pasaría también a recurrente si en Mailgun se confirma que dos cuotas de la misma unidad se aplican a la vez. En las demás filas con celdas sin verificar, la vía no cambia aunque la celda se resuelva. F-37 descansa en una regla provisional sobre qué variantes suman, y con otra regla bajaría a una sola API.

6.4. Propuestas de expresividad

6.4.1. P-01. La cuota declara volúmenes contratados

El caso. Los tres planes de pago de Postmark se anuncian con la fórmula *"Starting at 10,000 emails/month"* [49], y la estructura completa solo es observable desde el panel de una cuenta propia: seis volúmenes contratados, de 10.000 mensajes por 15 USD a 1.500.000 por 775, con negociación directa por encima del último. En el *tier* Basic el sobrecoste baja de 1,80 a 1,70 USD por millar al pasar de 125.000 a 300.000, y su valor en los demás cruces de *tier* y volumen no es observable con una cuenta sin aprobar (Sección 4.6). Resend exhibe la misma estructura y la publica: su plan Scale recorre seis volúmenes contratados, de 100.000 mensajes por 90 USD a 2.500.000 por 1.150, con el sobrecoste descendiendo de 0,90 a 0,46 USD por millar [50].

En Postmark la lista de volúmenes contratados solo era observable desde el panel de una cuenta propia, y en Resend está publicada en la página de precios, de modo que la misma carencia queda demostrada con una fuente citable y sin necesidad de acceso privilegiado.

Mailgun publica la misma lista de volúmenes contratados en un selector de su página de precios. En su plan Scale, el sobrecoste baja de 1,10 a 0,40 USD por millar al subir de volumen [54]. Con SendGrid y MailerSend, la carencia aparece en cinco APIs del catálogo (Tabla 6.1). Brevo vende también volúmenes contratados, pero sin sobrecoste, y esa variante no suma al recuento.

Qué hace v0.3. Una cuota declara un `value` escalar y un `overage_cost` único, de modo que no cabe ni la lista de volúmenes contratados ni el sobrecoste propio de cada uno. Las Datasheets de Postmark, Resend y Mailgun modelan el volumen mínimo de cada plan. La lista completa solo queda en el registro de procedencia, en prosa que ninguna herramienta lee.

Consecuencia. La Datasheet de Resend afirma que Scale son 100.000 mensajes con un sobrecoste de 0,90 USD por millar. Es cierto en el primer volumen contratado y falso en el último, donde el mismo plan son 2.500.000 mensajes a 0,46. Comparar ese plan con el de un proveedor de volumen fijo usando el único número serializado falsea la comparación de precios, que es precisamente lo que el método de comparación de este trabajo se propone hacer bien.

Propuesta. Que una cuota admita una lista de volúmenes contratados, cada uno con su precio y su sobrecoste, más una entrada explícita para el tramo negociado. El Extracto de código 6.1 muestra la forma sobre el caso de Resend.

Extracto de código 6.1: P-01: volúmenes contratados dentro de un plan, con precio y sobrecoste propios (caso Resend, plan Scale; se muestran tres de los seis).

```
quota_email_transactional_monthly_scale:
  unit: emails
  period: {value: 1, unit: MONTH}
  window_type: fixed_billing_cycle
  contracted_volumes:
    - value: 100000
```

```
      price: 90
      overage_cost: {price: 0.90, value: 1000}
    - value: 500000
      price: 350
      overage_cost: {price: 0.70, value: 1000}
    - value: 2500000
      price: 1150
      overage_cost: {price: 0.46, value: 1000}
```

Efecto sobre las herramientas. PRIME4API calcula la curva de coste a partir del precio del plan y del sobrecoste de la cuota. Con un único par de valores, la curva es una recta y el resultado es correcto solo en el volumen modelado. Con la lista, la curva pasa a tener una pendiente distinta en cada volumen contratado, que es lo que el proveedor cobra en realidad. La recomendación de plan, que el TFM de Gavira Sánchez ya ejecuta (su Tabla 5.12) [1], depende directamente de esa curva.

6.4.2. P-02 y P-03. El `workload` deja de ser una terna escalar

Los elementos no soportados F-03 y F-06 recurren por separado y piden el mismo cambio de fondo: que el consumo de una petición deje de expresarse con una sola terna `unit`, `min`, `max`. Se presentan juntas porque afectan al mismo punto del esquema y porque separarlas obligaría a proponer dos veces la misma reforma.

El caso de P-02. El endpoint `/emails/batch` de Resend acepta *"Trigger up to 100 batch emails at once"* y cada mensaje admite hasta cincuenta direcciones en el campo `to` [56]. Hasta aquí es una ambigüedad de modelado. Lo que la convierte en prueba es que el proveedor declara por escrito cómo se contabiliza: *"Multiple To, CC, or BCC recipients count as separate emails"*. El consumo de una llamada es el producto de los dos factores. Postmark presenta la misma estructura en `/email/batch` y `/email/batchWithTemplates`, con quinientos mensajes por llamada y cincuenta destinatarios totales por mensaje, límite este último que la documentación declaraba por campo y que el sondeo empírico del piloto corrigió (Sección 4.6). Lo que Postmark nunca ha declarado por escrito es cómo se contabiliza ese consumo [45]. Amazon SES es el tercer caso pleno: su envío en lote recibe una lista de entradas con sus propios destinatarios. Su cuota cuenta destinatarios y no mensajes [63]. La variante acotada es la de SendGrid, MailerSend y Brevo: un tope sobre el total de destinatarios de la petición, que no suma al recuento (Tabla 6.1).

Consecuencia de P-02. La Datasheet serializa `max: 100`, que es el número de mensajes y no el de unidades consumidas. El factor de consumo por petición alimenta las preguntas del motor sobre agotamiento de cuota y coste de sobrecoste, y con ese valor las respuestas salen mal por un factor de hasta cincuenta. El fallo da un número plausible, que el consumidor no tiene cómo distinguir de uno correcto. En Postmark el error se deducía. En Resend lo refuta la propia documentación del proveedor.

El caso de P-03. El endpoint `/email/bulk` de Postmark no tiene tope de mensajes: el proveedor documenta *"as many recipients as you need, as long as your total payload (including attachments) stays under 50 MB"* [45]. Resend acota los adjuntos en *"max 40MB per email, after Base64 encoding of the attachments"* [56]. En los dos

casos la cota efectiva no está en la unidad del `workload` sino en otra métrica, y medida después de codificar, de modo que el tope en contenido útil depende de la codificación. F-06, la de P-03, aparece en las ocho APIs. Mailchimp Transactional acota cada llamada por el tamaño de su JSON, que debe ocupar menos de 10 MB, y no por el número de mensajes. Amazon SES acota cada mensaje en 40 MB tras la codificación [63]. Brevo acota el correo entero en 20 MB. Mailgun acota la misma llamada a la vez por destinatarios, por tamaño de la petición y por tamaño de las opciones, con tres cifras distintas. El Capítulo 5 cita los literales de Mailchimp, Brevo y Mailgun en el apartado de cada una.

Consecuencia de P-03. En Postmark y Resend, el tope real está en otra métrica, y el campo `max` de la unidad propia queda sin cifra que escribir. Todas las Datasheets del catálogo salvo las de Postmark y Resend declaran el tope de tamaño como una carga aparte en `MBs`, y la de Mailgun usa además `MiBs`. En MailerSend y en Amazon SES esa carga no tiene cuota propia, y es la que hace fallar la curva acumulada del motor (Sección 6.5).

Propuesta. Que `workload` admita composición entre dimensiones anidadas y que `max` pueda expresarse en una métrica distinta de la unidad declarada. El Extracto de código 6.2 muestra las dos formas sobre los casos anteriores.

Extracto de código 6.2: P-02 y P-03: composición de dimensiones anidadas y cota expresada en otra métrica.

```
# P-02. El consumo es el producto de los dos factores.
workload:
  - unit: messages
    min: 1
    max: 100
    composes:
      unit: emails
      max_per_unit: 50
# P-03. El tope real esta en otra metrica, medida tras codificar.
workload:
  - unit: emails
    min: 1
    max_by: {unit: MBs, value: 40, measured_after: base64}
```

Efecto sobre las herramientas. Las dos formas alimentan el mismo cálculo. El factor de consumo por petición deja de ser un entero y pasa a ser una función de la petición, lo que obliga a PRIME4API a evaluar el consumo máximo y no solo el declarado. Es un cambio de mayor calado que P-01 y afecta a las operaciones de navegación además de a las de evaluación.

6.4.3. P-06. El estado de la suscripción y la reputación se declaran una vez

El caso. MailerSend publica cuatro estados de reputación con nombre, y da cifra para dos de ellos [58]. Cuando la cuenta entra bajo revisión, `/v1/email` pasa de 120 a 10 peticiones por minuto y el lote de 10 a 1, y ese estado no depende del plan. El sondeo del Apartado 5.2.4 midió esas mismas cifras sobre una cuenta recién creada, sin aprobar

ni plan contratado, que es un estado de la suscripción. Los límites medidos fueron 10 peticiones por minuto y 1 lote por minuto. El sondeo midió también la cuota diaria de 100 peticiones. La mensual de 100 correos es la publicada y no se midió (Tabla 5.6). Cada envío admitía una dirección en `to` y cada lote cinco objetos. La cuenta recién abierta que midió el sondeo no es la que describe el valor por defecto de la Datasheet.

El caso obliga a separar tres nociones. La cuenta es la identidad del cliente ante el proveedor, y puede tener varias suscripciones. Resend, por ejemplo, muestra bajo la misma cuenta una suscripción transaccional y otra de *marketing* (Apartado 5.2.2). El «estado de la suscripción» es la fase en que el proveedor sitúa la suscripción del cliente: en modo de prueba, sin aprobar o en el entorno de pruebas. La «reputación» es la valoración que hace el proveedor de cómo consume el cliente, por ejemplo por los errores que provocan sus peticiones. Es un factor que amplía o reduce la capacidad sin cambiar de suscripción. En MailerSend, la cuenta sin aprobar es un estado de la suscripción, y la revisión que baja `/v1/email` a 10 peticiones por minuto es un efecto de la reputación.

En seis de las ocho APIs del catálogo la capacidad depende del estado de la suscripción o de la reputación, y no del plan (Tabla 6.1). Por el estado de la suscripción, Postmark limita a quién se envía mientras la cuenta está en modo de prueba, y Amazon SES limita el caudal en el entorno de pruebas [63]. MailerSend reduce los topes en la cuenta sin aprobar, y la cuenta de demostración de Mailchimp tiene una cuota horaria fija. Por la reputación, MailerSend reduce el caudal bajo revisión, Amazon SES publica tres valores de `EnforcementStatus` sin cifra, y en Mailgun el caudal crece con la reputación del remitente. Resend amplía el caudal a los remitentes de confianza bajo petición y sin publicar cuánto [56], y Mailchimp ata a la reputación la cuota horaria de las cuentas de pago. Los literales de Postmark, Mailchimp y Mailgun están en los Apartados 5.2.1, 5.2.8 y 5.2.7.

Qué hace v0.3. Más de lo que este trabajo supuso al empezar el catálogo. La guía del esquema documenta *aliases* de estado dentro de un endpoint, y usa esta misma API como caso [43]. Su vocabulario de ejemplo tiene dos nombres: `healthy_reputation` y `under_review_reputation`. El catálogo no los había empleado en ninguna Datasheet, y esta pasada los adopta con un tercer estado para la cuenta sin aprobar, que el proveedor documenta y el ejemplo de la guía no recoge. Las cifras del Capítulo 5 no cambian, porque el estado sano conserva el valor por defecto. Lo que cambia es que el motor devuelve una fila por estado donde antes devolvía una sola. El elemento no soportado deja de ser «el esquema no lo expresa» y pasa a ser hasta dónde llega el mecanismo.

Qué no cubre. Tres carencias, y las tres se ven al aplicar el mecanismo, no al leerlo.

La primera es que el alias no tiene campo para el disparador. Un `trigger` escalar dentro del alias se ignora en silencio. Escrito al nivel del endpoint, que es donde parece natural ponerlo, el motor lo lee como un estado más, porque trata como estado todo diccionario cuya clave no sea una de las cuatro que conoce. El daño depende de dónde se escriba. En un endpoint que ya declara estados y no lleva `rate` propio, el estado fantasma sale vacío y las filas reales aguantan. En un endpoint que lleva su `rate` al nivel del endpoint, el endpoint deja de tener fila propia: el comodín `/*` devolvía 60 peticiones por minuto sin estado y pasa a devolver esas mismas 60 bajo un estado que

no existe. Ninguna de las tres comprobaciones del ciclo avisa: el validador de formato da 16 conformidades y 2 avisos, y la verificación por uso 20 aciertos y 1 error, exactamente lo mismo que con la Datasheet limpia.

La segunda carencia está en la guía: un alias admite también el `workload` y el código de referencia lo recoge, pero la guía solo documenta el *rate* y la cuota. La consecuencia se midió aquí. La primera pasada dio por fuera del mecanismo el cambio de carga por petición, que en esta API va de setenta destinatarios a dos, y lo hizo por leer la guía. Cabía. Mientras eso no se escriba, el mecanismo se usa por debajo de lo que da.

La tercera es que un estado de la suscripción se declara siete veces. El estado no depende del plan ni del endpoint, y el esquema no tiene dónde declararlo una sola vez para toda la Datasheet. La referencia se repite en los siete bloques de endpoint y nada liga las copias. Cambiar el estado obliga a tocar siete sitios, y nada detecta que uno se quede atrás. El recuento de semáforos hereda el problema. De los valores en verde de esta Datasheet, el tope de 50 MB del lote aparece tres veces, una por estado, copiado del endpoint. Solo una nota del registro de procedencia impide leer esas tres copias como tres evidencias. Esta tercera carencia recurre. La Datasheet de Amazon SES repite su alias de *sandbox* en ocho bloques y la de Mailchimp el de demostración en tres. El disparador y la carga por alias, en cambio, solo se han medido en MailerSend.

Propuesta. Tres cambios sobre el mecanismo de la guía, que lo extienden y no lo sustituyen. El primero es un atributo de disparador con sitio propio, que diga si el cambio de estado lo impone el proveedor o lo elige el cliente y que lleve su condición en prosa. El segundo es documentar el `workload` por alias, que la implementación ya admite. El tercero es una forma de declarar un estado una sola vez para toda la Datasheet, con las referencias de cada endpoint apuntando a esa declaración. Los tres no son la misma clase de cambio: el primero y el tercero extienden el esquema, el segundo rectifica la guía sin tocar el esquema, y la corrección que cierra esta subsección es del motor. El Extracto de código 6.3 muestra los tres sobre el caso de MailerSend.

Extracto de código 6.3: P-06: los estados de la suscripción y de la reputación declarados una vez, con su disparador y con la carga por petición que la guía no documenta (caso MailerSend).

```
# Declaracion unica, fuera de los endpoints.
account_states:
  unapproved_account:
    trigger: {mode: provider_managed, condition: "cuenta sin dominio
       verificado"}
    rate: rate_request_email_unapproved
    quota: quota_email_monthly_unapproved
    workload: workload_email_unapproved   # admitido por el motor, no
       documentado
# Cada endpoint referencia el estado en lugar de repetirlo.
/v1/email:
  states: [healthy_reputation, under_review_reputation,
     unapproved_account]
```

La reputación como campo genérico. Los estados de ejemplo de la guía llevan la reputación en el nombre, pero ningún campo dice cómo se mueve ni cuánto

cambia la capacidad. Se propone como extensión un campo de reputación genérico: un factor que amplía o reduce la capacidad declarada, con su efecto y sus disparadores. Con él, el esquema podría registrar si el proveedor publica su reputación de forma completa y determinista. Es completa si da la capacidad de cada estado de reputación. Es determinista si dice qué hace cambiar de estado y con qué umbral. La Tabla 6.4 aplica los dos criterios a las ocho APIs con lo que consta en sus registros, sin inferir lo que no consta. Ninguna publica su reputación de forma completa y determinista. Resend, MailerSend, SES, Mailchimp, Mailgun y SendGrid la publican incompleta y no determinista: dicen que la reputación mueve la capacidad, pero ninguna da a la vez la cifra de cada estado y el umbral que lo dispara. Son opacas Postmark y Brevo, que solo la nombran para su infraestructura de envío.

API	**Estados publicados**	**Umbral**	**Efecto con cifra**	**Veredicto**
Postmark	No consta	No consta	No consta	opaca
Resend	Ninguno: amplía el caudal a remitentes de confianza, a petición	No	No	incompleta y no determinista
SendGrid	Ninguno: bloqueo temporal 403 por peticiones erróneas	No	No	incompleta y no determinista
MailerSend	Cuatro con nombre	No: indicios como los 429, sin umbral ni ventana	En dos de los cuatro, y la pausa del envío en *Sending paused*	incompleta y no determinista
Brevo	No consta	No consta	No consta	opaca
Amazon SES	Tres valores de `EnforcementStatus`	No: aumento automático con un criterio sin umbral	No, salvo la pausa del envío en `SHUTDOWN`	incompleta y no determinista
Mailchimp	Niveles en el panel	No	No en las cuentas de pago	incompleta y no determinista
Mailgun	Ninguno: los caudales suben con la reputación	No	No	incompleta y no determinista

Tabla 6.4: La reputación en las ocho APIs del catálogo. Completa: el proveedor da la capacidad de cada estado. Determinista: dice qué hace cambiar de estado y con qué umbral. Opaca: los registros de la API no recogen que la reputación cambie su capacidad.

Efecto sobre las herramientas. El motor ya devuelve una fila por estado, así que leer la capacidad por estado no le exige ningún cambio. Lo que cambia es qué se le puede preguntar. Con el disparador declarado, una consulta puede separar los estados que el cliente elige de los que le impone el proveedor, que es la diferencia entre una capacidad contratable y una contingencia. Con la declaración única, el recuento de

evidencia deja de contar copias. Y hace falta una corrección más, que es del motor y no del esquema: rechazar la clave que no reconoce en lugar de convertirla en estado. Con la revisión del motor usada en este trabajo, una clave desconocida no produce ningún error, y el estado fantasma solo se ve leyendo la respuesta. El motor responde mal, además, en un listado al que le falta el campo para preguntar por estado de la suscripción. Tampoco ese fallo es del esquema, porque los canales de cálculo leen bien lo que la Datasheet declara.

6.4.4. P-07. El sobrecoste declara su tope

El caso. Desde marzo de 2026, SendGrid limita el envío mensual a diez veces el volumen contratado. Una cuenta con el plan Essentials de 50.000 correos puede enviar 500.000 al mes, y por encima recibe un error de créditos agotados [57]. El dato no está en la página de precios ni en el artículo de sobrecostes. Está en uno de resolución de errores.

Qué hace v0.3. `overage_cost` es un precio unitario sin límite de cantidad. La Datasheet declara los 50.000 correos incluidos y el precio de cada correo extra, y eso se lee como capacidad ilimitada a ese precio. El tope solo cabe en el registro de procedencia, en prosa, donde ningún consumidor lo evalúa. La guía define el campo como un objeto con precio y valor, y no le da ningún subcampo que acote la cantidad [43].

La traza. Pedida al motor la recomendación de presupuesto para 600.000 correos sobre ese plan, el motor responde 734,95 USD y declara el objetivo alcanzable en 59 segundos. El punto de control por debajo del tope, 400.000 correos, devuelve 474,95 USD con la misma forma: el motor cruza los 500.000 sin que nada cambie. El motor sabe negarse cuando el esquema se lo permite: con un presupuesto de 100 USD declara el objetivo inalcanzable. La respuesta falsa no viene del motor, viene del documento que lee.

Propuesta. Que `overage_cost` admita un máximo opcional, expresado como factor sobre el volumen contratado o en unidades absolutas. Con él, la Datasheet de SendGrid declara diez veces el plan y el motor deja de prometer un volumen que el proveedor rechaza. La extensión es retrocompatible, porque un documento que no declare el campo se lee igual que con v0.3. La extensión emparenta con F-13, que pide que el sobrecoste declare su modo. Las dos juntas le darían al sobrecoste un tope y un modo, en lugar del precio unitario abierto de v0.3. Lo que se retiene por encima de una cuota sin cobrarlo es otra carencia, y la trata P-09 (Apartado 6.4.5).

6.4.5. P-09. La cuota declara qué hace con lo que la supera

El caso. La cuota del plan gratuito de Brevo es de 300 correos al día. Al agotarla, el correo transaccional no se corta en seco: «once you reach your daily limit, up to 1,000 additional emails are held in a retry queue. Emails beyond this queue are not delivered» [46]. La cuota tiene así dos umbrales, el de la cifra publicada y el de la cola, y lo que ocurre entre los dos solo vale para el correo transaccional. El literal no dice si

el cliente recibe algún error por los correos que la cola ya no admite, y la cola no se ha observado (Apartado 5.2.5).

Qué hace v0.3. De los campos de la cuota, solo `overage_cost` habla de su borde [43]. Sin él, la cuota se lee como un muro. Con precio cero, se lee como un exceso que no se cobra ni se acaba. Ninguna de las dos lecturas es la de Brevo. La Datasheet declara los 300 correos sin sobrecoste, que en v0.3 se escribe igual que un muro.

La traza. Con la Datasheet sin manipular, el motor devuelve para esa cuota 300 correos al día sin sobrecoste y sin rastro de la cola. Es la misma forma que devuelve para la prueba gratuita de SendGrid, que sí es un muro. El caso de control es el plan Essentials de SendGrid: la Datasheet le declara sobrecoste, y el motor lo devuelve con su precio. El motor distingue un muro de un exceso que se cobra, pero no un muro de una cola, porque la Datasheet no tiene cómo decirlo.

Propuesta. Que la cuota admita un bloque `overflow` con las unidades que retiene por encima de su valor y lo que hace con ellas. Si falta, la cuota es un muro, y un documento que no declare el bloque se lee igual que con v0.3. Si hay sobrecoste, sigue en `overage_cost`, y el bloque nuevo solo describe la franja que no se cobra. La propuesta emparenta con P-04, porque las dos piden que el esquema diga qué clase de límite declara, además de su número. El Extracto de código 6.4 muestra la forma sobre el caso de Brevo.

Extracto de código 6.4: P-09: una cuota que retiene lo que la supera hasta un tope fijo (caso Brevo, plan gratuito, correo transaccional).

```
quota_email_daily_free:
  value: 300
  unit: emails
  period: {value: 1, unit: DAY}
  overflow:              # P-09: lo que la cuota hace al superarse
    capacity: 1000       # envios retenidos en la cola de reintento
    behavior: queued     # en cola, y el exceso no se entrega
```

Frontera con P-07. P-07 acota lo que se cobra por encima de la cuota, y P-09 describe lo que se retiene sin cobrarlo. Las dos tocan el mismo borde, y reunirlas en una sola forma es decisión de la guía. F-27 también toca ese borde, con un exceso que cambia el plan, pero no promueve (Sección 6.8).

6.4.6. P-08. Un cupo que cuenta destinatarios distintos y no se repone

El caso. La cuenta de MailerSend sin aprobar puede escribir a dos direcciones distintas, y solo a esas dos [58]. El cupo es de la cuenta, no un tope por petición. Solo lo levanta la aprobación, y la documentación no le da ventana. El listado de destinatarios devuelve cero, así que el cupo no se lee por ese canal. Los sondeos de este trabajo lo encontraron alcanzado sin haber enviado nunca un correo, y de ahí se infiere que lo gastaron intentos que no enviaron nada. Qué petición lo gastó no se determinó.

Qué hace v0.3. Una cuota tiene valor, unidad y periodo, y aquí fallan los tres. La unidad no es correos ni peticiones, sino direcciones distintas. No hay periodo, porque

el cupo no se renueva. Y, según esa inferencia, lo consume el intento y no el envío. El esquema no distingue los dos. Tampoco es uno de los recursos que pide F-11: un dominio o un token se tienen o no se tienen, y este cupo se gasta. La Datasheet lo deja en el registro de procedencia, en prosa. La guía da como ejemplos de unidad de una cuota peticiones, correos, SMS, registros, gigabytes y tokens, y sus formatos estándar añaden horas y minutos [43]. Ninguno cuenta entidades distintas. Una unidad nueva, como la `MBs` del catálogo, no bastaría: la cuota seguiría sin decir su ámbito, su falta de periodo ni qué la consume.

La traza. El motor lee la Datasheet del catálogo y atribuye al estado no aprobado una capacidad acumulada de 110 correos en un mes, frente a los 100 de su cuota mensual. Y declara alcanzable un objetivo de tres destinatarios, sin coste y en cero segundos. La medición del 14 de septiembre no envió ni un correo, y las dos cuotas modeladas quedaron intactas. El control confirma que el motor sabe declarar inalcanzable un objetivo con esa misma Datasheet. La prueba tiene un alcance acotado: esa respuesta no sería falsa en una cuenta recién abierta. Lo es en cuanto el cupo se ha gastado, que es el estado en que quedó la cuenta de este trabajo tras probar la API sin la aprobación.

Propuesta. Que una cuota pueda declarar cuatro cosas que v0.3 no declara: que su unidad son entidades distintas, que su ámbito es la cuenta, que no tiene periodo, y qué la consume, si el intento o el resultado. En el catálogo, el cupo es de MailerSend: las otras siete APIs se comprobaron y no lo tienen (Tabla 6.1), y por eso la propuesta entra por traza.

Frontera con P-06. Este cupo se registró al principio dentro de F-18, porque aparece en la cuenta sin aprobar. Comparten dónde salen, no qué piden. F-18 es un problema de selector: el esquema sabe decir el número de cada estado y no sabe decir cuál manda ni qué dispara el cambio. F-21 es un problema de dimensión, porque el límite no cabe ni con selector ni sin él. Se separan para que ninguna carencia cuente dos veces y cada propuesta pida una sola forma.

6.4.7. F-07 recurre y queda fuera

Los *add-ons* que modifican dimensiones de capacidad aparecen en las ocho APIs. En las cuatro primeras la IP dedicada exige un plan mínimo, y en Postmark, Resend y MailerSend también un umbral de volumen. Las tres que publican precio lo dan al mes. Postmark cobra 50 USD por la IP dedicada y la condiciona a superar 300.000 mensajes al mes y a estar en Pro. Resend cobra 30 USD y la condiciona a superar 3.000 mensajes al día y a estar en Scale. SendGrid la incluye en Pro y Premier, pero no en Essentials, y cobra 30 USD por cada IP adicional. Y la página de precios de MailerSend la reserva a Enterprise, para cuentas que envían más de 100.000 correos a la semana, y no publica precio. Las páginas de producto que la cobran a 50 USD desde 500.000 correos al mes quedan como incoherencia registrada. En las cuatro últimas la forma cambia de una API a otra. Brevo la cobra al año y la ofrece como complemento del plan Professional [53]. Amazon SES la cobra por mes y por IP [44]. Mailchimp la ofrece por 29,95 USD al mes sin umbral de volumen publicado [55]. Mailgun cobra 59

USD al mes por cada IP adicional y da acceso a ella desde los 50.000 correos [54]. El elemento no soportado recurre en las ocho y aun así no se propone extensión.

La razón es de alcance y no de evidencia. La Datasheet no tiene bloque de *add-ons* porque el esquema los deriva al canal web, que este trabajo no modela por la decisión de alcance tomada en la Sección 4.1.2. Proponer aquí un bloque de *add-ons* sería resolver en el canal API un problema que pertenece a la vinculación entre los dos canales, que el trabajo que introdujo el modelo dejó explícitamente como línea futura [1]. Lo que este catálogo aporta es un argumento con ocho casos para que esa vinculación se aborde, no una extensión unilateral que la haga innecesaria.

6.5. Defectos de contrato

Las ocho Datasheets del catálogo superan el validador de formato de este trabajo, que comprueba su estructura contra la guía del esquema. Sometidas a la herramienta de verificación, ninguna de las ocho es plenamente consumible por PRIME4API. La Tabla 6.5 recoge el resultado de la verificación por uso.

Datasheet	**Correctas**	**Con error**	**Causa de los errores**
Postmark	11	10	Diez por valor ausente (P-04)
Resend	8	13	Cinco con el error del valor ausente (P-04) y ocho con el del precio no público (P-05). Cada causa sola da doce o trece
SendGrid	8	13	Trece por precio no público (P-05)
MailerSend	20	1	Uno por curva acumulada sobre un solo caudal
Brevo	9	13	Trece por precio no público (P-05)
Amazon SES	20	1	Uno por curva acumulada sobre un solo caudal
Mailchimp Transactional	8	10	Diez por valor ausente (P-04)
Mailgun	20	1	Uno por curva acumulada con el mismo mensaje que MailerSend y Amazon SES

Tabla 6.5: Resultado de ingerir las ocho Datasheets del catálogo en PRIME4API con la herramienta de verificación. Diecisiete comprobaciones fijas por fichero más una por plan. El validador de formato de este trabajo daba cero errores en las ocho.

La distancia entre el validador de formato, que no da ningún error, y la columna de errores de la tabla es el argumento de esta sección. Un fichero puede ser conforme a la norma y dejar sin responder operaciones enteras del motor, porque entre el esquema y su consumidor hay un contrato implícito que la norma no escribe.

Mailchimp Transactional y Mailgun se sometieron a la herramienta el 24 de septiembre de 2026, con PRIME4API en la revisión `8e6011c`, después que las otras seis. Mailchimp da diez errores, con la conversión de un valor nulo que la Tabla 6.5 atribuye a P-04 (Apartado 6.5.3), el defecto que la dejaba sin respuesta en catorce de las diecisiete llamadas al motor de la Sección 5.4. Mailgun contestó a las diecisiete llamadas y la herramienta le encontró un error. Su caso muestra que responder a esas llamadas no equivale a superar las comprobaciones.

Las filas de MailerSend, Amazon SES y Mailgun son de otra naturaleza, y conviene despacharlas aquí porque no producen propuesta de esquema. El único error de MailerSend lo causa una dimensión de `workload` acotada solo por caudal. El tope de 50 MB del cuerpo del lote no tiene cuota propia. El motor construye entonces un escenario con un caudal y ninguna cuota, y falla al dividir milisegundos por una duración sin convertirla. Solo cae la curva acumulada, y el resto de la superficie pasa. Es el caso normal de F-06: el proveedor solo publica un tope de tamaño. SendGrid lo esquivó mientras su endpoint de envío repetía el *rate* del plan, porque con dos caudales el motor toma otro camino. El catálogo dejó de repetir límites el 14 de septiembre de 2026 (Sección 5.3). Desde entonces la herramienta solo registra en SendGrid los trece errores del precio no público, y su escenario de un solo caudal no se ha aislado. El remedio cabe en una línea del motor. Amazon SES reproduce el fallo con la misma traza, y al ejecutar sus escenarios uno a uno el fallo alcanza a 52 de 84.

Mailgun reproduce el mismo mensaje de error, con 20 de 21 comprobaciones correctas (Tabla 6.5). Su Datasheet declara dos dimensiones de tamaño que ninguna cuota acota: 52 MiB en `/v3/{domain_name}/messages` y 25 MB en `/v3/{domain_name}/bounces/import`. El envío lleva además el *rate* del endpoint, y con dos caudales el motor tomaría el otro camino, el que convierte la duración. La importación de rebotes solo recibe el del plan, que es la forma que produce el fallo en MailerSend. El escenario concreto no se ha aislado, y atribuirle la misma causa es inferencia. Sin las Datasheets de MailerSend y de Amazon SES, el error de Mailgun no tendría ninguna causa candidata.

6.5.1. Un defecto que no produce error: la cuota que no cambia de unidad

El caso anterior falla y se ve. Este responde, y la respuesta es incompleta. MailerSend declara en su plan Hobby dos cuotas en unidades distintas, 5.000 correos al mes y 1.000 peticiones al día, y su endpoint de envío declara además un `workload` de hasta setenta correos por petición. Ese factor es el que el motor usa para convertir entre las dos unidades.

Pedida la curva en correos, la respuesta trae el caudal y la cuota mensual, y no la diaria. Pedida en peticiones, trae los tres límites: la cuota diaria tal cual y la mensual dividida por el factor. El motor sí cruza las unidades, y lo hace con el factor que la Datasheet declara. De las tres conversiones que ese factor permite hace dos. Multiplica el caudal de peticiones por k para darlo en correos, y divide la cuota mensual de correos por k para darla en peticiones. La cuota diaria de peticiones no la multiplica por nada.

La consecuencia se mide en la propia respuesta. Con $k = 1$ la curva en correos

alcanza los 5.000 del plan a los 41 minutos. Con la cuota diaria dentro, los alcanza el quinto día, que es la escalera de la Figura 5.2. Son dos órdenes de magnitud de diferencia, y quien pregunta en la dimensión del dominio se lleva la cifra optimista.

Nada de esto produce un error. Las comprobaciones de la fila de MailerSend en la Tabla 6.5 pasan, salvo la de la curva acumulada, que es el caso anterior. La Datasheet declara los tres límites con su unidad y la pieza que haría falta ya está en el fichero, porque el factor de conversión es el mismo que el motor aplica en los otros dos casos. Por eso esto es defecto de contrato y no carencia de expresividad: el remedio es del consumidor, y el esquema no tiene nada que añadir.

El renderizador agrava el hueco en vez de compensarlo. Pedido el documento sin filtrar dimensión, el motor devuelve cuatro figuras y las cuatro son de correos. La dimensión de peticiones, la única que lleva la cuota diaria, solo se dibuja si se pide con un parámetro cuyo valor por defecto es falso, porque esa serie suele ser redundante con un límite nativo en peticiones. En MailerSend no lo es. Y filtrada esa dimensión sin el parámetro, el servicio responde con un documento de tres kilobytes y ninguna figura dentro. El endpoint de datos, en cambio, devuelve las dos dimensiones sin que se le pida nada, de modo que la ausencia está solo en el dibujo.

A diferencia del caso anterior, que Amazon SES reproduce y cuyo mensaje repite Mailgun, este solo se ve con la Datasheet de MailerSend. MailerSend es la única de las ocho que declara cuotas en dos unidades, así que en las otras siete no hay conversión entre cuotas que el motor pueda omitir.

6.5.2. Una ambigüedad previa: la forma de `workload`

Antes de las dos propuestas conviene despejar un caso que no es carencia del modelo sino desacuerdo entre sus fuentes. La guía del esquema documenta `workload` como un mapa [43]. La Datasheet de referencia del mismo repositorio lo serializa como una lista, y el trabajo que introdujo el modelo la usa también en sus extractos y en el anexo de SendGrid [1]. PRIME4API tolera las dos formas al calcular y solo acepta la lista al navegar, de modo que una Datasheet escrita siguiendo la guía produce números correctos y falla al recorrerla, que es justo lo que hacen el agente conversacional y la plataforma de publicación.

Un mapa solo puede declarar una métrica, y el TFM de Gavira Sánchez sostiene que «cualquier endpoint cuya petición consuma métricas distintas de forma simultánea necesita declararlas por separado» [1]. La lista es entonces un requisito del modelo, y las ocho Datasheets del catálogo usan esa forma. La propuesta es corregir la guía y, opcionalmente, normalizar en el motor para que los ficheros escritos contra la redacción actual no queden inservibles.

6.5.3. P-04. Distinguir «sin límite» de «límite no publicado»

El caso. Resend no publica el *rate* de su plan negociado, y su Datasheet deja el campo que lo declara presente y vacío. Ahí el campo vacío dice que el límite existe y su

valor no se conoce. En Postmark y en Mailchimp Transactional la misma forma recoge lo contrario: la fuente no pone tope en la unidad del campo. Postmark anuncia para `/email/bulk` «as many recipients as you need» (el literal de P-03, Apartado 6.4.2) mientras la carga total no pase de 50 MB, y ese endpoint no se ha medido. Mailchimp niega por escrito el tope de destinatarios en la API, «there's no recipient limitation», y acota el JSON de cada llamada por debajo de 10 MB. En el sondeo, una llamada con 5.000 destinatarios devolvió HTTP 200 sin rechazo, aunque con esta cuenta la respuesta no informa de ningún tope (Apartado 5.2.8). En los dos casos la cota publicada está en otra métrica, los bytes de la llamada (P-03, Apartado 6.4.2). Leído solo el campo, el consumidor no puede saber cuál de las dos lecturas le toca, y esa es la ambigüedad que P-04 pide cerrar.

P-04 se apoya en las dos vías, cada una sobre un conjunto distinto de APIs. La recurrencia es la de F-01, un *rate* que existe sin cifra, en cuatro APIs (Tabla 6.1). En Mailchimp la aporta la cuota horaria de la cuenta de pago, que el proveedor no cifra, y no el tope de destinatarios. La traza es la de un campo vacío en tres Datasheets, y el campo no siempre es un *rate*.

La traza. El motor convierte esos valores a número sin comprobar nulos y falla con un error de conversión de tipo. En Postmark caen por esta causa las cuatro operaciones de cálculo, la navegación por rangos del factor de consumo y las cinco comprobaciones de la superficie de análisis del motor: los diez errores del fichero. En Resend el mismo error aparece en cinco comprobaciones y el de P-05 en las otras ocho. El fichero lleva a la vez el campo vacío y las cadenas `custom` de P-05, y un control retira cada defecto por separado. Con el campo vacío relleno, la herramienta da trece errores, todos por la cadena `custom`. Con las cadenas sustituidas por números, da doce, todos por el campo vacío. Cualquiera de los dos defectos tumba por sí solo casi las mismas comprobaciones que los dos juntos.

Mailchimp Transactional reproduce el fallo en la tanda de diecisiete llamadas al motor de la Sección 5.4. El motor responde con `HTTP 500` y el mismo error de conversión en catorce de las diecisiete, y solo contestan las consultas que no calculan tiempo. Con el máximo relleno en memoria, sin tocar el fichero, el error pasa a ser una división por cero. La causa es un segundo campo vacío, el periodo de la franquicia de 500 correos de la cuenta de demostración. Con los dos rellenos, la misma llamada devuelve `200`. La herramienta de verificación le da los diez errores con el mismo mensaje (Tabla 6.5).

Por qué el esquema no ofrece salida. Las dos alternativas disponibles tienen coste. Omitir el campo hace que la Datasheet afirme implícitamente que no hay límite. En Resend, donde el límite existe, omitirlo es lo contrario de la verdad: produciría una respuesta optimista y falsa. En Postmark y en Mailchimp, omitirlo coincidiría con lo que dice el proveedor en la unidad del campo, pero el consumidor leería la misma ausencia que deja un límite que sí existe. La otra alternativa, declararlo vacío, deja sin responder operaciones enteras del motor. La Datasheet de Amazon SES toma la primera para el caudal de una cuenta en producción: no lo declara y anota la ausencia en su registro `not_found`. No hay tercera opción en v0.3, y esta traza, repetida en tres Datasheets, muestra que la decisión de modelado tiene consecuencias medibles. El registro de ausencias que usan las ocho Datasheets dice qué falta en el documento, pero no en qué campo. Que el catálogo haya necesitado ese registro apoya la propuesta,

porque la marca diría en el propio campo qué falta. Con la marca, las dos lecturas se separan. Resend la lleva en su campo. Postmark y Mailchimp pueden omitirlo, y la omisión dice lo mismo que su fuente: no hay tope en esa unidad. La cota en bytes la declara la Datasheet de Mailchimp como carga aparte, y la de Postmark necesita P-03 (Apartado 6.4.2).

Propuesta. Una marca explícita de valor no publicado junto al campo, de modo que el consumidor pueda omitir esa dimensión y seguir evaluando el resto en lugar de abandonar el documento. La corrección se reparte: el esquema declara la marca y el motor la respeta. La distinción entre las dos lecturas tiene que alcanzar también al periodo de una cuota, que es el segundo campo vacío de Mailchimp. Ahí, como en su tope de destinatarios, no falta una cifra: la franquicia de la cuenta de demostración no caduca ni se renueva (Apartado 5.2.8).

6.5.4. P-05. Declarar un plan cuyo precio existe y no es público

El caso. Resend no publica cifras del plan Enterprise, ni el precio ni la cuota, y la Datasheet recoge en los dos campos el valor `custom`. Es lo que publica el proveedor, y es la forma que la guía documenta para un valor negociable [43].

La traza. El motor intenta convertir esa cadena a número y falla. El fallo no queda acotado al plan afectado: como la evaluación recorre todos los planes del documento, un único valor no numérico impide evaluar también los tres planes publicados al detalle, que son los que cualquiera va a consultar. Con ese error fallan las siete operaciones de evaluación y el precio del plan Enterprise, que son ocho de los trece errores de esa Datasheet. Si se rellena el campo vacío, los trece errores son de esta causa. SendGrid y Brevo lo reproducen con trece errores cada una, todos con el mismo mensaje: la cadena `custom` no se convierte a número (Tabla 6.5).

Por qué importa. En siete de las ocho APIs del catálogo hay al menos un precio que no se publica. En seis es un plan, y en Mailchimp el precio por encima de cuatro millones de correos al mes. Las Datasheets de Resend, SendGrid y Brevo escriben ese plan con `custom`, como admite la guía, y el motor falla en las tres. Las demás lo dejan fuera de sus planes.

Propuesta. Dos cambios separables. En el esquema, una forma explícita de declarar un plan que tiene precio aunque no lo publique. Sustituiría a la cadena `custom` que la guía admite para un valor negociable y que el motor no sabe leer. En el motor, aislar el fallo por plan para que un plan no evaluable devuelva su error y los demás sigan calculándose. El segundo cambio es independiente del primero y por sí solo recupera tres de los cuatro planes de la Datasheet de Resend.

6.6. Observaciones que no son carencias del esquema

La construcción del catálogo deja además un registro de observaciones numeradas que no proponen nada sobre ACM2YAML, y que se recogen aquí por dos razones: su destinatario es otro, y varias condicionan cómo se construyen las Datasheets que sí alimentan las propuestas.

O-06 es sobre la especificación formal que publican los proveedores. La OpenAPI de SendGrid, en el repositorio `twilio/sendgrid-oai`, declara mil elementos como tope de `personalizations`, `cc` y `bcc`, y no declara ninguno para `to` ni menciona el tamaño máximo del mensaje o los caudales. El tope total de mil destinatarios solo existe en la prosa de la referencia de la API [57]. La de MailerSend, publicada en SwaggerHub, tampoco declara el tope de `to`, omite otros cinco campos del cuerpo y no menciona la respuesta `429`, y anuncia además un `200` donde la documentación del proveedor dice `202` [58]. La de Brevo, un fichero archivado de 207 rutas, no menciona el caudal ni una sola vez, y la documentación en prosa sí lo publica [59]. En Amazon SES, el contrato legible por máquina de la versión 2 no declara ningún límite (Apartado 5.2.6), y la página de límites sí los publica [63]. La consecuencia es de método: una Datasheet construida solo desde la especificación formal sería peor que la construida desde la prosa en las cuatro APIs donde se ha comprobado.

O-07 es que un mismo nombre de cabecera llega con dos formas de valor. En SendGrid `X-RateLimit-Reset` trae un número pequeño en tres endpoints y una marca de tiempo Unix en otros tres, y la documentación solo muestra la segunda forma [65]. Leer el número pequeño como segundos restantes es una inferencia, coherente con los tres instantes medidos: cada valor coincide con los segundos que faltaban para el siguiente cambio de minuto. Para una herramienta que lea cabeceras y calcule cuándo reintentar, el mismo nombre con dos formas de valor es una trampa sin aviso.

O-09 es sobre el propio ecosistema. La Datasheet de MailerSend que el repositorio de la guía trae como ejemplo define caudales de 15 y 30 peticiones por minuto que después no asigna. Al lote le asigna el de 10 en los cuatro planes, aunque el proveedor publica 15 para Starter y 30 para Professional. Declara además ventana deslizante en las cuotas mensuales, que ninguna fuente del proveedor dice [43]. La ruta del lote que usa tampoco existe. No es carencia del esquema. Son divergencias de modelado del ejemplo, y no hay evidencia de que sean cambios en el tiempo, porque el propio fichero ya definía los caudales de 15 y 30. Lo que sí caduca en ese ejemplo son los precios y los sobrecostes, y en ellos se apoya el argumento de F-09 de la Sección 6.8.

O-11 es sobre el canal por el que un cliente observa su consumo. MailerSend publica la regla (cada petición descuenta una unidad de la cuota diaria) y dos formas de observarla, una cabecera en cada respuesta y un endpoint dedicado [58]. La regla se cumple y el canal va por detrás: el contador expuesto se actualiza por lotes y con retardo, y la reposición de la ventana diaria tampoco es inmediata. Once horas y veinticuatro minutos después de la medianoche que repone la cuota, el endpoint seguía sirviendo el saldo del día anterior con la fecha de reposición del siguiente. Qué lo pone al día no se midió. Aquí no hay nada que proponer, porque v0.3 no modela el canal de observación del consumo de ninguna API. Lo que hay es un límite del método de medición: un sondeo no puede usar el contador de la respuesta siguiente como coste de

la anterior, ni fiarse de él para saber en qué ventana está.

O-20 es sobre lo que cuesta un sondeo. Los sondeos de Brevo usaban un destinatario que no se puede entregar, para que el proveedor rechazara cada petición y la cifra se leyera en el cuerpo del error. Con la cabecera de modo *sandbox*, el envío devolvió `201`, porque ese modo solo valida el formato de la petición (Apartado 5.2.5). Que una petición no se pueda entregar no implica que el proveedor la rechace, y en Brevo las dos cosas se separan. Que un sondeo no envíe correo ni gaste cuota no se deduce, por tanto, de su diseño, y hay que comprobarlo en cada API.

O-22 es sobre el mismo método en Mailchimp Transactional. Con diez destinatarios o menos, la API rechaza en la respuesta el envío de una cuenta sin dominio verificado. Desde once destinatarios, o con un adjunto (el menor probado, de 5 MB), deja el envío en cola sin decir si saldrá. Por encima de ese umbral, que el proveedor no publica, deja de existir la vía de medir haciendo que el proveedor rechace la petición (Apartado 5.2.8).

Otras observaciones del registro no están en esta sección porque ya tienen sitio en el capítulo. O-08 sostiene lo que v0.3 ya ofrece en P-06, los *aliases* de estado. O-12 es la medición de la primera carencia de P-06, el disparador sin sitio propio. No valió como prueba de F-18 porque describe un punto ciego del método de verificación. Y O-10 es el fallo del motor que la Sección 6.5 trata tras la Tabla 6.5.

6.7. Compatibilidad y vía de contribución

Las nueve propuestas no tienen el mismo coste para quien ya use v0.3. Todas salvo P-02, P-03 y P-05 son retrocompatibles: añaden campos opcionales que un documento existente puede no declarar sin dejar de ser válido, y un consumidor que los ignore se comporta igual que con v0.3. P-02 y P-03 no lo son en el mismo grado. Cambian la naturaleza del factor de consumo por petición, que deja de ser un entero para pasar a ser una expresión, y eso obliga a tocar las operaciones que lo leen. P-05 sustituye la cadena `custom` que la guía admite para un valor negociable, y las Datasheets de Resend, SendGrid y Brevo, que la usan, tendrían que cambiar ese campo. La corrección de la forma de `workload` es una rectificación de la guía y no una extensión, y su coste recae sobre los documentos escritos contra la redacción actual.

La ausencia de mecanismo se establece sobre el texto de la guía, que es lo que pide la condición L3 a las que entran por traza. En P-06 se contrastó también el código de referencia, que recoge el `workload` por alias. La guía no tiene ninguno de los mecanismos que piden P-06, P-07, P-08 y P-09. P-06 pide un sitio para el disparador del estado de la suscripción y de la reputación, y P-07 un tope para el sobrecoste. P-08 pide una cuota sin ventana que cuente entidades distintas, y P-09 una forma de decir qué hace una cuota con lo que la supera. La propia guía deja la puerta abierta a este tipo de trabajo, porque declara el esquema preliminar y anticipa que habrá APIs donde haya que adaptar o extender los campos [43].

Dos matices sobre P-06. Su parte de documentación no cuesta nada, porque describe lo que la implementación de referencia ya hace. Su corrección del motor sí

rompe algo: rechazar la clave desconocida invalida cualquier documento que la lleve, y ninguna Datasheet del catálogo la lleva. Sin esa corrección, una clave desconocida no produce error, y solo se detecta al leer la respuesta del motor.

Sobre la vía de contribución, el trabajo se atiene a una regla previa: ninguna discrepancia se comunica sin comprobar antes si ya está reportada. El rastreo realizado sobre los cuatro repositorios del ecosistema devuelve un solo *issue* pertinente al esquema, abierto en el motor por Ramón Gavira Sánchez, que pide aceptar listas de *rates* y cuotas [7]. Como el código ya lo implementa, merece un comentario en ese *issue* antes que uno nuevo. El repositorio de la guía tiene veinticinco *issues*, todos peticiones de modelado de una API concreta. El rastreo es del 10 de agosto de 2026 y no cubre lo aparecido después. Ninguna de las carencias propuestas hasta esa fecha estaba reportada. Lo que el capítulo documenta después sigue sin contrastar contra los *issues* abiertos, y ese contraste es condición previa para comunicarlo.

6.8. Lo que queda fuera

Entre los veinticuatro elementos no soportados promovidos hay dieciséis sin número de propuesta. Estos dieciséis entran por recurrencia, así que su patrón es del dominio. Un número de propuesta pide además una forma concreta y su contraste con el texto de la guía, y esta sección dice por qué cada uno de ellos no lo tiene. Las ocho primeras propuestas salen de las cuatro primeras APIs, y la ampliación a ocho solo añade P-09. La numeración sigue, por tanto, el orden en que se trabajó cada forma y no la recurrencia: F-11, en seis APIs, no lleva número, y F-21, en una, sí. F-07 y F-08 ya quedan justificadas en el Apartado 6.4.7 y en la Sección 6.3.

F-04 plantea si pertenece a la capacidad contractual un límite que el proveedor declara y que la respuesta a la petición no refleja. Es una pregunta conceptual sobre la definición del modelo y no un problema de serialización. Por eso se discute aquí y no se convierte en propuesta. La variante que añade MailerSend la agudiza en lugar de resolverla. El endpoint de lote acepta con un `202` un envío que la validación diferida rechaza entero. El estado final de los dos lotes medidos da cinco errores de validación y ningún correo enviado. Un consumidor que mida su consumo por la respuesta de la petición cuenta cinco correos que nunca salieron. También está medida en Amazon SES y en Mailchimp Transactional. Amazon SES declara una petición por segundo en sus acciones de gestión [63], y respondió con `200` a doce peticiones seguidas a 5,7 por segundo. Mailchimp Transactional declara que el JSON de cada llamada debe ocupar menos de 10 MB, y respondió con `200` a una llamada de unos 40 MB (Apartado 5.2.8). El registro de elementos no soportados esboza una salida: distinguir en el esquema el límite publicado del aplicado, con la fuente de cada uno. En Amazon SES y en Mailchimp Transactional la cifra publicada y la aplicada difieren, así que la pregunta tiene además una cara práctica: hay que elegir qué cifra va al fichero. Este capítulo no hace esa elección, que queda como trabajo futuro (Capítulo 7).

F-05 corresponde a una dimensión que el propio esquema declara fuera de su versión operativa, así que proponerla sería reiterar una línea futura ya reconocida.

El registro de elementos no soportados esboza una forma para once de las doce

restantes, y la de F-12 la enuncia este capítulo. La Tabla 6.6 recoge las doce. Este capítulo no las trabaja ni las contrasta con el texto de la guía, y por eso no llevan número de propuesta. F-09, en cinco APIs, y F-10, en tres, tienen además argumentos que no dependen de la frecuencia. El de F-09 es que el propio repositorio de la guía conserva una Datasheet con valores caducados a los cinco meses. El de F-10, que un servicio con dos unidades de consumo distintas obliga a modelar uno de los dos productos sin poder declarar en el fichero que solo cubre ese. Con el catálogo cerrado, las formas de las doce siguen sin fijar. Fijarlas es trabajo futuro (Capítulo 7), y la tabla deja el punto de partida.

Dónde	Id	Carencia	APIs	Forma esbozada
Documento	F-09	El documento caduca entero, y sus datos caducan uno a uno	5/8	Fecha de verificación por afirmación en el registro de procedencia
Documento	F-10	Dos productos de precio con unidades de consumo distintas	3/8	Varios catálogos por servicio, o un campo que acote qué producto se modela
Plan	F-11	Límites de recursos por plan, que no son consumo	6/8	Bloque de recursos en el plan, con nombre, valor y unidad y sin periodo
Plan	F-13	El sobrecoste es opcional y no existe en el plan gratuito	5/8	Modo del sobrecoste declarado en el plan
Plan	F-14	El plan tiene fecha de caducidad	2/8	Vigencia del plan y plan al que cae al expirar
Precio	F-26	Un precio que depende de la geografía y una sola divisa en el fichero	3/8	Precio como lista de ofertas, cada una con su divisa
Cuota	F-12	La recepción consume la cuota de envío	2/8	Declarar que un evento entrante reenviado descuenta de la cuota de envío, con un factor que depende de la ruta
Carga	F-34	Dos topes de tamaño sobre la misma llamada, uno del mensaje y otro de la petición	2/8	Carga que declara el objeto medido además de la unidad
Canal	F-15	Dos canales, dos `workload` distintos para la misma cuota	5/8	Canales declarados, o carga declarada por canal
Acceso	F-28	Una lista de IP autorizadas decide si la llamada entra	3/8	Condiciones de admisión declarables en el plan o en la credencial
Bloque nuevo	F-37	Un tope de capacidad que fija el cliente, no el proveedor	2/8	Bloque de límites que configura el cliente, fuera de la capacidad contractual
Operación	F-22	Una credencial que solo se gestiona desde la web	4/8	Qué canal ofrece cada operación, registrado por operación

Tabla 6.6: Promovidos sin número de propuesta y con una forma esbozada en el registro de elementos no soportados de cada API, salvo F-12, cuya forma enuncia este capítulo. Dónde: la parte del esquema que tocaría la forma. APIs: recuento sobre las ocho del catálogo. Las formas no se han trabajado ni contrastado con la guía en este capítulo.

F-22 queda en la frontera del objeto del esquema. Describe qué operaciones ofrece cada canal de un producto, y el esquema describe precios, cuotas y límites. Su forma entra en la tabla porque el registro la esboza, pero no amplía la capacidad contractual que el esquema modela.

Los once elementos no soportados sin promover cuentan todos en una sola API, así que solo podían entrar por traza. La columna de vía de las Tablas 6.1 y 6.2 dice

qué condición le falta a cada uno según el veredicto de su fila. En F-23 y F-25 falla L3, porque un mecanismo de la guía cubre al menos una de sus dos mitades, y en F-29 la comprobación de L3 está pendiente. En F-32, F-33, F-36 y F-38 falta L2, porque ninguna ejecución archivada muestra su efecto sobre la Datasheet sin manipularla. En F-27 y F-31 también falta L2, aunque hay ejecución archivada: ninguno de los valores que devuelve se puede dar por falso. En F-30 y F-35 faltan L2 y L3. En F-32, F-33 y F-36 L3 no tiene veredicto escrito, y en F-38 está en disputa. En F-27, el exceso que cambia el plan, la ejecución archivada devuelve un coste plano, y ninguna fuente dice qué pasa al agotar el volumen en los planes de pago. Una curva plana no distingue un muro de un cambio de plan, y el disparador, que cuenta contactos, no entra en una consulta hecha en correos. La diferencia con F-24 está en la fuente: allí un literal publica la cola de reintento, y el muro que devuelve el motor lo contradice.

F-36 y F-38 piden que una cuota o un *rate* admita varios valores de la misma unidad. Es el *issue* del motor que abrió Ramón Gavira Sánchez en abril de 2026, «Soportar listas de rates y cuotas en datasheet» [7], y que la Sección 6.7 ya menciona. El motor ya lo implementa en su código, y la guía no lo documenta. Como mucho, el catálogo le aporta dos casos, y ninguno cumple el criterio, porque ninguno tiene ejecución archivada. Un tercero, F-17, descansaba en una frase del centro de ayuda que la página de precios del mismo proveedor contradice, y se retiró.

Y queda una pregunta que el motor plantea y este capítulo no convierte en propuesta. Una Datasheet con límites en dos unidades no declara cuál de las dos mide el plan, y quien la consume elige la dimensión al preguntar. El Apartado 6.5.1 la trata como defecto del consumidor, porque el factor de conversión está en el fichero y el motor lo aplica en los otros dos casos. Si se quisiera que lo dijera la Datasheet, la forma sería declarar en qué dimensión mide el proveedor. La misma pregunta aparece en el precio. La Tabla 5.18 del capítulo anterior responde al tiempo mínimo con dos cifras, según se trate la cuota con sobrecoste como un tope duro o se la expanda con un presupuesto. Nada en el fichero dice cuál de los dos regímenes describe el plan. Son F-13 y F-16 vistos desde el motor. F-16 ya tiene forma en P-07, y F-13 queda con los promovidos sin forma fijada de la Tabla 6.6.

6.9. Respuesta a la pregunta de investigación

La RQ2 pregunta si ACM2YAML es suficientemente expresivo para representar la capacidad contractual de las APIs de correo, y qué extensiones requiere. Con las ocho APIs del catálogo, la respuesta a la primera mitad es que no. La respuesta a la segunda son las nueve propuestas de la Tabla 6.3, siete de expresividad y dos de contrato, más la rectificación de la guía.

Interesa dónde está la insuficiencia, porque no está solo donde cabía esperar. La dimensión que falta y que el catálogo documenta, los resultados por petición, aparece en siete APIs (F-05), y queda fuera porque el esquema ya la reconoce como línea futura. Las carencias de P-01, P-02 y P-03 no hablan de dimensiones que falten en el metamodelo. Gavira Sánchez ya declara que v0.3 captura dos de las cuatro dimensiones de capacidad del metamodelo [1]. Hablan de las dimensiones que sí existen: se declaran

con valores escalares mientras el dominio las vende por listas, las consume por productos o las acota en otra métrica. La que sostiene P-06 es de otro tipo. El dominio añade un eje que el esquema solo sabe declarar dentro de cada endpoint y repetido en todos: la capacidad depende también del estado de la suscripción y de la reputación. El mecanismo que la guía ya ofrece para ese eje llega hasta la cifra y no hasta lo que la dispara. Con las ocho APIs, ese eje aparece en seis, y F-18, el elemento no soportado de P-06, promueve por recurrencia.

El exceso sobre una cuota es el tercer frente. P-07 le pide un tope y P-09 que declare qué hace con lo que la supera, y las dos tocan el mismo borde de la cuota. El modo del sobrecoste, que pide F-13, queda esbozado en la Tabla 6.6. Fuera de ese frente, P-08 pide una cuota que cuente entidades distintas y no se reponga. Los elementos no soportados de P-07, P-08 y P-09 entran por traza, con una sola API cada una.

Y las dos carencias de contrato no son de expresividad en absoluto: el esquema admite el valor que describe al proveedor, y el consumidor de referencia falla al leerlo. En P-04 el campo vacío no dice si el límite no se publica o no existe, y el motor falla al convertirlo a número. En P-05 la guía admite la cadena `custom` para un valor negociable, y el motor falla al leerla.

El capítulo entrega, por tanto, nueve propuestas y una rectificación de la guía sobre un inventario de treinta y cinco elementos no soportados y las ocho APIs del catálogo. La Tabla 6.6 recoge otros doce promovidos con una forma esbozada y sin trabajar en este capítulo. De ellos, F-22 queda en la frontera del objeto, porque describe qué operaciones ofrece cada canal. El resto amplía el inventario de lo que el esquema no expresa, pero sin forma fijada no llega a propuesta. Los cuatro promovidos restantes, F-04, F-05, F-07 y F-08, quedan fuera por los motivos de la Sección 6.8. Al pasar de cuatro APIs a ocho, ninguna propuesta pierde su prueba. El límite de la respuesta es de validez externa: la recurrencia se mide sobre ocho APIs de un solo dominio y con un solo consumidor de referencia. Qué parte de la respuesta vale para otras APIs de correo y para otros dominios queda para el Capítulo 7.

7. Conclusiones

El capítulo se organiza del siguiente modo. En primer lugar se responde a las cuatro preguntas de investigación, y a continuación se evalúa cada contribución frente a los objetivos de los que sale. Después se agrupan las amenazas a la validez y se describe el paquete de replicación. Por último se recoge el trabajo futuro y se cierra con las conclusiones técnicas del trabajo.

7.1. Respuesta a las preguntas de investigación

Cada pregunta se reproduce con el enunciado de la Sección 1.3, y la respuesta remite al apartado que la sostiene.

RQ1. *¿Qué APIs de correo deben incluirse en un análisis representativo del dominio, y bajo qué criterios reproducibles?*

El catálogo reúne ocho APIs, y entre ellas aparece al menos una vez cada una de las cuatro formas de cobro que fija el primer criterio de la consulta (Sección 4.1). Dos de esas formas son planes, por volumen mensual o con un límite diario adicional, y las otras dos son el pago por uso puro y el producto vendido como añadido de otro. La Sección 4.7 cierra la RQ1 con la condición completa: el conjunto debe cubrir esas cuatro formas, y sus APIs deben diferir en el rasgo diferenciador de límites que pide el cuarto criterio (Sección 4.1.2). Ese rasgo es, por ejemplo, un *rate* publicado por *endpoint*, un caudal negociado no publicado o un límite diario. Cada API se documentó con la trazabilidad que exige la Sección 4.3. Ninguna API se sustituyó, así que la lista de reserva (Apartado 4.1.3) no llegó a usarse. El Capítulo 5 analiza las mismas ocho de las que parte la selección.

Los criterios son reproducibles, pero no en la misma medida. La cobertura se comprueba contra las páginas de precios de cada proveedor. La consulta al modelo de lenguaje que completó la lista está en el Anexo A, que reproduce el prompt literal y la respuesta íntegra, con una ficha que declara además lo que no consta. El repositorio de datos que acompaña al trabajo conserva el registro original. Constan también la fecha de la consulta y el modelo usado, pero repetir la consulta no garantiza la misma respuesta.

La respuesta tiene límites de dos clases. La primera clase está en las fuentes: la primera del criterio, las APIs de un Trabajo Fin de Máster previo del mismo programa, no pudo ejecutarse, y la consulta la sustituyó. La segunda clase son tres premisas de la respuesta que la construcción corrigió en la ventana de toma de datos (Apartado 4.1.4). Amazon SES entró en el catálogo el 1 de septiembre de 2026 como pago por uso puro, y el proveedor vendía planes desde el 21 de julio de 2026 (Apartado 5.2.6). Su plan à la carte no cobra suscripción y cobra por correo (Tabla 5.9), y de ahí se infiere que la forma sigue representada. La respuesta presentaba además la combinación de cuota diaria y mensual como aportación de Brevo, y esa combinación ya estaba en el plan

Free de Resend. La Sección 4.7 recoge además otros dos límites de la primera clase: los cinco criterios solo decidieron cuatro de las ocho APIs, y el tamaño del catálogo no se deriva de la cobertura.

La tercera premisa es la de Mailgun. La respuesta registrada la eligió como el caso limpio de *rate* documentado, pero su documentación de desarrollador no cifra el umbral (Sección 6.3). El único caudal que el proveedor publica para su API REST, 1.500 peticiones por segundo, está en sus preguntas frecuentes y acotado a una IP de cliente. Esa premisa es la que justificó su entrada, porque la respuesta le atribuye la misma forma de cobro que a Postmark, SendGrid y MailerSend. La construcción encontró otro rasgo de límites: Mailgun es la única del catálogo que contesta por escrito sobre la concurrencia de su API REST (Sección 4.7).

RQ2. *¿Es ACM2YAML suficientemente expresivo para representar la capacidad contractual de las APIs de correo? ¿Qué extensiones requiere?*

Con las ocho APIs del catálogo, la respuesta a la primera pregunta es que no. La respuesta a la segunda son las nueve propuestas de la Tabla 6.3, siete de expresividad y dos de contrato, más una rectificación de la guía (Sección 6.9). Las carencias de P-01, P-02 y P-03 están en dimensiones que v0.3 ya modela: el esquema las declara con valores escalares mientras el dominio las vende por listas, las consume por productos o las acota en otra métrica. P-06 añade el estado de la suscripción y la reputación, un eje que el esquema solo sabe declarar dentro de cada *endpoint* y repetido en todos, y que aparece en seis de las ocho APIs. P-07 y P-09 tratan el exceso sobre la cuota, y P-08 pide una cuota que cuente entidades distintas y que no se reponga. Los dos defectos de contrato, P-04 y P-05, son de otro tipo: el esquema admite el valor y el consumidor de referencia falla al leerlo. El dominio pide más de lo que llega a propuesta. La Tabla 6.6 recoge otros 12 elementos promovidos con la forma esbozada y sin fijar, y la Sección 6.8 deja fuera otros cuatro con su motivo. Si esta respuesta vale fuera del catálogo se discute en la Sección 7.3.3.

RQ3. *¿Cómo se relacionan los límites de uso y los rates en las APIs de correo, y qué implicaciones tiene esa relación para el consumidor?*

La cuota es la restricción activa cuando el *rate* y la cuota tienen valor en la Datasheet, publicado o medido, y la cuenta está sana (Sección 5.3). Eso ocurre en seis de las ocho APIs. En su ventana, el *rate* admite entre 2,6 veces la cuota (MailerSend Professional 2M) y 129.600.000 veces (el Free de Postmark, con el *rate* en ámbar). Las otras dos APIs quedan fuera de la regla. SES no publica la cuota ni el caudal de producción, así que la relación no se puede calcular antes de sacar la cuenta del entorno de pruebas. En Mailchimp Transactional la relación se invierte: en la cuenta de prueba, la franquicia de 500 correos con un tope de 25 por hora tarda 20 horas en agotarse, y el caudal es lo único que fija el tiempo.

Para el consumidor, la relación tiene tres implicaciones. En esas seis APIs puede dimensionar el volumen sin mirar el *rate*, siempre que la cuenta esté sana. El estado de la suscripción y la reputación explican más de la capacidad observada que el plan, y en tres APIs el sondeo midió la capacidad de un estado y no la del plan. Y solo en la mitad del catálogo, la que ofrece un endpoint de introspección, puede verificar la relación antes de enviar.

RQ4. *¿Cómo pueden compararse las APIs de correo de forma sistemática para responder a preguntas de decisión que involucran simultáneamente requisitos funcionales y no funcionales?*

Las APIs se comparan con un catálogo de 32 preguntas de decisión (29 no funcionales y 3 funcionales) construido sobre la gramática de Pricing Intelligence [2]. El procedimiento resuelve cada pregunta API a API y registra la fuente de cada respuesta (Secciones 5.4 y 5.5). Por su clase, el instrumental del canal API alcanza 25 de las 29 no funcionales. Las preguntas de HARVEY4API, 4 de esas 25, se clasifican por especificación, porque la herramienta no se ha ejecutado. Sobre cada Datasheet, la cobertura va de 16 preguntas en Brevo a 22 en Resend (Sección 5.5). La matriz de *features* por plan responde dos de las tres funcionales, fuera del esquema, y la tercera, la retención, espera sitio en él. La parte funcional se resuelve por lectura porque el canal API que fija el alcance del Capítulo 4 no tiene instrumento para ella. Una de las funcionales mezcla un requisito funcional con uno no funcional: pregunta qué API da IP dedicada al menor precio para 200.000 correos al mes. Se responde componiendo a mano la matriz y la Datasheet (Apartado 5.5.1), y responderla con instrumento exigiría el vínculo entre el iPricing del canal web y la Datasheet del canal API. En 12 preguntas hay respuesta para las ocho APIs a la vez.

7.2. Evaluación de las contribuciones frente a los objetivos

La Tabla 7.1 da el estado de cada objetivo y de la contribución que le corresponde, con las secciones de la memoria que lo sostienen. Los párrafos que siguen justifican cada estado. Empiezan por OBJ-03 y OBJ-04, donde el Capítulo 1 sitúa el componente de investigación.

Objetivo	**RQ**	**Contrib.**	**Estado del objetivo**	**Estado de la contribución**	**Secciones que lo sostienen**
OBJ-01	RQ1	C-01	cumplido en parte	cumplida en parte	4.1, 5.2
OBJ-02	RQ2–RQ4	C-01	cumplido	cumplida en parte	4.3, 6.1, 6.5
OBJ-03	RQ2	C-02	cumplido en parte	cumplida	6.3, 6.8, 6.9
OBJ-04	RQ3	C-03	cumplido	cumplida	5.3, 5.6
OBJ-05	RQ4	C-04	cumplido	cumplida	5.4, 5.5

Tabla 7.1: Estado de cada objetivo específico y de la contribución que le corresponde. RQ: la pregunta de investigación de la que se deriva el objetivo. OBJ-02 sirve de medio a RQ2, RQ3 y RQ4, porque el Capítulo 1 presenta el catálogo como condición necesaria. C-01 aparece dos veces porque responde a OBJ-01 y a OBJ-02. Los párrafos de esta sección justifican cada estado.

OBJ-03 y C-02. El criterio de promoción de la Sección 4.5, aplicado en el Capítulo 6, promueve 24 de los 35 elementos no soportados del inventario (Sección 6.3). Las nueve propuestas de la Tabla 6.3 dan forma a ocho de esos promovidos, y la novena, P-05, no sale de ningún elemento del inventario. La Tabla 6.6 recoge otros 12 promovidos con una forma esbozada y sin fijar. Los cuatro promovidos restantes quedan fuera por los motivos de la Sección 6.8. OBJ-03 queda cumplido en parte porque pide la extensión que corresponde a cada carencia identificada, y los promovidos de la Tabla 6.6 no tienen todavía la forma fijada. De las nueve, siete son de expresividad y dos de contrato, P-04 y P-05. C-02 se cumple: cada una de las siete de expresividad responde a un caso real que v0.3 no representa, con su prueba de recurrencia o de traza. Las dos de contrato quedan fuera del enunciado de C-02, porque v0.3 sí representa el valor y es el consumidor de referencia el que falla. La rectificación corrige la guía y no extiende el esquema.

OBJ-04 y C-03. La Sección 5.3 caracteriza la relación entre límites y *rates* en las ocho APIs. En seis de ellas la cuota es la restricción activa, y las Tablas 5.16 y 5.17 lo muestran con el cociente entre *rate* y cuota de cada plan. La lectura geométrica de la capacidad [34] se aplica con las curvas $C(t)$ que PRIME4API dibuja en local para cuatro APIs (Figura 5.1). En Mailgun, con dos contadores sobre la misma ventana, y en Mailchimp Transactional, con una cuota sin ventana, la capacidad no cabe en una curva de ese tipo (Sección 5.3). La caracterización responde a la tercera observación del Capítulo 3: en seis APIs la cuota enmascara al *rate*, y en Mailchimp Transactional el *rate* enmascara a la cuota. OBJ-04 y C-03 se cumplen, con las salvedades de la Sección 7.3.

C-01 frente a OBJ-01 y OBJ-02. OBJ-02 se cumple, con la salvedad de la Sección 7.3.1. Todas las Datasheets del catálogo siguen ACM2YAML v0.3, y cada valor lleva la URL de procedencia y la fecha de acceso, junto a la cita literal que lo sostiene (Sección 4.3). Superan además el validador de formato de este trabajo, aunque usan dos formas que la guía no define: la unidad `MBs` y la lista `not_found` (Sección 6.1). OBJ-01 se cumple en parte. La primera fuente del criterio no se ejecutó, y la premisa por la que entró SES no se sostuvo. Un tercero tampoco puede repetir la lista entera, que es lo que OBJ-01 le pide, porque la consulta registrada no garantiza la misma respuesta. Postmark, además, no entra por ninguna de las tres fuentes: es el piloto con el que se calibró el método (Sección 4.6). C-01 se cumple en parte. El catálogo sale revisado del ciclo de cada API, está versionado y cada dato remite a su fuente. La contribución a SPHERE está preparada [5]: los ocho pricings en Pricing2YAML pasan el analizador que usa la plataforma, y las 20 limitaciones que Pricing2YAML deja fuera quedan documentadas. La carga por la interfaz de SPHERE no se ha probado, y la publicación depende de los responsables de la plataforma. Que sea reutilizable por el resto de herramientas del ecosistema, la otra promesa de C-01, se sostiene en parte. PRIME4API lee en local las ocho Datasheets, pero ninguna es plenamente consumible (Tabla 6.5). En MailerSend, Amazon SES y Mailgun falla una sola comprobación, y el Capítulo 6 no deriva de ese fallo ninguna propuesta de esquema. HARVEY4API no se ha ejecutado.

OBJ-05 y C-04. El catálogo de preguntas existe, y el procedimiento se ha repetido sobre las ocho APIs (Sección 5.5). OBJ-05 y C-04 se cumplen con tres límites declarados. Las preguntas de HARVEY4API se clasifican por especificación, y las pre-

guntas de PRIME4API de Mailchimp Transactional están calculadas a mano. Dos de las funcionales se responden leyendo la matriz de *features*, y la tercera, la retención, espera sitio en el esquema. El vínculo con el iPricing que daría instrumento a la pregunta funcional mixta queda como trabajo futuro.

7.3. Amenazas a la validez

Cada apartado del Capítulo 5 declara sus límites al cerrar, y el Capítulo 6 hace lo mismo con los suyos. Esta sección los agrupa en cuatro ejes de validez, uno por apartado. Cada amenaza parte de un caso real y dice qué sesga. Después dice qué la mitiga en este trabajo y qué queda abierto.

7.3.1. Validez de constructo

La validez de constructo pregunta si lo medido es lo que se dice medir. El sondeo midió en tres APIs un estado de la suscripción y no el plan. En MailerSend era una cuenta Sandbox sin aprobar. En SES era el entorno de pruebas, y en Mailchimp Transactional una cuenta de prueba. En Brevo y Mailgun la cuenta era del plan gratuito. Brevo describe, sin cifra, un estado de la suscripción pendiente de aprobación, y Mailgun un arranque de cuenta nueva cuyo caudal sube con la reputación. Lo medido vale para el plan gratuito en el estado de esa cuenta, y la Datasheet de Mailgun referencia el caudal medido desde sus cuatro planes. El sesgo está en la capacidad observada, que es la del estado. MailerSend publica 120 peticiones por minuto para una cuenta sana, y la cuenta Sandbox sin aprobar del sondeo devolvió 10, la cifra que el proveedor publica para la reputación bajo revisión. Lo mitiga que las tres Datasheets declaren el estado medido, y que la de MailerSend conserve aparte el valor documental del plan. Queda abierto medir en cuentas de producción, y salir de esos estados exige alta de cuenta o gasto, que este trabajo no decide.

En Postmark y Mailgun el *rate* se adoptó desde una cabecera, sin llegar al rechazo. El sondeo de Postmark envió 200 peticiones sin obtener un 429, y en Mailgun alcanzarlo exige 80.000 peticiones en 10 segundos, una carga para la que el sondeo no se diseñó. En SendGrid, el caudal de envío se publica en una página de preguntas frecuentes y ninguna cabecera lo deja observar, así que solo se contrastaría con una campaña de carga. El sesgo está en la cifra: la de Postmark y Mailgun es la que anuncia la cabecera, la de SendGrid es la de la página, y en ninguna consta la que el proveedor aplica al rechazar. Lo mitiga que las cifras de Postmark y Mailgun vayan en ámbar en su Datasheet. La respuesta a RQ3 no cambia mientras el *rate* aplicado siga por encima de lo que la cuota deja enviar en su ventana. Es inferencia y no medida. Queda abierto el cuarto peldaño de la escalera de validación, la medición formal [48].

La ventana de las cuotas tiene cuatro APIs en rojo, cuatro en ámbar y ninguna en verde para todas sus cuotas, y la ventana del caudal no la declara ninguna API. La unidad de la cuota tiene respaldo en cuatro APIs, y en dos de ellas, Brevo y SES, lo aporta un sondeo. En las otras cuatro no consta que un destinatario cuente como un correo a efectos de contador. El sesgo alcanza a los dos pasos del cálculo de RQ3: el

cociente entre *rate* y cuota se toma sobre la ventana de la cuota, y los correos salen de multiplicar peticiones por destinatarios. Lo mitiga que el semáforo de confianza dé color a la ventana de cada cuota. El respaldo de la unidad lo declara API a API el Capítulo 5, así que ninguno de los dos datos se presenta como publicado sin serlo. Queda abierto medir la unidad en Postmark, MailerSend, Mailgun y Mailchimp Transactional, las cuatro sin respaldo.

La trazabilidad de cada valor descansa en su cita literal, y la cita solo se comprueba de forma mecánica si se archivó el volcado de su página. En MailerSend, la última comprobación registrada del verificador de citas deja 70 citas sin volcado en la carpeta contra la que se contrastan. La mayoría son de páginas leídas sin guardar su volcado, y el resto son literales del sondeo, cuyo volcado es de otra fecha. Postmark no tiene ningún volcado archivado, así que ninguna de sus citas se puede comprobar de forma mecánica (Sección 7.4). El sesgo está en la comprobación: las de páginas sin volcado no se pueden contrastar con la página tal como era el día de acceso. Lo mitiga que la URL y la fecha de acceso consten en cada valor. Queda abierto releer esas páginas y archivar su volcado.

Lo que este trabajo llama carencia de v0.3 depende de con qué se contrasta la Datasheet, y no existe validador oficial de Datasheets (Sección 6.1). La conformidad se comprueba con el validador de formato de este trabajo y con la verificación por uso sobre PRIME4API. Las dos formas que la guía no define, `MBs` y `not_found`, están declaradas. Un validador propio corre el riesgo de medir contra su propia lectura de la guía. Lo mitiga, en los defectos de contrato y en los elementos no soportados que entran por traza, usar como consumidor el motor del ecosistema, que no es de este trabajo. Las carencias que entran solo por recurrencia, las de P-01, P-02, P-03 y P-06, descansan en la lectura de la guía que hace este trabajo. P-06 muestra el riesgo. La primera pasada, hecha sobre la guía, dio por fuera del mecanismo el cambio de carga por petición, y el código lo admitía. La ausencia de mecanismo se establece sobre el texto de la guía, y en P-06 se contrastó también el código de referencia (Sección 6.7). El rastreo de las carencias ya reportadas es del 10 de agosto de 2026. Queda abierto contrastar las carencias con el código de la guía y con lo reportado después del 10 de agosto.

El catálogo de preguntas de decisión es de este trabajo, y la cobertura del instrumento depende de qué preguntas incluye. El sesgo tiene dirección: quien escribe las preguntas conoce el instrumental, y eso puede subir la cobertura. Es inferencia, pero basta para no leer la cobertura de este catálogo, un 78 %, contra el 54 % que el paper de Pricing Intelligence [2] obtiene sobre su banco de 56 plantillas. Lo mitiga que las preguntas sigan su gramática, y que ni sus clases ni su denominador se muevan al pasar de cuatro APIs a ocho. Queda abierto declarar el criterio con el que se generó el catálogo y por qué se detiene en 32 preguntas.

7.3.2. Validez interna

La validez interna pregunta si los resultados dependen de las Datasheets o del montaje con que se obtienen. La herramienta de verificación clona PRIME4API sin fijar la revisión, así que el resultado de la Tabla 6.5 depende de la revisión que se

clone (Sección 6.1). La copia local del motor se clonó el 4 de septiembre de 2026, y su último cambio es del 18 de agosto. Un cambio del motor entre dos ejecuciones puede cambiar el número de errores sin que cambie la Datasheet. Las ocho ejecuciones no son simultáneas: Mailchimp Transactional y Mailgun se sometieron a la herramienta el 24 de septiembre de 2026, después de las otras seis. Las de Mailchimp Transactional y Mailgun registran la misma revisión `8e6011c` que las de Postmark, Resend, SendGrid y MailerSend. Lo mitiga, en parte, que esas seis ejecuciones con revisión registrada puedan repetirse contra ella. Para las de Brevo y SES no hay mitigación, porque su revisión no consta. Queda abierto fijar la revisión en la herramienta y registrarla en cada ejecución.

El Capítulo 6 recoge tres observaciones que son límites del propio método de sondeo (Sección 6.6). En MailerSend el contador expuesto se actualiza por lotes y con retardo, y qué lo pone al día no se midió (O-11). Un sondeo no puede usar el contador de la respuesta siguiente como coste de la anterior. En Brevo, un destinatario que no se puede entregar no asegura el rechazo, porque con la cabecera de modo *sandbox* el envío devolvió `201` (O-20). En Mailchimp Transactional, desde 11 destinatarios o con un adjunto (el menor probado, de 5 MB), el envío queda en cola, y deja de existir la vía de medir por rechazo (O-22). Las tres observaciones pueden hacer pasar por coste o por límite una señal que no lo es. En Brevo lo mitiga un sondeo previo que mide qué guarda deja ver el rechazo, junto a un libro de envíos que descuenta de un tope declarado cada destinatario de una petición aceptada. En MailerSend, la segunda ejecución del sondeo leyó la cuota al final, sin fiarse del contador de cada respuesta. En Mailchimp Transactional no hay mitigación: por encima del umbral de la cola, el sondeo no mide. Queda abierto qué pone al día el contador de MailerSend y cómo medir en Mailchimp Transactional por encima del umbral de la cola.

PRIME4API se ejecutó como servicio en local, no en el despliegue del ecosistema [1]. Las 10 preguntas P de Mailchimp Transactional están calculadas a mano y sin contraste, porque el motor falla en 14 de sus 17 llamadas por dos huecos que el esquema admite. El sesgo está en el instrumento: un despliegue con otra imagen podría responder de otro modo. Lo mitiga que el servicio local sea la misma imagen de PRIME4API que despliega SPHERE en local (Sección 5.3). No consta qué imagen ejecuta el despliegue del ecosistema. En Mailchimp Transactional lo mitiga que el fallo esté declarado como fallo del instrumento, con los dos huecos que lo causan. Queda abierto repetir las ejecuciones en el despliegue del ecosistema.

Los valores modelados dependen también de qué fuente se toma cuando dos discrepan. La regla de la página de precios solo resuelve las divergencias en que la página es uno de los lados: 37 de las 82 registradas. No resuelve las siete en que la página se contradice a sí misma. Las demás siguen registrándose sin elegir. Si la página publica un redondeo, el valor modelado es el redondeo, como el sobrecoste de Essentials 50K en SendGrid, que pasa de 0,00133 a 0,0013 USD.

Las Datasheets y el recuento de recurrencia los hicieron agentes de inteligencia artificial, y el agente que construye no es el que verifica (Anexo A). El riesgo es un dato que el agente dé por leído sin que la fuente lo contenga. Lo mitigan la cita literal de cada valor, que una comprobación automática contrasta con la página archivada, y un agente verificador distinto del constructor, que vuelve a abrir cada fuente. Queda

abierto reunir lo que refutaron las verificaciones de las ocho APIs.

7.3.3. Validez externa

La validez externa pregunta hasta dónde valen los resultados fuera del catálogo y de la toma de datos. La respuesta a RQ2 se mide sobre ocho APIs de un solo dominio y con un solo consumidor de referencia (Sección 6.9). Para otras APIs de correo vale el «no» de la primera pregunta, que no depende del tamaño de la muestra porque basta un caso que v0.3 no represente. Al pasar de cuatro APIs a ocho, además, ninguna propuesta perdió su prueba. Lo que depende de la muestra es la promoción, y no depende igual en todas las propuestas. Es inferencia: las que recurren en más APIs, P-03 en las ocho, P-06 en seis y P-01 en cinco, son las que menos cambiarían con otro catálogo. P-07, P-08 y P-09 descansan en una API cada una, así que su prueba no depende de la muestra y su alcance en el dominio queda sin medir. Los cuatro elementos no soportados que cumplen el umbral con dos APIs justas son los que otro catálogo movería primero (Sección 7.3.4).

P-04 y P-05 dependen del consumidor, no del dominio. Es inferencia y no medida: cualquier Datasheet que declare un valor vacío o la cadena `custom` fallaría ante la misma revisión del motor. Las demás propuestas nacen de casos de correo y no se han probado en otros dominios, así que la extensibilidad que anticipa el Capítulo 1 queda sin probar. HARVEY4API no se ha ejecutado, y no hay un segundo consumidor que confirme los defectos de contrato. Lo mitiga que cada propuesta lleve el caso que la motiva, de modo que otro dominio puede comprobarla caso a caso. Queda abierto repetir el inventario en otro dominio y con un segundo consumidor.

La respuesta a RQ3 tiene una parte que no depende de la muestra. Que la cuota sea la restricción activa sale del cociente, grande por construcción cuando el proveedor cobra por volumen mensual y publica un *rate* por segundo o por minuto (Sección 5.6). Las demás afirmaciones dependen de lo observado, y el paso de cuatro APIs a ocho debilitó tres de ellas. El sesgo está en esas afirmaciones observadas, que otra muestra puede no repetir. La comparación de RQ4 tiene el mismo límite: su base son ocho APIs y el estado de la suscripción desde el que se midió. Para las afirmaciones observadas y para la comparación de RQ4 no hay mitigación en este trabajo. Queda abierto repetir las afirmaciones observadas sobre otras APIs de correo.

La condición de representatividad de la Sección 4.7 deja fuera de la prueba las formas de cobrar que no están en las ocho APIs. Las cuatro formas salen del primer criterio de la consulta, que fijó el autor, y no de un censo del sector. La respuesta registrada nombra además en la reserva una variante de cobro por créditos prepagados que el catálogo no incluye. Lo mitiga que el criterio sea explícito y comprobable contra las páginas de precios, y que la consulta que completó la lista esté registrada. Queda abierto si repetir la consulta daría la misma lista.

El catálogo modela solo el producto transaccional de cada proveedor, y el de *marketing* queda fuera (Sección 4.1). Las conclusiones sobre límites y *rates* no se trasladan a ese producto, porque se cobra y se limita con otras unidades: Resend lo factura por contactos y no por correos enviados. Lo mitiga que el ciclo de construcción no dependa

del producto que se modela, así que el de *marketing* puede recorrerlo igual. Queda abierto construir sus Datasheets y comparar las dos respuestas.

La oferta de los proveedores cambia sin aviso, y el catálogo la describe tal como era en la ventana de toma de datos (Apartado 4.1.4). La propia selección lo muestra con el elemento no soportado de la fuente que caduca sin avisar (F-09): SES vendía planes antes de que la consulta lo describiera como pago por uso puro. Un precio o un límite correcto en la toma de datos puede haber cambiado cuando se lea la memoria. Lo mitiga que cada valor lleve su URL y su fecha de acceso (Sección 4.3). Queda abierto un procedimiento de vigencia que diga cuándo reverificar cada Datasheet.

7.3.4. Validez de conclusión

La validez de conclusión pregunta si los recuentos que sostienen las conclusiones están bien hechos y bien acotados. El recuento de recurrencia deja 31 celdas sin verificar y 1 en suspenso (Tablas 6.1 y 6.2). Postmark y Resend reúnen 15 de esas celdas. Una celda sin verificar solo puede sumar, así que cada recuento es un mínimo respecto de esas celdas, y en ellas el sesgo va en una sola dirección. Verificarlas no retira ninguna promoción, pero podría hacer recurrentes a F-31, F-35 y F-36, y pasar F-16 de la traza a la recurrencia. Confirmar en Mailgun que dos cuotas de la misma unidad se aplican a la vez haría recurrente F-38. El umbral de dos APIs se cumple justo en cuatro filas, F-12, F-14, F-34 y F-37, así que en ellas la recurrencia descansa en dos celdas. F-37 depende además de una regla provisional sobre qué variantes suman, y con otra regla bajaría a una sola API. Lo mitiga que cada celda esté marcada fila a fila en las tablas del inventario. Queda abierto verificar las 31 celdas y fijar la regla de variantes.

El recuento es del 23 de septiembre de 2026, y su fichero declara que las cuentas se comprobaron dos veces, a mano y con un script que las recalcula desde las celdas. Ese script no está en el repositorio, así que la segunda comprobación no se puede repetir. La doble comprobación acota los errores de aritmética, no el juicio sobre cada celda. El 25 de septiembre de 2026 cada registro de elementos no soportados se alineó con el recuento en la columna de su propia API, y cada celda que cambió de lectura quedó anotada junto a la anterior. Las copias de columnas de otras APIs que repiten las tablas acumulativas conservan su valor anterior. En Mailgun, por ejemplo, la celda de F-09 pasó de presente a ausente, y la de F-07, de sin verificar a presente. Siete celdas del recuento siguen sin fila en el registro de elementos no soportados de su API. Ninguna suma a una promoción: cinco son ausencias, una es parcial y la séptima es una variante que tampoco cuenta. Queda abierto versionar el script y crear esas siete filas.

Las coberturas del Capítulo 5 dependen de dos criterios de recuento. «No publicado» cuenta como respuesta cuando la pregunta lo admite, y en las preguntas de comparación una API cuenta cuando aporta su lado. El primero no se aplica igual en todas las preguntas: en Postmark cuenta en la pregunta por el *rate* máximo, y no en las que preguntan si hay límite diario o si el *rate* sube con el plan. El sesgo está en la cobertura de Postmark, que se movería en una o dos preguntas si la regla se unificara. No hay mitigación en este trabajo: la regla se dejó como estaba y quedó anotada. Queda abierto fijar esa regla en el catálogo de preguntas.

Las coberturas cuentan igual una respuesta en ámbar que una en verde. En Postmark van en ámbar 13 de sus 18 respuestas y en Mailgun 12 de 20, frente a 1 de 22 en Resend y 1 de 20 en SendGrid. El sesgo está en la comparación entre APIs: la cobertura dice cuánto se responde y no con qué confianza. Lo mitiga que el Capítulo 5 dé el reparto del ámbar por API. Queda abierto dar la cobertura de cada API separada por color.

La otra convención aritmética es la del mes, que se toma de 30 días. Con meses de 28 o de 31 días, los cocientes entre *rate* y cuota mensual cambian un 6,7 % o un 3,3 %. Con 28 días, el menor cociente de las cuentas sanas pasa de 2,6 a 2,4 y sigue por encima de uno. La única fila por debajo de uno, MailerSend Starter 1M bajo revisión, sigue por debajo con los dos meses, así que ninguna conclusión se mueve. Lo mitiga ese recálculo. En esta convención no queda nada abierto.

7.4. Paquete de replicación

El repositorio de datos que acompaña al trabajo contiene lo que sostiene los Capítulos 4 a 6, y está disponible en `https://github.com/danvelcam/tfm-pricing-apis-correo`. La Tabla 7.2 recoge qué contiene cada ruta y qué parte de la memoria sostiene.

Ruta	Contenido	Sostiene
03_datos/apis/	Una carpeta por API: Datasheet, registro de fuentes, registro de elementos no soportados (fricciones.md), acta de revisión, matriz de *features* y evidencias de los sondeos. Las copias fechadas de las páginas del proveedor no se publican: PAGINAS.md da la huella sha256 de cada una	Capítulo 5, Tablas 6.1 y 6.2
03_datos/SELECCION_APIS_2026-09-01.md	Consulta de selección: prompt literal, respuesta íntegra, fecha y modelo	Sección 4.1
03_datos/RECURRENCIA_2026-09-23.md	Recuento de recurrencia celda a celda	Tablas 6.1 y 6.2
03_datos/CRITERIOS.md	Reglas de modelado y de promoción	Capítulos 4 y 6
03_datos/DISCREPANCIAS_v03.md	Discrepancias entre la guía v0.3, PRIME4API y el repositorio del ecosistema	Sección 6.7
03_datos/preguntas/	Catálogo de preguntas de decisión	Sección 5.4
03_datos/_plantilla/	Plantilla de la Datasheet y lista de comprobación de la recolección	Sección 4.2
05_salida/_cap5/	Hechos por API, análisis y ejecuciones del motor	Capítulo 5
verificar_prime4api.py	Herramienta de verificación contra PRIME4API	Tabla 6.5
gen_datasheet_viewer.py	Validador de formato y estación de revisión	Sección 6.5
verificar_citas.py	Comprobación de cada cita literal de la Datasheet contra el volcado archivado de su página	Sección 4.3
ejecutar_prime4api_PI.py	Ejecución de las preguntas P contra PRIME4API	Tabla 5.22
gen_fig_capacidad.py	Curvas de capacidad con la biblioteca de PRIME4API	Figura 5.1
probe_*, medir_l2_*.py	Sondeos por API y comprobación de la condición de traza	Apartados de la Sección 5.2, Sección 6.1
lib_*	Bibliotecas que cargan los sondeos: lectura de credenciales, guarda de destinatarios y tope de gasto	Apartados de la Sección 5.2
tests/	Pruebas de las herramientas	—

Tabla 7.2: Contenido del repositorio de datos que acompaña al trabajo. Las herramientas, desde verificar_prime4api.py hasta tests/, están en 05_salida/_herramientas/. Sostiene: la parte de la memoria que se apoya en cada ruta.

La herramienta de verificación se ejecuta sobre una Datasheet con una sola orden desde la raíz del repositorio.

```
python3 05_salida/_herramientas/verificar_prime4api.py \
```

`03_datos/apis/postmark/datasheet.yaml`

La primera ejecución clona PRIME4API desde su repositorio público [7]. Necesita red y las dependencias de Python que lista la cabecera de la herramienta. La orden devuelve 0 si pasan todas las comprobaciones y 1 si alguna falla. Con la Datasheet de Postmark, lo esperado es la fila de la Tabla 6.5: 11 comprobaciones superadas y 10 fallidas por valor ausente. Esa fila se obtuvo con la revisión `8e6011c` del motor, cuyo último cambio es del 18 de agosto de 2026. La herramienta clona la última revisión cuando no tiene copia, así que reproducir la fila exige situar antes la copia en esa revisión (Sección 7.3.2).

El repositorio no incluye las credenciales de las APIs, que viven fuera de él. Repetir un sondeo exige cuentas propias, y el resultado depende del estado de esas cuentas (Sección 7.3.1). Tampoco incluye PRIME4API, que la herramienta clona al ejecutarse, ni las memorias de otros trabajos académicos citados en la bibliografía. Sí incluye las copias fechadas que se archivaron de las páginas de los proveedores, que son la evidencia de las citas que el verificador comprueba (Sección 7.3.1). De Postmark no se archivó ninguna.

7.5. Trabajo futuro

Los Capítulos 5 y 6 dejan cinco líneas de investigación abiertas. La primera es distinguir el límite publicado del aplicado (F-04). SES declara una petición por segundo en sus acciones de gestión y respondió con `200` a 12 peticiones seguidas a 5,7 por segundo. Mailchimp Transactional declara que el JSON de cada llamada debe ocupar menos de 10 MB, y respondió con `200` a una llamada de unos 40 MB. El registro de elementos no soportados esboza la salida, que es distinguir en el esquema los dos límites con la fuente de cada uno, y queda por elegir qué cifra va al fichero (Sección 6.8). La segunda es fijar las 12 formas esbozadas de la Tabla 6.6 y contrastarlas con la guía. La forma de la credencial que solo se gestiona desde la web (F-22) queda en la frontera del objeto del esquema, porque describe qué operaciones ofrece cada canal. La tercera es el vínculo entre iPricing y la Datasheet, que daría instrumento a la pregunta funcional mixta. Coincide con la línea «Pricing de servicios multicanal» que deja abierta Gavira Sánchez [1], y este trabajo le aporta el caso del correo. En ese caso, la multicanalidad se lee como el conjunto de interfaces de cada servicio, con la definición del NIST (Sección 3.4). La cuarta es la medición formal, el cuarto peldaño de la escalera de validación [48], que mediría en una campaña de carga el 429 de Postmark y Mailgun y el caudal de SendGrid. La quinta es ejecutar las preguntas H con HARVEY4API y repetir las preguntas P en el despliegue del ecosistema.

Dos líneas que Gavira Sánchez deja abiertas reciben de este trabajo un caso cada una [1]. Completar la serialización ACM2YAML, que pide recoger los límites de petición individual, recibe el caso de los resultados por petición, documentados en siete APIs (F-05). Validarla formalmente recibe la herramienta de verificación por uso. Lo que este trabajo aporta a SPHERE (Capítulo 3) abre otras dos. La primera es construir en la plataforma la sección de términos de consumo, con su pestaña de versiones y la procedencia de cada dato. La segunda es asociar a cada interfaz sus límites de

consumo, porque las *skills* de Ruiz López ya toman la interfaz como unidad de análisis (Sección 4.1.2). Postmark recomienda diez conexiones por IP en el relé SMTP, pero no aplica ese tope. Su API REST anuncia 5.000 peticiones por segundo en las cabeceras de respuesta. El YAML del analizador nombra cada interfaz, pero no tiene un atributo para su límite.

Quedan además cuatro tareas del propio trabajo.

- Fijar la revisión del motor en la herramienta de verificación y registrarla en cada ejecución.
- Verificar las 31 celdas del recuento y fijar la regla provisional de variantes.
- Contrastar las propuestas con el código de la guía y con lo reportado en los repositorios del ecosistema después del 10 de agosto de 2026.
- Definir un procedimiento de vigencia que diga cuándo reverificar cada Datasheet ante la fuente que caduca sin avisar (F-09).

Las demás amenazas de la Sección 7.3 dejan su parte abierta en su propio párrafo.

7.6. Conclusiones técnicas

En seis de las ocho APIs, la cuota es la restricción activa. El *rate* sigue fijando el tiempo mínimo para enviar el volumen que la cuota deja (Tablas 5.16 y 5.17), y en la cuenta de prueba de Mailchimp Transactional es lo único que lo fija.

El estado de la suscripción y la reputación explican más de la capacidad observada que el plan. En seis APIs hay al menos un estado publicado que condiciona la capacidad, y tres de ellas publican una cifra distinta por estado. Un consumidor que compare planes sin mirar el estado compara cifras que su cuenta puede no tener. P-06 da a ese eje el disparador y la declaración única que el mecanismo de *aliases* de la guía no tiene.

Con las ocho APIs, ACM2YAML v0.3 no basta, y las carencias de P-01 a P-03 están en dimensiones que ya modela. El tope de carga expresado en otra métrica, que motiva P-03, aparece en las ocho APIs, y la dimensión que lo contiene, el `workload`, ya existe en v0.3. La segunda observación del Capítulo 3 partía de las dimensiones que v0.3 no serializa, como la concurrencia o los resultados por petición. Los resultados por petición aparecen en siete APIs (F-05), y no llevan propuesta porque el esquema ya los reconoce como línea futura. En las tres propuestas, el esquema declara con valores escalares dimensiones que el dominio vende por listas, consume por productos o acota en otra métrica.

La conformidad con la guía no basta para que el motor consuma la Datasheet. Todas las Datasheets del catálogo superan el validador de formato de este trabajo, y ninguna de las ocho es plenamente consumible. En P-04 y P-05 el esquema admite el valor que describe al proveedor, y el consumidor de referencia falla al leerlo. Por eso el método añade la verificación por uso a la de formato.

Con las ocho Datasheets, el instrumental del canal API alcanza por clase 25 de las 29 preguntas no funcionales. Cada Datasheet devuelve su propia cobertura, y esa diferencia entre APIs es el primer resultado de la comparación, antes de cualquier cifra de precio. Con cuatro Datasheets había respuesta común en 17 preguntas. Las cinco que se pierden con ocho caen por dos huecos de la página de precios de Brevo: ningún plan de pago documenta qué ocurre al superar la cuota, y los escalones intermedios de volumen no tienen precio.

Con ocho APIs, el dominio muestra formas que las cuatro primeras no tenían. Con cuatro APIs, el catálogo describía la parte regular del dominio. Mailchimp Transactional no tiene planes ni tope de destinatarios en la API, y SES no tiene cuota incluida. En tres APIs no hay sobrecoste. La primera observación del Capítulo 3 pedía confrontar el modelo con un conjunto representativo de las APIs de un sector, elegido con un criterio declarado, incluidas las que no encajan con naturalidad. La respuesta es parcial en dos sentidos. Las ocho APIs cubren cuatro formas de cobrar, no el sector entero (Sección 7.3.3). Y la Datasheet las describe con dos formas que la guía no define, mientras el motor no consume sin error ninguna de las ocho.

8. Bibliografía

[1] R. Gavira Sánchez, «Pricing Intelligence: Modelo y motor de análisis de la capacidad de pricing de APIs,» Máster en Ingeniería del Software: Cloud, Datos y Gestión TI, Trabajo Fin de Máster, Universidad de Sevilla, 2026.

[2] F. J. Cavero, A. García-Fernández, J. A. Parejo, M. Resinas y A. Ruiz-Cortés, «Pricing Intelligence: Rethinking IS Engineering in Volatile SaaS Environments,» en *Advanced Information Systems Engineering (CAiSE 2026)*, ép. Lecture Notes in Computer Science, vol. 16558, Springer, 2026, págs. 42-58. DOI: 10.1007/978-3-032-28110-4_3

[3] A. García-Fernández, J. A. Parejo, P. Trinidad y A. Ruiz-Cortés, «Automated Analysis of Pricings in SaaS-Based Information Systems,» en *Advanced Information Systems Engineering (CAiSE 2025)*, ép. Lecture Notes in Computer Science, vol. 15702, Springer, 2025, págs. 223-239. DOI: 10.1007/978-3-031-94571-7_13

[4] F. J. Cavero, J. A. Parejo, J. C. Alonso y A. Ruiz-Cortés, «A-MINT: An LLM Pipeline for Automated Modeling of iPricings from SaaS Pricing Pages,» en *Service-Oriented Computing (ICSOC 2025)*, ép. Lecture Notes in Computer Science, vol. 16321, Springer, 2026, págs. 83-91. DOI: 10.1007/978-981-95-5015-9_6

[5] ISA Group, *SPHERE: A SaaS Platform for API Pricing Evaluation and Management*, 2026. visitado 2 de ago. de 2026. dirección: https://sphere.score.us.es/

[6] D. Ruiz López, «HARVEY4API: Modelado, extracción y análisis de pricing de APIs,» En preparación, Trabajo Fin de Máster, Universidad de Sevilla, 2026.

[7] R. Gavira Sánchez, *PRIME4API: motor de análisis de la capacidad contractual de APIs*, 2026. visitado 10 de ago. de 2026. dirección: https://github.com/API-TaskForce/prime4api

[8] Mailtrap, *6 Best Transactional Email Services Compared*, 2026. visitado 2 de ago. de 2026. dirección: https://mailtrap.io/blog/transactional-email-services/

[9] MailSlurp, *Transactional Email Service Comparison and Evaluation Guide*, 2026. visitado 2 de ago. de 2026. dirección: https://www.mailslurp.com/blog/transaction-email-services-compared/

[10] A. R. Hevner, S. T. March, J. Park y S. Ram, «Design Science in Information Systems Research,» *MIS Quarterly*, vol. 28, n.º 1, págs. 75-105, 2004.

[11] K. Peffers, T. Tuunanen, M. A. Rothenberger y S. Chatterjee, «A Design Science Research Methodology for Information Systems Research,» *Journal of Management Information Systems*, vol. 24, n.º 3, págs. 45-77, 2007. DOI: 10.2753/MIS0742-1222240302

[12] Universidad de Sevilla, *Trabajo Fin de Máster (51790028). Máster Universitario en Ingeniería del Software: Cloud, Datos y Gestión de las Tecnologías de la Información*, 2026. visitado 29 de sep. de 2026. dirección: https://www.us.es/estudiar/que-estudiar/oferta-de-masteres/master-universitario-en-ingenieria-del-software-cloud/51790028

[13] Ministerio de Educación, Cultura y Deporte, *Real Decreto 1125/2003, de 5 de septiembre, por el que se establece el sistema europeo de créditos y el sistema de calificaciones en las titulaciones universitarias de carácter oficial y validez*

en todo el territorio nacional, Boletín Oficial del Estado, núm. 224, de 18 de septiembre de 2003, 2003. visitado 29 de sep. de 2026. dirección: https://www.boe.es/buscar/act.php?id=BOE-A-2003-17643

[14] Agencia Digital de Andalucía, *Instrucción 1/2023, de 4 de mayo de 2023, de la Agencia Digital de Andalucía sobre perfiles, precios de referencia y desglose de costes en contratos de bienes y servicios TIC*, 2023. visitado 29 de sep. de 2026. dirección: https://www.juntadeandalucia.es/haciendayadministracionpublica/apl/pdc_sirec_documentacion/rest/descargar/documento/129086

[15] Xataka, *Huawei MateBook D 16 (2024), análisis*, 2024. visitado 29 de sep. de 2026. dirección: https://www.xataka.com/analisis/huawei-matebook-d-16-2024-analisis-caracteristicas-precio-especificaciones

[16] A. García-Fernández, J. A. Parejo, F. J. Cavero y A. Ruiz-Cortés, «Racing the Market: An Industry Support Analysis for Pricing-Driven DevOps in SaaS,» en *Service-Oriented Computing (ICSOC 2024)*, ép. Lecture Notes in Computer Science, vol. 15405, Springer, 2025, págs. 273-288. DOI: 10.1007/978-981-96-0808-9_19

[17] A. García-Fernández, J. A. Parejo, F. J. Cavero y A. Ruiz-Cortés, «Trends in Industry Support for Pricing-Driven DevOps in SaaS,» *IEEE Transactions on Services Computing*, vol. 19, n.º 1, págs. 726-737, 2026. DOI: 10.1109/TSC.2025.3634801

[18] D. Benavides, S. Segura y A. Ruiz-Cortés, «Automated analysis of feature models 20 years later: A literature review,» *Information Systems*, vol. 35, n.º 6, págs. 615-636, 2010. DOI: 10.1016/j.is.2010.01.001

[19] A. García-Fernández, J. A. Parejo y A. Ruiz-Cortés, «Pricing4SaaS: Towards a Pricing Model to Drive the Operation of SaaS,» en *Intelligent Information Systems (CAiSE 2024 Forum)*, ép. Lecture Notes in Business Information Processing, vol. 520, Springer, 2024, págs. 47-54. DOI: 10.1007/978-3-031-61000-4_6

[20] A. García-Fernández, J. A. Parejo, P. Trinidad y A. Ruiz-Cortés, *SaaS Pricing Evaluation Corpus for Temporal Representation & Analysis (SPECTRA)*, Zenodo, Versión 1.1.0, 2025. DOI: 10.5281/zenodo.17237051

[21] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine, 2000.

[22] OpenAPI Initiative, *OpenAPI Specification v3.2.0*, 2025. visitado 2 de ago. de 2026. dirección: https://spec.openapis.org/oas/v3.2.0.html

[23] A. Gámez-Díaz, P. Fernández y A. Ruiz-Cortés, «An Analysis of RESTful APIs Offerings in the Industry,» en *Service-Oriented Computing (ICSOC 2017)*, ép. Lecture Notes in Computer Science, vol. 10601, Springer, 2017, págs. 589-604. DOI: 10.1007/978-3-319-69035-3_43

[24] A. Gámez-Díaz et al., «The Role of Limitations and SLAs in the API Industry,» en *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019)*, ACM, 2019, págs. 1006-1014. DOI: 10.1145/3338906.3340445

[25] A. Andrieux et al., «Web Services Agreement Specification (WS-Agreement),» Open Grid Forum, inf. téc. GFD-R-P.107, 2007.

[26] A. Gámez-Díaz, P. Fernández y A. Ruiz-Cortés, «Automating SLA-Driven API Development with SLA4OAI,» en *Service-Oriented Computing (ICSOC 2019)*,

ép. Lecture Notes in Computer Science, vol. 11895, Springer, 2019, págs. 20-35. DOI: 10.1007/978-3-030-33702-5_2

[27] R. Fresno-Aranda, P. Fernández, A. Gámez-Díaz, A. Durán y A. Ruiz-Cortés, «Pricing4APIs: A Rigorous Model for RESTful API Pricings,» *Computer Standards & Interfaces*, vol. 91, pág. 103 878, ene. de 2025, ISSN: 0920-5489. DOI: 10.1016/j.csi.2024.103878

[28] A. Grummit, *Capacity Management – A Practitioner Guide* (ITSM Library). Van Haren Publishing, 2010.

[29] M. Fowler y J. Lewis, *Microservices*, 2014. visitado 2 de ago. de 2026. dirección: https://martinfowler.com/articles/microservices.html

[30] R. Fresno-Aranda, P. Fernández, A. Durán y A. Ruiz-Cortés, «Semi-automated Capacity Analysis of Limitation-Aware Microservices Architectures,» en *Economics of Grids, Clouds, Systems, and Services (GECON 2022)*, ép. Lecture Notes in Computer Science, vol. 13430, Springer, 2023, págs. 75-88. DOI: 10.1007/978-3-031-29315-3_7

[31] R. Fresno-Aranda, P. Fernández y A. Ruiz-Cortés, «Smart LAMA API: Automated Capacity Analysis of Limitation-Aware Microservices Architectures,» en *Actas de las XVII Jornadas de Ciencia e Ingeniería de Servicios (JCIS 2022)*, Sistedes, 2022.

[32] R. Fresno-Aranda, «Automated Capacity Analysis of Limitation-Aware Microservices Architectures,» Tesis doct., Universidad de Sevilla, 2025.

[33] R. Fresno-Aranda, P. Fernández y A. Ruiz-Cortés, «A Catalogue of Analysis Operations for API Pricing Plans,» en *Actas de las XVIII Jornadas de Ciencia e Ingeniería de Servicios (JCIS 2023)*, Sistedes, 2023.

[34] A. Ruiz-Cortés y M. Ruiz-López, «Gestión de la capacidad y el rendimiento de APIs bajo el enfoque de caja negra,» en *Actas de las XX Jornadas de Ciencia e Ingeniería de Servicios (JCIS 2025)*, J. Boubeta-Puig, ed., Sistedes, 2025. dirección: https://hdl.handle.net/11705/JCIS/2025/47

[35] D. Ruiz López y R. Gavira Sánchez, *Pricing4API: A Python Library for Modeling and Reasoning about API Pricing Plans*, 2024. visitado 2 de ago. de 2026. dirección: https://github.com/PRICING4API/Pricing4API

[36] R. Gavira Sánchez, «Modelo visual y analítico de la capacidad de APIs desde la perspectiva del cliente,» Trabajo Fin de Grado, Escuela Técnica Superior de Ingeniería Informática, Universidad de Sevilla, 2025.

[37] GitHub, *Rate limits for the REST API*, 2026. visitado 2 de ago. de 2026. dirección: https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api

[38] Anthropic, *Introducing the Model Context Protocol*, 2024. visitado 2 de ago. de 2026. dirección: https://www.anthropic.com/news/model-context-protocol

[39] D. Ruiz López, `provider-service-resolver` *(v1.1) y* `saas-interface-channel-analyzer` *(v3.0): skills para caracterizar servicios SaaS*, Material sin publicar, facilitado por su autor en octubre de 2026, 2026.

[40] Joint Task Force, «Security and Privacy Controls for Information Systems and Organizations,» National Institute of Standards y Technology, NIST Special Publication 800-53 Rev. 5, 2020. DOI: 10.6028/NIST.SP.800-53r5

[41] *Apuntes de la asignatura FIS*, Máster en Ingeniería del Software: Cloud, Datos y Gestión TI, Universidad de Sevilla.

[42] Postmark, *What are Transactional emails? — Postmark Support Center*, 2026. visitado 28 de sep. de 2026. dirección: https://postmarkapp.com/support/article/804-what-are-transactional-emails

[43] API-TaskForce, *apis_consumption_model: API Datasheets — guía del esquema ACM2YAML y datasheets de referencia*, 2026. visitado 3 de ago. de 2026. dirección: https://github.com/API-TaskForce/apis_consumption_model

[44] Amazon Web Services, *Amazon Simple Email Service (SES) Pricing*, 2026. visitado 18 de sep. de 2026. dirección: https://aws.amazon.com/ses/pricing/

[45] Postmark, *Postmark Developer Documentation: API Overview*, 2026. visitado 2 de ago. de 2026. dirección: https://postmarkapp.com/developer/api/overview

[46] Brevo, *FAQs: What are the limits of the Free plan? — Brevo Help Center*, 2026. visitado 16 de sep. de 2026. dirección: https://help.brevo.com/hc/en-us/articles/208580669-FAQs-What-are-the-limits-of-the-Free-plan

[47] Mailchimp, *Fundamentals — Mailchimp Transactional API*, 2026. visitado 21 de sep. de 2026. dirección: https://mailchimp.com/developer/transactional/docs/fundamentals/

[48] API-TaskForce, *datasheet-usage-testing: banco de pruebas para la medición empírica de límites de uso de APIs*, 2026. visitado 3 de ago. de 2026. dirección: https://github.com/API-TaskForce/datasheet-usage-testing

[49] Postmark, *Postmark Pricing*, 2026. visitado 2 de ago. de 2026. dirección: https://postmarkapp.com/pricing

[50] Resend, *Resend Pricing*, 2026. visitado 5 de ago. de 2026. dirección: https://resend.com/pricing

[51] Twilio SendGrid, *Twilio SendGrid: Email API Pricing*, 2026. visitado 6 de sep. de 2026. dirección: https://www.twilio.com/en-us/products/email-api/pricing

[52] MailerSend, *MailerSend Pricing*, 2026. visitado 8 de sep. de 2026. dirección: https://www.mailersend.com/pricing

[53] Brevo, *Brevo Pricing*, 2026. visitado 16 de sep. de 2026. dirección: https://www.brevo.com/pricing/

[54] Mailgun, *Transactional Email API Service For Developers — Mailgun Pricing*, 2026. visitado 21 de sep. de 2026. dirección: https://www.mailgun.com/pricing/

[55] Mailchimp, *Transactional Email Pricing — Mailchimp*, 2026. visitado 21 de sep. de 2026. dirección: https://mailchimp.com/pricing/transactional-email/

[56] Resend, *Resend Developer Documentation: API Reference*, 2026. visitado 5 de ago. de 2026. dirección: https://resend.com/docs/api-reference/introduction

[57] Twilio SendGrid, *Twilio SendGrid Developer Documentation: Mail Send API Reference*, 2026. visitado 6 de sep. de 2026. dirección: https://www.twilio.com/docs/sendgrid/api-reference/mail-send

[58] MailerSend, *General API resources — MailerSend Developers*, 2026. visitado 8 de sep. de 2026. dirección: https://developers.mailersend.com/general.html

[59] Brevo, *API rate limits — Brevo Developers*, 2026. visitado 16 de sep. de 2026. dirección: https://developers.brevo.com/docs/api-limits

[60] Brevo, *About Brevo's pricing plans — Brevo Help Center*, 2026. visitado 16 de sep. de 2026. dirección: https://help.brevo.com/hc/en-us/articles/208589409-About-Brevo-s-pricing-plans

[61] Brevo, *Add an attachment to a transactional email — Brevo Help Center*, 2026. visitado 16 de sep. de 2026. dirección: https://help.brevo.com/hc/en-us/articles/4402811730962-Add-an-attachment-to-a-transactional-email

[62] Brevo, *Transactional Email Service with 99 % Deliverability — Brevo*, 2026. visitado 16 de sep. de 2026. dirección: https://www.brevo.com/products/transactional-email/

[63] Amazon Web Services, *Service quotas in Amazon SES — Amazon SES Developer Guide*, 2026. visitado 18 de sep. de 2026. dirección: https://docs.aws.amazon.com/ses/latest/dg/quotas.html

[64] Mailgun, *API Overview — Mailgun Developer Documentation*, 2026. visitado 21 de sep. de 2026. dirección: https://documentation.mailgun.com/docs/mailgun/api-reference/api-overview

[65] Twilio SendGrid, *Twilio SendGrid Developer Documentation: Rate Limits*, 2026. visitado 6 de sep. de 2026. dirección: https://www.twilio.com/docs/sendgrid/api-reference/how-to-use-the-sendgrid-v3-api/rate-limits

A. Uso de inteligencia artificial

Este anexo declara el uso de inteligencia artificial en el trabajo. La primera sección describe su alcance, y la segunda reproduce íntegra la consulta con la que se completó la selección de APIs (Sección 4.1).

A.1. Alcance del uso

El trabajo se ha hecho con Claude, de Anthropic, en sus entornos Cowork y Claude Code, con la suscripción que imputa la Sección 2.3. Además de la consulta de la selección, el asistente interviene en seis tareas.

Construcción de las Datasheets. Un agente lee las fuentes y construye cada Datasheet aplicando el libro de criterios del catálogo. Otro agente, que solo ve la Datasheet terminada, vuelve a abrir cada URL citada y contrasta los datos uno a uno. Un dato entra en el catálogo solo si los dos lo leyeron igual, y además pasa comprobaciones automáticas de formato y de citas contra la página archivada.

Criterios no previstos. Las dudas de modelado que el libro de criterios no resuelve las delibera un consejo de agentes. Cada veredicto se escribe como regla nueva del libro, y solo llega al autor el empate que el consejo no resuelve.

Sondeos empíricos. Desde el 16 de septiembre de 2026 los ejecutan los agentes, con envíos limitados a tres direcciones del autor. Antes los ejecutaba el autor.

Análisis. El recuento de recurrencia de las fricciones y las ejecuciones sobre PRIME4API los preparan agentes, con los resultados versionados.

Herramientas. Los programas del repositorio, como el validador de formato, la verificación de citas o el cálculo de horas del Capítulo 2, se programaron con el asistente y tienen pruebas automáticas.

Redacción. El asistente redacta borradores de la memoria y de los documentos de trabajo con el registro del autor, medido sobre su Trabajo Fin de Grado. Los revisan otros agentes: uno por capítulo lee como un tribunal, un verificador adversarial contrasta cada hallazgo y un modelo auxiliar vota con él qué arreglos se aplican.

El autor fija el alcance, las preguntas de investigación, el índice, los criterios de selección y todo lo que afecta a la relación con los tutores. Decide también lo que los agentes no resuelven y lee el documento completo antes de la entrega.

A.2. La consulta de la selección

Esta sección registra la consulta a un modelo de lenguaje con la que se completó la selección de APIs del catálogo. Reproduce el prompt y la respuesta tal como se registraron, sin corregir su contenido. Lo que la construcción de las Datasheets corrigió después se indica al final de la sección.

La Tabla A.1 recoge las condiciones de la consulta que constan en el registro, y también las que no constan.

Campo	**Valor**
Fecha	1 de septiembre de 2026
Modelo	`claude-fable-5-1`, de Anthropic
Entorno	La sesión de trabajo del proyecto, a petición del autor
Criterios	Fijados por el autor antes de la consulta e incluidos en el prompt
Catálogo previo	Postmark, Resend, SendGrid y MailerSend, que el prompt excluye
Respuesta pedida	Cuatro APIs seleccionadas y cuatro de reserva, ordenadas, y para cada una el tipo de modelo de precios, lo que aporta y el riesgo principal, sin inventar cifras
No consta en el registro	La hora, el prompt de sistema, los parámetros de generación, el número de intentos y la transcripción de la sesión

Tabla A.1: Ficha de la consulta, según el registro de la selección.

La transcripción conserva la negrita y la cursiva del original, igual que los términos en letra de código. Solo cambia el marcado: los encabezados de la respuesta pasan a negrita, y su tabla final se reproduce como Tabla A.2.

A.3. Prompt literal

> Actúa como analista de APIs SaaS. Necesito seleccionar cuatro APIs de correo electrónico transaccional para completar un catálogo de ocho, destinado a un Trabajo Fin de Máster sobre Pricing Intelligence que modela la **capacidad contractual** de cada API (cuotas, rates, volumen contratado, sobrecoste) en Datasheets ACM2YAML v0.3 y las analiza con PRIME4API. Ya están en el catálogo: Postmark, Resend, SendGrid y MailerSend. Excluye esas cuatro.
>
> Criterios, por orden de peso:
> 1. **Diversidad del modelo de precios.** El catálogo debe cubrir formas distintas de cobrar: planes por volumen mensual, pago por uso puro, producto vendido como añadido de otro, y planes con límite diario además del mensual. Prioriza las que añadan una forma que aún no esté cubierta.

2. **Documentación pública y sin registro** de precios y de límites (rate limits, tamaños máximos, destinatarios por petición). Sin esto la datasheet no se puede construir con trazabilidad.
3. **Adopción real** en producción, para que la comparación tenga interés práctico.
4. **Rasgo diferenciador** para el análisis de la relación entre límites de uso y rates: rate publicado por endpoint, caudal negociado no publicado, límite diario, etc.
5. **Diversidad geográfica y de tamaño de proveedor**, con al menos una europea.

Devuelve exactamente cuatro APIs seleccionadas y cuatro de reserva, ordenadas, y para cada una: modelo de precios (tipo), qué aporta al catálogo que las demás no, y el riesgo principal para construir su datasheet. No inventes cifras: describe el tipo de modelo, no los precios; los precios se verificarán contra la documentación al construir cada datasheet.

A.4. Respuesta íntegra

Seleccionadas

1. **Amazon SES** — *Pago por uso puro, sin planes ni cuotas mensuales.* Es la única forma de cobro que el catálogo no cubre todavía y la que más tensiona el esquema: no hay «volumen contratado», el coste es lineal y los límites (cuota de envío diaria y tasa máxima por segundo) se asignan por cuenta y crecen con la reputación. Adopción máxima en producción. Riesgo: los límites son por cuenta y no por plan, así que la datasheet tendrá que documentar el valor inicial publicado y el mecanismo de ampliación.
2. **Mailgun** — *Planes por volumen mensual con rate limits publicados por endpoint y sobrecoste por exceso.* Aporta el caso limpio de rate documentado, que es justo la relación límite-rate que el trabajo quiere caracterizar, y es la candidata natural al sondeo empírico. Riesgo: el catálogo de planes ha cambiado varias veces; hay que fijar fecha de captura.
3. **Brevo** (antes Sendinblue) — *Planes por volumen mensual con límite diario en el plan gratuito.* Añade la combinación cuota diaria + cuota mensual, que obliga a modelar dos ventanas de tiempo sobre la misma métrica. Europea (Francia). Riesgo: el producto mezcla marketing y transaccional; hay que acotar la datasheet a la API transaccional, como se hizo con Resend (Q1).
4. **Mailchimp Transactional (Mandrill)** — *Producto vendido como añadido de otro:* solo se contrata con un plan de pago de Mailchimp y se cobra por bloques de envíos. Es una forma de cobro que ninguna otra del catálogo tiene y que pone a prueba el campo `type` (D-003) y la noción de volumen contratado. Riesgo: la dependencia del plan base complica el cálculo del coste total.

Reserva, por orden

5. **Mailjet** — planes por volumen con rate documentado; europea; mismo grupo que Mailgun (Sinch), por lo que aporta menos diversidad que Brevo.
6. **Zoho ZeptoMail** — cobro por créditos prepagados, otra forma de pago por uso.
7. **SparkPost** (Bird) — planes por volumen, muy documentada; menos adopción reciente.

8. **SMTP2GO** — planes por volumen con límites diarios, proveedor pequeño; útil como caso de diversidad de tamaño si cae alguna de las anteriores.

Cobertura de modelos de precio resultante (8 APIs)

Modelo de precios	**APIs**
Planes por volumen mensual	Postmark, SendGrid, MailerSend, Mailgun
Planes por volumen con caudal negociado no publicado	Resend
Planes por volumen con límite diario adicional	Brevo
Pago por uso puro	Amazon SES
Añadido de otro producto, por bloques	Mailchimp Transactional

Tabla A.2: Tabla incluida en la respuesta de la consulta, reproducida literalmente.

La consulta se hizo en la sesión de trabajo del proyecto (Tabla A.1), y la respuesta usa ese contexto: menciona, con sus identificadores internos, dos decisiones previas que el prompt no recoge. Una acota Resend a su producto transaccional, y la otra fija el criterio del campo `type`.

A.5. Cómo se aplicó

La selección no cambió después de la consulta: ninguna API de la reserva sustituyó a una del catálogo (Apartado 4.1.3). La respuesta no da precios ni límites con cifra, así que ninguno de los que recoge el catálogo procede de ella. Cada uno sale de la documentación del proveedor, de su panel o de un sondeo, en la ventana de toma de datos (Apartado 4.1.4). La construcción corrigió tres afirmaciones de la respuesta (sobre Amazon SES, Mailgun y Brevo), y la Sección 4.7 las detalla.

El 16 de octubre de 2026 se hizo una segunda consulta, ajena a la selección. En ella se preguntó cómo se generalizaría cada uno de los doce elementos no soportados que el criterio no promovía o dejaba en suspenso en esa fecha, y la Sección 4.5 resume su respuesta. La respondió Claude Opus 5.5, en una sesión a la que solo se le permitió leer un fichero, sin usar la web. Ese fichero describe cada caso sin su veredicto. El prompt, esa entrada y la respuesta íntegra están en el repositorio del trabajo. La respuesta no cambia ningún dato del catálogo ni ningún veredicto del Capítulo 6.